

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
BỘ MÔN ĐIỀU KHIỂN HỌC**



ĐỒ ÁN TỐT NGHIỆP

Đề tài:

**NGHIÊN CỨU, THIẾT KẾ XE TỰ HÀNH ỨNG
DỤNG AI TRONG GIÁM SÁT HƯ HỎNG MẶT
ĐƯỜNG BÊ TÔNG NHỰA NÓNG**

Sinh viên thực hiện: Nguyễn Mạnh Dũng

Người hướng dẫn: TS. Lê Thị Thuý Nga

Hà Nội – 2026

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
BỘ MÔN ĐIỀU KHIỂN HỌC**



ĐỒ ÁN TỐT NGHIỆP

CHUYÊN NGÀNH: KỸ THUẬT ROBOT VÀ TRÍ TUỆ NHÂN TẠO

Đề tài:

**NGHIÊN CỨU, THIẾT KẾ XE TỰ HÀNH ỨNG
DỤNG AI TRONG GIÁM SÁT HƯ HỎNG MẶT
ĐƯỜNG BÊ TÔNG NHỰA NÓNG**

Sinh viên thực hiện: Nguyễn Mạnh Dũng

Lớp: Kỹ sư Kỹ thuật Robot và Trí tuệ nhân tạo K62

Người hướng dẫn: TS. Lê Thị Thuý Nga

Hà Nội – 2026

NHIỆM VỤ THIẾT KẾ TỐT NGHIỆP

MỤC LỤC

NHIỆM VỤ THIẾT KẾ TỐT NGHIỆP.....	i
MỤC LỤC	ii
DANH MỤC CÁC TỪ VIẾT TẮT	v
DANH MỤC CÁC HÌNH VẼ.....	vii
DANH MỤC CÁC BẢNG BIỂU	ix
MỞ ĐẦU	1
CHƯƠNG 1. TỔNG QUAN CHUNG VỀ CÔNG TÁC KIỂM TRA, ĐÁNH GIÁ CHẤT LƯỢNG MẶT ĐƯỜNG	2
1.1. Những khó khăn thường gặp trong công tác kiểm tra và bảo trì đường bộ truyền thống	2
1.2. Các hệ thống và robot kiểm tra mặt đường hiện nay trên thị trường. 4	
1.2.1. Hệ thống kiểm tra chuyên dụng gắn trên xe ô tô	4
1.2.2. Robot mặt đất cỡ nhỏ.....	5
1.2.3. Giải pháp sử dụng thiết bị bay không người lái.....	6
1.2.4. Tình hình nghiên cứu và ứng dụng tại Việt Nam.....	7
1.3. Xây dựng bài toán cho mô hình robot kiểm tra mặt đường.....	7
1.3.1. Mục tiêu của đề án	7
1.3.2. Giới hạn của đề án	8
1.3.3. Yêu cầu cụ thể với các khối chức năng.....	8
1.3.4. Tiêu chí đánh giá	9
Kết luận chương 1.....	10
CHƯƠNG 2. THIẾT KẾ HỆ THỐNG PHẦN CỨNG CHO MÔ HÌNH ROBOT KIỂM TRA MẶT ĐƯỜNG.....	11
2.1. Thiết kế cơ khí.....	11
2.1.1. Các thành phần tự gia công.....	11
2.1.2. Một số linh kiện cơ khí khác	12
2.2. Thiết kế mạch điện phần cứng	13
2.2.1. Sơ đồ khối chức năng, giải thích các khối trong sơ đồ	14
2.2.2. Lựa chọn các thiết bị tương ứng với các khối chức năng.....	15

2.2.3. Sơ đồ đầu nối mạch điện.....	23
2.3. Kết quả thiết kế.....	25
Kết luận chương 2.....	25
CHƯƠNG 3. XÂY DỰNG THUẬT TOÁN ĐIỀU KHIỂN VÀ GIÁM SÁT	27
3.1. Yêu cầu điều khiển.....	27
3.1.1. Điều khiển động cơ.....	27
3.1.2. Các chế độ điều khiển	30
3.2. Thuật toán xử lý ảnh phát hiện làn đường và nhận diện ổ gà.....	33
3.2.1. Mô hình TwinLiteNet cho bài toán phát hiện làn đường	33
3.2.2. Mô hình YOLOv11 cho bài toán nhận diện ổ gà.....	41
3.3. Giải pháp truyền thông	45
3.3.1. Truyền thông nội bộ trong xe	46
3.3.2. Truyền thông từ xe tới PC	48
3.3.3. Truyền thông từ xe tới điều khiển.....	50
3.4. Xây dựng giao diện giám sát.....	51
3.4.1. Giao diện video	52
3.4.2. Giao diện dữ liệu.....	52
3.4.3. Giao diện bản đồ.....	53
3.5. Lưu đồ thuật toán	54
3.5.1. Lưu đồ thuật toán khối xử lý trung tâm	55
3.5.2. Lưu đồ thuật toán khối xử lý giữ làn đường và truyền thông	61
3.5.3. Lưu đồ thuật toán khối nhận diện ổ gà và lưu trữ.....	66
Kết luận chương 3.....	70
CHƯƠNG 4. THỬ NGHIỆM MÔ HÌNH, ĐÁNH GIÁ KẾT QUẢ VÀ HƯỚNG PHÁT TRIỂN	71
4.1. Thử nghiệm mô hình	71
4.1.1. Thử nghiệm bám làn đường.....	71
4.1.2. Thử nghiệm phát hiện và đo kích thước ổ gà	73
4.1.3. Thử nghiệm truyền thông.....	74
4.2. Đánh giá kết quả	75

4.2.1. Kết quả đạt được	75
4.2.2. Các vấn đề còn tồn tại	75
4.3. Hướng phát triển trong tương lai.....	76
Kết luận chương 4.....	77
TÀI LIỆU THAM KHẢO	78
PHỤ LỤC	79
Phụ lục 1. Chương trình code trên ESP32	79
Phụ lục 2. Chương trình code trên Raspberry Pi 5.....	97
Phụ lục 3. Chương trình code nhận diện ổ gà và lưu trữ trên PC	108

DANH MỤC CÁC TỪ VIẾT TẮT

Từ viết tắt	Tiếng Anh	Tiếng Việt
ADC	Analog-to-Digital Converter	Bộ chuyển đổi tương tự – số
AI	Artificial Intelligence	Trí tuệ nhân tạo
ARAN	Automatic Road Analyzer	Máy phân tích đường bộ tự động
BEV	Bird's-Eye View	Ảnh nhìn từ trên xuống
BLE	Bluetooth Low Energy	Bluetooth năng lượng thấp
CAM	Channel Attention Module	Module attention theo kênh
CNC	Computer Numerical Control	Điều khiển số bằng máy tính
CPU	Central Processing Unit	Bộ xử lý trung tâm
DAC	Digital-to-Analog Converter	Bộ chuyển đổi số – tương tự
DC	Direct Current	Dòng điện một chiều
EMA	Exponential Moving Average	Đường trung bình động hàm mũ
FPS	Frames Per Second	Số khung hình trên giây
GPIO	General Purpose Input/Output	Các chân đầu vào/ra đa năng
GPS	Global Positioning System	Hệ thống định vị toàn cầu
HDMI	High-Definition Multimedia Interface	Giao diện đa phương tiện độ nét cao
HTTP	HyperText Transfer Protocol	Giao thức truyền tải siêu văn bản
ID	Identifier	Mã định danh
IP	Internet Protocol	Giao thức Internet
JSON	JavaScript Object Notation	Định dạng dữ liệu JavaScript Object
mAP50	mean Average Precision at IoU=0.50	Độ chính xác trung bình tại ngưỡng IoU=0.5

mAP50-95	mean Average Precision over IoU thresholds 0.50 to 0.95	Độ chính xác trung bình qua các ngưỡng IoU 0.5-0.95
mIoU	mean Intersection over Union	Độ tương giao trung bình
PAM	Position Attention Module	Module attention theo vị trí
PCSS	Pavement Condition Survey System	Hệ thống khảo sát tình trạng mặt đường
PD	Power Delivery	Cung cấp nguồn
PID	Proportional-Integral-Derivative	Bộ điều khiển tỉ lệ – tích phân – vi phân
PWM	Pulse Width Modulation	Điều chế độ rộng xung
PyQt5	Python bindings for Qt5	Bộ công cụ Python cho Qt5
ROI	Region of Interest	Vùng quan tâm
RPM	Revolutions Per Minute	Số vòng quay trên phút
SPI	Serial Peripheral Interface	Giao diện ngoại vi nối tiếp
TCP	Transmission Control Protocol	Giao thức điều khiển truyền vận
UART	Universal Asynchronous Receiver/Transmitter	Bộ thu/phát bất đồng bộ đa năng
UDP	User Datagram Protocol	Giao thức gói dữ liệu người dùng
USB	Universal Serial Bus	Bus nối tiếp đa năng
VRAMP	Vietnam Road Asset Management Project	Dự án quản lý tài sản đường bộ Việt Nam
WGS-84	World Geodetic System 1984	Hệ thống trắc địa thế giới 1984
YOLO	You Only Look Once	Mô hình phát hiện đối tượng "You Only Look Once"

DANH MỤC CÁC HÌNH VẼ

Hình 1.1. Ảnh minh họa công nhân đang kiểm tra ổ gà trực tiếp trên đường.....	2
Hình 1.2. Xe khảo sát mặt đường ARAN.....	5
Hình 1.3. Robot RABIT trang bị camera và cảm biến để kiểm tra mặt đường.....	6
Hình 1.4. Drone được sử dụng để chụp ảnh và phát hiện hư hỏng mặt đường....	6
Hình 1.5. Hệ thống thiết bị khảo sát tình trạng mặt đường (PCSS).....	7
Hình 1.6. Sơ hình thử nghiệm cho đồ án.....	8
Hình 2.1. Bản vẽ khung gầm xe.....	11
Hình 2.2. Thiết kế các chi tiết in 3D và lắp ráp vào khung gầm của xe.....	12
Hình 2.3. Giá treo camera.....	12
Hình 2.4. Bánh xe và khớp lực giác	13
Hình 2.5. Bộ truyền đai GT2.....	13
Hình 2.6. Trục bánh sau và vòng bi	13
Hình 2.7. Sơ đồ khối chức năng phần cứng.....	14
Hình 2.8. Động cơ DC Servo JGB37-520 176RPM	16
Hình 2.9. Pin Lipo 3S Tiger	17
Hình 2.10. Động cơ servo MG996R.....	17
Hình 2.11. Mạch hạ áp DC-DC LM2596 3A	18
Hình 2.12. Driver BTS7960	18
Hình 2.13. Cảm biến HY-SRF05	19
Hình 2.14. Camera DFRobot 0.3MP	19
Hình 2.15. Module GPS G-Mouse VK162	20
Hình 2.16. Raspberry Pi 5.....	20
Hình 2.17. Mạch nguồn 52Pi PD Power	21
Hình 2.18. Màn hình LCD Waveshare 7 inch HDMI.....	22
Hình 2.19. Pin sạc dự phòng Energizer.....	22
Hình 2.20. Vi điều khiển ESP32	23
Hình 2.21. Sơ đồ đấu nối hệ thống.....	24
Hình 2.22. Kết quả thiết kế mô hình.....	25
Hình 3.1. Đồ thị đáp ứng vận tốc của xe với $K_p=1,86$; $K_i=18,6$; $K_d=0,046$...	28
Hình 3.2. Đồ thị đáp ứng vận tốc của xe với $K_p=1,86$; $K_i=4,5$; $K_d=0,046$	29
Hình 3.3. Giao diện WebSocket điều khiển xe	32
Hình 3.4. Kiến trúc của mô hình học sâu TwinLiteNet.....	33
Hình 3.5. Kết quả so sánh của mô hình	34
Hình 3.6. Gán nhãn làn đường và vùng đường	34
Hình 3.7: Đồ thị biểu diễn sự biến thiên của hàm Loss và chỉ số mIoU trong quá trình huấn luyện	36

Hình 3.8. So sánh YOLOv11 với các phiên bản trước và những mô hình khác trên tập dữ liệu COCO	41
Hình 3.9 Các module kiến trúc chính trong YOLOv11	42
Hình 3.10. Tập dữ liệu để train mô hình.....	43
Hình 3.11. Đồ thị huấn luyện của mô hình YOLOv11.....	44
Hình 3.12. Sơ đồ truyền thông giữa các khối	46
Hình 3.13. Giao diện video ở tab Camera.....	52
Hình 3.14. Giao diện tab Dữ liệu.....	53
Hình 3.15. Giao diện tab Bản đồ	54
Hình 3.16. Lưu đồ thuật toán khối xử lý trung tâm	55
Hình 3.17. Lưu đồ thuật toán module Quản lý trạng thái kết nối và chuyển đổi chế độ.....	56
Hình 3.18. Lưu đồ thuật toán module Nhận lệnh từ Pi 5.....	57
Hình 3.19. Lưu đồ thuật toán module Chế độ an toàn chủ động.....	58
Hình 3.20. Lưu đồ thuật toán module Gửi gói tin cho Raspberry Pi 5 qua UART	59
Hình 3.21. Lưu đồ thuật toán module Duy trì WiFi, xử lý HTTP/WebSocket.....	60
Hình 3.22. Lưu đồ thuật toán khối xử lý giữ làn đường và truyền thông	61
Hình 3.23. Lưu đồ thuật toán module TelemetryThread	62
Hình 3.24. Lưu đồ thuật toán module BEVThread	63
Hình 3.25. Lưu đồ thuật toán module AIThread	64
Hình 3.26. Lưu đồ thuật toán module UDPSenderThread	65
Hình 3.27. Lưu đồ thuật toán khối Nhận diện ổ gà và lưu trữ	66
Hình 3.28. Lưu đồ thuật toán module Nhận gói UDP từ xe, ghép khung hình & giải nén	67
Hình 3.29. Lưu đồ thuật toán module Nhận diện & lưu trữ ổ gà	68
Hình 3.30. Lưu đồ thuật toán module Cập nhật đồ họa UI.....	69
Hình 4.1. Giao diện camera trên xe đang chạy khi ở chế độ tự động	71
Hình 4.2. Quá trình xe di chuyển trên đường sa hình	72
Hình 4.3. Thử nghiệm nhận diện làn đường và vùng đường ngoài thực tế.....	72
Hình 4.4. Ảnh chụp quá trình đo thủ công ổ gà bằng thước dây.....	73

DANH MỤC CÁC BẢNG BIỂU

<i>Bảng 3.1. Bảng thông số kỹ thuật động cơ JGB37-520 176RPM.....</i>	<i>27</i>
<i>Bảng 3.2. Các mô hình có sẵn của YOLOv11 cho tác vụ phát hiện đối tượng...</i>	<i>42</i>
<i>Bảng 3.3. Kết quả đánh giá mô hình trên tập xác thực</i>	<i>44</i>
<i>Bảng 4.1. Kết quả đo kích thước ổ gà bằng hệ thống sau 10 lần đo</i>	<i>73</i>

MỞ ĐẦU

Trong những năm qua, cùng với sự phát triển mạnh mẽ của kinh tế xã hội, hệ thống hạ tầng giao thông đường bộ tại Việt Nam đã được đầu tư mở rộng và nâng cấp không ngừng. Tuy nhiên, dưới áp lực từ lưu lượng phương tiện ngày càng lớn cùng những tác động tiêu cực của điều kiện thời tiết, bề mặt đường giao thông thường xuyên xuất hiện các hư hỏng, đặc biệt là tình trạng ổ gà. Hiện nay, công tác kiểm tra và đánh giá chất lượng mặt đường vẫn chủ yếu dựa vào phương pháp thủ công, điều này không chỉ gây tốn kém về thời gian, nhân lực mà còn tiềm ẩn nhiều nguy cơ mất an toàn cho cán bộ kỹ thuật khi làm việc trực tiếp trên đường. Trước thực tế đó, việc tìm kiếm một giải pháp tự động hóa để giám sát hạ tầng giao thông trở thành một yêu cầu thực tiễn và cấp thiết.

Đề tài “Nghiên cứu, thiết kế xe tự hành ứng dụng AI trong giám sát hư hỏng mặt đường bê tông nhựa nóng” được thực hiện với mục tiêu xây dựng một hệ thống thông minh có khả năng tự động hóa quy trình tuần tra đường bộ. Hệ thống tập trung vào việc thiết kế robot có khả năng tự bám làn đường ổn định thông qua thuật toán thị giác máy tính TwinLiteNet được thực thi trên bộ vi xử lý Raspberry Pi 5. Song song với đó, luồng dữ liệu hình ảnh được truyền về máy chủ để ứng dụng mô hình Deep Learning YOLO nhằm nhận diện chính xác các ổ gà trên mặt đường. Việc tích hợp thêm công nghệ định vị vệ tinh GPS giúp hệ thống tự động ghi lại tọa độ hư hỏng và truyền về giao diện giám sát theo thời gian thực qua giao thức UDP, phục vụ hiệu quả cho công tác quản lý và duy tu đường bộ.

Nội dung nghiên cứu của đề án ngoài phần mở đầu, kết luận và tài liệu tham khảo được cấu trúc qua các chương chính như sau:

- Chương 1: Tổng quan chung về công tác kiểm tra, đánh giá chất lượng mặt đường.
- Chương 2: Thiết kế hệ thống phần cứng cho mô hình robot kiểm tra mặt đường.
- Chương 3: Xây dựng thuật toán điều khiển và giám sát.
- Chương 4: Thử nghiệm mô hình, đánh giá kết quả và hướng phát triển.

Để hoàn thành đề án này, em xin bày tỏ lòng biết ơn sâu sắc tới TS. Lê Thị Thúy Nga, người đã tận tình hướng dẫn, chỉ bảo và tạo mọi điều kiện thuận lợi cho em trong suốt quá trình làm đề án. Em cũng xin gửi lời cảm ơn chân thành tới các thầy cô giáo trong bộ môn Điều khiển học, khoa Điện - Điện tử đã truyền đạt những kiến thức quý báu làm nền tảng cho em thực hiện đề tài. Cuối cùng, em xin cảm ơn gia đình và bạn bè đã luôn đồng hành, giúp đỡ em trong quá trình học tập và hoàn thiện đề án tốt nghiệp này.

CHƯƠNG 1. TỔNG QUAN CHUNG VỀ CÔNG TÁC KIỂM TRA, ĐÁNH GIÁ CHẤT LƯỢNG MẶT ĐƯỜNG

1.1. Những khó khăn thường gặp trong công tác kiểm tra và bảo trì đường bộ truyền thống

Công tác kiểm tra và bảo trì đường bộ là một nhiệm vụ quan trọng nhưng đầy thách thức, đặc biệt khi vẫn còn phụ thuộc chủ yếu vào các phương pháp thủ công và truyền thống. Trong nhiều năm qua, các cơ quan quản lý đường bộ tại Việt Nam và nhiều nước đang phát triển vẫn sử dụng hình thức nhân công trực tiếp kết hợp với các dụng cụ đo lường đơn giản như thước dây, thước thẳng, máy ảnh kỹ thuật số cầm tay và ghi chép sổ sách. Dù có ưu điểm là dễ thực hiện và không đòi hỏi thiết bị đắt tiền, phương pháp này bộc lộ rất nhiều khó khăn và hạn chế, đặt ra yêu cầu cấp thiết phải có giải pháp tự động hóa hiện đại.

Một trong những khó khăn lớn nhất là sự phụ thuộc hoàn toàn vào nhân lực, kéo theo chi phí vận hành cao và rủi ro mất an toàn lao động nghiêm trọng. Một đội kiểm tra truyền thống thường cần ít nhất 3-5 người, bao gồm kỹ thuật viên đo đạc, người ghi chép, người chụp ảnh và lái xe. Họ phải làm việc trực tiếp trên mặt đường, thường xuyên đối diện với các phương tiện giao thông đang lưu thông. Dù có đặt biển báo và rào chắn, tai nạn vẫn có thể xảy ra, đặc biệt trên các tuyến quốc lộ đông đúc hoặc cao tốc. Điều này tiềm ẩn nhiều nguy hiểm bất ngờ do tốc độ xe cộ cao cũng như khả năng quan sát hạn chế của người lái xe. Rõ ràng, yếu tố an toàn là một thách thức không thể xem nhẹ.



Hình 1.1. Ảnh minh họa công nhân đang kiểm tra ổ gà trực tiếp trên đường

Tiếp theo, tính chủ quan và sai số lớn là một nhược điểm cố hữu của phương pháp thủ công. Việc xác định kích thước chiều dài, chiều rộng, độ sâu của ổ gà hay vết nứt phụ thuộc vào cảm quan và kinh nghiệm của từng người. Cùng một hư hỏng, người này đánh giá là nghiêm trọng, người kia cho là trung bình. Số liệu

ghi chép bằng tay dễ bị nhầm lẫn, tẩy xóa, và khó đồng bộ với ảnh chụp. Khi tập hợp dữ liệu từ nhiều tổ, nhiều đợt về văn phòng, sai số tích lũy có thể lên tới 10-20%, ảnh hưởng trực tiếp đến việc lập dự toán sửa chữa và ưu tiên xử lý các điểm hư hỏng nguy hiểm.

Không kém phần quan trọng là thời gian xử lý chậm, không đáp ứng yêu cầu giám sát liên tục. Quy trình truyền thống thường được thực hiện theo định kỳ (ví dụ 3 tháng, 6 tháng một lần). Sau khi thu thập số liệu thực địa, các kỹ thuật viên phải quay về văn phòng, nhập liệu vào Excel, xử lý ảnh, vẽ sơ đồ và viết báo cáo quản lý. Toàn bộ chu trình này thường mất từ 2 đến 4 tuần. Trong khoảng thời gian đó, các hư hỏng có thể trở nên trầm trọng hơn, gây nguy cơ tai nạn cho người tham gia giao thông, hoặc các điểm hỏng mới phát sinh không được ghi nhận kịp thời. Đặc biệt sau mùa mưa lũ, tình trạng ổ gà xuất hiện hàng loạt, phương pháp thủ công hoàn toàn lúng túng trong việc cập nhật trạng thái mặt đường theo thời gian thực.

Bên cạnh đó, các phương pháp truyền thống hầu như không tận dụng được các công nghệ mới như định vị vệ tinh GPS, xử lý ảnh số, thị giác máy tính và trí tuệ nhân tạo. Việc xác định vị trí hư hỏng chủ yếu dựa vào mốc lý trình (km số + m) hoặc các địa danh gần nhất, dễ bị nhầm lẫn và thiếu chính xác so với tọa độ địa lý. Ảnh chụp chỉ dừng ở dạng tư liệu tham khảo, không được phân tích tự động để đếm số lượng, phân loại hay tính kích thước. Kết quả kiểm tra không được số hóa thành bản đồ hư hỏng trực quan, gây cản trở lớn cho việc lưu trữ lâu dài, truy xuất và chia sẻ dữ liệu giữa các đơn vị quản lý đường bộ, thanh tra giao thông và đội bảo trì.

Cuối cùng, không thể không kể đến tác động tiêu cực đến môi trường và hiệu quả kinh tế - xã hội. Việc huy động nhiều xe chuyên dụng (xe tải chở công nhân, xe cảnh báo, xe chở thiết bị) trong thời gian kéo dài, chạy trên nhiều cung đường làm gia tăng đáng kể lượng khí thải CO₂ và bụi mịn. Hơn nữa, khi phải đóng một phần làn đường để kiểm tra, giao thông bị ùn tắc, thời gian di chuyển của người dân bị kéo dài, làm chậm sự phát triển của xã hội và lãng phí ngân sách.

Tóm lại, các khó khăn kể trên cho thấy việc duy trì phương pháp kiểm tra và bảo trì mặt đường truyền thống là không còn phù hợp trong bối cảnh hạ tầng giao thông ngày càng hiện đại và yêu cầu quản lý ngày càng khắt khe. Những vấn đề về an toàn lao động, tính chính xác, tốc độ xử lý, khả năng tiếp cận, ứng dụng công nghệ và hiệu quả kinh tế - xã hội đã đặt ra yêu cầu bức thiết phải nghiên cứu, phát triển các giải pháp tự động hóa, đặc biệt là sử dụng robot tự hành kết hợp trí tuệ nhân tạo, để thay thế hoặc hỗ trợ đắc lực cho con người trong nhiệm vụ giám sát và đánh giá chất lượng mặt đường. Đây chính là cơ sở thực tiễn cho việc lựa

chọn đề tài "Nghiên cứu, thiết kế xe tự hành ứng dụng AI trong giám sát hư hỏng mặt đường bê tông nhựa nóng" trong đề án này.

1.2. Các hệ thống và robot kiểm tra mặt đường hiện nay trên thị trường

Nhận thức được những hạn chế của phương pháp thủ công, các quốc gia phát triển trên thế giới đã nghiên cứu và ứng dụng nhiều hệ thống cơ giới hóa, tự động hóa vào công tác kiểm tra mặt đường. Dựa trên phương thức hoạt động và quy mô ứng dụng, các hệ thống này có thể được phân chia thành ba nhóm chính: hệ thống kiểm tra chuyên dụng gắn trên xe ô tô (xe khảo sát đường bộ), robot mặt đất cỡ nhỏ và các giải pháp sử dụng thiết bị bay không người lái (drone).

1.2.1. Hệ thống kiểm tra chuyên dụng gắn trên xe ô tô

Đây là nhóm giải pháp lâu đời và được sử dụng rộng rãi nhất ở các nước có hệ thống giao thông phát triển, đặc biệt phục vụ kiểm tra định kỳ đường cao tốc và quốc lộ. Thiết bị tiêu biểu cho nhóm này là hệ thống ARAN (Automatic Road Analyzer) do hãng Fugro Roadware (Canada) sản xuất. Với bề dày kinh nghiệm từ năm 1969, ARAN được coi là “tiêu chuẩn thực tế” trong lĩnh vực thu thập dữ liệu mặt đường trên toàn thế giới.

ARAN là một xe tải hoặc xe van được trang bị hệ thống cảm biến đa dạng, bao gồm camera độ phân giải cao, laser scanner và hệ thống định vị quán tính (INS) kết hợp GPS. Hệ thống này có thể hoạt động ở tốc độ giao thông bình thường, vừa di chuyển vừa tự động thu thập các chỉ số như độ nhám, độ lún vệt bánh xe (rutting), vết nứt, kết cấu hình học của đường và chụp ảnh toàn cảnh tuyến đường. Dữ liệu sau đó được xử lý bởi phần mềm Vision để tạo ra các báo cáo và bản đồ số phục vụ quản lý. Mô-đun hóa là đặc điểm nổi bật của ARAN, với 3 dòng sản phẩm chính cung cấp từ 2 đến 32 hệ thống con khác nhau, có thể dễ dàng nâng cấp hoặc chuyển đổi giữa các xe.

Tuy nhiên, nhược điểm của các hệ thống dòng này là chi phí đầu tư rất lớn (hàng trăm nghìn đến hàng triệu đô la Mỹ), yêu cầu vận hành chuyên nghiệp và thường chỉ phù hợp với các cơ quan quản lý đường bộ cấp quốc gia hoặc các bang, thành phố lớn. Thêm vào đó, mặc dù thu thập dữ liệu rất nhanh, nhưng việc xử lý và phân tích dữ liệu sau đó thường mất thời gian, dẫn đến độ trễ trong công tác bảo trì.



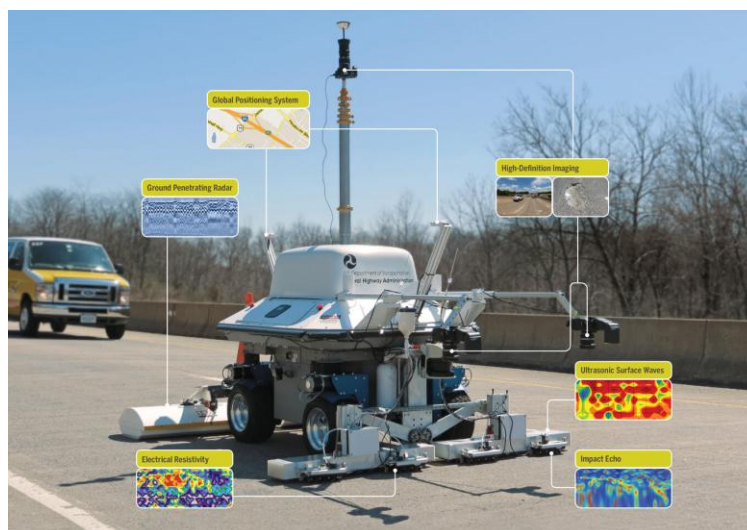
Hình 1.2. Xe khảo sát mặt đường ARAN

1.2.2. Robot mặt đất cỡ nhỏ

Đây là nhóm giải pháp nổi lên mạnh mẽ trong vòng một vài năm trở lại đây, nhờ sự phát triển của vi điều khiển, cảm biến chi phí thấp và trí tuệ nhân tạo. Các robot mặt đất có kích thước nhỏ gọn hơn nhiều so với xe tải khảo sát, thường được thiết kế dạng bánh lốp hoặc bánh xích, có khả năng di chuyển linh hoạt và thực hiện kiểm tra chi tiết ở tốc độ chậm.

Các dự án nghiên cứu điển hình có thể kể đến như robot PAVIA của Đại học Toronto nghiên cứu về chiến lược kiểm tra tự động, hay phương pháp phát hiện vết nứt bằng robot phối hợp drone-robot của ScienceDirect. Robot RABIT (Robotics Assisted Bridge Inspection Tool) cũng được phát triển để kiểm tra cầu đường với tốc độ thu thập dữ liệu tự động nhanh gấp 3 lần so với đội 5 kỹ thuật viên thủ công. Nhiều nghiên cứu gần đây tập trung vào việc tích hợp AI và thị giác máy tính trực tiếp trên robot, cho phép nhận diện và phân loại hư hỏng theo thời gian thực mà không cần gửi dữ liệu về trung tâm xử lý ngay lập tức. Robot có thể được trang bị thêm GPS để gắn tọa độ và các cảm biến khác như lidar để tránh chướng ngại vật.

Nhóm giải pháp này phù hợp với các chiến dịch kiểm tra định kỳ cho các tuyến đường đô thị, đường tỉnh lộ hoặc các khu vực có mật độ giao thông cao. Tuy nhiên, thời gian vận hành pin và tốc độ di chuyển chậm là những rào cản khi cần khảo sát trên phạm vi rất lớn.

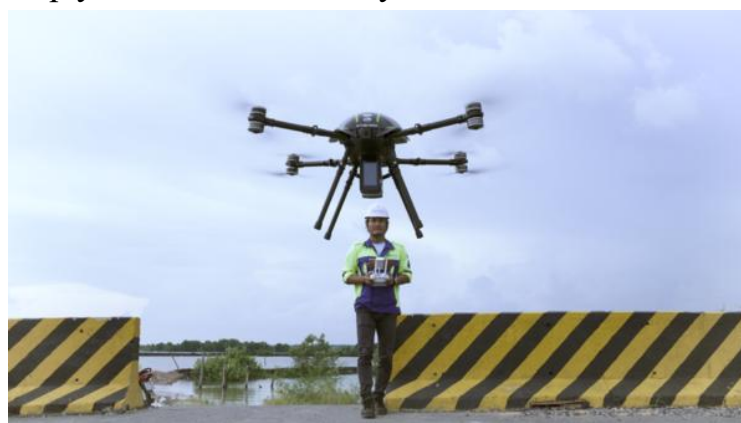


Hình 1.3. Robot RABIT trang bị camera và cảm biến để kiểm tra mặt đường

1.2.3. Giải pháp sử dụng thiết bị bay không người lái

Trong những năm gần đây, cùng với sự phổ cập của drone, các giải pháp kiểm tra mặt đường từ trên không ngày càng thu hút sự quan tâm nghiên cứu. Drone có khả năng bay phía trên tuyến đường, ghi hình toàn cảnh với độ phân giải cao, thuận lợi cho việc phát hiện các dạng hư hỏng có diện tích lớn như vết nứt dài, lún lõm hay ổ gà. Kết hợp với các thuật toán học sâu như YOLO, drone có thể tự động xác định và khoanh vùng các điểm bất thường trên mặt đường ngay trong quá trình bay. Một hệ thống gồm drone và robot mặt đất phối hợp cũng đã được đề xuất để tăng hiệu suất kiểm tra.

Tuy nhiên, giải pháp drone cũng có những hạn chế cố hữu. Thời gian bay của mỗi chuyến drone thường chỉ kéo dài 20-30 phút do hạn chế về pin, gây gián đoạn cho quá trình kiểm tra. Bên cạnh đó, ảnh chụp từ trên cao khó có thể phát hiện chính xác các hư hỏng có kích thước nhỏ hơn hoặc yêu cầu đo đạc chi tiết về độ sâu. Ngoài ra, việc vận hành drone đòi hỏi người điều khiển có tay nghề và phải tuân thủ các quy định về an toàn bay, đặc biệt ở các khu vực đông dân cư.



Hình 1.4. Drone được sử dụng để chụp ảnh và phát hiện hư hỏng mặt đường

1.2.4. Tình hình nghiên cứu và ứng dụng tại Việt Nam

Tại Việt Nam, công tác kiểm tra mặt đường chủ yếu vẫn là thủ công. Một số dự án lớn như VRAMP đã đưa vào sử dụng hệ thống khảo sát chuyên dụng PCSS để thu thập dữ liệu hư hỏng, nhưng chi phí cao và chỉ giới hạn ở các dự án trọng điểm.



Hình 1.5. Hệ thống thiết bị khảo sát tình trạng mặt đường (PCSS)

Trong lĩnh vực robot và AI, một số doanh nghiệp đã bắt đầu phát triển giải pháp Vision AI để phát hiện ổ gà, vết nứt. Các nghiên cứu trong nước chủ yếu ở cấp độ đề tài khoa học, đồ án sinh viên. Nhìn chung, chưa có sản phẩm robot kiểm tra mặt đường giá rẻ, ứng dụng AI nào được thương mại hóa rộng rãi.

Do đó, đề tài “Nghiên cứu, thiết kế xe tự hành ứng dụng AI trong giám sát hư hỏng mặt đường” có ý nghĩa thực tiễn, phù hợp với điều kiện Việt Nam.

1.3. Xây dựng bài toán cho mô hình robot kiểm tra mặt đường

1.3.1. Mục tiêu của đồ án

Đề tài hướng đến việc xây dựng một mô hình robot tự hành cỡ nhỏ, hoạt động trong môi trường có kiểm soát, với các mục tiêu chính sau:

- Thiết kế và chế tạo robot có khả năng tự bám làn đường dựa trên thị giác máy tính.
- Tích hợp mô hình AI phát hiện ổ gà xử lý trên PC.
- Xây dựng giao diện giám sát đơn giản nhận dữ liệu từ robot qua UDP.

Toàn bộ hệ thống nhằm chứng minh tính khả thi của giải pháp robot ứng dụng AI trong điều kiện phòng thí nghiệm, làm cơ sở mở rộng sau này.

1.3.2. Giới hạn của đồ án

Do nguồn lực và thời gian có hạn, đồ án xác định các giới hạn rõ ràng:

Môi trường thử nghiệm: Đường bằng phẳng, vạch kẻ rõ, không có phương tiện tham gia giao thông. Chỉ thử nghiệm trong đường sa hình.



Hình 1.6. Sa hình thử nghiệm cho đồ án

Đối tượng hư hỏng: Chỉ nhận diện ổ gà và đo đặc kích thước, độ sâu; chưa xử lý vết nứt hay bong tróc.

Tốc độ tối đa: $\leq 0,7$ m/s đối với chế độ điều khiển thủ công và $\leq 0,15$ m/s đối với chế độ tự động, ưu tiên độ ổn định của thuật toán.

Quy trình xử lý AI:

- Huấn luyện mô hình YOLO v11 (phát hiện ổ gà) và TwinLiteNet (phát hiện làn đường) trên PC dựa trên dữ liệu thu thập từ sa hình.
- Xuất mô hình sang định dạng ONNX để tối ưu hiệu suất cho Raspberry Pi 5.
- Raspberry Pi 5 chỉ chạy mô hình TwinLiteNet để phát hiện làn đường và tự động điều khiển xe song song với việc gửi video và dữ liệu cảm biến về PC để chạy mô hình YOLO phát hiện ổ gà.

Truyền thông: Gửi gói tin UDP gồm các thông tin độ sâu ổ gà, dữ liệu điều khiển xe, video realtime về PC.

Định vị: Module GPS G-Mouse, chấp nhận sai số 5–15m do đặc thù của dòng module đơn tần, dễ bị ảnh hưởng bởi hiệu ứng đa đường (Multipath) trong môi trường đô thị và sự nhiễu loạn của tầng khí quyển.

1.3.3. Yêu cầu cụ thể với các khối chức năng

Các chức năng chính của robot được xác định như sau:

Bám làn đường: Robot tự động cân chỉnh hướng đi theo vạch kẻ đường. Sử dụng mô hình TwinLiteNet chạy trên Raspberry Pi 5. Từ ảnh camera, mô hình xác định đường đi và xử lý tính toán gửi lệnh điều khiển góc lái.

Phát hiện ổ gà:

- Huấn luyện YOLO-v11-seg trên PC với dữ liệu thu thập trên sa hình.
- Xây dựng giao diện nhận diện ổ gà dạng segmentation từ đó xác định được diện tích thực tế của ổ gà.

- Ổ gà phải đảm bảo nằm trong vùng đường được nhận diện bằng mô hình TwinLiteNet.

Định vị: Module GPS ghi tọa độ (kinh độ, vĩ độ) liên tục để truyền về PC.

Truyền thông:

- Gửi gói tin UDP từ Raspberry Pi 5 về máy tính giám sát qua Wifi được phát từ PC.
- Mỗi gói tin chứa frame ảnh, phần trăm pin, chế độ điều khiển, thời gian, tốc độ, góc lái, tọa độ GPS, dữ liệu 3 cảm biến siêu âm.
- Gửi dữ liệu liên tục về PC theo thời gian thực.

An toàn:

- Cảm biến siêu âm phía trước và sau liên tục đo khoảng cách đến vật cản. Khi phát hiện vật cản ở khoảng cách an toàn, hệ thống tự động giảm tốc; nếu ở khoảng cách nguy hiểm, robot sẽ dừng khẩn cấp. Sau khi vật cản rời khỏi vùng ảnh hưởng, robot tự động tăng tốc trở lại bình thường.
- Hệ thống giám sát liên tục tình trạng kết nối WiFi. Nếu mất kết nối trong một khoảng thời gian nhất định, robot tự động dừng động cơ và đưa góc lái về vị trí trung tâm, chỉ hoạt động trở lại khi kết nối được phục hồi.
- Có cơ chế chống đảo chiều đột ngột, khi nhận lệnh đảo chiều trong lúc đang di chuyển, robot sẽ dừng hẳn rồi mới tăng tốc theo hướng mới, tránh gây sốc cho hệ thống truyền động.
- Khi robot dừng, hệ thống duy trì một lực hãm nhỏ để chống hiện tượng trôi do quán tính hoặc địa hình dốc, giúp robot đứng yên ổn định.

1.3.4. Tiêu chí đánh giá

Hệ thống được đánh giá dựa trên các tiêu chí sau:

- Robot tự động di chuyển ổn định trên các đoạn đường thẳng và cong nhẹ trong sa hình, không cần can thiệp thủ công, sai số bám làn ở mức chấp nhận được.
- Hệ thống nhận diện được phần lớn ổ gà mô phỏng đặt trên làn đường, đo được diện tích và độ sâu tương đối chính xác.
- Các mô hình AI chạy ổn định trên Raspberry Pi 5 và PC, đảm bảo xử lý ảnh theo thời gian thực, không gây giật lag.
- Dữ liệu video và cảm biến được gửi liên tục từ Raspberry Pi 5 về PC qua UDP với độ trễ thấp, ít mất gói tin.
- Module G-Mouse cập nhật tọa độ liên tục, sai số ở mức cho phép đối với thiết bị định vị thông thường.

- Robot phản ứng kịp thời với vật cản, xử lý tốt các tình huống mất kết nối WiFi, đảo chiều êm dịu và giữ ổn định khi dừng. Thời gian hoạt động liên tục đáp ứng yêu cầu thử nghiệm.

Kết luận chương 1

Chương 1 đã trình bày tổng quan về công tác kiểm tra và bảo trì mặt đường, chỉ ra những khó khăn, hạn chế của phương pháp thủ công truyền thống như mất an toàn lao động, sai số lớn, thời gian xử lý chậm và thiếu ứng dụng công nghệ hiện đại. Qua khảo sát các hệ thống kiểm tra mặt đường trên thế giới cũng như tình hình nghiên cứu trong nước, có thể thấy tiềm năng lớn trong việc phát triển robot tự hành giá rẻ ứng dụng trí tuệ nhân tạo để tự động hóa quy trình giám sát hư hỏng đường bộ.

Trên cơ sở đó, bài toán “Nghiên cứu, thiết kế xe tự hành ứng dụng AI trong giám sát hư hỏng mặt đường bê tông nhựa nóng” được xây dựng với các mục tiêu, giới hạn, yêu cầu chức năng cụ thể và tiêu chí đánh giá rõ ràng. Đây là cơ sở để triển khai các nội dung thiết kế phần cứng, xây dựng thuật toán và thử nghiệm mô hình trong các chương tiếp theo.

CHƯƠNG 2. THIẾT KẾ HỆ THỐNG PHẦN CỨNG CHO MÔ HÌNH ROBOT KIỂM TRA MẶT ĐƯỜNG

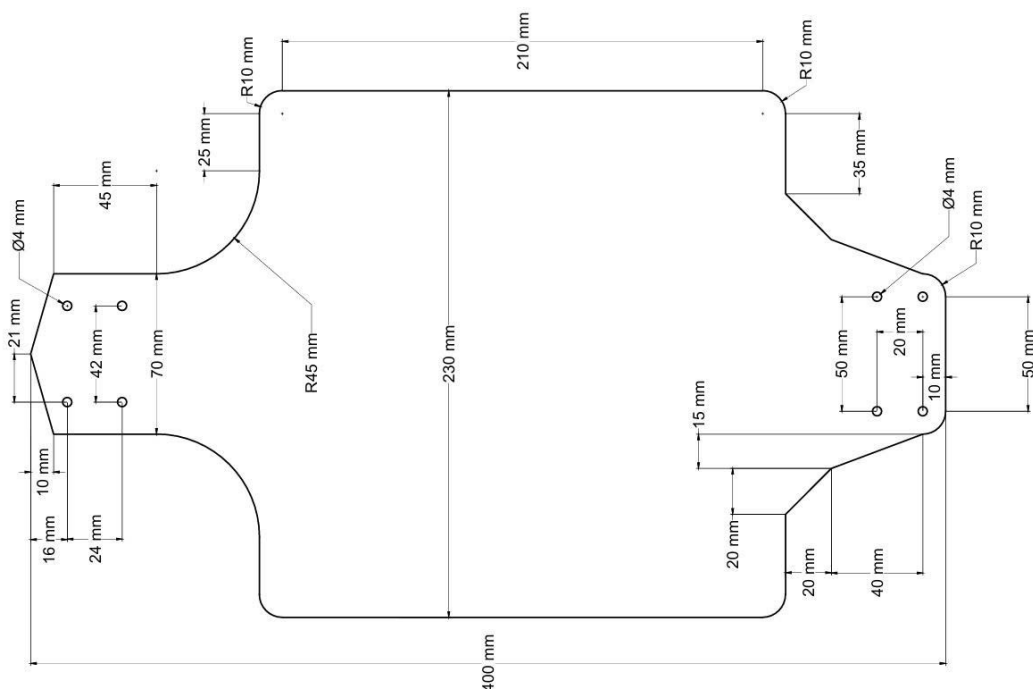
2.1. Thiết kế cơ khí

Hệ thống cơ khí là nền tảng để robot vận hành ổn định, đặc biệt quan trọng đối với bài toán xử lý ảnh yêu cầu hạn chế rung lắc. Robot được thiết kế với kết cấu hỗn hợp: khung chính bằng thép CNC dày 2 mm, các chi tiết phụ trợ in 3D từ nhựa PETG, kết hợp với linh kiện tiêu chuẩn (bánh xe, dây đai, vòng bi). Toàn bộ hệ thống đảm bảo độ cứng vững, trọng tâm thấp, và khả năng lắp đặt gọn gàng các mô-đun điện tử.

2.1.1. Các thành phần tự gia công

Khung gầm chính:

- Vật liệu gia công: Tấm thép CT3 dày 2 mm, cắt chính xác bằng máy CNC.
- Kích thước: Dài 400 mm, rộng 230 mm.
- Vai trò: Tạo nên bộ khung xương cứng vững, chịu tải trọng 4,1 kg mà không bị võng hay biến dạng.
- Lợi ích: Trọng lượng thép giúp hạ thấp trọng tâm, giảm rung chấn truyền lên camera và vi xử lý; đảm bảo robot di chuyển đầm chắc trên sa hình.

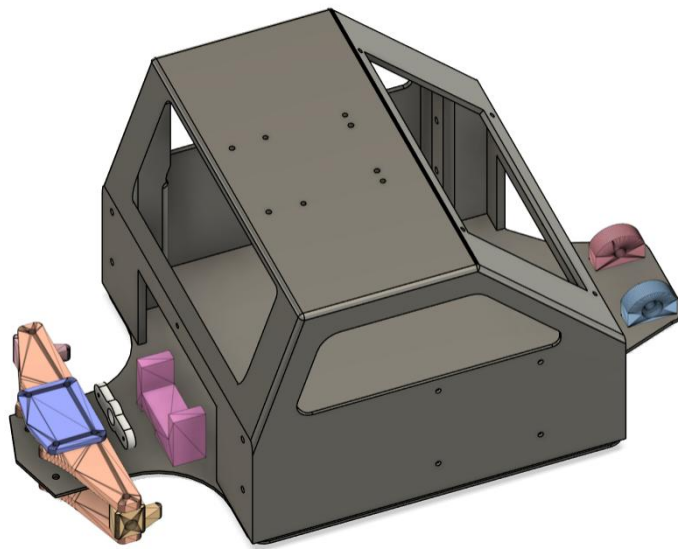


Hình 2.1. Bản vẽ khung gầm xe

Chi tiết in 3D

- Loại nhựa sử dụng: PETG

- Lý do lựa chọn: Độ bền cơ học cao, dẻo dai, chịu nhiệt tốt; không giòn gãy như PLA khi robot hoạt động liên tục.
- Các bộ phận cụ thể:
 - + Giá đỡ servo: Cố định servo lái lên khung thép; kết hợp với thanh nối và tay link lái để truyền chuyển động chính xác đến bánh trước.
 - + Thanh đòn trước (arm) và ụ đỡ bánh sau: Liên kết bánh xe, trực truyền động với thân xe một cách đồng bộ; có lỗ lắp vòng bi, đảm bảo đồng tâm.
 - + Vỏ cabin: Bảo vệ linh kiện, đồng thời làm giá đỡ cho màn hình, Raspberry Pi 5, ESP32 và mạch hạ áp; giúp bố trí nội thất gọn gàng, tránh bụi và va đập.



Hình 2.2. Thiết kế các chi tiết in 3D và lắp ráp vào khung gầm của xe

2.1.2. Một số linh kiện cơ khí khác

Giá treo camera

- Cấu tạo: Gậy inox dạng ống lồng (que chỉ bằng), có thể điều chỉnh chiều cao và khóa chặt kết hợp với đầu bi xoay 360 giúp tinh chỉnh góc nghiêng và hướng của camera một cách dễ dàng.
- Ưu điểm: Giữ cố định độ cao camera, từ đó giữ nguyên góc nhìn, giúp mô hình AI bám làn và phát hiện ổ gà ổn định hơn.



Hình 2.3. Giá treo camera

Hệ thống truyền động và bánh xe

- Bánh xe:

- + Lớp cao su bám tốt, đường kính 75 mm, rộng 33 mm, mâm kim loại.
- + Kết nối trục qua khớp lực góc 12 mm.



Hình 2.4. Bánh xe và khớp lực góc

- Bộ truyền đai răng GT2:

- + Chiều dài đai 200 mm.
- + Puly 20 răng (lỗ 6 mm trên động cơ, lỗ 5 mm trên trục bánh sau); puly 40 răng được đặt ở giữa để căng dây đai.
- + Chức năng: Dẫn động lực kéo từ động cơ đến bánh xe.



Hình 2.5. Bộ truyền đai GT2

- Trục bánh xe:

- + Đối với trục sau sử dụng trục có đường kính 5mm, độ dài 200mm; đối với trục trước gắn trực tiếp với ụ bánh trước.
- + Sử dụng vòng bi 695zz và 693zz để giảm độ ma sát giữa trục và chi tiết nhựa.

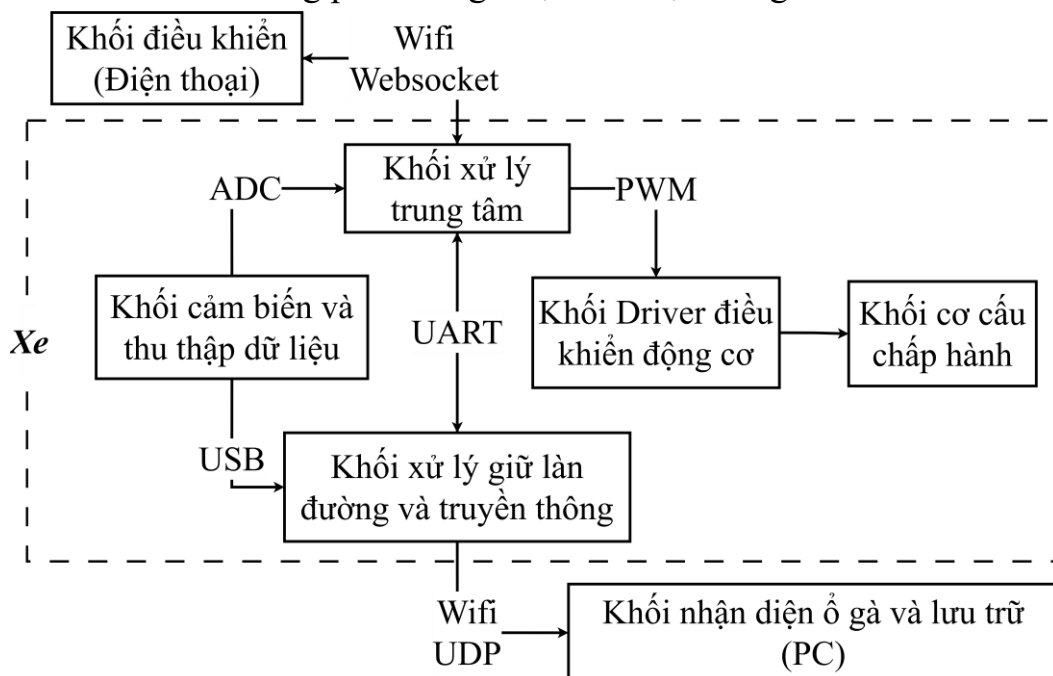


Hình 2.6. Trục bánh sau và vòng bi

2.2. Thiết kế mạch điện phân cứng

2.2.1. Sơ đồ khối chức năng, giải thích các khối trong sơ đồ

Hệ thống được tổ chức theo mô hình phân cấp với hai khối xử lý chính: khối xử lý trung tâm đảm nhận các tác vụ thời gian thực và an toàn; khối xử lý giữ làn đường và truyền thông chuyên trách các thuật toán thị giác máy và gửi dữ liệu về PC. Sơ đồ khối chức năng phần cứng được thể hiện trong Hình 2.7.



Hình 2.7. Sơ đồ khối chức năng phần cứng

a. Khối cơ cấu chấp hành

Khối này gồm các cơ cấu chấp hành nhận tín hiệu điều khiển từ trung tâm điều khiển:

- Động cơ dẫn động bánh chủ động.
- Động cơ thay đổi góc bánh dẫn hướng.

b. Khối Driver điều khiển động cơ

Khối này đóng vai trò là cầu nối trung gian công suất giữa khối xử lý trung tâm và khối cơ cấu chấp hành. Khối này có nhiệm vụ tiếp nhận các tín hiệu điều khiển logic dòng nhỏ (như tín hiệu PWM, tín hiệu điều hướng) từ vi điều khiển để khuếch đại thành dòng điện và điện áp công suất lớn, trực tiếp vận hành động cơ quay thuận/ngược và tăng/giảm tốc độ theo yêu cầu. Đồng thời, khối driver còn thực hiện chức năng cách ly nhiễu và bảo vệ mạch xử lý trung tâm trước các nguy cơ quá dòng, quá nhiệt trong quá trình hệ thống vận hành.

c. Khối cảm biến và thu thập dữ liệu

Khối này bao gồm tất cả các thiết bị đầu vào phục vụ cho việc thu thập dữ liệu môi trường và trạng thái robot:

- Camera: Ghi nhận hình ảnh mặt đường phía trước, kết nối với khối xử lý giữ làn đường và truyền thông qua cổng USB.
- Cảm biến siêu âm: Sử dụng ba module, trong đó một module hướng xuống để đo độ sâu ổ gà, hai module hướng trước và sau phát hiện vật cản. Tín hiệu từ các cảm biến này được đưa về trung tâm điều khiển.
- Module định vị: Cập nhật tọa độ (kinh độ, vĩ độ) theo chu kỳ, gửi dữ liệu về khối xử lý giữ làn đường và truyền thông qua USB.

d. Khối xử lý giữ làn đường và truyền thông

Khối này luôn thực hiện việc thu thập dữ liệu từ camera, GPS và các cảm biến, đóng gói và truyền lên khối nhận diện ổ gà và lưu trữ (PC). Riêng việc chạy mô hình AI để phát hiện vùng đường và gửi lệnh điều khiển xuống khối trung tâm chỉ được kích hoạt khi khối trung tâm báo chế độ tự động:

- Nhận luồng dữ liệu từ camera và module định vị (qua UART hoặc USB).
- Biến đổi phối cảnh để chuẩn hóa dữ liệu đầu vào.
- Chạy mô hình học sâu phát hiện vùng đường đi được.
- Tính toán lệnh điều khiển (tốc độ, góc lái) từ kết quả nhận dạng.
- Gửi lệnh điều khiển xuống trung tâm điều khiển qua kết nối nối tiếp.
- Đóng gói và truyền dữ liệu hình ảnh cùng các thông số vận hành lên máy tính giám sát qua giao diện mạng (Wi-Fi).

e. Khối xử lý trung tâm

Khối này đảm nhiệm toàn bộ các tác vụ điều khiển thời gian thực và giám sát an toàn:

- Tiếp nhận lệnh điều khiển từ xa qua kênh truyền thông không dây (Wifi) từ khối điều khiển (Điện thoại).
- Đọc tín hiệu từ các cảm biến vòng kín và các cảm biến môi trường.
- Thực hiện điều khiển duy trì tốc độ động cơ theo giá trị đặt.
- Tạo tín hiệu điều khiển gửi đến các cơ cấu chấp hành.
- Giám sát và thực thi các cơ chế an toàn (giảm tốc/dừng khi vật cản, xử lý mất kết nối, chống đảo chiều đột ngột).
- Giao tiếp hai chiều với khối xử lý giữ làn đường và truyền thông.

2.2.2. Lựa chọn các thiết bị tương ứng với các khối chức năng

a. Khối cơ cấu chấp hành

Để robot hoạt động hiệu quả, hệ thống cần đáp ứng các thông số kỹ thuật tối thiểu sau:

- Khối lượng thực tế là 4,1 kg, thiết kế dự phòng cho $M = 5 \text{ kg}$.
- Để thu thập hình ảnh ổn định cho xử lý AI, vận tốc cần đạt $v = 0,6 \text{ m/s}$.

- Để thắng lực ma sát lăn khoảng $\mu = 0,2$ cần một lực kéo:

$$F = Mg\mu = 5 * 9,8 * 0,2 = 9,8 \text{ N} \quad (2.1)$$

- Tại bánh xe bán kính $r = 0,0375 \text{ m}$ cần momen xoắn mục tiêu:

$$\begin{aligned} \tau_{bánh xe} &= F * r = 9,8 * 0,0375 = 0,3675 \text{ Nm} \\ &\approx 0,3675 * \frac{1}{0,01 * 9,8} = 3,75 \text{ kg.cm} \end{aligned} \quad (2.2)$$

- Tốc độ vòng quay mục tiêu:

$$RPM = \frac{v * 60}{2\pi r} = \frac{0,6 * 60}{2\pi * 0,0375} \approx 153 \text{ vòng/phút} \quad (2.3)$$

Từ các yêu cầu tính toán động lực học và thông số kỹ thuật thực tế của thiết bị, hệ thống quyết định lựa chọn phối hợp hai dòng động cơ này cho Khối cơ cấu chấp hành. Đối với hệ thống truyền động chủ động, động cơ DC Servo JGB37-520 (12V - 176 RPM) hoàn toàn đáp ứng tốt khả năng kéo tải nặng nhờ mô-men định mức lớn (vượt 1.7 lần nhu cầu thực tế của xe), đồng thời tích hợp sẵn Encoder 11 xung 2 kênh A-B giúp vi điều khiển phản hồi và kiểm soát vận tốc robot một cách chính xác theo thời gian thực để phục vụ quá trình thu thập ảnh ổn định. Song song với đó, đối với cơ cấu bẻ lái dẫn hướng, động cơ Servo MG996R được lựa chọn nhờ sở hữu bộ bánh răng hoàn toàn bằng kim loại bền bỉ cùng mô-men xoắn mạnh mẽ từ 9.4 kg/cm đến 11 kg/cm, đảm bảo cung cấp đủ lực bẻ góc chính xác dưới tải trọng thực tế của xe và tích hợp sẵn mạch phản hồi vòng kín giúp tối giản hóa thuật toán điều khiển góc lái trên vi điều khiển trung tâm.

Động cơ DC Servo JGB37-520 176RPM

Thông số kỹ thuật:

- Tỷ số truyền: 1:56
- Tốc độ không tải: 178 vòng/phút (RPM).
- Tốc độ định mức: 140 vòng/phút (RPM).
- Mô-men xoắn định mức: 6,5 kg.cm.
- Mô-men xoắn tối đa: 9,0 kg.cm.
- Chiều dài hộp số: 24 mm.
- Điện áp hoạt động: 12VDC.
- Dòng điện không tải: $\leq 120\text{mA}$.
- Dòng điện định mức: $\leq 1\text{A}$.
- Dòng điện kẹt/dừng: 2,3A.
- Điện áp cấp cho Encoder hoạt động: 3,3-5VDC
- Đầu Encoder 11 xung, hai kênh A-B.



Hình 2.8. Động cơ DC Servo JGB37-520 176RPM

- Đường kính động cơ: 37mm.
- Đường kính trục: 6mm

Do động cơ hoạt động ở điện áp 12VDC nên cần cung cấp nguồn là pin 3S để đảm bảo động cơ hoạt động tốt nhất.

Pin Lipo 3S Tiger

Thông số kỹ thuật:

Nhãn hiệu: Tiger Dòng: 5400mAh

Điện áp: 11.1V

Dòng xả: 30C

Trọng lượng: 410g

Kích thước: 149x49x29mm



Hình 2.9. Pin Lipo 3S Tiger

Động cơ servo MG996R

Thông số kỹ thuật:

- Góc quay: 360 độ
- Trọng lượng sản phẩm: 55g
- Kích thước: 40,7 * 19,7 * 42,9mm
- Lực kéo: 9,4kg/cm (4,8V), 11kg/cm (6V)
- Điện áp làm việc: 4,8-7,2V
- Nhiệt độ hoạt động: 0 °C -55 °C
- Vật liệu bánh răng: Kim loại



Hình 2.10. Động cơ servo MG996R

Mạch hạ áp DC-DC LM2596 3A

Mạch hạ áp có nhiệm vụ chuyển đổi điện áp từ Pin Li-Po 3S xuống 5V để cấp nguồn cho động cơ Servo MG996R. Dòng điện khi có tải hoặc dòng định của động cơ đạt mức: $I_{MG996R} = 2,5A$.

Ngoài ra còn cần cung cấp một dòng điện nhỏ cho vi điều khiển và cảm biến.

Yêu cầu đặt ra cho bộ biến đổi là phải đáp ứng được ngưỡng dòng định lớn hơn 2,5A để tránh hiện tượng sụt áp mỗi khi Servo bẻ lái gấp. Do đó lựa chọn mạch hạ áp DC-DC LM2596 3A.

Thông số kỹ thuật:

- Điện áp đầu vào: Từ 3V đến 30V.
- Điện áp đầu ra: Điều chỉnh được trong khoảng 1.5V đến 30V.
- Dòng đáp ứng tối đa là 3A.
- Công suất: 15W



- Kích thước: 45 (dài) * 20 (rộng) * 14 (cao)
mm

Hình 2.11. Mạch hạ áp
DC-DC LM2596 3A

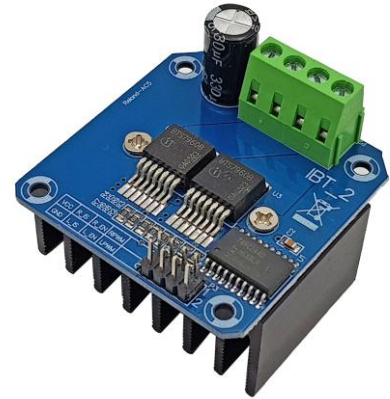
b. Khối Driver điều khiển động cơ

Với khối lượng xe khoảng 4,1 kg, động cơ có thể ăn dòng khởi động hoặc dòng kẹt (Stall current) lên tới 4-5A. Driver BTS7960 (chịu tải tối đa 43A) được lựa chọn để đảm bảo hệ số an toàn cực cao, tránh tình trạng quá nhiệt và cháy mạch khi xe di chuyển lâu hoặc trên địa hình có ma sát lớn.

Driver BTS7960

Thông số kỹ thuật:

- Điện áp đầu vào: 6~27V
- Dòng điện tải mạch: 43A
- Tín hiệu logic điều khiển: 3.3 ~ 5V.
- Tần số điều khiển tối đa: 25KHz.
- Tự động shutdown khi điện áp thấp: Để tránh điều khiển động cơ ở mức điện áp thấp thiết bị sẽ tự shutdown. Nếu điện áp < 5.5V, mạch điều khiển động cơ DC BTS7960 sẽ tự ngắt điện và sẽ mở lại sau khi điện áp > 5.5V.
- Bảo vệ quá nhiệt: BTS7960 bảo vệ chống quá nhiệt bằng cảm biến nhiệt tích hợp bên trong. Đầu ra sẽ bị ngắt khi có hiện tượng quá nhiệt.
- Kích thước: 40 x 50 x 12mm.



Hình 2.12. Driver BTS7960

d. Khối cảm biến và thu thập dữ liệu

Từ các yêu cầu thực tế về thu thập dữ liệu môi trường, nhận diện trạng thái bề mặt đường và định vị vị trí thời gian thực cho robot, một số phương án linh kiện tương đương đã được cân nhắc như cảm biến siêu âm HC-SR04, các dòng Webcam Logicoool tiêu chuẩn hoặc các module GPS dòng Neo-6M/Neo-8M giao tiếp UART. Tuy nhiên, sau khi đánh giá tổng thể về độ ổn định phần cứng, tính tương thích giao tiếp, hiệu suất xử lý và đặc biệt là nhằm tối ưu hóa chi phí đầu tư nghiên cứu, cấu hình được đề xuất lựa chọn bao gồm: Cảm biến siêu âm HY-SRF05, Camera DFRobot 0.3MP và Module GPS G-Mouse VK162.

Cảm biến HY-SRF05

Thông số kỹ thuật:

- Điện áp hoạt động: 5VDC
- Dòng tiêu thụ: 10~40mA
- Tín hiệu giao tiếp: TTL-
Chân tín hiệu: Echo, Trigger
(thường dùng) và Out (ít dùng).
- Góc quét: <15 độ
- Tần số phát sóng: 40Khz
- Khoảng cách đo được: 2~450cm (khoảng cách xa nhất đạt được ở điều kiện lý tưởng với không gian trống và bề mặt vật thể bằng phẳng, trong điều kiện bình thường cảm biến cho kết quả chính xác nhất ở khoảng cách <100cm).
- Sai số: 0,3cm (khoảng cách càng gần, bề mặt vật thể càng phẳng sai số càng nhỏ).
- Kích thước: 43mm x 20mm x 17mm



Hình 2.13. Cảm biến HY-SRF05

Tính toán cầu phân áp: Để an toàn cho chân GPIO (3,3V) của ESP32 khi nhận tín hiệu Echo (5V), hệ thống cần sử dụng cầu phân áp:

$$V_{out} = V_{in} * \frac{R_2}{R_2 + R_1} \quad (2.4)$$

$$\text{Do } V_{in} = 5V; V_{out} = 3,3V \rightarrow \frac{R_2}{R_2 + R_1} = 0,66$$

Chọn $R_1 = 10k\Omega$; $R_2 = 20k\Omega$

Camera DFRobot 0.3MP

Thông số kỹ thuật:

- Photosensitive Element: CMOS
- AGC/AEC/AWB: auto
- Infrared Filter: 650±10nm
- Hỗ trợ định dạng MJPG và YUV
- FPN: < 0,03% of VPEAK-TO-PEAK
- Cổng kết nối: USB 2.0
- Pixel: 0.3MP
- Độ phân giải: 640*480
- Điện áp hoạt động: 5V



Hình 2.14. Camera DFRobot 0.3MP

- Nhiệt độ hoạt động: -20°C-70°C
- FCC & CE Certification
- Kích thước: 30*25*21,4mm/1,18*0,98*0,84"
- Hỗ trợ các hệ điều hành như: Windows, iOS, Android, Linux

Module GPS G-Mouse VK162

Thông số kỹ thuật:

- Điện áp hỗ trợ: 3,3~5VDC
- Hướng: <0,5 Degrees
- Thời gian chính xác: 30ns
- Hệ tọa độ tham chiếu: WGS-84
- Độ cao tối đa: 50.000 m



Hình 2.15. Module GPS G-Mouse VK162

e. Khối xử lý giữ làn đường và truyền thông

Để hiện thực hóa các thuật toán thị giác máy tính và quản lý truyền thông cấp cao, một số giải pháp cấu hình phần cứng nhúng phổ biến hiện nay đã được đặt ra cân nhắc, bao gồm việc đặt trực tiếp một máy tính xách tay (Laptop) lên khung xe hoặc sử dụng các dòng mạch chuyên dụng cho AI như NVIDIA Jetson Nano. Mặc dù việc đặt Laptop mang lại hiệu năng tính toán cực mạnh và Jetson Nano sở hữu các lõi CUDA tối ưu rất tốt cho mạng nơ-ron, nhưng cả hai phương án này đều bộc lộ những hạn chế lớn về mặt thiết kế robot di động: Laptop quá cồng kềnh, tiêu thụ năng lượng lớn và làm tăng trọng lượng xe một cách không cần thiết; trong khi Jetson Nano (phiên bản cũ) lại có giá thành tương đối cao ở thời điểm hiện tại và cấu trúc CPU đã dần lỗi thời đối với một số thư viện học sâu mới. Do đó, cấu hình tối ưu được đề xuất bao gồm Máy tính nhúng Raspberry Pi 5 (phiên bản 8GB RAM) phối hợp cùng Màn hình LCD Waveshare 7 inch HDMI.

Raspberry Pi 5 8GB Ram

Thông số kỹ thuật:

- Bộ vi xử lý: Broadcom BCM2712, Quad-core ARM Cortex-A76 (2.4GHz) lõi tứ, tiến trình 16nm tích hợp tản nhiệt kim loại.
- Bộ nhớ đệm: 512KB L2 cache mỗi lõi và 2MB L3 cache dùng chung.
- Bộ nhớ RAM: 8GB LPDDR4X-4267.
- Đồ họa: VideoCore VII GPU (OpenGL ES 3.1, Vulkan 1.2); hỗ trợ 2 cổng micro-HDMI 4Kp60 HDR và bộ giải mã 4Kp60 HEVC.



Hình 2.16. Raspberry Pi 5

- Kết nối ngoại vi: 2 cổng USB 3.0 (5Gbps), 2 cổng USB 2.0 và 2 cổng MIPI 4-lane hỗ trợ Camera/Display.
- Lưu trữ: Khe MicroSD (SDR104) và hỗ trợ mở rộng SSD NVMe qua giao thức PCIe 2.0 x1.
- Truyền thông: Gigabit Ethernet (PoE+), Wi-Fi Dual-band (2.4/5GHz), Bluetooth 5.0 / BLE.
- Giao tiếp lập trình: 40 chân GPIO (UART, I2C, SPI), cổng UART debug và cổng quạt tản nhiệt PWM riêng biệt.
- Tiện ích hệ thống: Tích hợp nút nguồn vật lý và module RTC (Real-time Clock) có công gấn pin sạc.
- Nguồn: Yêu cầu nguồn 5V/5A qua USB-C (chuẩn PD)
- Kích thước: 88 x 56 x 17 mm.

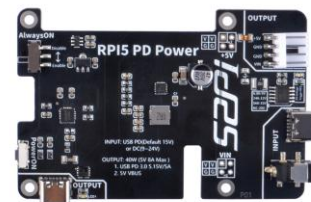
Để đảm bảo Raspberry Pi 5 hoạt động ổn định cần sử dụng mạch nguồn chuyên dụng *52Pi PD Power*.

Mạch nguồn 52Pi PD Power

Thông số kỹ thuật:

- Đầu vào nguồn (Input):

- + USB PD: Mặc định 15V qua cổng Type-C.
 - + DC: Dải điện áp 9V - 12V (Jack DC).
- Lưu ý:* Hai cổng vào này không được sử dụng đồng thời.



- Đầu ra nguồn (Output):

- + Cổng USB-C: Công suất tối đa 40W (5V/8A MAX).
- + PD 3.0: Đáp ứng chuẩn 5.15V 5A.
- + VBUS 5V: Chân cắm đầu ra 5V dự phòng.

- Tính năng điều khiển:

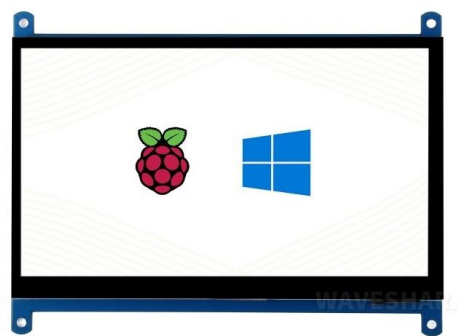
- + Công tắc Always-ON hỗ trợ tự động khởi động Raspberry Pi 5 ngay khi có nguồn cấp vào.
- + Nút nhấn PowerON để bật/tắt nguồn chủ động.
- + Tương thích hoàn toàn với kiểu dáng và yêu cầu dòng điện cao của Raspberry Pi 5.
- + Tích hợp quản lý điện năng thông minh, bảo vệ hệ thống khi hoạt động ở cường độ cao.

Hình 2.17. Mạch nguồn
52Pi PD Power

Màn hình LCD Waveshare 7 inch HDMI

Thông số kỹ thuật:

- Kích thước màn hình: 7 inch.
- Độ phân giải: 1024×600 pixels.
- Cổng kết nối: HDMI, USB
- Cảm ứng điện dung và cấp nguồn 5VDC qua cổng Micro USB.
- Thiết kế Low Power tiết kiệm năng lượng.



Hình 2.18. Màn hình LCD
Waveshare 7 inch HDMI

Do màn hình có kích thước lớn luôn hiển thị và cần sự ổn định, việc lựa chọn nguồn riêng cung cấp nguồn cho màn hình giúp tránh hiện tượng nhiễu và sụt áp khi khởi động Raspberry Pi 5 hoặc khi xe tăng tốc đột ngột.

Pin sạc dự phòng Energizer

Thông số kỹ thuật:

- Dung lượng pin: 10000mAh
- Công suất sạc: Có dây 18W, không dây: 10W
- Cổng sạc ra: USB-C, USB-A, không dây
- Cổng sạc vào: USB-C, Micro-USB
- Tiện ích: Đèn LED hiển thị dung lượng pin, Sạc nhiều thiết bị cùng lúc
- Hiệu suất sạc: 65%
- Lõi pin: Li-Polymer
- Kích thước: 14.3 x 6.8 x 1.5 cm
- Trọng lượng: 210 g
- Hãng sản xuất: Energizer



Hình 2.19. Pin sạc
dự phòng Energizer

f. Khối xử lý trung tâm

Để đảm nhiệm toàn bộ các tác vụ điều khiển thời gian thực và giám sát an toàn cho robot, một số giải pháp điều khiển công nghiệp và nhúng đã được cân nhắc như bộ điều khiển lập trình PLC, vi điều khiển STM32 hay Arduino. Tuy nhiên, qua tính toán sơ đồ kết nối phần cứng, hệ thống chỉ cần sử dụng khoảng 16 chân tín hiệu. Nếu sử dụng PLC, thiết bị sẽ rất cồng kềnh, giá thành cao và không nằm trong nội dung chương trình đào tạo chuyên ngành. Do đó, phương án tối ưu nhất là lựa chọn Vi điều khiển ESP32 (Phiên bản 30 chân), đây là dòng vi điều khiển rất quen thuộc, được ứng dụng nhiều trong các bài tập lớn và dự án IoT thuộc chương trình học tại trường nên dễ dàng tiếp cận và triển khai code phần mềm. Bên cạnh đó, ESP32 sở hữu cấu trúc CPU lõi kép mạnh mẽ đáp ứng tốt các tác vụ thời gian thực như đọc encoder, điều khiển driver động cơ, đồng thời tích

hợp sẵn kết nối Wi-Fi/Bluetooth giúp tiết kiệm chi phí mua module rồi khi triển khai kênh nhận lệnh điều khiển không dây từ xa.

Vi điều khiển ESP32

Thông số kỹ thuật:

- Số lượng chân: 30 pin
- Kết nối không dây: Wi-Fi (802.11 b/g/n) và Bluetooth (v4.2)
- Điện áp hoạt động: 3,3V
- Bộ nhớ: RAM 520KB, Flash từ 4MB đến 16MB tùy phiên bản
- GPIO: 34 chân GPIO có thể lập trình
- ADC: 12-bit ADC với 18 kênh
- DAC: 2 kênh DAC 8-bit
- PWM: Hỗ trợ PWM cho điều khiển động cơ và ánh sáng
- Giao tiếp: SPI, I2C, UART, CAN



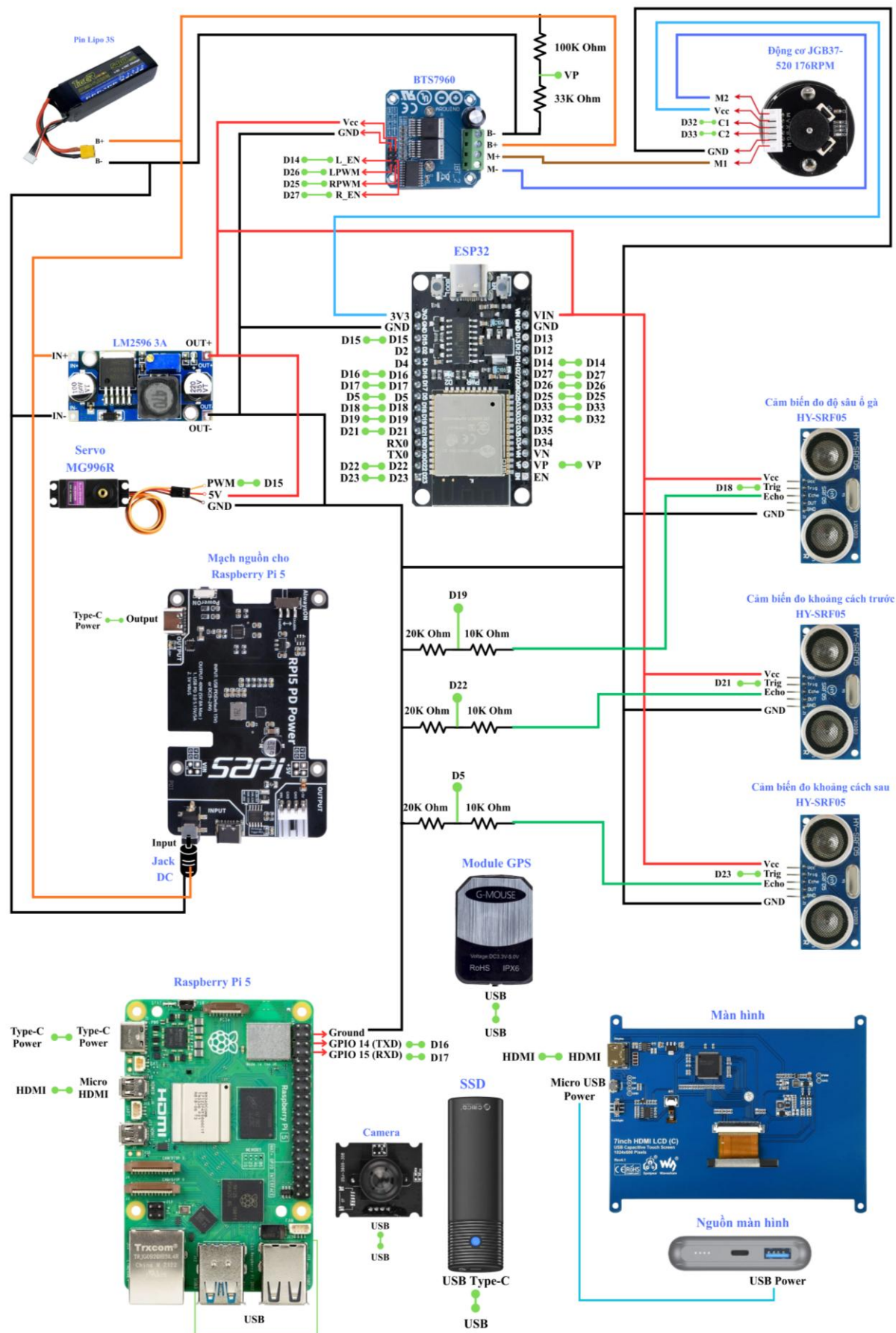
Hình 2.20. Vi điều khiển ESP32

Tính năng nổi bật:

- ESP32 hỗ trợ cả Wi-Fi và Bluetooth, cho phép xây dựng các ứng dụng IoT đa dạng.
- Với CPU Dual-core 240MHz, ESP32 mang lại hiệu suất xử lý tốt cho các tác vụ phức tạp.
- Có chế độ ngủ sâu giúp tiết kiệm điện năng trong các ứng dụng không cần kết nối liên tục.
- Hỗ trợ nhiều môi trường lập trình như Arduino IDE, PlatformIO, và ESP-IDF, giúp lập trình viên dễ dàng tiếp cận và phát triển.

2.2.3. Sơ đồ đấu nối mạch điện

Dựa trên các tính toán về công suất và yêu cầu đảm bảo các tác vụ điều khiển được xử lý kịp thời đáp ứng thời gian thực, sơ đồ đấu nối được xây dựng nhằm thể hiện chi tiết kết nối giữa các khối chức năng. Chi tiết được thể hiện trong Hình 2.21.



Hình 2.21. Sơ đồ đấu nối hệ thống

2.3. Kết quả thiết kế

Sau khi hoàn thiện quá trình tính toán, thiết kế tối ưu phần cơ khí và hệ thống mạch điều khiển điện tử, mô hình xe tự hành đã được chế tạo và lắp ráp thực tế. Kiểu dáng cũng như cách bố trí các module linh kiện trên xe được tính toán kỹ lưỡng nhằm đảm bảo tính thẩm mỹ, sự cân bằng giúp xe di chuyển ổn định trong quá trình làm việc.

Hình ảnh thực tế của mô hình xe tự hành sau khi hoàn thiện lắp ráp được thể hiện trong Hình 2.22 dưới đây:



Hình 2.22. Kết quả thiết kế mô hình.

Kết luận chương 2

Chương 2 đã tập trung thiết kế toàn bộ hệ thống phần cứng cho robot tự hành, bao gồm cả phần cơ khí và mạch điện.

Về cơ khí, robot được xây dựng với khung gầm chính bằng thép CNC dày 2 mm đảm bảo độ cứng vững và ổn định khi di chuyển. Các chi tiết phụ trợ như giá đỡ servo, tay đòn, ụ đỡ bánh xe và vỏ cabin được chế tạo bằng phương pháp in 3D từ nhựa PETG, vừa đảm bảo độ bền vừa tối ưu trọng lượng. Hệ thống truyền động sử dụng bộ truyền đai GT2, bánh xe cao su đường kính 75 mm cùng các vòng bi tiêu chuẩn giúp robot vận hành êm ái, giảm rung chấn ảnh hưởng đến chất lượng hình ảnh.

Về mạch điện, hệ thống được tổ chức theo mô hình phân cấp với ESP32 làm trung tâm điều khiển đảm nhận các tác vụ thời gian thực, an toàn; Raspberry Pi 5 giữ vai trò xử lý giữ làn đường và truyền thông. Các thiết bị được lựa chọn (cảm biến siêu âm HY-SRF05, camera USB 0,3 MP, GPS G-Mouse VK-162, động cơ JGB37-520, servo MG996R, driver BTS7960, nguồn pin 3S Tiger, mạch hạ áp LM2596, mạch nguồn PD cho Pi 5) đều được tính toán dòng tiêu thụ và công suất để đảm bảo tương thích. Sơ đồ nguyên lý đã thể hiện chi tiết các kết nối nguồn, tín hiệu, mạch phân áp bảo vệ chân GPIO, tạo nên một hệ thống điện hoàn chỉnh, ổn định và an toàn.

Những thiết kế trên là cơ sở vững chắc để triển khai các thuật toán điều khiển, xử lý ảnh và giám sát trong Chương 3.

CHƯƠNG 3. XÂY DỰNG THUẬT TOÁN ĐIỀU KHIỂN VÀ GIÁM SÁT

3.1. Yêu cầu điều khiển

3.1.1. Điều khiển động cơ

a. Tính toán PID điều khiển tốc độ động cơ JGB37-520 176RPM

Từ thông số kỹ thuật của động cơ JGB37-520 176RPM, ta có các thông số:

Bảng 3.1. Bảng thông số kỹ thuật động cơ JGB37-520 176RPM

Đại lượng	Ký hiệu	Giá trị
Điện áp định mức	U	12V
Tốc độ không tải	n_0	178 vòng/phút
Dòng điện không tải	I_{nl}	0,12A
Dòng điện ngắn mạch (stall)	I_s	2,3A

Tính toán hàm truyền

Điện trở phần ứng:

$$R = \frac{U}{I_s} = \frac{12}{2,3} = 5,2174\Omega \quad (3.1)$$

Tốc độ góc không tải:

$$\omega_0 = \frac{n_0 * 2\pi}{60} = \frac{178 * 2\pi}{60} = 18,6401 \text{ rad/s} \quad (3.2)$$

Hằng số sức điện động có tính sụt áp trên R khi không tải:

$$K_e = \frac{U - I_{nl}R}{\omega_0} = \frac{12 - 0,12 * 5,2174}{18,6401} = 0,6103 \text{ V.s/rad} \quad (3.3)$$

Trong hệ đơn vị SI, hằng số momen $K_t = K_e = 0,6103 \text{ N.m/A}$

Hệ số ma sát nhớt:

$$b = \frac{K_t I_{nl}}{\omega_0} = \frac{0,6103 * 0,12}{18,6401} = 0,00393 \text{ N.m.s/rad} \quad (3.4)$$

Hệ số khuếch đại tĩnh của động cơ:

$$K = \frac{\Omega(s)}{V(s)} = \frac{K_t}{Rb + K_t K_e} = \frac{0,6103}{5,2174 * 0,00393 + 0,6103^2} = 1,553 \text{ (rad/s)/V} \quad (3.5)$$

Giả sử chọn hằng số thời gian cơ học $T = 0,12 \text{ s}$

Hàm truyền gần đúng (bỏ qua điện cảm) là:

$$G(s) = \frac{1,553}{0,12s + 1} \quad (3.6)$$

Do hộp số và đáp ứng dòng điện gây ra một độ trễ nhỏ. Giả sử thời gian trễ $L = 0,05 \text{ s}$.

Mô hình đối tượng điều khiển có dạng:

$$G(s) = \frac{1,553}{0,12s + 1} e^{-0,05s} \quad (3.7)$$

Thiết kế bộ điều khiển PID theo phương pháp Ziegler-Nichols I

Bộ điều khiển PID trong miền thời gian được xác định bởi công thức:

$$u(t) = K_p * e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (3.8)$$

Phương pháp Ziegler-Nichols I cho hệ bậc nhất có trễ với công thức:

$$K_p = \frac{1,2T}{KL}; \quad T_i = 2L; \quad T_d = 0,5L; \quad K_i = \frac{K_p}{T_i}; \quad K_d = K_p T_d$$

Hệ số tỉ lệ:

$$K_p = \frac{1,2T}{KL} = \frac{1,2 * 0,12}{1,553 * 0,05} = 1,854 \approx 1,86 \quad (3.9)$$

Hệ số tích phân:

$$K_i = \frac{K_p}{T_i} = \frac{K_p}{2L} = \frac{1,854}{2 * 0,05} = 18,54 \approx 18,6 \quad (3.10)$$

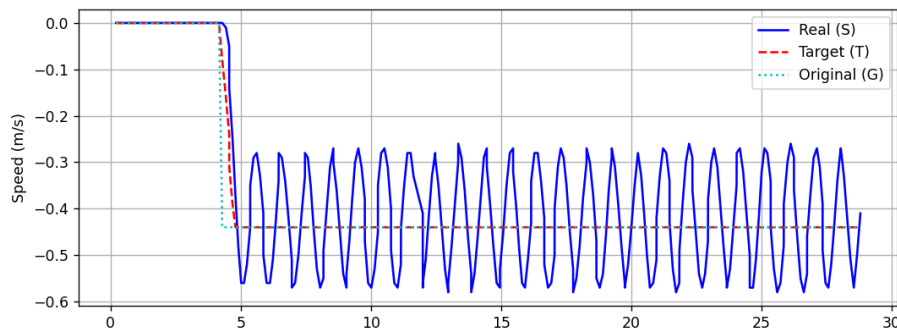
Hệ số vi phân:

$$K_d = K_p T_d = K_p * 0,5L = 1,854 * 0,5 * 0,05 = 0,04635 \approx 0,046 \quad (3.11)$$

Bộ tham số PID thu được:

$$K_p = 1,86; \quad K_i = 18,6; \quad K_d = 0,046$$

Thử nghiệm kết quả:

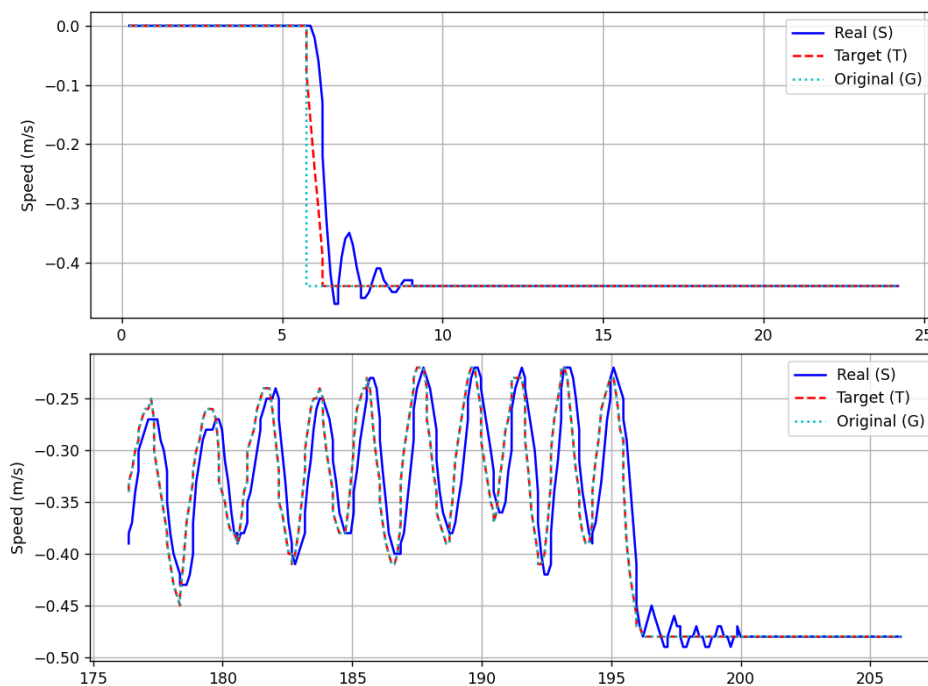


Hình 3.1. Đồ thị đáp ứng vận tốc của xe với $K_p=1,86$; $K_i=18,6$; $K_d=0,046$

Dựa trên đồ thị đáp ứng, bộ tham số $K_p=1,86$; $K_i=18,6$; $K_d=0,046$ cho thấy tốc độ đáp ứng tức thời rất cao nhưng hệ thống rơi vào trạng thái mất ổn định với dao động duy trì biên độ lớn quanh giá trị đặt. Cụ thể, khi Setpoint thay đổi đột ngột, tốc độ thực tế vọt lố mạnh lên đến khoảng 36% (đạt ngưỡng -0,6 m/s so với mục tiêu -0,44 m/s). Sau giai đoạn quá độ, hệ thống không thể xác lập mà liên tục dao động hình sin với chu kỳ ổn định, cho thấy thành phần tích phân K_i quá lớn khiến vòng điều khiển bị quá bù. Sai số tức thời dao động trong khoảng rộng từ -

0,28m/s đến -0,58m/s, minh chứng rằng bộ thông số hiện tại chưa đạt yêu cầu về độ ổn định cho hệ thống.

Thông qua quá trình thử nghiệm và tinh chỉnh trực tiếp trên phần cứng, giá trị $K_i = 4,5$ đã được lựa chọn vì cho đáp ứng tốt nhất: dập tắt hoàn toàn các dao động hình sin, giảm độ vọt lố và giúp vận tốc thực tế hội tụ nhanh chóng về đường Setpoint. Kết quả thực nghiệm cho thấy hệ thống đã đạt được độ ổn định cao và khả năng bám bắt giá trị đặt rất tốt. Vận tốc thực tế phản ứng kịp thời, chính xác theo các dao động của đường Target và triệt tiêu hoàn toàn hiện tượng vọt lố cũng như dao động biên độ lớn trước đó. Đặc biệt ở giai đoạn xác lập, sai số giữa giá trị thực và giá trị đặt là không đáng kể, chứng minh bộ tham số tinh chỉnh đã đáp ứng tốt yêu cầu điều khiển thực tế.



Hình 3.2. Đồ thị đáp ứng vận tốc của xe với $K_p=1,86$; $K_i=4,5$; $K_d=0,046$

b. Điều khiển góc động cơ servo MG996R

Góc lái bánh xe δ (độ) được thực hiện thông qua động cơ servo MG996R. Dựa trên cơ cấu lái của robot, góc servo θ (độ) liên hệ với góc lái δ theo tỷ lệ tuyến tính:

$$\theta = \theta_0 + k \cdot \delta \quad (3.12)$$

Trong đó:

- $\theta_0 = 88^\circ$ là góc servo ứng với góc lái 0° (xe đi thẳng).
- $k = \frac{135-88}{30} = \frac{47}{30} \approx 1,5667$ là hệ số khuếch đại (độ servo/độ lái).
- Dải góc lái $\delta \in [-30^\circ, +30^\circ]$, dải góc servo tương ứng $\theta \in [41^\circ, 135^\circ]$.

Khi nhận được giá trị góc lái mong muốn δ (từ bộ điều khiển hoặc từ lệnh thủ công), ESP32 tính toán góc servo θ theo công thức trên và gửi lệnh đến servo thông qua thư viện ESP32Servo. Để tránh thay đổi đột ngột, tốc độ biến thiên của góc servo được giới hạn ở $90^\circ/\text{s}$, đảm bảo góc lái thay đổi mượt mà.

3.1.2. Các chế độ điều khiển

Hệ thống điều khiển xe được thiết kế với hai chế độ hoạt động chính: thủ công và tự động. Việc chuyển đổi giữa hai chế độ chỉ được thực hiện qua giao diện WebSocket. Ngoài ra, xe còn được trang bị các cơ chế an toàn đa tầng và một giao diện điều khiển từ xa thân thiện.

a. Chế độ thủ công

Chế độ thủ công cho phép người dùng trực tiếp điều khiển tốc độ và góc lái của xe thông qua các thanh trượt trên giao diện web. Ở chế độ này:

- Người dùng kéo thanh trượt tốc độ và thanh trượt góc lái trên trình duyệt.
- Trình duyệt gửi lệnh WebSocket dạng dc:{value};sv:{value} đến ESP32, trong đó dc là tốc độ mong muốn (m/s) và sv là góc lái mong muốn (độ).
- ESP32 nhận lệnh, cập nhật trực tiếp giá trị đặt cho bộ PID tốc độ (g_setpoint_rpm) và góc servo đặt (g_servoTargetAbs). Các giá trị đặt này sau đó được đưa qua bộ lọc trung gian để thay đổi mượt mà trước khi đưa vào bộ điều khiển, tránh hiện tượng giật cục.
- Xe phản hồi theo đúng yêu cầu của người dùng, các cơ chế an toàn (giới hạn tốc độ, xử lý vật cản, mất kết nối) vẫn được duy trì.
- Khi chuyển từ chế độ tự động sang thủ công, ESP32 tự động dừng động cơ (tốc độ đặt = 0) và đưa servo về vị trí trung tâm trước khi nhận lệnh điều khiển tiếp theo.

b. Chế độ tự động

Chế độ tự động được kích hoạt khi người dùng bật công tắc “Autonomous” trên giao diện web, gửi lệnh mode:1 qua WebSocket. Ở chế độ này:

- Raspberry Pi 5 chạy mô hình TwinLiteNet để phát hiện vùng đường đi được và vạch làn, tính toán sai lệch tâm, từ đó sinh ra lệnh điều khiển (tốc độ, góc lái).
- Lệnh điều khiển được gửi xuống ESP32 qua UART dưới dạng S{cmd_speed}A{cmd_steer} với tần suất 30 Hz.
- ESP32 thực hiện điều khiển tốc độ động cơ DC và điều khiển servo để thực thi lệnh.

- Nếu không phát hiện được vùng đường hoặc phát hiện vật cản trước với khoảng cách dưới 15 cm, xe sẽ tự động dừng và dừng chạy mô hình phát hiện làn đường.
- Cơ chế boost được kích hoạt khi xe bị kẹt khi trong 10s tốc độ thực tế < 0,05 m/s và lệnh tốc độ > 0,1 m/s để thoát khỏi vị trí kẹt bằng cách cho xe di chuyển tiến – lùi – tiến.
- Trong chế độ tự động, các thanh trượt điều khiển thủ công bị vô hiệu hóa để tránh xung đột lệnh.

c. Cơ chế an toàn đa tầng

Hệ thống được trang bị nhiều lớp bảo vệ để đảm bảo an toàn cho xe và người xung quanh, được thực hiện trực tiếp trên ESP32, đảm bảo phản ứng tức thời không phụ thuộc vào Raspberry Pi 5.

An toàn va chạm: Xe sử dụng hai cảm biến siêu âm phía trước và phía sau để đo khoảng cách đến vật cản. Nếu khoảng cách phát hiện nhỏ hơn 10 cm, hệ thống lập tức dừng khẩn cấp (tốc độ đặt về 0). Nếu khoảng cách nằm trong vùng nguy hiểm từ 10 cm đến 30 cm, tốc độ mong muốn được giảm tuyến tính theo công thức:

$$v_{safe} = v_{desired} \cdot \frac{d - 10}{20} \quad (3.13)$$

An toàn kết nối: ESP32 giám sát liên tục tín hiệu heartbeat từ WebSocket và tình trạng kết nối WiFi. Nếu không nhận được heartbeat hoặc mất kết nối WiFi trong thời gian dài hơn 2 giây, hệ thống tự động dừng động cơ, đưa servo về vị trí trung tâm và chuyển sang chế độ an toàn, đợi kết nối được phục hồi.

Chống đảo chiều đột ngột: Khi nhận lệnh yêu cầu đảo chiều (ví dụ từ tiến sang lùi), bộ điều khiển không cho phép thay đổi hướng ngay lập tức. Thay vào đó, nó giảm tốc độ hiện tại về 0, duy trì tốc độ 0 trong một khoảng thời gian ngắn (khoảng 100 ms) rồi mới tăng tốc theo hướng mới. Cơ chế này tránh gây sốc cho hệ thống truyền động cơ khí.

Auto-hold khi dừng: Khi tốc độ đặt bằng 0, hệ thống PID không được kích hoạt; thay vào đó, một lực hãm nhỏ (giá trị PWM khoảng ± 15) được duy trì để chống hiện tượng trôi do dốc hoặc quán tính. Lực hãm được tự động điều chỉnh dựa trên tốc độ thực tế đọc từ encoder, đảm bảo xe đứng yên ổn định.

Giới hạn hoạt động: Tốc độ tối đa được giới hạn ở 0,70 m/s đối với chế độ thủ công và 0,15 m/s đối với chế độ tự động. Góc lái bánh xe bị chặn trong khoảng $\pm 30^\circ$, tương ứng với góc servo từ 41° đến 135° .

Giám sát điện áp pin: Điện áp của pin Li-Po 3S được đo thông qua cầu phân áp điện trở $R_1 = 100 \text{ k}\Omega$ và $R_2 = 33 \text{ k}\Omega$, đưa tín hiệu vào chân ADC của

ESP32 (GPIO36). Tỷ số phân áp là $R_2/(R_1 + R_2) \approx 0,248$. Giá trị điện áp thực tế được tính toán và lọc qua bộ lọc trung bình trượt kết hợp EMA (cửa sổ trượt 5 mẫu, sau đó trung bình 100 mẫu). Dung lượng pin được tính theo công thức:

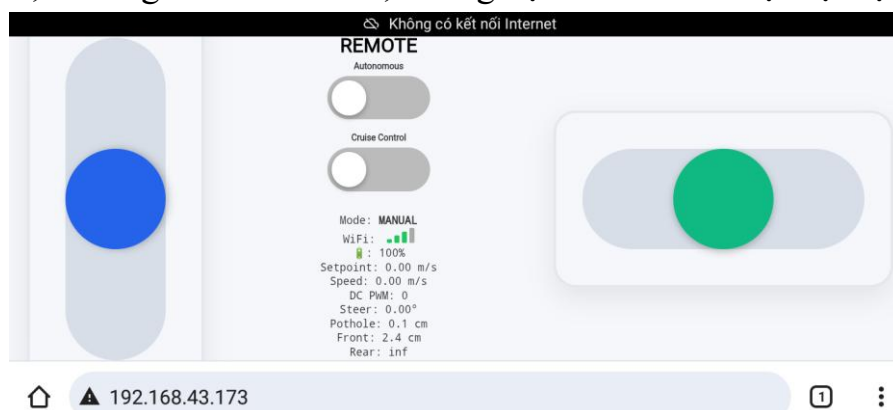
$$\text{BatPercent} = \frac{V_{bat} - V_{min}}{V_{max} - V_{min}} \times 100\% \quad (3.14)$$

với $V_{min} = 11,1$ V (ngưỡng pin yếu) và $V_{max} = 12,62$ V (pin đầy). Giá trị phần trăm pin được gửi lên Raspberry Pi qua UART (trong chuỗi FB:) và hiển thị trên giao diện WebSocket (trong chuỗi state:). Hệ thống chỉ có chức năng giám sát và hiển thị, không tự động giảm tốc hay cắt nguồn khi pin yếu. Người dùng cần theo dõi và chủ động sạc pin khi dung lượng xuống thấp để đảm bảo an toàn vận hành.

d. Giao diện điều khiển

Giao diện điều khiển từ xa được xây dựng dưới dạng trang web (HTML/CSS/JS) nhúng trong bộ nhớ của ESP32. Người dùng có thể truy cập bằng trình duyệt thông qua địa chỉ IP được cấp cho ESP32. Giao diện bao gồm:

- Thanh trượt tốc độ cho phép điều chỉnh từ -0,70 m/s đến +0,70 m/s, với bước nhảy 0,01 m/s.
- Thanh trượt góc lái cho phép điều chỉnh từ -30° đến +30°, bước nhảy 1°.
- Công tắc bật/tắt chế độ tự động, khi bật các thanh trượt bị vô hiệu hóa và xe chuyển sang điều khiển bằng AI.
- Công tắc Cruise Control cho phép giữ tốc độ không đổi khi thả tay khỏi thanh trượt.
- Khung hiển thị trạng thái hiển thị tốc độ thực tế, góc lái, dung lượng pin, khoảng cách cảm biến, cường độ WiFi và chế độ hiện tại.



Hình 3.3. Giao diện WebSocket điều khiển xe

Giao tiếp giữa trình duyệt và ESP32 sử dụng WebSocket với tần suất gửi lệnh 30 ms, đảm bảo phản hồi nhanh. Khi không có lệnh mới, trình duyệt tự động

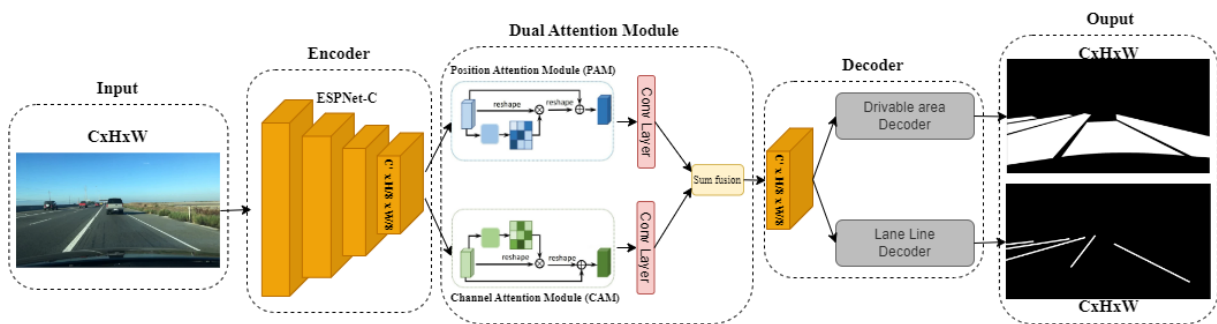
gửi gói tin hb (heartbeat) mỗi 250 ms để giữ kết nối. Nếu mất heartbeat quá 2 giây, ESP32 sẽ dừng xe để đảm bảo an toàn.

3.2. Thuật toán xử lý ảnh phát hiện làn đường và nhận diện ô gà

3.2.1. Mô hình TwinLiteNet cho bài toán phát hiện làn đường

a. Giới thiệu về mô hình TwinLiteNet

TwinLiteNet là một mô hình học sâu siêu nhẹ và rất hiệu quả, được thiết kế đặc biệt để giải quyết bài toán phát hiện làn đường với yêu cầu xử lý tốc độ cao. Mô hình này vận hành theo cơ chế Encoder-Decoder – một kiến trúc mượn ý tưởng từ các mạng phát triển trong lĩnh vực xử lý ngôn ngữ tự nhiên (NLP), vốn nổi tiếng với khả năng chuyển đổi chuỗi đầu vào thành chuỗi đầu ra. Trong đó, khối encoder đảm nhận việc trích xuất các đặc trưng đa lớp từ ảnh đầu vào. Những đặc trưng này sau đó được kết hợp và xây dựng lại ở khối decoder, để đầu ra cuối cùng là ảnh đã được phân đoạn làn đường. Nhờ cơ chế này, TwinLiteNet có thể xử lý đồng thời cả ngữ cảnh cục bộ và toàn cục của ảnh, nâng cao độ chính xác.



Hình 3.4. Kiến trúc của mô hình học sâu TwinLiteNet

Về cấu trúc bên trong, khối encoder của TwinLiteNet sử dụng ESP-Encoder – một thành phần được kế thừa từ mạng ESPNet, vốn cũng được xây dựng nhằm mục đích nhận diện làn đường nhanh chóng. ESP-Encoder tận dụng các phép tích chập phân tách và tích chập giãn cách để giảm thiểu số lượng tham số mà vẫn giữ được khả năng trích xuất đặc trưng đa tỷ lệ. Để tăng cường thêm độ chính xác, TwinLiteNet còn tích hợp cơ chế attention, cụ thể là hai module: PAM (Position Attention Module – tập trung vào mối quan hệ giữa các vị trí trong không gian ảnh) và CAM (Channel Attention Module – chú ý đến sự phụ thuộc giữa các kênh đặc trưng). Cả hai module này hoạt động song song, giúp mạng học cách “tập trung” vào những vùng quan trọng nhất trên ảnh đầu vào. Sau đó, các đầu ra từ PAM và CAM được kết hợp lại và đưa qua khối decoder. Nhiệm vụ của decoder là xây dựng lại ảnh phân đoạn làn đường từ các đặc trưng giàu thông tin đã được tăng cường.

Về hiệu năng, TwinLiteNet cho tốc độ xử lý nhanh hơn rõ rệt so với các mô hình khác cùng giải quyết bài toán như YOLOP, YOLOPv2 hay HybridNets, đồng thời không xảy ra sự đánh đổi giữa tốc độ và độ chính xác. Cụ thể, độ chính xác trên tác vụ phân đoạn vùng lái được đạt mIoU 91,3%, chỉ thua không đáng kể so với YOLOP và YOLOPv2. Trên tác vụ phân đoạn vạch làn đường, chỉ số IoU đạt 31,08%.

TABLE I: Computation Performance.

Network	Params	Speed (FPS)
YOLOP [9]	7.9M	93
YOLOPv2 [10]	38.9M	95
HybridNets [11]	13.8M	25
TwinLiteNet (our)	0.4M	415

TABLE II: Drivable Area Segmentation Results. TABLE III: Lane Detection Results.

Network	mIoU (%)
MultiNet [13]	71.6
DLT-Net [12]	71.3
PSPNet [17]	89.6
HybridNets [11]	90.5
YOLOP [9]	91.5
YOLOPv2 [10]	93.2
TwinLiteNet (our)	91.3 (-1.9)

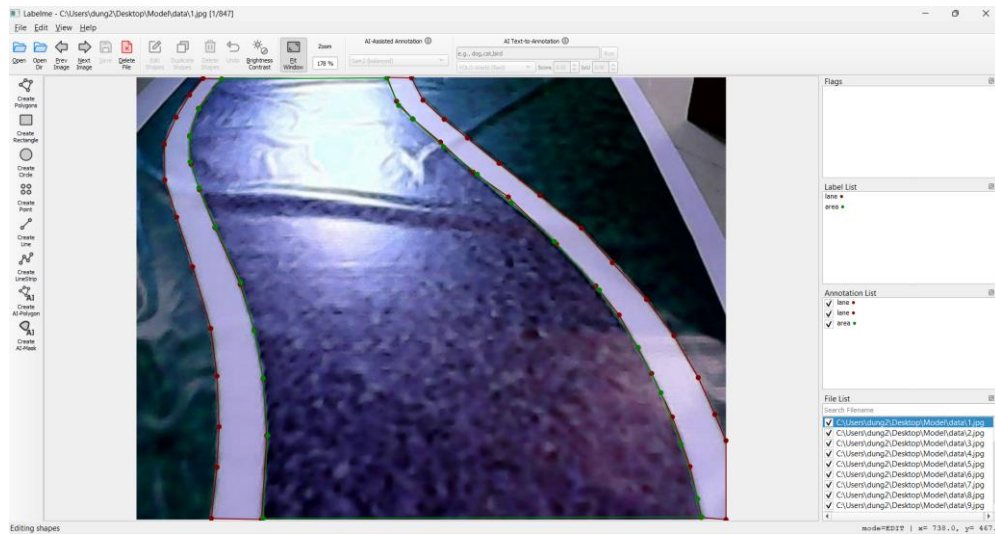
Network	IoU (%)
ENet [5]	14.64
SCNN [7]	15.84
R-101-SAD [8]	15.96
ENet-SAD [8]	16.02
YOLOP [9]	26.20
YOLOPv2 [10]	27.25
HybridNets [11]	31.6
TwinLiteNet (our)	31.08 (-0.52)

Hình 3.5. Kết quả so sánh của mô hình

b. Tập dữ liệu train mô hình

Tập dữ liệu bao gồm 847 ảnh màu được thu thập từ camera gắn trên xe trong các điều kiện ánh sáng khác nhau. Mỗi ảnh đi kèm một tệp chú thích LabelMe (định dạng JSON) chứa các đa giác (polygon) gán hai loại nhãn chính:

- area : vùng đường xe có thể lưu thông (drivable area).
- lane : vạch kẻ làn (lane line).



Hình 3.6. Gán nhãn làn đường và vùng đường

Các ảnh và JSON được đặt cùng thư mục data. Trong giai đoạn tiền xử lý, ảnh gốc và các mặt nạ (mask) được resize về kích thước 360×640 pixel, phù hợp với đầu vào của mô hình TwinLiteNet. Mask được tạo dưới dạng nhị phân hai kênh (nền và đối tượng) phục vụ bài toán phân đoạn hai lớp.

Để cải thiện khả năng tổng quát hóa, dữ liệu được tăng cường khi huấn luyện với các phép biến đổi ngẫu nhiên sau, mỗi phép có xác suất 0,5:

- Biến đổi phối cảnh (xoay, dịch chuyển, co giãn, nghiêng).
- Biến đổi không gian màu HSV (thay đổi hue, saturation, value).
- Lật ngang (horizontal flip).

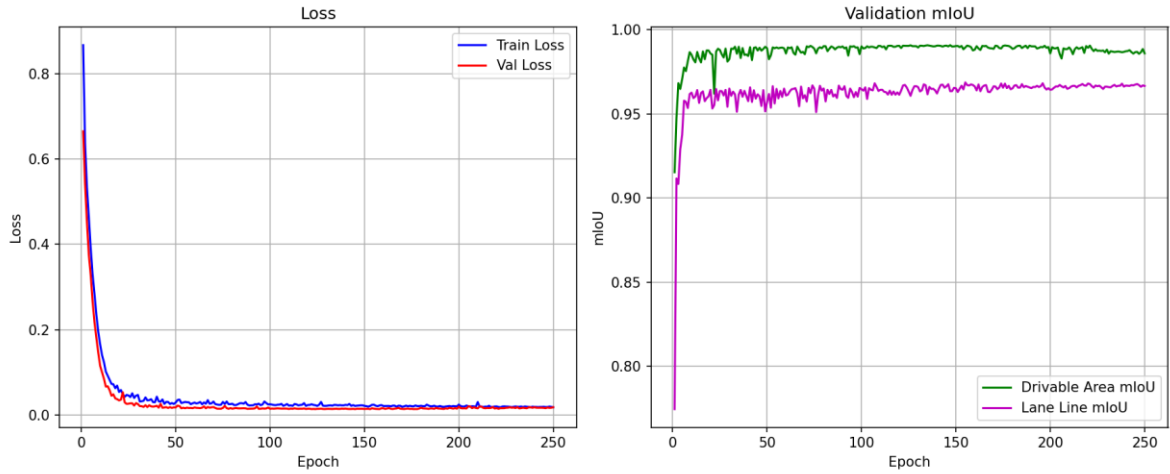
Tập dữ liệu được chia ngẫu nhiên thành 80% dùng cho huấn luyện (khoảng 678 ảnh) và 20% dùng cho đánh giá (validation) (khoảng 169 ảnh). Kích thước batch khi huấn luyện được thiết lập là 8 ảnh. Nhờ sự đa dạng và kích thước hợp lý, bộ dữ liệu đáp ứng được yêu cầu huấn luyện cho bài toán phân đoạn đường và làn xe.

c. Kết quả train mô hình

Quá trình huấn luyện mô hình TwinLiteNet được thực hiện trong 250 epochs với các tham số tối ưu hóa cụ thể như sau:

- Thuật toán tối ưu: Adam với hệ số suy giảm trọng số là 0.0005.
- Tốc độ học (Learning Rate): Khởi tạo ban đầu ở mức 0.0005, áp dụng bộ lập lịch giảm đa thức (Poly Learning Rate Scheduler) để giảm dần giá trị về gần 0 ở cuối chu kỳ huấn luyện nhằm đảm bảo tính hội tụ.
- Hàm mất mát tổng hợp (Total Loss): Phối hợp giữa Focal Loss (giảm thiểu tác động của hiện tượng mất cân bằng lớp) và Tversky Loss (gán trọng số $\alpha = 0.7$, $\beta = 0.3$ cho vùng di chuyển được; và $\alpha = 0.9$, $\beta = 0.1$ cho vạch kẻ làn). Cấu trúc này giúp tối ưu hóa trực tiếp chỉ số mIoU và hạn chế tối đa việc bỏ sót các mục tiêu kích thước nhỏ như vạch đường.

Kết quả sự biến thiên của hàm Loss và chỉ số mIoU qua 250 epochs được thể hiện chi tiết trong Hình 3.7.



Hình 3.7: Đồ thị biểu diễn sự biến thiên của hàm Loss và chỉ số mIoU trong quá trình huấn luyện

Dựa trên các đường đặc tính thu được từ đồ thị, kết quả huấn luyện được phân tích cụ thể:

- Đồ thị Loss: Trong 30 epochs đầu, cả Train Loss và Val Loss đều giảm mạnh từ 0.85 xuống dưới 0.05, cho thấy tốc độ hội tụ rất nhanh. Từ epoch 50 đến 250, hai đường này đi ngang ổn định và bám sát nhau. Khoảng cách Loss cực thấp chứng minh các phép tăng cường dữ liệu ngẫu nhiên (xoay phối cảnh, biến đổi HSV, lật ảnh) giúp mô hình tổng quát hóa xuất sắc, hoàn toàn không bị overfitting.
- mIoU Vùng di chuyển được (Drivable Area): Chỉ số mIoU nhanh chóng vượt ngưỡng 0.95 và ổn định ở mức tiệm cận tối đa (~99%), cho thấy khả năng phân tách bề mặt đường đạt độ chính xác gần như hoàn hảo.
- mIoU Vạch kẻ làn (Lane Line): Do diện tích pixel vạch làn rất nhỏ, đồ thị có dao động nhẹ ở giai đoạn đầu. Tuy nhiên, nhờ hàm phạt Tversky Loss định hướng tốt, mIoU vạch làn nhanh chóng tăng và duy trì vững chắc trong khoảng 96.5% – 97.0%.

Quá trình huấn luyện đạt trạng thái hội tụ lý tưởng, không bị bùng nổ gradient. Độ chính xác mIoU vượt trội (99% cho vùng đường, 96.5% cho vạch làn) khẳng định bộ trọng số tối ưu này đã đáp ứng đầy đủ tiêu chuẩn kỹ thuật để triển khai thuật toán điều khiển bám làn thời gian thực.

d. Thuật toán giữ làn đường

Biến đổi phối cảnh

Ảnh đầu vào $I(x, y)$ được ánh xạ sang không gian nhìn từ trên xuống bằng phép biến đổi phối cảnh với ma trận đồng nhất $M \in \mathbb{R}^{3 \times 3}$:

$$\begin{bmatrix} u \\ v \\ w \end{bmatrix} = M \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (3.15)$$

Tọa độ điểm trong ảnh phối cảnh được xác định bởi:

$$(x', y') = \left(\frac{u}{w}, \frac{v}{w} \right) \quad (3.16)$$

Trong đó:

- $I(x, y)$: ảnh đầu vào tại tọa độ pixel (x, y)
- M : ma trận biến đổi phối cảnh
- u, v, w : tọa độ đồng nhất sau biến đổi
- (x', y') : tọa độ điểm trong ảnh phối cảnh
- W, H : chiều rộng và chiều cao ảnh phối cảnh

Trong không gian ảnh phối cảnh: $W \times H = 640 \times 480$

Tâm ảnh theo phương ngang:

$$w_{\text{mid}} = \frac{W}{2} = 320 \quad (3.17)$$

Trong đó: w_{mid} là vị trí trung tâm ảnh dùng làm mốc tính sai số tám lần.

Xác định tâm đường theo từng hàng

Chọn $N = 10$ hàng ngang khảo sát trong ảnh phối cảnh. Tọa độ từng hàng được xác định:

$$y_i = \left\lfloor y_{\text{start}} - (i - 1) \cdot \frac{y_{\text{start}} - y_{\text{end}}}{N - 1} \right\rfloor \quad (3.18)$$

$$\text{với: } y_{\text{start}} = \lfloor 0.95H \rfloor; \quad y_{\text{end}} = \lfloor 0.80H \rfloor$$

Trong đó:

- y_i : tọa độ hàng khảo sát thứ i
- N : số hàng khảo sát
- y_{start} : hàng gần xe nhất
- y_{end} : hàng xa xe hơn
- $\lfloor \cdot \rfloor$: phép lấy phần nguyên

Tại mỗi hàng y_i cần xác định tập điểm thuộc vạch làn và vùng đường có thể di chuyển:

$$\begin{cases} \mathcal{L}(y_i) = \{x \mid \text{ll_mask}(x, y_i) = 255\} \\ \mathcal{D}(y_i) = \{x \mid \text{da_mask}(x, y_i) = 255\} \end{cases} \quad (3.19)$$

Trong đó:

- ll_mask : ảnh nhị phân vạch làn
- da_mask : ảnh nhị phân vùng đường
- giá trị 255: pixel thuộc lớp đối tượng

Biên trái và phải của vùng đường:

$$\begin{aligned} x_l(y_i) &= \min \mathcal{D}(y_i) \\ x_r(y_i) &= \max \mathcal{D}(y_i) \end{aligned} \quad (3.20)$$

Chia tập điểm vạch làn thành hai phía:

$$\begin{aligned} L_l &= \left\{ x \in \mathcal{L}(y_i) \mid x < \frac{x_l + x_r}{2} \right\} \\ L_r &= \left\{ x \in \mathcal{L}(y_i) \mid x > \frac{x_l + x_r}{2} \right\} \end{aligned} \quad (3.21)$$

Trong đó:

- L_l : tập điểm vạch làn bên trái
- L_r : tập điểm vạch làn bên phải

Tâm đường tại hàng y_i được xác định:

$$c(y_i) = \begin{cases} \frac{\mu(L_l) + \mu(L_r)}{2}, & L_l \neq \emptyset, L_r \neq \emptyset \\ \frac{\mu(L_l) + x_r}{2}, & L_l \neq \emptyset, L_r = \emptyset \\ \frac{x_l + \mu(L_r)}{2}, & L_l = \emptyset, L_r \neq \emptyset \\ \frac{x_l + x_r}{2}, & L_l = \emptyset, L_r = \emptyset \end{cases} \quad (3.22)$$

Trong đó:

- $c(y_i)$: tâm đường tại hàng y_i
- $\emptyset$: tập rỗng
- $\mu(\cdot)$: phép tính trung bình cộng

Lọc ngoại lai bằng nội suy

Sau khi quét tâm đường tại các hàng y_i , có thể xuất hiện một số điểm bị nhiễu hoặc phân đoạn sai, dẫn đến nhảy vọt bất thường. Các điểm này cần được phát hiện và thay thế bằng giá trị nội suy từ các điểm lân cận hợp lệ để đảm bảo đường tâm liên tục và mượt mà.

Gọi vector các tâm đường ban đầu: $\mathbf{c} = [c(y_1), c(y_2), \dots, c(y_N)]^T$

Một điểm được xem là ngoại lai nếu: $|c(y_i) - c(y_{i-1})| > \Delta_{\max}$ với $\Delta_{\max} = 35$ pixel

Trong đó: $\Delta_{\max}$ là ngưỡng sai khác tối đa giữa hai hàng liên tiếp

Các điểm ngoại lai sẽ được thay thế bằng nội suy tuyến tính từ những điểm hợp lệ lân cận. Giả sử tập chỉ số các điểm hợp lệ là $K = \{i_1, i_2, \dots, i_m\}$ thì giá trị nội suy:

$$c^*(y_i) = c(y_{i_k}) + \frac{i - i_k}{i_{k+1} - i_k} [c(y_{i_{k+1}}) - c(y_{i_k})] \text{ với: } i_k < i < i_{k+1} \quad (3.23)$$

Sau bước lọc:

$$\tilde{c}_i = c^*(y_i) \quad (3.24)$$

Trong đó:

- $c^*(y_i)$: giá trị sau nội suy

- $\tilde{c}_i$: tâm đường đã làm mượt

Tính sai số bám làn

Sai số tại từng hàng:

$$e_i = \tilde{c}_i - w_{\text{mid}} \quad (3.25)$$

Trong đó:

- e_i : sai số vị trí tại hàng i
- $\tilde{c}_i$: tâm đường sau lọc
- w_{mid} : tâm ảnh

Sai số tổng hợp:

$$\bar{e}(t) = \frac{\sum_{i=1}^N \alpha_i e_i}{\sum_{i=1}^N \alpha_i} \quad (3.26)$$

Trọng số của mỗi hàng được xác định:

$$\alpha_i = 1.0 - 0.7 \cdot \frac{i - 1}{N - 1} \quad (3.27)$$

Trong đó:

- $\bar{e}(t)$: sai số bám làn tổng hợp
- α_i : trọng số của hàng thứ i

Các hàng gần xe có trọng số lớn hơn nhằm ưu tiên phản ứng nhanh với quỹ đạo hiện tại.

Bộ điều khiển PD

Tín hiệu góc lái:

$$u(t) = K_P \bar{e}(t) + K_D \frac{\bar{e}(t) - \bar{e}(t - \Delta t)}{\Delta t} \quad (3.28)$$

Trong đó:

- $u(t)$: góc lái điều khiển
- $K_P = 0.35$: hệ số tỉ lệ
- $K_D = 0.12$: hệ số vi phân
- $\Delta t \approx 0.033$ s: chu kỳ điều khiển

Giới hạn góc lái:

$$u_{\text{cmd}}(t) = \max(-U_{\text{max}}, \min(U_{\text{max}}, u(t))) \text{ với: } U_{\text{max}} = 30^\circ \quad (3.29)$$

Trong đó:

- $u_{\text{cmd}}(t)$: góc lái cuối cùng sau giới hạn
- U_{max} : góc lái tối đa cho phép

Điều khiển tốc độ

Tốc độ đặt phụ thuộc độ lớn góc lái

$$v_{cmd}(t) = \begin{cases} v_{max}, & |u_{cmd}(t)| \leq \theta_{slow} \\ v_{min} + (v_{max} - v_{min}) \left(1 - \frac{|u_{cmd}(t)| - \theta_{slow}}{U_{max} - \theta_{slow}}\right)^2, & |u_{cmd}(t)| > \theta_{slow} \end{cases} \quad (3.30)$$

Trong đó:

- $v_{cmd}(t)$: tốc độ đặt
- $v_{max} = 0.15$ m/s: tốc độ tối đa
- $v_{min} = 0.10$ m/s: tốc độ tối thiểu
- $\theta_{slow} = 15^\circ$: ngưỡng bắt đầu giảm tốc

Cơ chế Boost khi xe bị kẹt

Xe được xem là bị kẹt nếu trong khoảng $\Delta t \geq T_{stall}$ thỏa mãn:

$$\begin{cases} v_{actual}(t) < v_{move} \\ v_{cmd}(t) > 0.1 \\ |u_{cmd}(t)| \leq U_{boost} \end{cases} \quad \text{với: } \begin{cases} T_{stall} = 10.0 \text{ s} \\ v_{move} = 0.05 \text{ m/s} \\ U_{boost} = 15^\circ \end{cases}$$

Trong đó:

- $v_{actual}(t)$: tốc độ thực tế đo được
- v_{move} : ngưỡng xác định xe đang di chuyển
- U_{boost} : giới hạn góc lái cho phép kích hoạt Boost

Giai đoạn 1 – Tăng tốc mạnh

$$\begin{aligned} T_f &= 1.0 \text{ s} \\ v_{final}(t) &= v_f = 0.70 \text{ m/s} \end{aligned} \quad (3.31)$$

Giai đoạn 2 – Lùi ngắn

$$\begin{aligned} T_b &= 0.1 \text{ s} \\ v_{final}(t) &= v_b = -0.10 \text{ m/s} \end{aligned} \quad (3.32)$$

Giai đoạn 3 – Giảm tốc tuyến tính

$$\begin{aligned} T_{ramp} &= 0.05 \text{ s} \\ v_{final}(t) &= v_f - \frac{t'}{T_{ramp}} (v_f - v_{cmd}) \end{aligned} \quad (3.33)$$

Trong đó: t' là thời gian đã trôi qua trong giai đoạn giảm tốc.

Sau đó: $v_{final}(t) = v_{cmd}(t)$

Boost bị hủy nếu: $|u_{cmd}(t)| > U_{boost}$

Điều kiện dừng khẩn cấp

Gọi $A_{da}(t)$ là diện tích vùng đường hợp lệ sau lọc nhiễu.

Nếu $A_{da}(t) < A_{min}$ với $A_{min} = 1000$ pixel hoặc không tồn tại bất kỳ $\tilde{c}_i$ hợp lệ nào thì:

$$\begin{aligned} v_{final}(t) &= 0 \\ u_{final}(t) &= 0 \end{aligned}$$

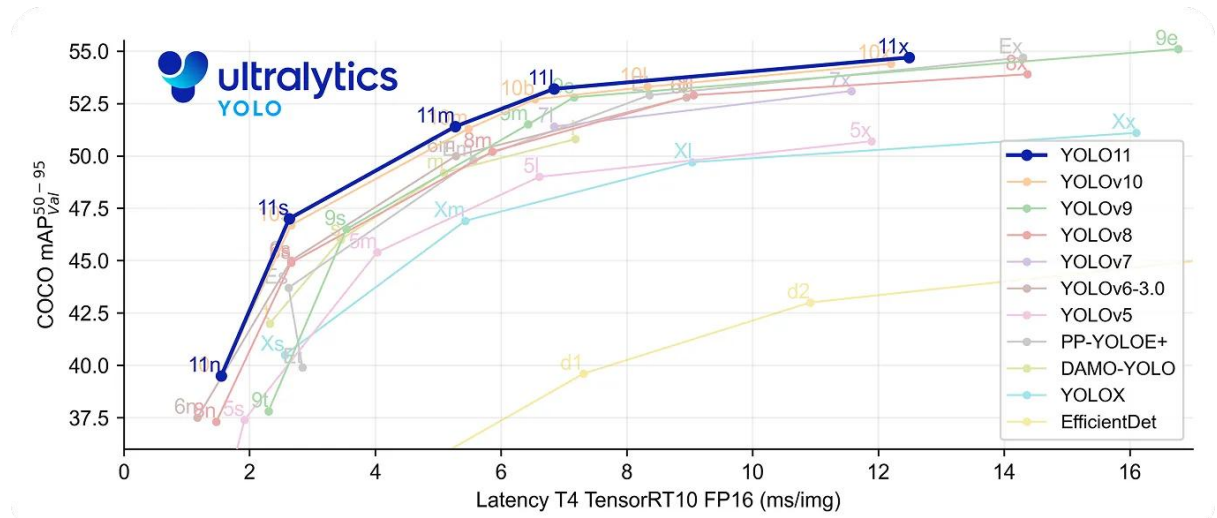
Trong đó:

- $A_{da}(t)$: diện tích vùng đường
- A_{min} : ngưỡng diện tích tối thiểu
- $u_{final}(t)$: góc lái cuối cùng
- $v_{final}(t)$: vận tốc cuối cùng

3.2.2. Mô hình YOLOv11 cho bài toán nhận diện ô gà

a. Giới thiệu về mô hình YOLOv11

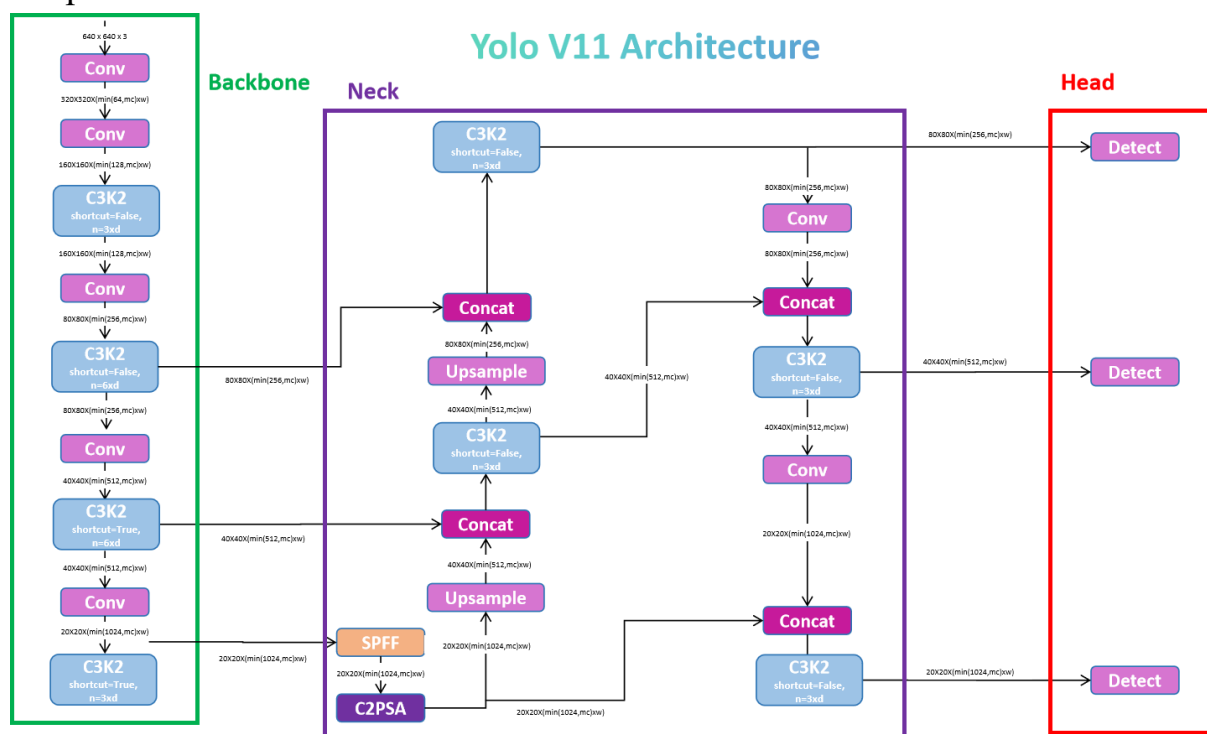
YOLOv11 là một phiên bản của dòng mô hình phát hiện đối tượng theo thời gian thực được phát triển bởi Ultralytics vào năm 2024, kế thừa kiến trúc của YOLOv8 và được tối ưu vượt bậc về cả tốc độ lẫn độ chính xác.



Hình 3.8. So sánh YOLOv11 với các phiên bản trước và những mô hình khác trên tập dữ liệu COCO

Kiến trúc cốt lõi của YOLO gồm ba thành phần chính: backbone trích xuất đặc trưng đa tỷ lệ, neck tổng hợp và tăng cường biểu diễn đặc trưng, head tạo ra dự đoán cuối cùng về vị trí và phân loại đối tượng. YOLOv11 được xây dựng dựa trên YOLOv8 nhưng có những cải tiến đáng kể về cấu trúc và tối ưu tham số, mang lại hiệu suất phát hiện vượt trội. Cụ thể, trong backbone, YOLOv11 thay thế khối C2F bằng khối C3K2 (sử dụng hai phép tích chập nhỏ thay vì một phép lớn để tối ưu hiệu quả tính toán), đồng thời giữ lại khối SPPF và bổ sung khối C2PSA giúp tăng cường khả năng tập trung vào đặc trưng không gian. Ở neck, khối C3K2 giúp quá trình tổng hợp đặc trưng nhanh và hiệu quả hơn, kết hợp với C2PSA để cải thiện độ chính xác khi phát hiện vật thể nhỏ hoặc bị che khuất. Tại head, module C3K2 được sử dụng để xử lý và tinh chỉnh bản đồ đặc trưng, cho phép tùy chỉnh kích thước kernel nhờ tham số C3K2, từ đó nâng cao khả năng thích ứng với các đối tượng có kích thước và hình dạng khác nhau. Nhờ những cải tiến này, YOLOv11 không chỉ giảm số lượng tham số, tối ưu tốc độ xử lý mà

còn mở rộng khả năng triển khai trên nhiều môi trường khác nhau, vượt trội so với phiên bản trước.



Hình 3.9 Các module kiến trúc chính trong YOLOv11

Nhờ những cải tiến này, YOLOv11 hỗ trợ đa dạng tác vụ như phát hiện đối tượng, phân đoạn thể hiện, ước tính tư thế, phân loại ảnh, phát hiện đối tượng theo hướng và theo dõi đối tượng. Để phù hợp với nhiều nền tảng phần cứng khác nhau (từ thiết bị biên đến GPU mạnh), YOLOv11 cung cấp năm phiên bản: nano (n), small (s), medium (m), large (l) và extra large (x). Bảng 3.2 dưới đây so sánh chi tiết hiệu năng của các mô hình YOLOv11 Detect trên tập dữ liệu COCO, bao gồm độ chính xác (mAP), tốc độ suy luận trên CPU và GPU, số lượng tham số và FLOPs.

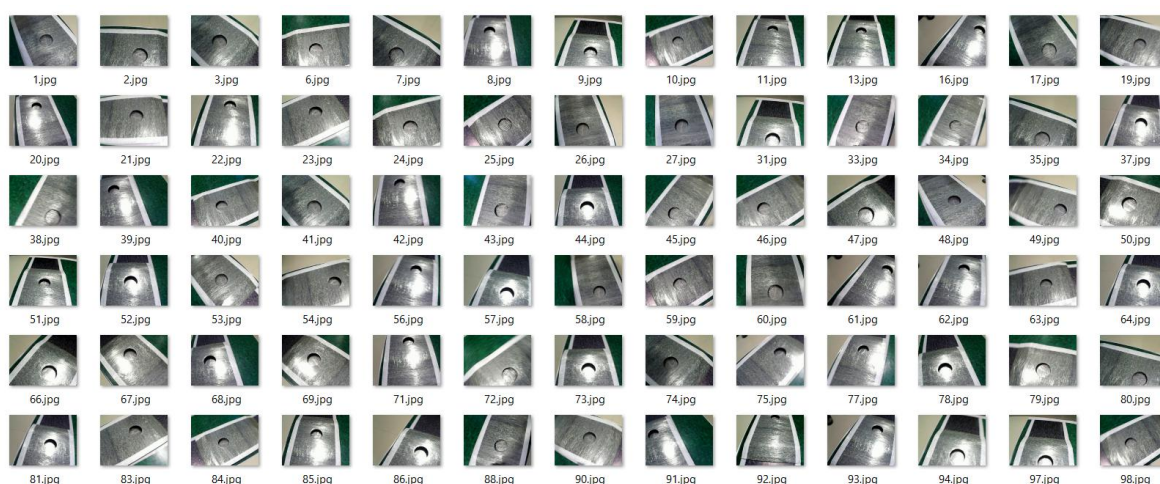
Bảng 3.2. Các mô hình có sẵn của YOLOv11 cho tác vụ phát hiện đối tượng.

Model	size (pixels)	mAPval 50-95	Speed CPU ONNX (ms)	Speed T4 TensorRT10 (ms)	params(M)	FLOPs(B)
YOLOv11n	640	39,5	56,1 ± 0,8	1,5 ± 0,0	2,6	6,5
YOLOv11s	640	47,0	90,0 ± 1,2	2,5 ± 0,0	9,4	21,5
YOLOv11m	640	51,5	183,2 ± 2,0	4,7 ± 0,1	20,1	68,0
YOLOv11l	640	53,4	238,6 ± 1,4	6,2 ± 0,1	25,3	86,9
YOLOv11x	640	54,7	462,8 ± 6,7	11,3 ± 0,2	56,9	194,9

b. Tập dữ liệu train mô hình

Để phục vụ bài toán phát hiện và phân đoạn ổ gà, tập dữ liệu được xây dựng từ các ảnh chụp trên sa hình. Toàn bộ 98 ảnh được thu thập trong các điều kiện ánh sáng và góc chụp khác nhau nhằm tăng tính đa dạng cho mô hình. Trên mỗi ảnh, ranh giới của ổ gà được gán nhãn thủ công dưới dạng đa giác (polygon) bằng phần mềm LabelMe – công cụ phổ biến cho bài toán phân đoạn.

Sau khi gán nhãn, các tệp JSON do LabelMe sinh ra được chuyển đổi sang định dạng YOLO segmentation. Quá trình chuẩn hoá này bao gồm: đọc tọa độ các điểm từ file JSON, chuẩn hóa chúng về khoảng $[0,1]$ theo chiều rộng và chiều cao thực tế của ảnh, sau đó ghi vào tệp .txt với mỗi dòng có cấu trúc `<class_id> <x1> <y1> <x2> <y2> ... <xN> <yN>`. Lớp duy nhất được sử dụng có mã 0 tương ứng với nhãn "pothole".

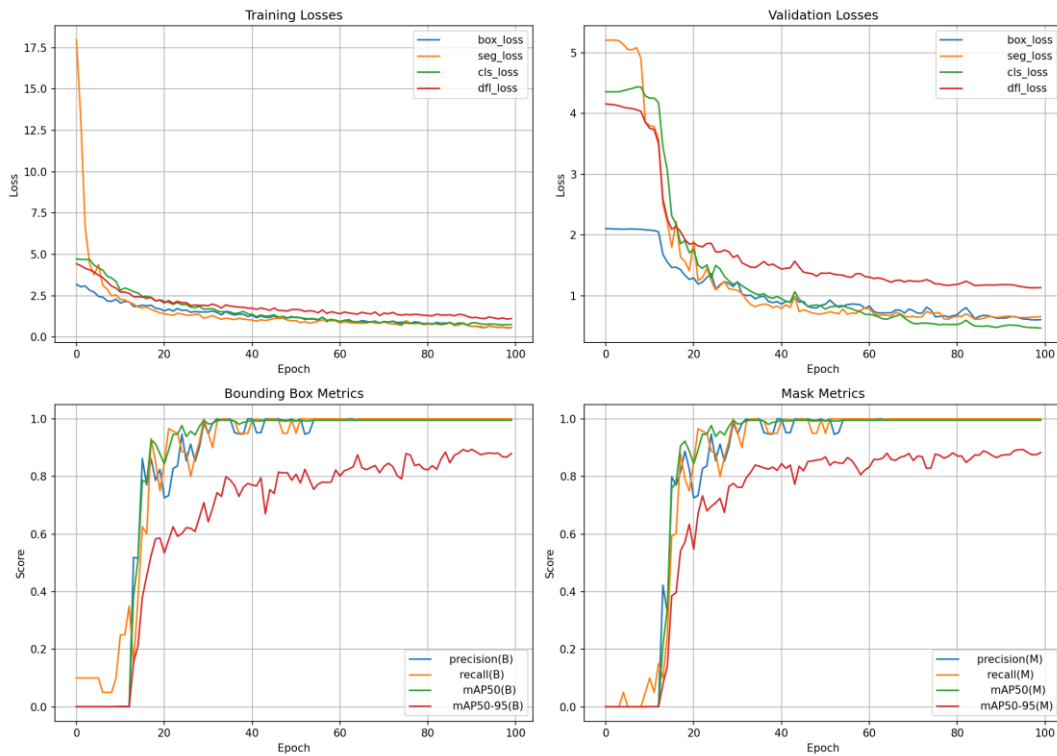


Hình 3.10. Tập dữ liệu để train mô hình

Cuối cùng, tập dữ liệu được phân chia ngẫu nhiên với tỷ lệ 80% cho huấn luyện và 20% cho xác thực, tương ứng với 78 ảnh dùng để train và 20 ảnh dùng để val. Kích thước ảnh đầu vào của mô hình được thống nhất là 640×640 pixel. Cấu trúc thư mục đầu ra tuân thủ đúng chuẩn của thư viện Ultralytics YOLO, sẵn sàng cho quá trình huấn luyện mô hình YOLOv11 ở bước tiếp theo.

c. Kết quả train mô hình

Mô hình YOLOv11-seg (phiên bản nano) được huấn luyện từ đầu trên tập dữ liệu 78 ảnh train và 20 ảnh val, qua 100 epoch với kích thước ảnh 640×640 , batch size 8. Toàn bộ diễn biến của quá trình huấn luyện được thể hiện qua Hình 3.11 dưới đây.



Hình 3.11. Đồ thị huấn luyện của mô hình YOLOv11

Cả bốn loại loss trên tập huấn luyện và xác thực đều giảm nhanh trong 20 epoch đầu. Điển hình, seg_loss giảm từ ~18 xuống dưới 2. Sau epoch 60, các giá trị ổn định, không có dấu hiệu overfitting. Ở epoch 100, các loss đạt: box_loss=0,58, seg_loss=0,58, cls_loss=0,76, dfl_loss=1,12 trên tập train, và tương ứng 0,60; 0,65; 0,47; 1,13 trên tập val.

Ở giai đoạn đầu (0–14 epoch), các metrics gần bằng 0. Từ epoch 14 trở đi, precision, recall, mAP50 tăng vọt. Kết quả chính ở epoch 100 được tổng hợp trong Bảng 3.3.

Bảng 3.3. Kết quả đánh giá mô hình trên tập xác thực

Chỉ số	Phát hiện (Bbox)	Phân đoạn (Mask)
Precision	0,998	0,998
Recall	1,000	1,000
mAP50	0,995	0,995
mAP50-95	0,879	0,883

Mô hình đạt độ phủ 100%, độ chính xác 99,8% và mAP50 là 99,5% trên cả hai tác vụ. Chỉ số mAP50-95 đạt ≈0,88, cho thấy khả năng định vị và phân đoạn chi tiết ở ngưỡng IoU cao. Với chỉ 78 ảnh huấn luyện, kết quả này khẳng định kiến trúc YOLOv11 cùng các kỹ thuật augmentation giúp mô hình học rất hiệu quả dữ liệu ồ ạt trên sa hình, không bị overfitting và sẵn sàng ứng dụng thực tế.

d. Thuật toán tính kích thước ổ gà

Hệ thống xác định ba thông số hình học của ổ gà được xử lý trên PC gồm: diện tích bề mặt, độ sâu tối đa và thể tích ước tính. Quy trình dựa trên ảnh biến đổi phối cảnh nhận từ xe đã hiệu chỉnh phối cảnh và dữ liệu từ một cảm biến đo độ sâu duy nhất.

Mô hình YOLOv11-seg tạo mặt nạ nhị phân (mask) cho mỗi ổ gà trên ảnh phối cảnh. Gọi A_{px} là số pixel thuộc mask (diện tích trên ảnh). Quá trình hiệu chỉnh camera xác định: 60 cm thực tế ứng với 400 pixel trên ảnh BEV, do đó hệ số quy đổi diện tích là:

$$k = \left(\frac{60}{400}\right)^2 = 0,0225 \text{ cm}^2/\text{pixel}. \quad (3.34)$$

Diện tích thực của ổ gà được tính bằng:

$$A_{\text{cm}^2} = A_{px} \times k. \quad (3.35)$$

Trong quá trình theo dõi (tracking), khi ổ gà di chuyển qua vùng ROI1, hệ thống liên tục cập nhật diện tích và lưu lại giá trị lớn nhất đạt được.

Xe chỉ có một cảm biến đo độ sâu đặt cố định, cảm biến này chỉ đo được độ sâu tại một điểm trên mặt đường. Để có được độ sâu đặc trưng cho toàn bộ ổ gà, thuật toán thực hiện như sau:

- Vùng ROI2 được định nghĩa là dải hẹp phía trước xe, nơi tia cảm biến có khả năng giao với ổ gà.
- Khi tâm bounding box của ổ gà nằm trong ROI2, hệ thống ghi nhận giá trị độ sâu $d_{\text{đo}}$ từ cảm biến.
- Mỗi track (ổ gà) lưu lại độ sâu lớn nhất từ tất cả các lần đo:

$$d_{\text{max}} = \max \{d_{\text{đo},1}, d_{\text{đo},2}, \dots \}. \quad (3.36)$$

Việc lấy giá trị lớn nhất giúp bù đắp cho hạn chế chỉ có một điểm đo: khi ổ gà có độ sâu không đều, chỉ cần có lần nào tia cảm biến trúng vào vùng sâu nhất thì giá trị đó được ghi nhận, phản ánh đúng mức độ nghiêm trọng của ổ gà.

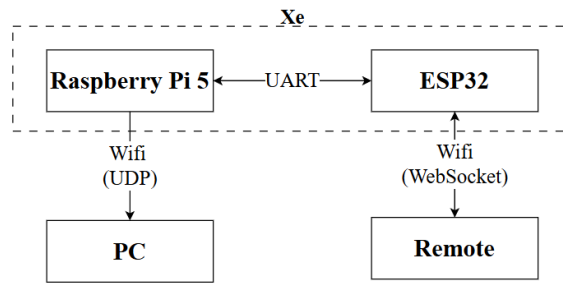
Thể tích của ổ gà được tính gần đúng theo mô hình hình trụ, với đáy là diện tích bề mặt và chiều cao là độ sâu tối đa:

$$V_{\text{cm}^3} = A_{\text{cm}^2} \times d_{\text{max}}. \quad (3.37)$$

Nhờ cơ chế lấy giá trị lớn nhất qua nhiều lần đo, hệ thống chỉ với một cảm biến độ sâu vẫn ước lượng được kích thước ổ gà một cách đáng tin cậy. Các thông số này được lưu cùng metadata vào cơ sở dữ liệu phục vụ thống kê và hiển thị.

3.3. Giải pháp truyền thông

Hệ thống sử dụng ba kênh truyền thông chính để đảm bảo luồng dữ liệu từ cảm biến, xử lý trung tâm, điều khiển từ xa và hiển thị trên PC. Hình dưới đây mô tả tổng thể các kết nối:



Hình 3.12. Sơ đồ truyền thông giữa các khối

3.3.1. Truyền thông nội bộ trong xe

a. Giới thiệu về chuẩn truyền thông UART

Khái niệm: UART (Universal Asynchronous Receiver/Transmitter) là một giao thức truyền thông phần cứng nối tiếp, bất đồng bộ, cho phép hai thiết bị trao đổi dữ liệu với nhau. Giao thức này sử dụng hai đường tín hiệu độc lập: TX (truyền) và RX (nhận). Điểm đặc trưng của UART là không cần xung clock đồng bộ giữa các bên; tốc độ truyền được xác định trước và phải giống nhau ở cả hai thiết bị.

Nguyên lý hoạt động: Dữ liệu được truyền dưới dạng các khung (frame) tuần tự. Mỗi khung bao gồm:

- 1 bit Start (bắt đầu), kéo đường TX từ mức cao xuống thấp báo hiệu bắt đầu khung.
- 5–9 bit dữ liệu (thường 8 bit), được gửi từ bit thấp nhất đến bit cao nhất.
- 1 bit Parity (kiểm tra chẵn lẻ, tùy chọn) giúp phát hiện lỗi truyền.
- 1 hoặc 2 bit Stop, đưa đường TX về mức cao, kết thúc khung.

Các bit được truyền nối tiếp với tốc độ baud (bit/giây) được cài đặt trước (ví dụ 9600, 115200). Giao tiếp UART có thể hoạt động ở chế độ simplex (một chiều), half-duplex (hai chiều luân phiên) hoặc full-duplex (hai chiều đồng thời). Tín hiệu thường ở mức 3,3V hoặc 5V. Mỗi kết nối UART chỉ hỗ trợ hai thiết bị (master – slave), không cho phép nhiều slave trên cùng một bus.

Ưu và nhược điểm:

- **Ưu điểm:** Chỉ cần hai dây tín hiệu, không cần clock ngoài, cấu trúc khung linh hoạt, dễ triển khai, có cơ chế kiểm tra lỗi parity, được hỗ trợ rộng rãi trong các vi điều khiển.
- **Nhược điểm:** Kích thước khung dữ liệu nhỏ (tối đa 9 bit), không hỗ trợ nhiều slave, tốc độ hai bên phải đồng bộ với sai số dưới 10%.

Ứng dụng chung: UART được sử dụng phổ biến để kết nối vi điều khiển với máy tính (qua cáp USB-UART), giao tiếp giữa các module GPS, Bluetooth, cảm biến, màn hình hiển thị, điều khiển robot, và trong các hệ thống đo lường.

Nhờ tính đơn giản và tin cậy, UART là một trong những giao thức truyền thông nội bộ được ưa chuộng nhất trong các hệ thống nhúng.

b. Ứng dụng vào bài toán

Trong hệ thống xe, UART được sử dụng để kết nối Raspberry Pi 5 (bộ xử lý AI) với ESP32 (bộ điều khiển động cơ và thu thập cảm biến). Cụ thể:

Kết nối phần cứng và cấu hình

- Chân kết nối trên ESP32: TX là GPIO17, RX là GPIO16. Các chân này được nối trực tiếp với UART của Raspberry Pi (TX Pi nối RX ESP32, RX Pi nối TX ESP32).
- Tốc độ baud: 115200, đủ nhanh để truyền lệnh điều khiển và nhận phản hồi cảm biến với chu kỳ 50 ms.
- Chế độ truyền: Full-duplex, cho phép hai bên gửi và nhận đồng thời.

Luồng dữ liệu từ Raspberry Pi xuống ESP32 (lệnh điều khiển)

Raspberry Pi gửi lệnh dưới dạng chuỗi text theo cú pháp: *S<speed>A<steer>\r\n*

Trong đó:

<speed>: tốc độ mong muốn (m/s), có thể âm để lùi, giá trị nằm trong khoảng $\pm 0,70$.

<steer>: góc đánh lái mong muốn (độ), dương là rẽ phải, âm là rẽ trái, giới hạn $\pm 30^\circ$.

Ví dụ: *S0.35A-12.00\r\n* yêu cầu xe chạy 0,35 m/s và đánh lái 12° sang trái.

Lệnh được gửi từ Raspberry Pi mỗi khi có kết quả điều khiển mới (từ AI hoặc từ remote), tần suất khoảng 20–30 Hz.

Luồng dữ liệu từ ESP32 lên Raspberry Pi (phản hồi trạng thái)

ESP32 đọc các cảm biến và gửi phản hồi định kỳ (mỗi 50 ms) theo định dạng:

FB:S=<real_speed>|A=<real_steer>|B=<battery_percent>|P=<pothole_dept h_cm>|F=<front_cm>|R=<rear_cm>|M=<mode>\n

Trong đó:

S: Tốc độ thực tế (m/s) tính từ encoder.

A: Góc lái thực tế (độ) tính từ phản hồi servo.

B: Dung lượng pin còn lại (%).

P: Độ sâu ổ gà đo được từ cảm biến siêu âm hướng xuống (cm).

F: Khoảng cách từ cảm biến trước đến vật cản (cm), inf nếu không có vật cản.

R: Khoảng cách từ cảm biến sau đến vật cản (cm), inf nếu không có.

M: Chế độ hoạt động (0: manual, 1: auto).

Ví dụ: $FB:S=0.34|A=-11.8|B=87|P=5.2|F=inf|R=inf|M=1$

Raspberry Pi nhận chuỗi này, phân tích và cập nhật các biến trạng thái (tốc độ, góc lái, pin, độ sâu, khoảng cách vật cản) để phục vụ cho điều khiển vòng kín và hiển thị trên giao diện.

Nhờ kết nối UART ổn định, độ trễ thấp, giải pháp này đáp ứng tốt yêu cầu điều khiển thời gian thực của xe tự hành.

3.3.2. Truyền thông từ xe tới PC

a. Giới thiệu về giao thức UDP

Khái niệm: UDP (User Datagram Protocol) là một trong hai giao thức quan trọng thuộc tầng vận chuyển của bộ giao thức TCP/IP. UDP được thiết kế để truyền dữ liệu nhanh, không thiết lập kết nối trước khi gửi, và không đảm bảo độ tin cậy. Điều này làm cho UDP phù hợp với các ứng dụng yêu cầu độ trễ thấp, cho phép mất một phần dữ liệu, như phát lại video, gọi thoại/video, chơi game trực tuyến, hoặc tra cứu DNS.

Đặc điểm chính:

- Không có quá trình bắt tay (handshake) giữa hai bên trước khi truyền dữ liệu.
- Các gói tin có thể đến sai thứ tự, bị trùng, hoặc bị mất mà không có cơ chế truyền lại.
- Độ trễ thấp, chi phí nhỏ do không có cơ chế kiểm soát luồng, kiểm soát lỗi hay xác nhận gói tin.
- Tiêu đề cố định 8 byte bao gồm các trường: Cổng nguồn (2B), Cổng đích (2B), Độ dài (2B), Checksum (2B). So với TCP (tiêu đề 20–60 byte), UDP nhẹ hơn nhiều.

So sánh UDP với TCP:

- TCP: Đáng tin cậy, có thứ tự, có kiểm soát tắc nghẽn – dùng cho truyền file, web, email.
- UDP: Nhanh hơn, không tin cậy – dùng cho streaming video/audio, DNS, game online.

Ứng dụng chung của UDP:

- Truyền video theo thời gian thực (RTSP, WebRTC).
- Giao thức DHCP, DNS (vì yêu cầu phản hồi nhanh).
- Các ứng dụng multicast (phát đa hướng).

Trong hệ thống phát hiện ổ gà, yêu cầu đặt lên hàng đầu là độ trễ thấp để PC có thể xử lý và hiển thị video kịp thời; việc mất một vài khung hình là chấp nhận được. Vì vậy, UDP là lựa chọn tối ưu.

b. Ứng dụng vào bài toán

Trong hệ thống, Raspberry Pi 5 (trên xe) đóng vai trò client UDP, PC (máy tính trung tâm) đóng vai trò server lắng nghe trên cổng 5557. Quy trình thực hiện như sau:

- *Tạo ảnh BEV và metadata trên Raspberry Pi*: Raspberry Pi xử lý ảnh từ camera, biến đổi phối cảnh thành ảnh BEV kích thước 400×600 pixel, đồng thời tổng hợp metadata dạng JSON bao gồm: tọa độ GPS, độ sâu ổ gà, tốc độ, góc lái, dung lượng pin, khoảng cách vật cản, góc quay thân xe. Các dữ liệu này được lấy từ ESP32 qua UART và từ chính Raspberry Pi.

- *Nén ảnh và đóng gói dữ liệu*: Ảnh BEV được nén thành JPEG với hệ số chất lượng 85%. Toàn bộ gói tin có cấu trúc:

[meta_len (4B)] [metadata (JSON)] [frame_len (4B)] [JPEG data]

Trong đó:

meta_len: độ dài chuỗi JSON (little-endian).

frame_len: độ dài dữ liệu JPEG.

Do gói tin thường vượt quá 1500 byte do vậy cần phân mảnh.

- *Phân mảnh và truyền qua UDP*: Mỗi gói UDP con có cấu trúc:

frame_id (2B) | num_chunks (2B) | chunk_idx (2B) | chunk_data

Trong đó:

frame_id: định danh khung hình.

num_chunks: tổng số mảnh.

chunk_idx: chỉ số mảnh (0...num_chunks-1).

chunk_data: một phần của gói tổng hợp.

Raspberry Pi gửi lần lượt các chunk đến địa chỉ IP của PC ở cổng 5557, với tốc độ khoảng 15 frame/giây.

- *Nhận và xử lý trên PC*: PC chạy luồng UDPVideoThread liên tục nhận các gói UDP. Dựa vào frame_id, nó tập hợp các chunk. Khi đủ số chunk cho một frame, PC giải nén JPEG, phân tích metadata, và phát tín hiệu đến chương trình PotholeProcessor để xử lý phát hiện ổ gà. Nếu quá thời gian chờ, các mảnh chưa hoàn chỉnh sẽ bị hủy.

- *Xử lý mất kết nối*: Nếu PC không nhận được bất kỳ gói tin nào trong 10 giây, nó phát tín hiệu Disconnected, xóa màn hình, reset chỉ số FPS, và thông báo mất kết nối. Khi kết nối được khôi phục, luồng video tự động tiếp tục.

Ưu điểm của giải pháp:

- Độ trễ thấp (thường dưới 50 ms), đáp ứng yêu cầu thời gian thực.

- Cơ chế phân mảnh cho phép truyền các khung hình lớn hơn đơn vị truyền tối đa.

- Dễ dàng mở rộng cho nhiều xe bằng cách thay đổi IP hoặc cổng.

Tóm lại, kết hợp WiFi và UDP là giải pháp tối ưu để truyền video từ Raspberry Pi 5 đến PC, đảm bảo tốc độ và độ trễ thấp, phù hợp với bài toán phát hiện ở gà thời gian thực.

3.3.3. Truyền thông từ xe tới điều khiển

a. Giới thiệu về giao thức WebSocket

Khái niệm: WebSocket là một giao thức truyền thông cung cấp các kênh liên lạc song công hoàn toàn qua một kết nối TCP duy nhất giữa máy khách và máy chủ. Không giống như HTTP truyền thống tuân theo mô hình phản hồi yêu cầu, giao thức này cho phép giao tiếp hai chiều. Điều này có nghĩa là máy khách và máy chủ có thể gửi dữ liệu cho nhau bất cứ lúc nào, giúp dữ liệu được truyền đi nhanh chóng mà không cần phải tải lại trang web.

Cách thức hoạt động: Về bản chất, WebSocket có tính chất hai chiều, nghĩa là giao tiếp diễn ra qua lại giữa máy khách-máy chủ. Do đó, quá trình hoạt động của giao thức này vẫn cần bắt đầu bằng một "bắt tay" (handshake) đặc biệt, sau đó là duy trì một kết nối liên tục, cho phép truyền dữ liệu một cách nhanh chóng và hiệu quả. Toàn bộ quá trình hoạt động của WebSocket sẽ diễn ra như sau:

- Đầu tiên, trình duyệt gửi một yêu cầu HTTP đặc biệt đến máy chủ, yêu cầu mở một kết nối WebSocket. Yêu cầu này bao gồm một tiêu đề HTTP/1.1 thông báo cho máy chủ biết rằng trình duyệt muốn "nâng cấp" kết nối từ HTTP sang WebSocket.

- Nếu máy chủ có hỗ trợ giao thức WebSocket, nó sẽ phản hồi với một tiêu đề HTTP để xác nhận việc chuyển đổi. Khi tiêu đề này được trình duyệt tiếp nhận, kết nối WebSocket sẽ chính thức được thiết lập.

- Sau khi kết nối được thiết lập, máy chủ và trình duyệt đã có thể gửi dữ liệu lẫn nhau mà không cần yêu cầu hoặc phản hồi. Điều này giúp giảm thiểu độ trễ và tăng tốc độ truyền dữ liệu. Dữ liệu được truyền qua WebSocket được đóng gói trong các frame. Mỗi khung có thể chứa dữ liệu văn bản hoặc nhị phân, trong đó mỗi bên lại có thể gửi và nhận dữ liệu độc lập với nhau.

- Khi có bất kỳ bên nào chủ động đóng kết nối, giao thức WebSocket sẽ được đóng lại và quá trình truyền tin sẽ kết thúc.

Ứng dụng chung của WebSocket:

- Ứng dụng trò chuyện trực tuyến (chat).
- Trò chơi trực tuyến (đồng bộ trạng thái).
- Ứng dụng tài chính, chứng khoán (cập nhật giá theo thời gian thực).
- Truyền phát video, âm thanh.

- IoT (Internet of Things): Kết nối liên mạch giữa các thiết bị thông minh.

Hạn chế:

- Không tương thích với một số trình duyệt cũ.
- Yêu cầu duy trì kết nối liên tục, có thể gây quá tải tài nguyên nếu số lượng client lớn.
- Triển khai phức tạp hơn HTTP truyền thống.

b. Ứng dụng vào bài toán

Trong hệ thống điều khiển xe từ xa, ESP32 chạy một WebSocket server (cổng 81) cùng với một Web server (cổng 80). Mọi thiết bị (điện thoại, máy tính bảng, laptop) kết nối vào cùng mạng WiFi với ESP32 đều có thể truy cập giao diện web và điều khiển xe.

- Giao diện web nhúng sẵn: Toàn bộ mã HTML/CSS/JS được lưu trong bộ nhớ ESP32. Người dùng truy cập địa chỉ IP của ESP32 qua trình duyệt, tải trang có các thanh trượt (tốc độ, góc lái) và nút bật/tắt chế độ tự động.
- Kết nối WebSocket: Sau khi tải trang, trình duyệt mở kết nối WebSocket tới ws://<ESP32_IP>:81/. Người dùng kéo thanh trượt → JavaScript gửi lệnh với tần suất cao (~30 lần/giây) nhờ WebSocket, đảm bảo điều khiển mượt mà.
- Xử lý lệnh trên ESP32: Hàm handleWsMsg() nhận và phân tích lệnh. Nếu ở chế độ manual, lệnh được chuyển đến bộ điều khiển động cơ; nếu ở chế độ auto, lệnh thủ công bị bỏ qua. Ngoài ra, ESP32 gửi gói state: (tốc độ thực, pin, cảm biến) qua WebSocket mỗi 150–500 ms để cập nhật giao diện.
- An toàn mất kết nối: ESP32 có timeout 2 giây; nếu không nhận heartbeat hoặc lệnh, nó tự động dừng xe. Trên giao diện, WebSocket tự động thử kết nối lại khi mất kết nối.

Nhờ WebSocket, hệ thống đạt được:

- Độ trễ thấp: Thao tác từ remote đến xe gần như tức thì.
- Cập nhật trạng thái thời gian thực: Người lái luôn biết tốc độ, pin, khoảng cách vật cản.
- Triển khai nhẹ: ESP32 vừa chạy WebSocket server vừa xử lý điều khiển động cơ mà không quá tải.

3.4. Xây dựng giao diện giám sát

Giao diện giám sát được xây dựng bằng PyQt5, hoạt động trên máy tính trung tâm (PC), nhận dữ liệu từ Raspberry Pi qua UDP và hiển thị trực quan kết quả phát hiện ổ gà, thông số xe, bản đồ vị trí, cùng khả năng quản lý dữ liệu lịch

sử. Giao diện gồm ba tab chính: Camera (hiển thị ảnh BEV và các ổ gà phát hiện), Dữ liệu (quản lý bản ghi) và Bản đồ (hiển thị vị trí các ổ gà trên bản đồ số).

3.4.1. Giao diện video

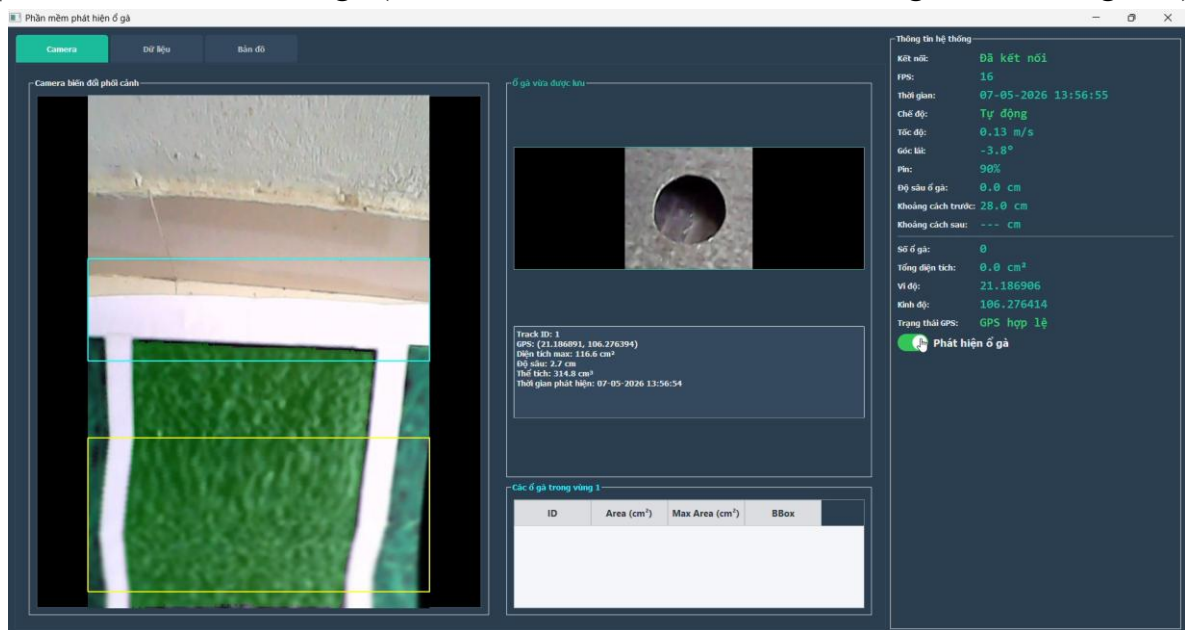
Tab Camera là giao diện chính khi vận hành, gồm ba phần chính:

- *Khu vực hiển thị ảnh BEV*: Ảnh BEV nhận từ xe qua UDP được hiển thị với hai vùng ROI (màu lục lam và vàng) tương ứng với vùng đo diện tích (ROI1) và vùng đo độ sâu (ROI2). Khi phát hiện ổ gà, bounding box và ID theo dõi được vẽ trực tiếp lên ảnh. Nếu bật chế độ phát hiện, các ROI sẽ hiện; nếu tắt, chúng được ẩn để tránh rối mắt.

- *Khu vực ổ gà vừa lưu*: Khi một ổ gà được ghi nhận đủ điều kiện (đã qua ROI2, ra khỏi ROI2, có ảnh crop), một ảnh crop của ổ gà (mở rộng 30px) được hiển thị cùng thông tin track ID, GPS, diện tích lớn nhất, độ sâu, thể tích và thời gian phát hiện. Nhấp chuột vào ảnh sẽ tự động chuyển sang tab Bản đồ và zoom vào vị trí của ổ gà.

- *Bảng thống kê vùng ROI1*: Hiển thị các ổ gà hiện đang nằm hoàn toàn trong vùng ROI1, với các cột: ID, diện tích hiện tại (cm^2), diện tích lớn nhất ghi nhận và tọa độ bounding box. Bảng được cập nhật mỗi khi nhận frame mới.

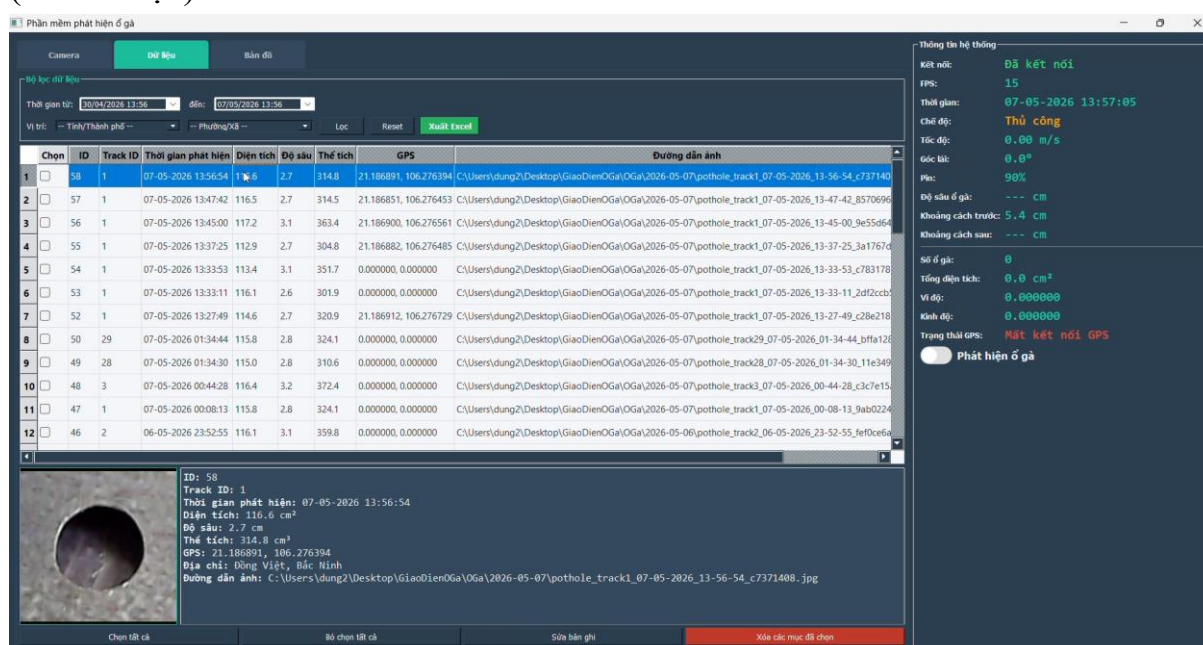
Ngoài ra, trên thanh bên phải là Metadata Panel hiển thị trạng thái kết nối, FPS, thời gian hệ thống, chế độ (auto/manual), tốc độ, góc lái, pin, độ sâu ổ gà, khoảng cách cảm biến trước/sau, tọa độ GPS, và số ổ gà hiện diện trong ROI1 cùng tổng diện tích. Nút gạt "Phát hiện ổ gà" cho phép bật/tắt toàn bộ quá trình phát hiện và theo dõi ổ gà (khi tắt, chỉ hiển thị ảnh thuần, không vẽ bounding box).



Hình 3.13. Giao diện video ở tab Camera

3.4.2. Giao diện dữ liệu

Tab Dữ liệu cho phép xem, lọc, sửa, xóa và xuất danh sách các ổ gà đã được lưu vào cơ sở dữ liệu SQLite. Giao diện gồm các thành phần:



3.4.3. Giao diện bản đồ

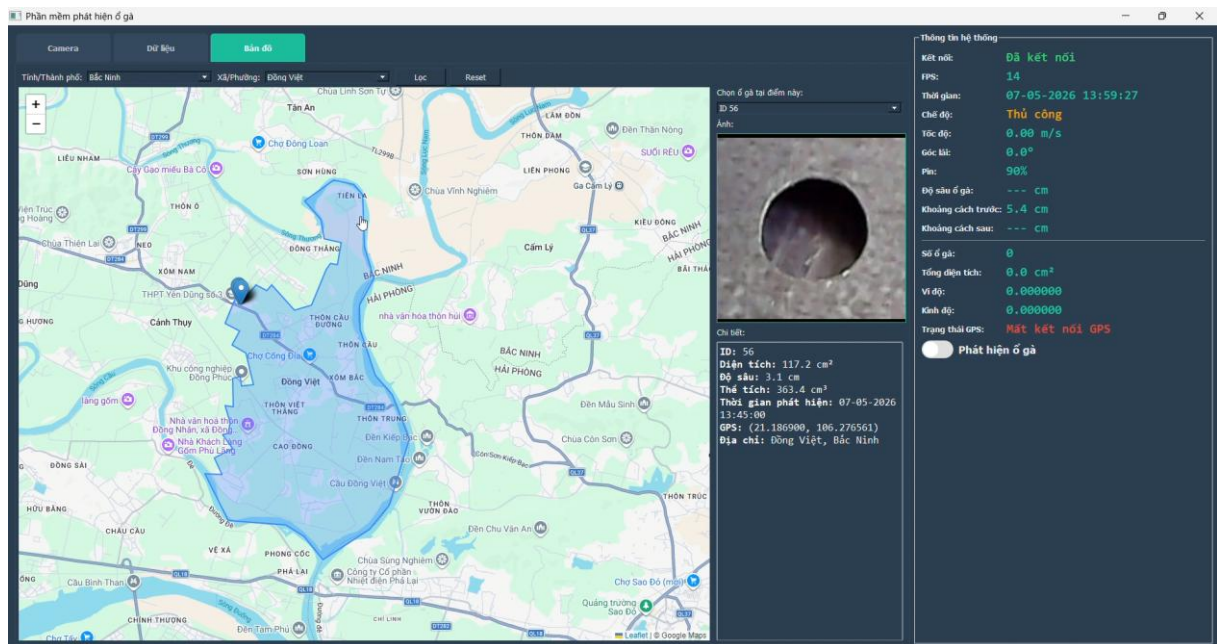
Tab Bản đồ sử dụng Leaflet (JavaScript) nhưng trong QWebEngineView, hiển thị bản đồ nền từ Google Maps, và cho phép tương tác hai chiều giữa Python và JavaScript qua QWebChannel.

- Hiển thị marker nhóm: Các vị trí ổ gà được lấy từ cơ sở dữ liệu (chỉ những bản ghi có GPS hợp lệ). Khi người dùng di chuyển hoặc zoom bản đồ, MapBridge gọi đến database để lấy các marker trong khung nhìn hiện tại, sau đó nhóm các marker có cùng tọa độ (sai số $< 1e-6$ độ) lại để hiển thị số lượng ổ gà tại điểm đó. Mỗi marker khi click sẽ gửi tín hiệu `select_pothole(id)` để hiển thị chi tiết.

- Lọc theo địa danh: Có hai combobox cho phép chọn tỉnh/thành phố và phường/xã. Khi chọn và nhấn *Lọc*, hệ thống lấy geometry của địa danh từ file GeoJSON, hiển thị vùng (polygon) lên bản đồ với màu xanh trong suốt, đồng thời chỉ hiển thị các marker nằm trong khu vực đó. Nút *Reset* trả về toàn bộ marker.

- Chi tiết ổ gà: Khi click marker, bên phải xuất hiện combo box liệt kê tất cả các ổ gà có cùng tọa độ (sai số $1e-6$ độ). Chọn từ combo sẽ hiển thị ảnh crop, diện tích, độ sâu, thể tích, thời gian, GPS và địa chỉ (nếu có). Nhấp vào ảnh sẽ zoom bản đồ đến điểm đó và hiển thị popup tạm thời.

- Đồng bộ từ các tab khác: Khi người dùng click ảnh có GPS hợp lệ ở tab Dữ liệu, chương trình sẽ chuyển sang tab Map và zoom đến vị trí ổ gà, hiển thị popup trong 8 giây, đồng thời tự động chọn bộ lọc địa danh phù hợp nếu có.

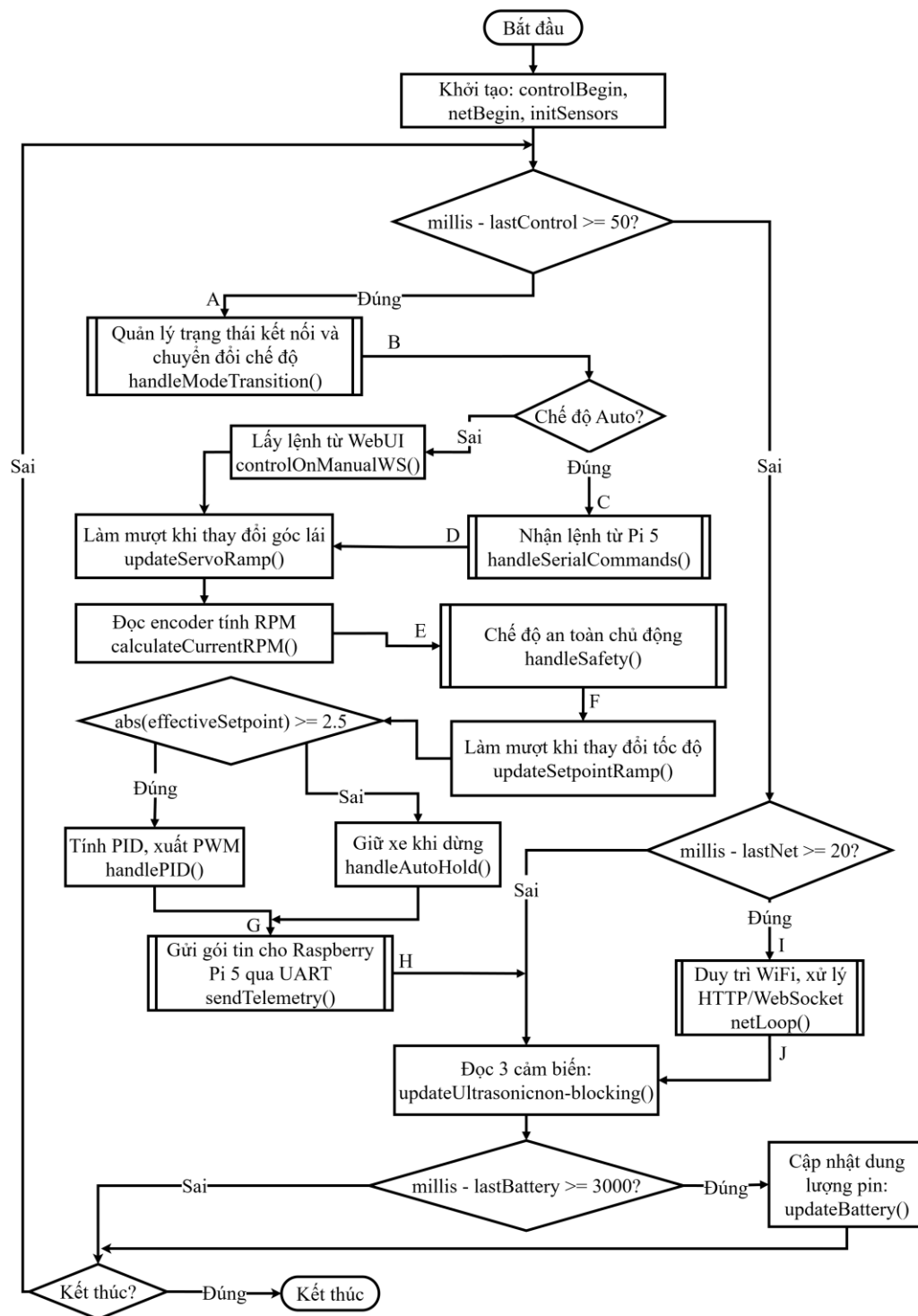


Hình 3.15. Giao diện tab Bản đồ

3.5. Lưu đồ thuật toán

3.5.1. Lưu đồ thuật toán khối xử lý trung tâm

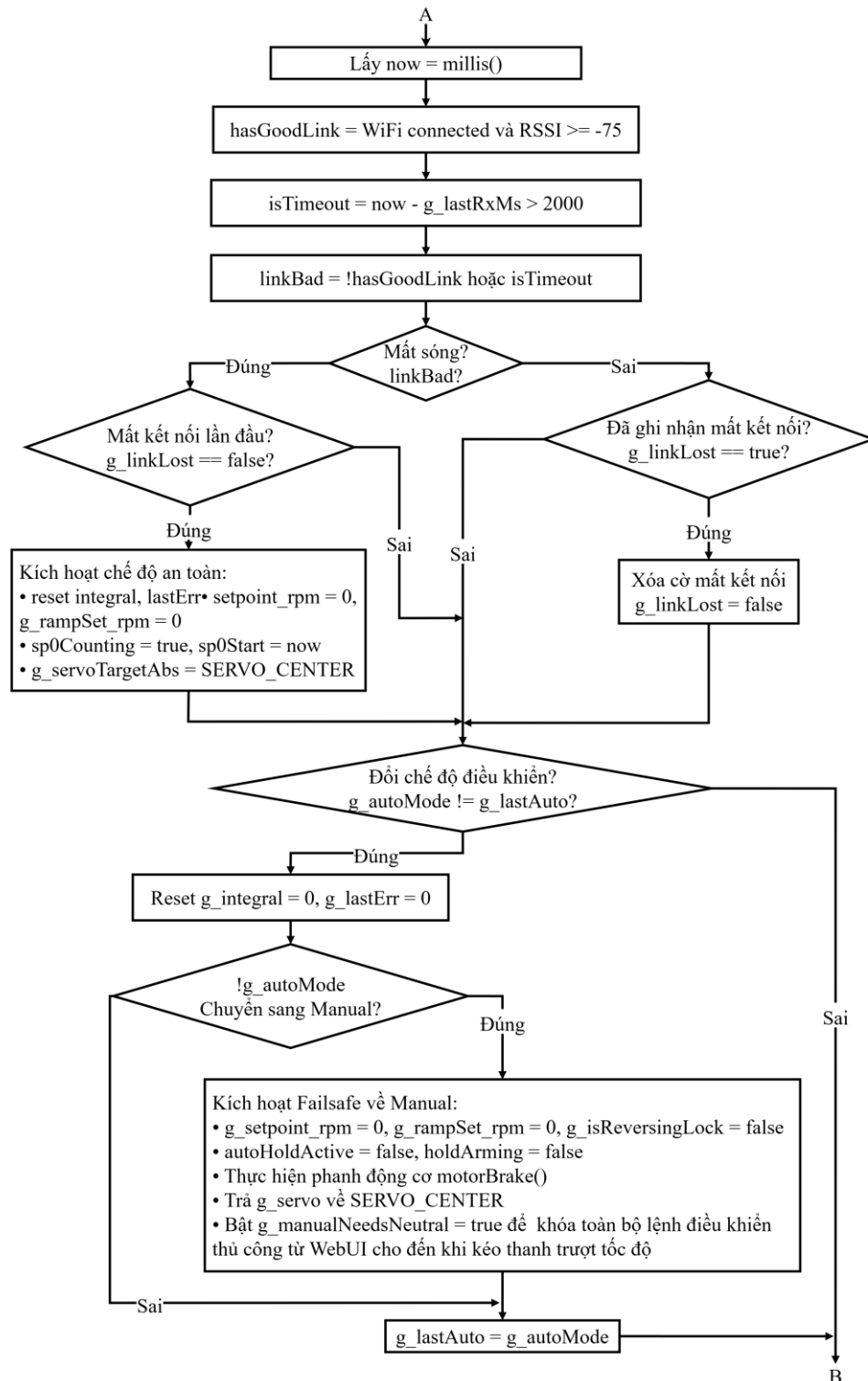
Lưu đồ thuật toán khối xử lý trung tâm (ESP32)



Hình 3.16. Lưu đồ thuật toán khối xử lý trung tâm

Lưu đồ này được cụ thể hoá bằng ngôn ngữ C++ trong file main.cpp ở Phụ lục 1.

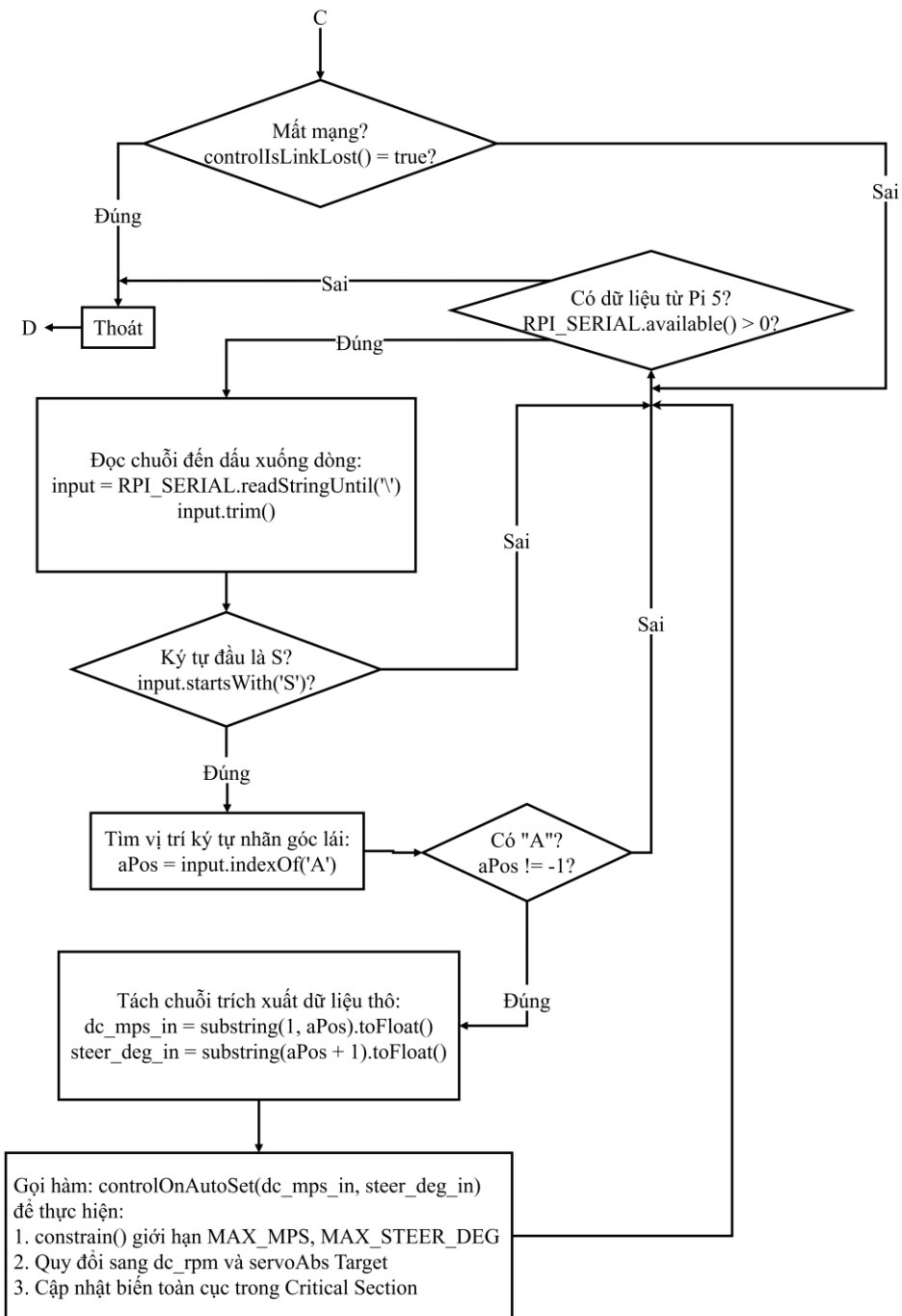
**Lưu đồ thuật toán module Quản lý trạng thái kết nối và chuyển đổi chế độ
handleModeTransition()**



Hình 3.17. Lưu đồ thuật toán module Quản lý trạng thái kết nối và chuyển đổi chế độ

Lưu đồ này được cụ thể hoá bằng ngôn ngữ C++ trong file Control.cpp ở Phụ lục 1.

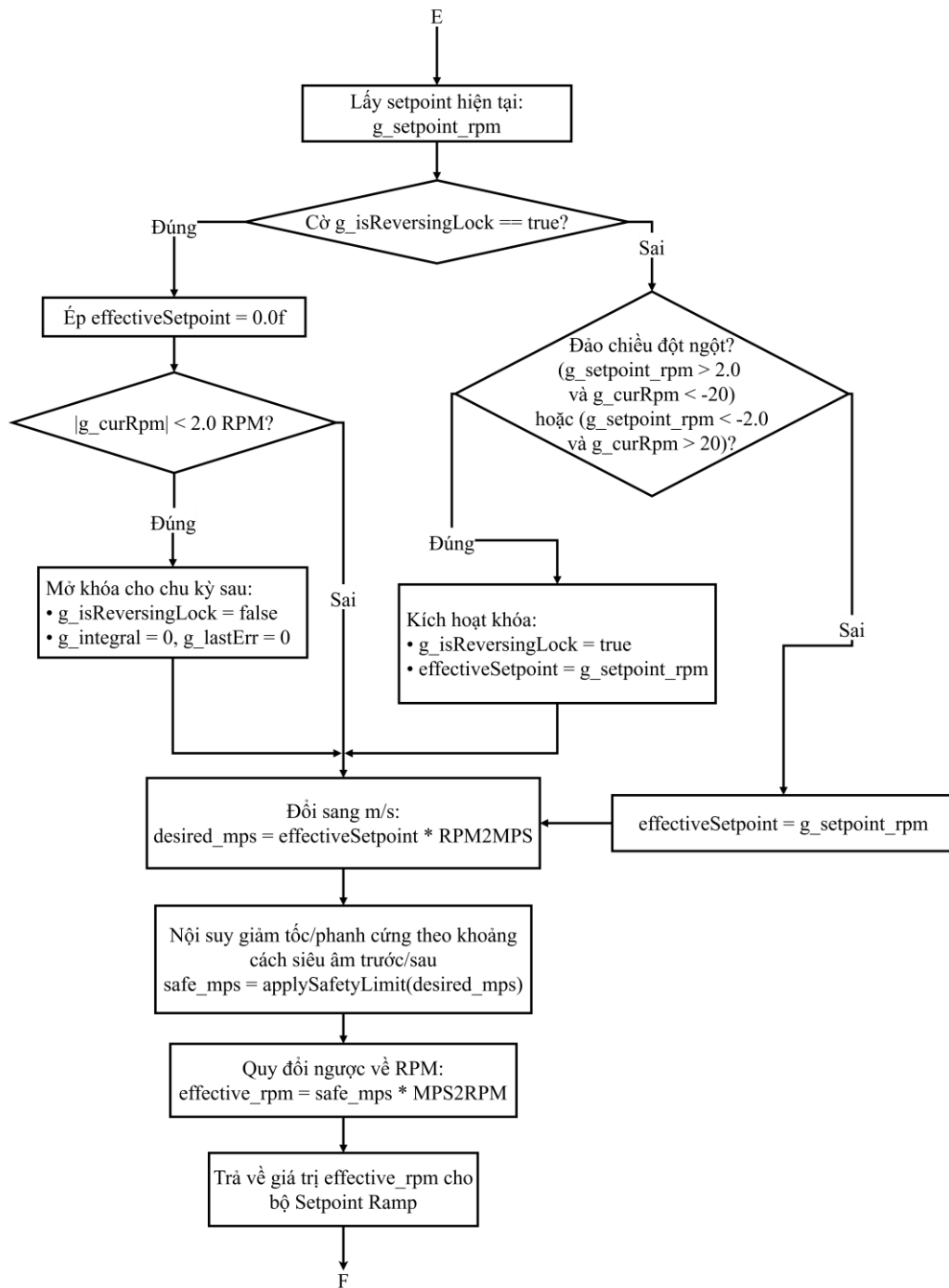
Lưu đồ thuật toán module Nhận lệnh từ Pi 5 handleSerialCommands()



Hình 3.18. Lưu đồ thuật toán module Nhân lệnh từ Pi 5

Lưu đồ này được cụ thể hoá bằng ngôn ngữ C++ trong file Control Parser.cpp ở Phụ lục 1.

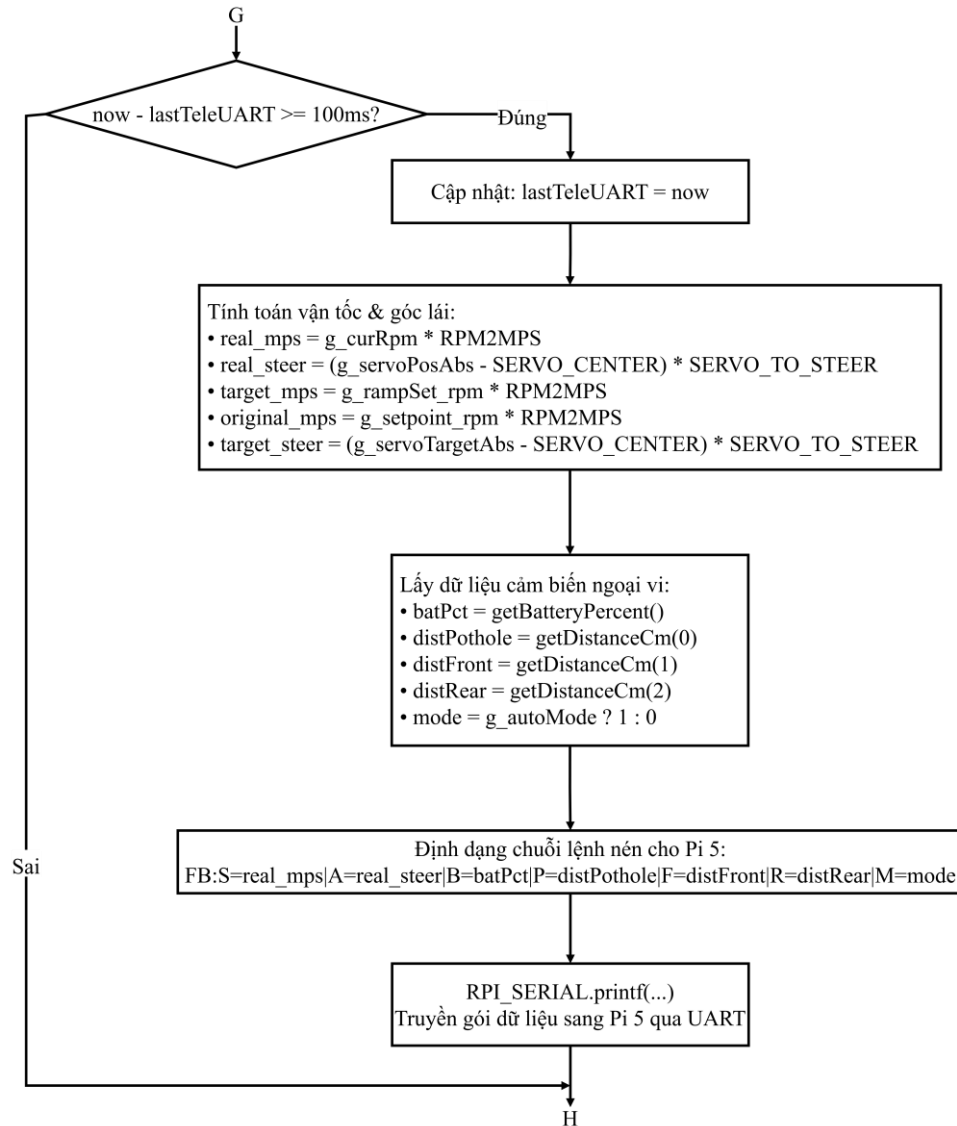
**Lưu đồ thuật toán module Chế độ an toàn chủ động
handleSafety()**



Hình 3.19. Lưu đồ thuật toán module Chế độ an toàn chủ động

Lưu đồ này được cụ thể hoá bằng ngôn ngữ C++ trong file Control_Safety.cpp ở Phụ lục 1.

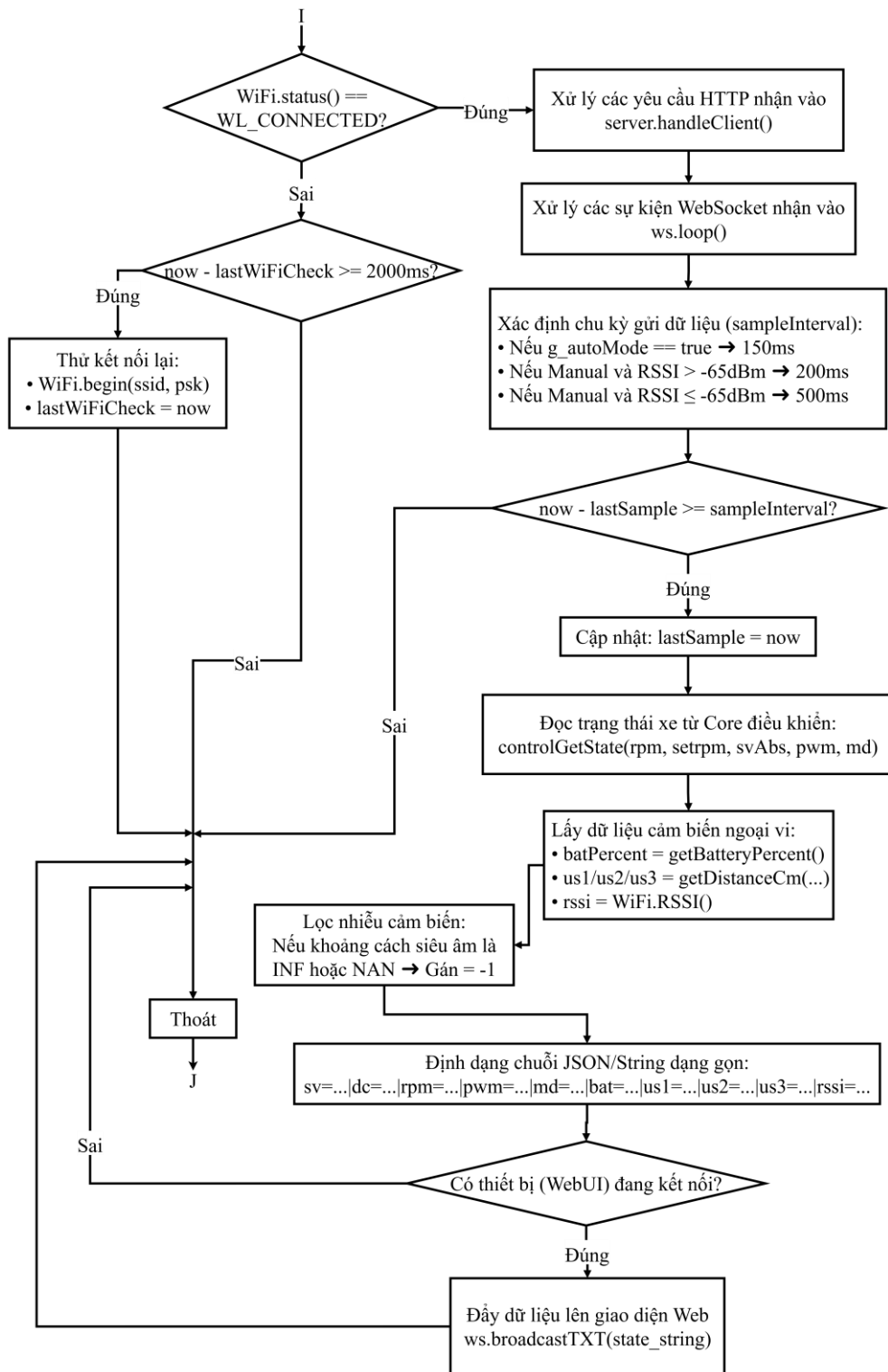
**Lưu đồ thuật toán module Gửi gói tin cho Raspberry Pi 5 qua UART
sendTelemetry()**



Hình 3.20. Lưu đồ thuật toán module Gửi gói tin cho Raspberry Pi 5 qua UART

Lưu đồ này được cụ thể hoá bằng ngôn ngữ C++ trong file Control.cpp ở Phụ lục 1.

Lưu đồ thuật toán module Duy trì WiFi, xử lý HTTP/WebSocket netLoop()

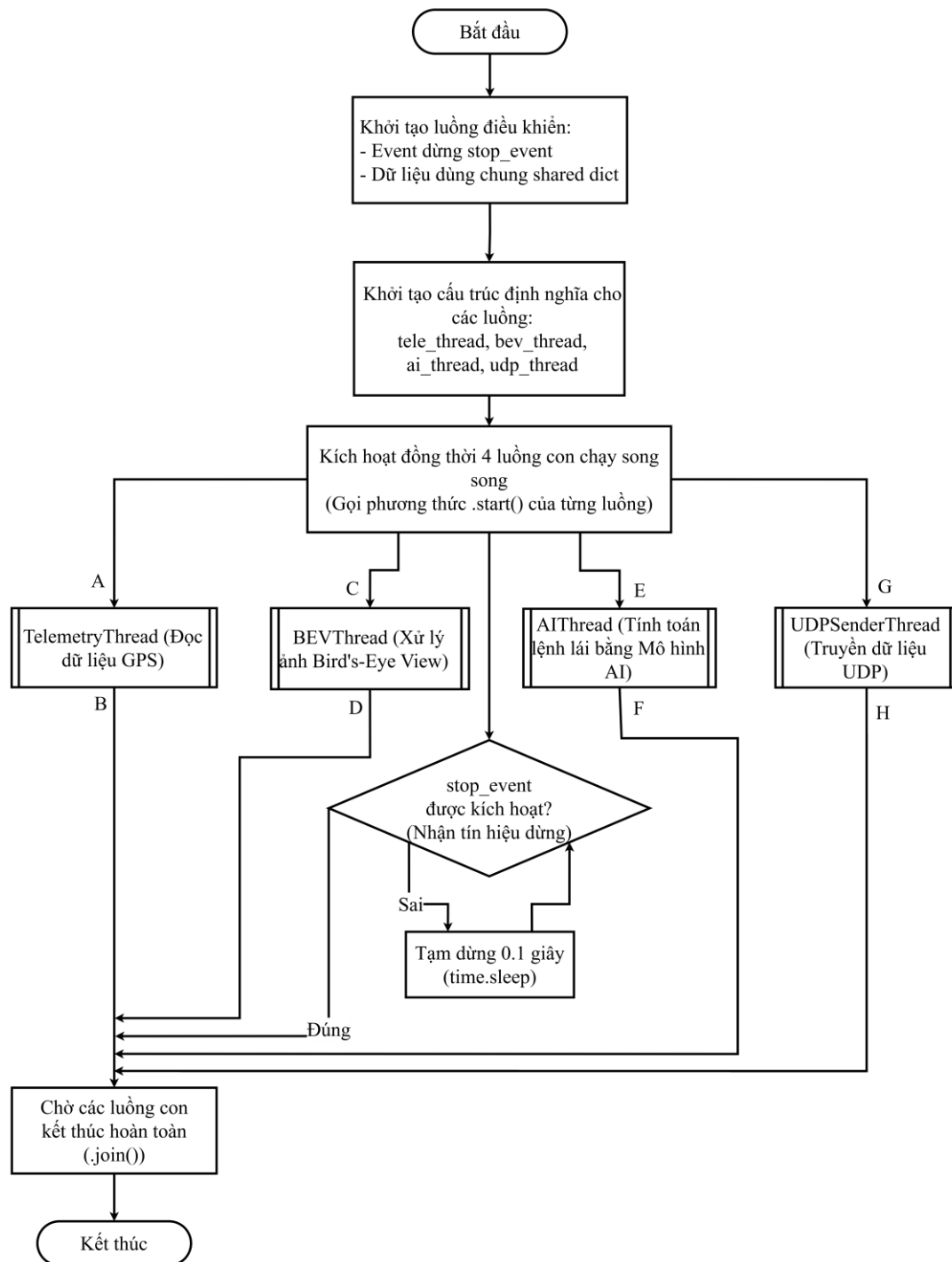


Hình 3.21. Lưu đồ thuật toán module Duy trì WiFi, xử lý HTTP/WebSocket

Lưu đồ này được cụ thể hoá bằng ngôn ngữ C++ trong file `Net.cpp` ở Phụ lục 1.

3.5.2. Lưu đồ thuật toán khối xử lý giữ làn đường và truyền thông

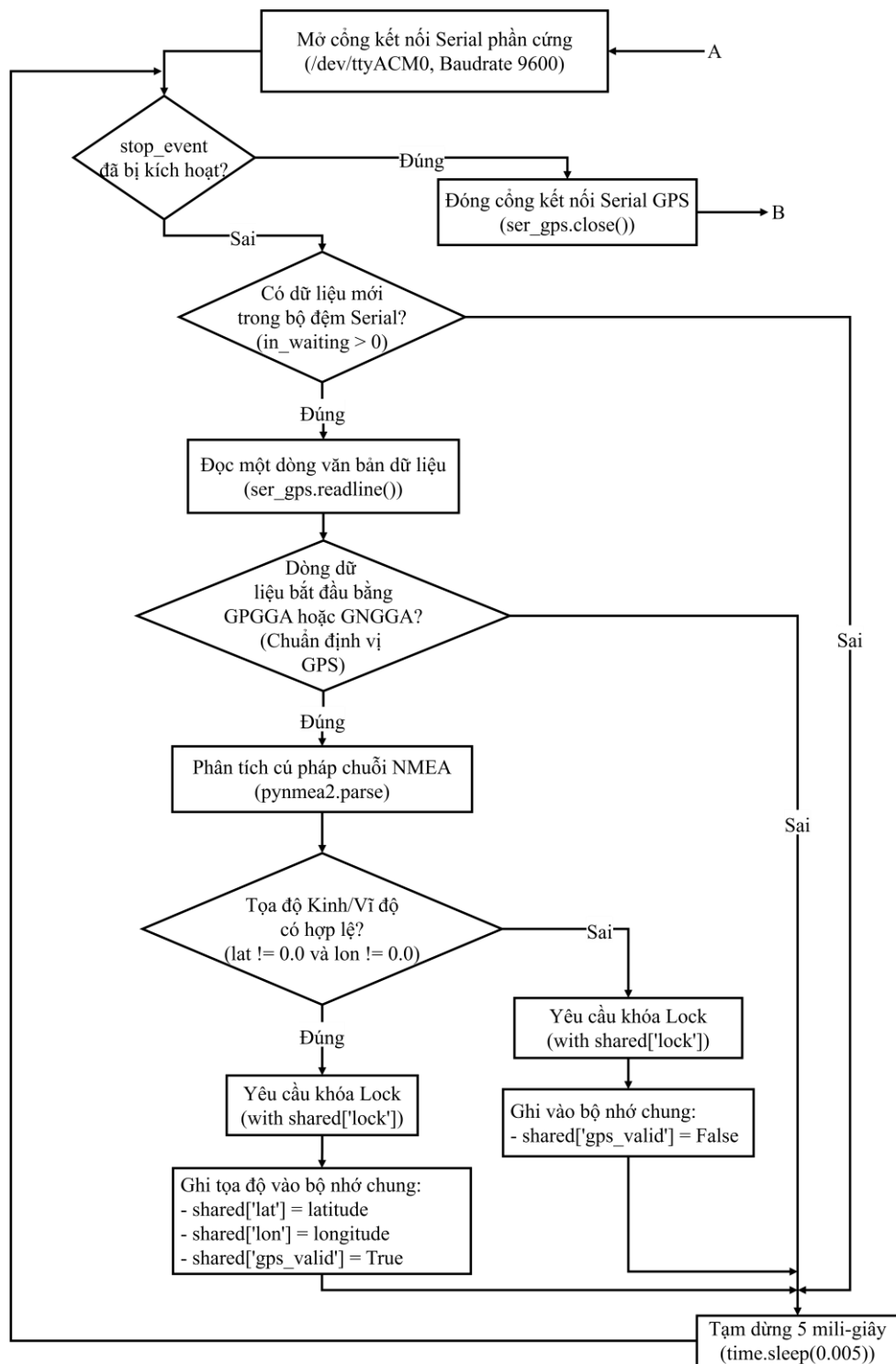
Lưu đồ thuật toán khối xử lý giữ làn đường và truyền thông (Raspberry Pi 5)



Hình 3.22. Lưu đồ thuật toán khối xử lý giữ làn đường và truyền thông

Lưu đồ này được cụ thể hoá bằng ngôn ngữ Python trong file main.py ở Phụ lục 2.

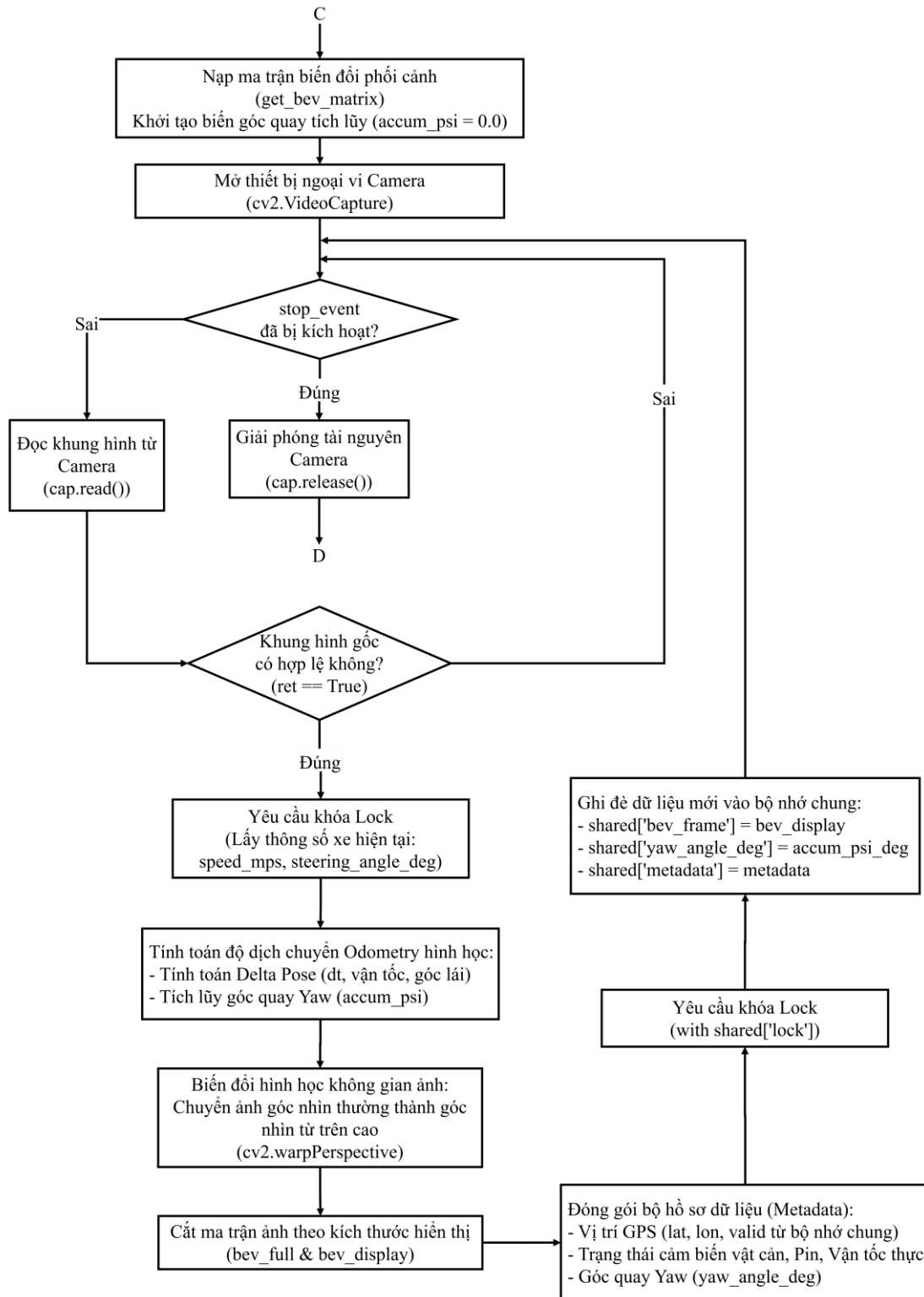
Lưu đồ thuật toán module TelemetryThread (Đọc dữ liệu GPS)



Hình 3.23. Lưu đồ thuật toán module *TelemetryThread*

Lưu đồ này được cụ thể hoá bằng ngôn ngữ Python trong file `telemetry.py` ở Phụ lục 2.

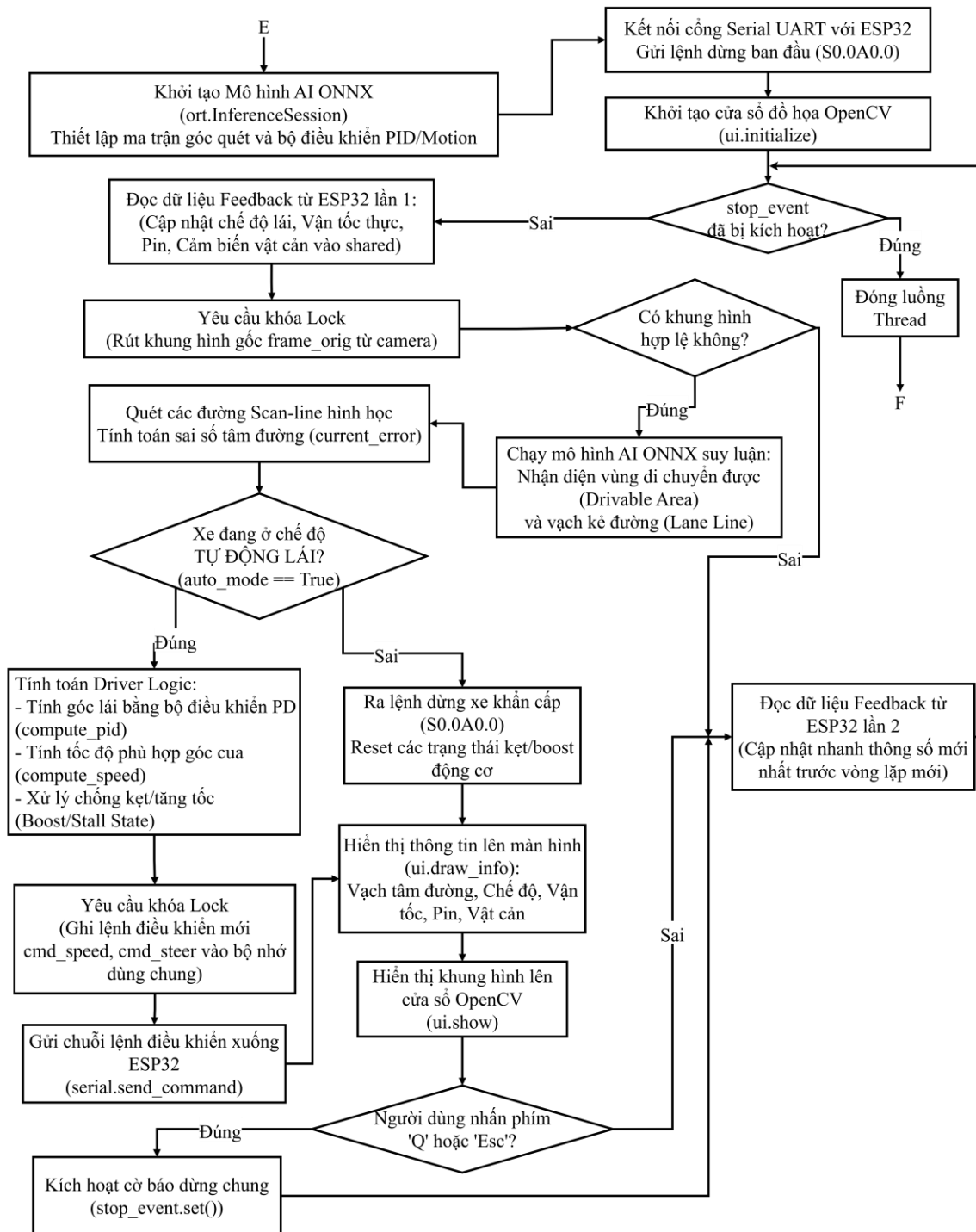
Lưu đồ thuật toán module BEVThread (Xử lý ảnh Bird's-Eye View)



Hình 3.24. Lưu đồ thuật toán module BEVThread

Lưu đồ này được cụ thể hoá bằng ngôn ngữ Python trong file `bev_thread.py` ở Phụ lục 2.

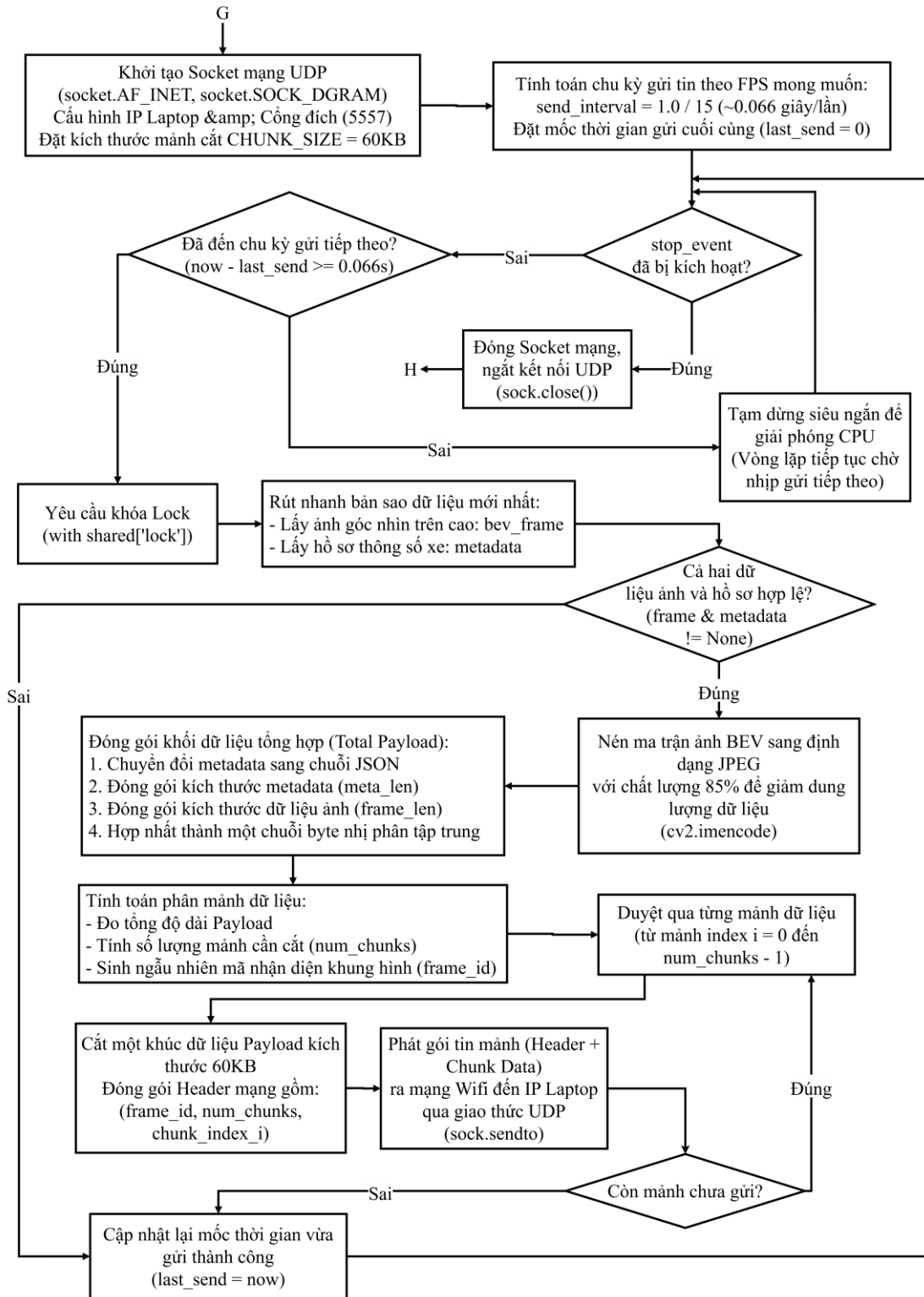
Lưu đồ thuật toán module AIThread (Tính toán lệnh lái bằng Mô hình AI)



Hình 3.25. Lưu đồ thuật toán module AIThread

Lưu đồ này được cụ thể hoá bằng ngôn ngữ Python trong file ai_thread.py ở Phụ lục 2.

Lưu đồ thuật toán module UDPSenderThread (Truyền dữ liệu UDP)

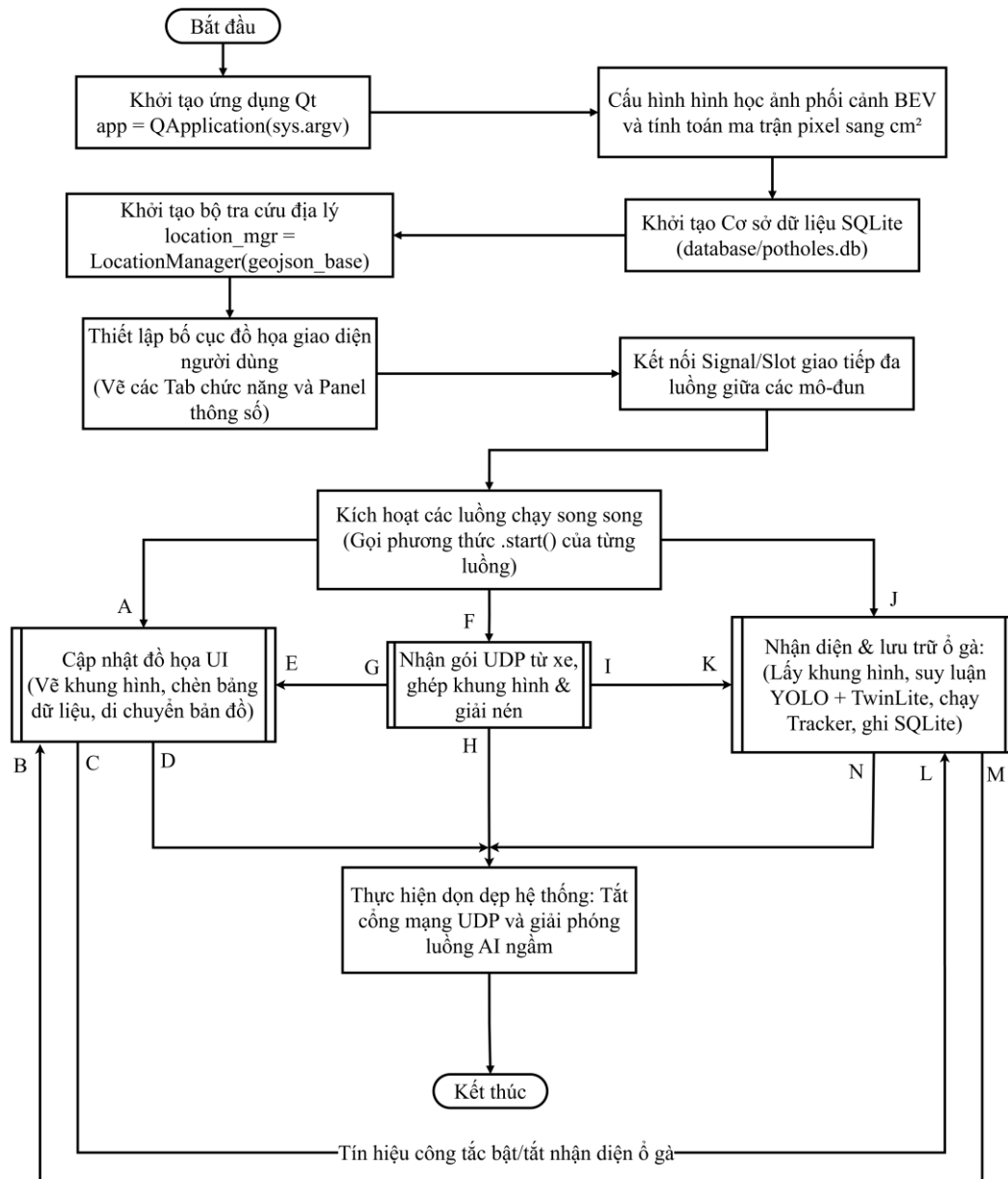


Hình 3.26. Lưu đồ thuật toán module UDPSenderThread

Lưu đồ này được cụ thể hoá bằng ngôn ngữ Python trong file udp_sender.py ở Phụ lục 2.

3.5.3. Lưu đồ thuật toán khối nhận diện ô gà và lưu trữ

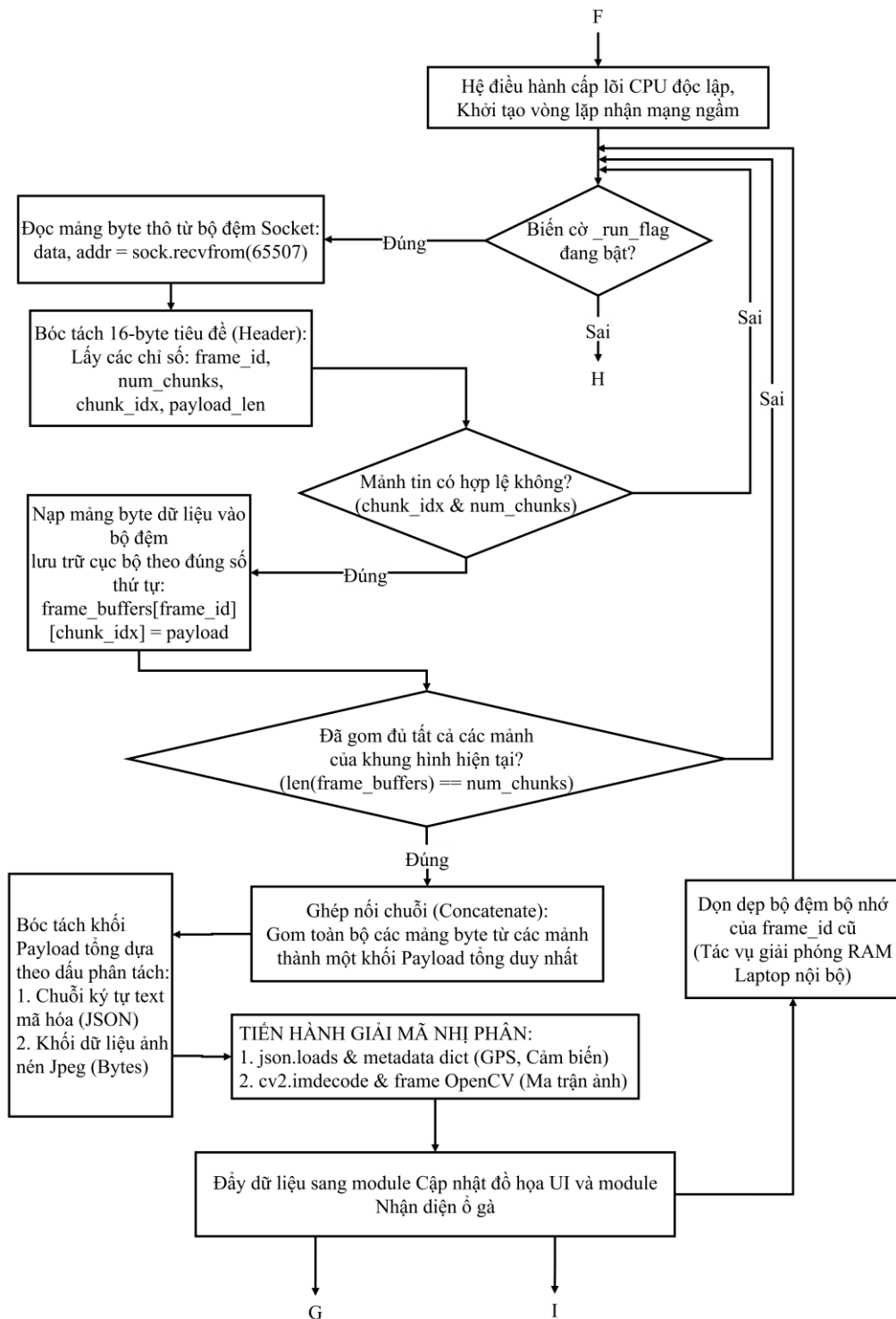
Lưu đồ thuật toán khối Nhận diện ô gà và lưu trữ (PC)



Hình 3.27. Lưu đồ thuật toán khối Nhận diện ô gà và lưu trữ

Lưu đồ này được cụ thể hoá bằng ngôn ngữ Python trong file main.py ở Phụ lục 3.

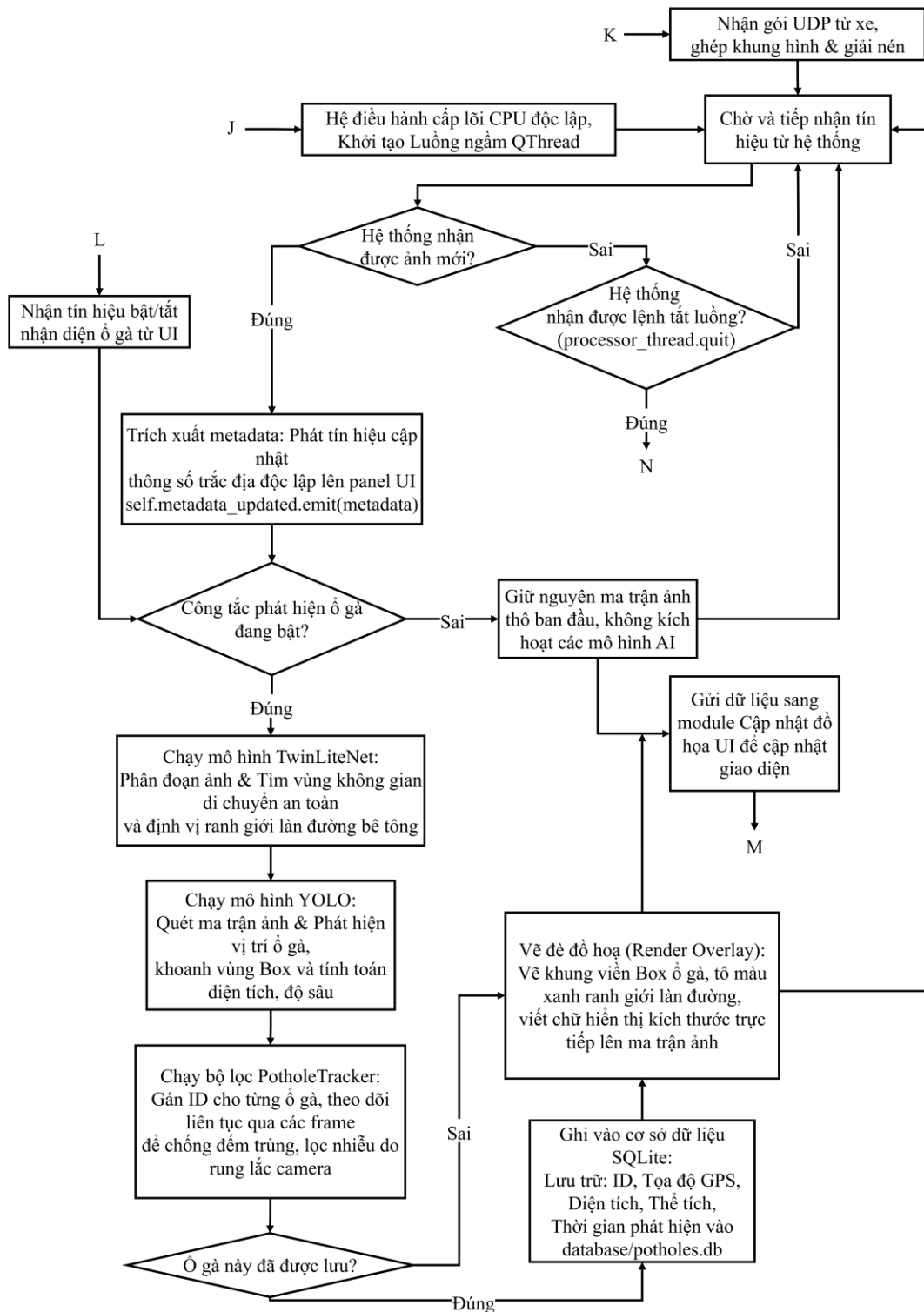
Lưu đồ thuật toán module Nhận gói UDP từ xe, ghép khung hình & giải nén



Hình 3.28. Lưu đồ thuật toán module Nhận gói UDP từ xe, ghép khung hình & giải nén

Lưu đồ này được cụ thể hoá bằng ngôn ngữ Python trong file udp_thread.py ở Phụ lục 3.

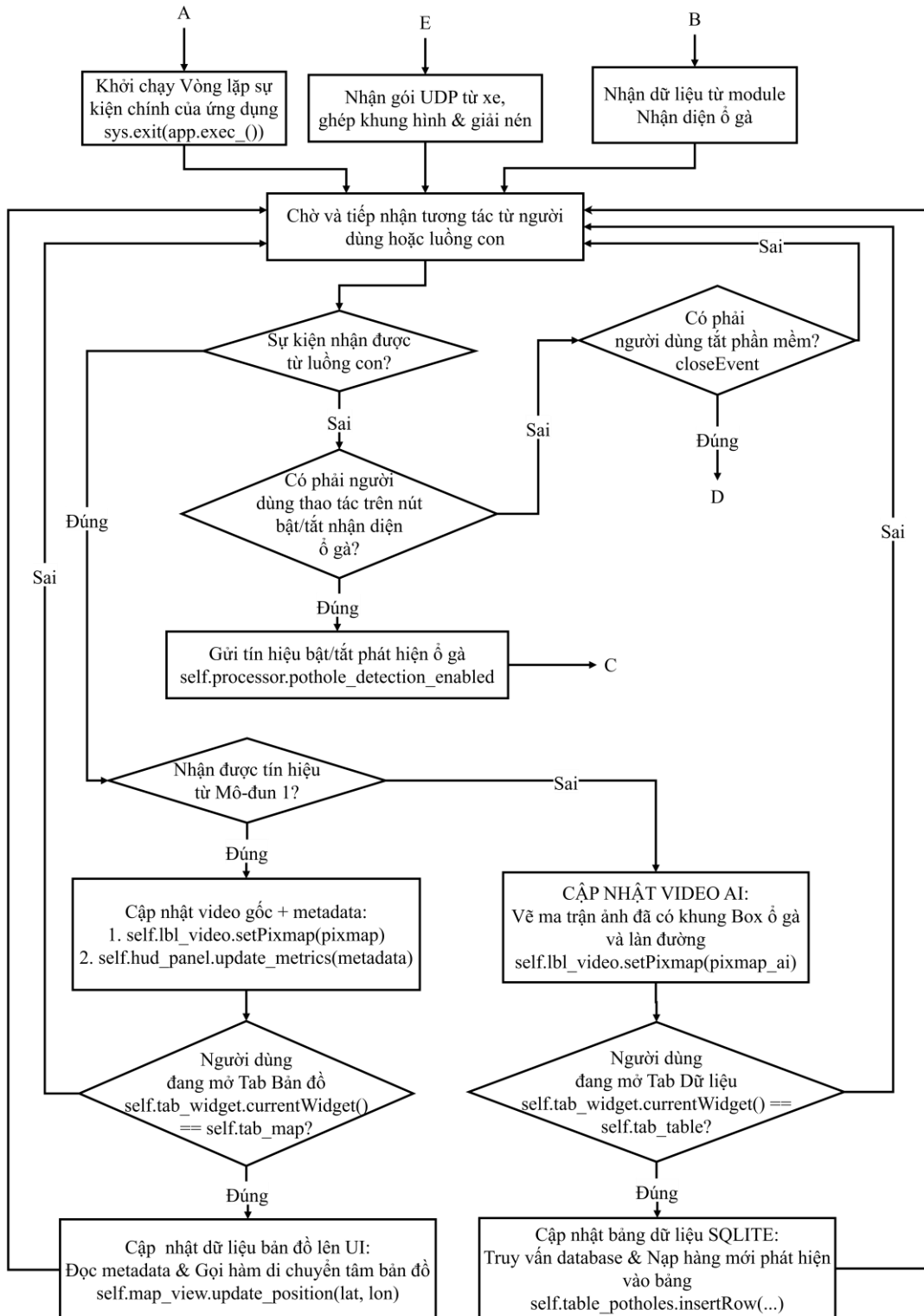
Lưu đồ thuật toán module Nhận diện & lưu trữ ổ gà



Hình 3.29. Lưu đồ thuật toán module Nhận diện & lưu trữ ổ gà

Lưu đồ này được cụ thể hoá bằng ngôn ngữ Python trong file `processor_thread.py` ở Phụ lục 3.

Lưu đồ thuật toán module Cập nhật đồ họa UI



Hình 3.30. Lưu đồ thuật toán module Cập nhật đồ họa UI

Lưu đồ này được cụ thể hoá bằng ngôn ngữ Python trong file `main_window.py` ở Phụ lục 3.

Kết luận chương 3

Chương 3 đã tập trung xây dựng toàn bộ các thuật toán điều khiển và giải pháp giám sát cho hệ thống xe tự hành. Các nội dung chính đạt được bao gồm:

- Điều khiển động cơ và an toàn: Hệ thống được trang bị bộ điều khiển PID tốc độ với tham số đã được tinh chỉnh ($K_p = 1,86$; $K_i = 4,5$; $K_d = 0,046$) giúp động cơ bám tốc độ ổn định, không dao động. Đồng thời, các cơ chế an toàn đa tầng (xử lý va chạm, mất kết nối, chống đảo chiều đột ngột, auto-hold và boost khi kẹt) được triển khai trực tiếp trên ESP32, đảm bảo phản ứng tức thời.

- Xử lý ảnh và nhận diện hư hỏng: Mô hình TwinLiteNet siêu nhẹ chạy trên Raspberry Pi 5 cho phép xe tự bám làn với độ chính xác cao (mIoU vùng đường >99%, vạch làn ~96,5%). Mô hình YOLOv11-seg (phiên bản nano) được huấn luyện trên 98 ảnh thu thập từ sa hình, đạt kết quả vượt trội (mAP50 = 0,995, recall = 100%) cho phát hiện và phân đoạn ổ gà. Thuật toán tính kích thước ổ gà (diện tích, độ sâu tối đa, thể tích) được xây dựng dựa trên ảnh BEV và chỉ một cảm biến độ sâu, tận dụng cơ chế lấy giá trị lớn nhất để bù đắp hạn chế của cảm biến.

- Giải pháp truyền thông: Ba kênh truyền thông được thiết lập đồng bộ: UART (Raspberry Pi 5 – ESP32) với cấu trúc lệnh S...A... và phản hồi FB:...; UDP (Raspberry Pi 5 – PC) có phân mảnh để truyền ảnh BEV và metadata thời gian thực; WebSocket (ESP32 – thiết bị điều khiển từ xa) cho phép người dùng điều khiển và giám sát trạng thái xe qua giao diện web.

- Giao diện giám sát: Phần mềm trên PC được xây dựng bằng PyQt5, gồm ba tab (Camera, Dữ liệu, Bản đồ) và các module xử lý độc lập (nhận UDP, phát hiện ổ gà, quản lý SQLite, tương tác bản đồ Leaflet). Lưu đồ tổng thể và các lưu đồ con đã thể hiện rõ luồng xử lý và sự phối hợp giữa các module.

Tóm lại, Chương 3 đã hoàn thiện phần lý thuyết và thiết kế thuật toán cho hệ thống, tạo cơ sở vững chắc để chuyển sang giai đoạn thử nghiệm mô hình và đánh giá kết quả trong Chương 4.

CHƯƠNG 4. THỬ NGHIỆM MÔ HÌNH, ĐÁNH GIÁ KẾT QUẢ VÀ HƯỚNG PHÁT TRIỂN

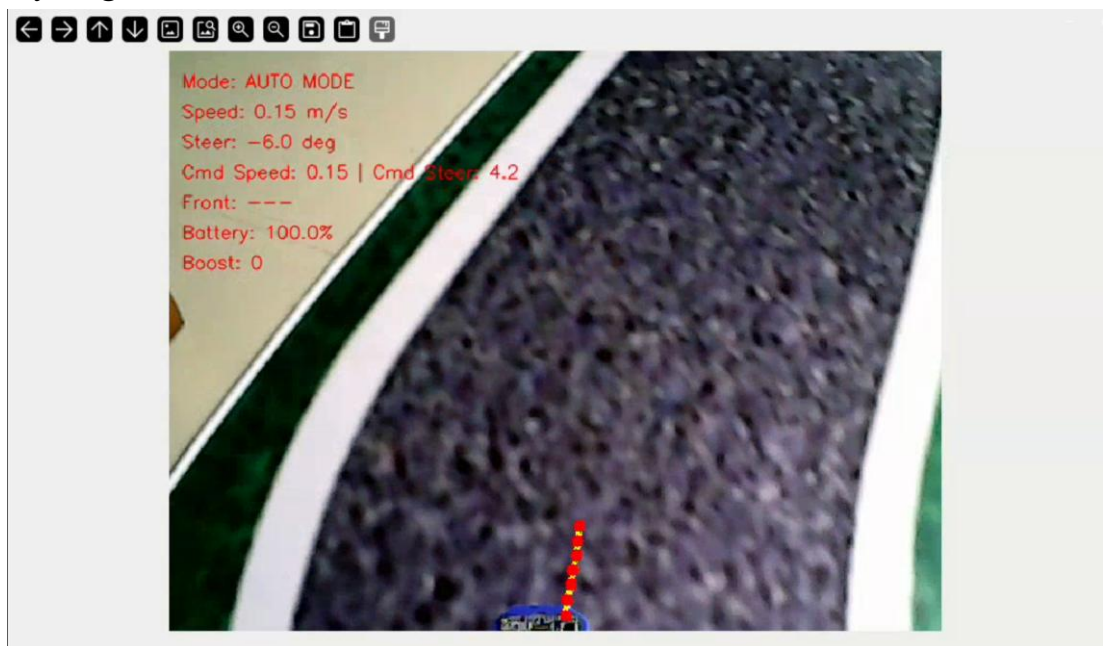
4.1. Thử nghiệm mô hình

4.1.1. Thử nghiệm bám làn đường

a. Thử nghiệm bám làn đường trên sa hình mô phỏng

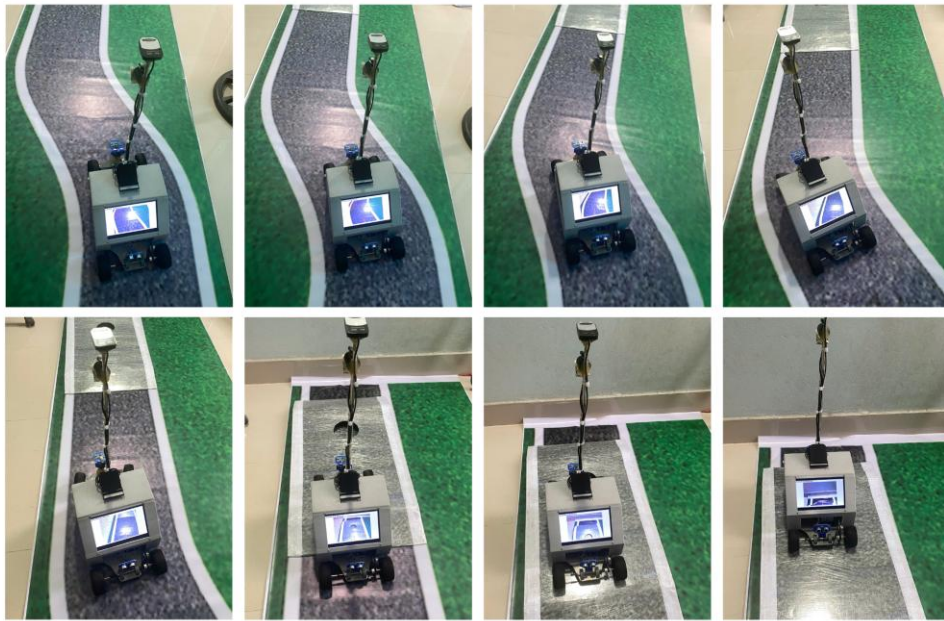
Xe được đặt ở chế độ tự động, chạy trên sa hình dài 5 m, bao gồm cả đường thẳng và đường cong. Mô hình TwinLiteNet trên Raspberry Pi 5 xử lý ảnh từ camera resize về kích thước 240×426 , đưa ra lệnh góc lái gửi xuống ESP32 qua UART. Quan sát và ghi nhận:

- Trên đoạn thẳng: Xe bám làn ổn định, không chạm vạch hai bên dù. Sai số lệch tâm ước lượng khoảng 2–4 cm.
- Trên đoạn cong: Robot vẫn bám làn thành công, góc lái tự động điều chỉnh để bám theo khúc cua. Tốc độ được giảm theo công thức (3.30) để đảm bảo an toàn.
- Thời gian đáp ứng: Khi xe lệch khỏi tâm làn, phản ứng điều khiển góc lái xảy ra gần như tức thời.



Hình 4.1. Giao diện camera trên xe đang chạy khi ở chế độ tự động

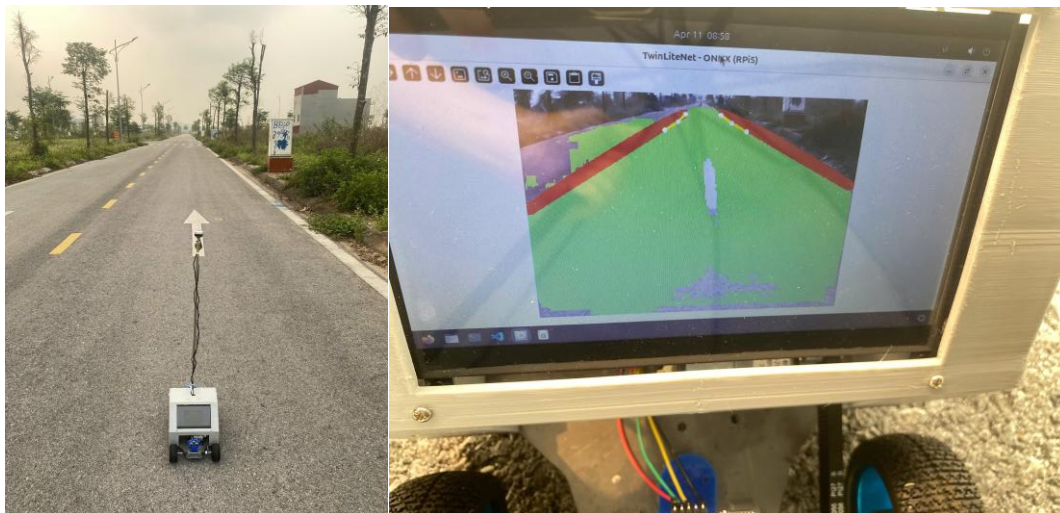
Quá trình xe di chuyển trên đường sa hình được ghi nhận qua các thời điểm thể hiện cách hoạt động của thuật toán giữ làn đường được minh họa trong Hình 4.2.



Hình 4.2. Quá trình xe di chuyển trên đường sa hình

b. Thử nghiệm nhận diện làn đường trên đường thực tế

Để đánh giá nâng cao năng lực của mô hình thị giác máy tính dưới tác động của môi trường thực tế, hệ thống được đưa vào vận hành thử nghiệm tại một tuyến đường nhựa nội bộ có vạch kẻ phân làn rõ ràng và không gian thông thoáng dưới điều kiện ánh sáng tự nhiên.



Hình 4.3. Thử nghiệm nhận diện làn đường và vùng đường ngoài thực tế

Do thử nghiệm ngoài thực địa này chỉ tập trung vào việc kiểm chứng khả năng nhận diện làn đường của giải thuật, hệ thống ghi nhận luồng dữ liệu phân đoạn (segmentation) hiển thị trực quan trên màn hình LCD nhúng cho thấy: Mô hình TwinLiteNet hoạt động ở mức độ tương đối ổn định, phân tách và bao phủ được vùng đường đi được (vùng màu xanh) cũng như bám theo vạch kẻ làn giới hạn (vùng màu đỏ). Dưới tác động của cường độ ánh sáng tự nhiên ngoài trời, mô hình vẫn duy trì khả năng nhận diện, các điểm bám vạch kẻ đường hỗ trợ tốt cho

việc định hình hướng di chuyển, ít bị nhầm lẫn bởi kết cấu nhám đặc trưng của bề mặt đường nhựa hay các vật bóng đổ nhẹ từ môi trường xung quanh. Tuy nhiên, do môi trường thực tế có biên độ thay đổi ánh sáng lớn và xuất hiện nhiều yếu tố nhiễu ngoại cảnh phức tạp hơn so hình phòng thí nghiệm, hệ thống vẫn cần phải tiếp tục cải tiến và tối ưu hóa thêm về cả thuật toán lẫn phần cứng thu nhận ảnh trong tương lai để đạt được độ chính xác và tin cậy cao hơn.

4.1.2. Thử nghiệm phát hiện và đo kích thước ổ gà

a. Chuẩn bị và đo thủ công

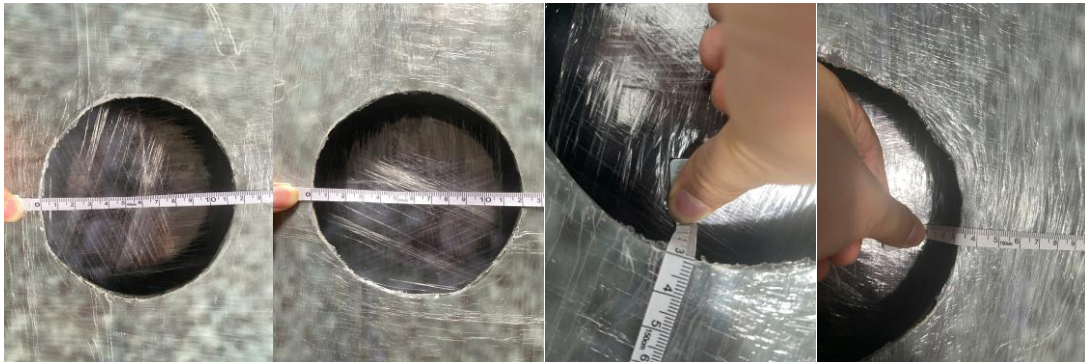
Một ổ gà mô phỏng có dạng hình tròn được đặt cố định trên làn đường, nằm trong sa hình. Đo thủ công để có giá trị tham chiếu:

- Đường kính: $D = 11,8 \text{ cm}$.
- Diện tích tham chiếu (hình tròn):

$$S_{tc} = \pi \times \left(\frac{D}{2}\right)^2 = \pi \times (5,9)^2 \approx 109,36 \text{ cm}^2.$$

- Độ sâu: $d_{tc} = 2.9 \text{ cm}$.
- Thể tích tương đối của ổ gà:

$$V_{tc} = S_{tc} * d_{tc} = 109,36 \times 2.9 = 317,14 \text{ (cm}^3\text{)}$$



Hình 4.4. Ảnh chụp quá trình đo thủ công ổ gà bằng thước dây

b. Quy trình đo bằng hệ thống

Xe chạy qua ổ gà ở chế độ tự động. Ảnh BEV và độ sâu từ cảm biến siêu âm được gửi về PC qua UDP. Trên PC, YOLOv11-seg phát hiện, phân đoạn và tracking ổ gà. Quá trình lặp lại 10 lần. Mỗi lần, PC ghi nhận:

- Diện tích lớn nhất S_{auto} (khi ổ gà trong ROI1).
- Độ sâu lớn nhất d_{auto} (từ metadata, được ESP32 lấy giá trị max khi tâm track qua ROI2).

Bảng 4.1. Kết quả đo kích thước ổ gà bằng hệ thống sau 10 lần đo

Lần đo	Diện tích (cm ²)	Độ sâu (cm)	Thể tích (cm ³)
1	122.31	2.7	330.237
2	125.0775	2.8	350.217

3	123.4575	2.7	333.3353
4	122.3775	2.7	330.4193
5	119.1825	3.1	369.4658
6	121.68	2.8	340.704
7	120.105	1.7	204.1785
8	120.8475	1.9	229.6103
9	121.6575	2.7	328.4753
10	121.41	2.3	279.243

Trung bình: $\bar{S}_{auto} = 121,81 \text{ cm}^2$, $\bar{d}_{auto} = 2,54 \text{ cm}$, $\bar{V}_{auto} = 309,59 \text{ cm}^3$.

c. So sánh và đánh giá sai số

So sánh giá trị trung bình đo bằng hệ thống với giá trị tham chiếu đo thủ công:

- *Diện tích*: $\bar{S}_{auto} = 121,81 \text{ cm}^2$ so với $S_{tc} = 109,36 \text{ cm}^2$.

Sai số tuyệt đối: $\Delta S = +12,45 \text{ cm}^2 (\approx +11,4\%)$.

- *Độ sâu*: $\bar{d}_{auto} = 2,54 \text{ cm}$ so với $d_{tc} = 2,9 \text{ cm}$.

Sai số tuyệt đối: $\Delta d = -0,36 \text{ cm} (\approx -12,4\%)$.

- *Thể tích*: $\bar{V}_{auto} = 309,59 \text{ cm}^3$ so với $V_{tc} = 317,14 \text{ cm}^3$.

Sai số tuyệt đối: $\Delta V = -7,55 \text{ cm}^3 (\approx -2,4\%)$.

Phân tích sai số:

Diện tích xe đo lớn hơn thực tế khoảng 11%. Nguyên nhân chính do xe di chuyển và camera có tốc độ chụp (FPS) thấp, gây hiện tượng kéo dài ảnh (motion blur), mask phân đoạn bị kéo giãn theo hướng chuyển động. Ngoài ra, ánh sáng tạo bóng và biến dạng phối cảnh nhẹ cũng góp phần.

Độ sâu xe đo nhỏ hơn thực tế khoảng 12%. Có hai nguyên nhân chính:

- Tấm nhựa giả lập ổ gà bị lún dưới trọng lượng xe khi bánh xe đi qua, làm giảm độ sâu tạm thời so với trạng thái tĩnh.

- Các lần đo thứ 7 và 8 có độ sâu rất thấp (1,7 và 1,9 cm) do xe bị kích hoạt chế độ boost (tăng tốc vượt dốc) khi phát hiện xe bị kẹt nhẹ. Khi boost, tốc độ đột ngột tăng lên 0,7 m/s trong 1 giây, khiến cảm biến siêu âm không kịp đo hoặc đo sai lệch do thay đổi góc chiếu. Hai lần đo này làm giảm đáng kể giá trị trung bình độ sâu. Nếu loại bỏ lần 7 và 8, độ sâu trung bình của 8 lần còn lại là $\approx 2,68 \text{ cm}$, sai số chỉ còn $-0,22 \text{ cm} (\approx -7,6\%)$.

- Thể tích sai số nhỏ (chỉ -2,4%) do sự bù trừ giữa diện tích lớn hơn và độ sâu nhỏ hơn, tạo ra sản phẩm tương đối chính xác. Khi loại bỏ hai lần đo nhiễu, thể tích trung bình 8 lần là $\approx 322,6 \text{ cm}^3$, sai số +1,7%.

4.1.3. Thử nghiệm truyền thông

Các thử nghiệm truyền thông được thực hiện trong phòng có kích thước khoảng 20 m², sử dụng mạng WiFi nội bộ (kết nối giữa Raspberry Pi 5 trên xe và PC, cũng như giữa ESP32 với điện thoại).

- *Truyền video UDP từ Raspberry Pi 5 lên PC*: Trong phạm vi phòng thí nghiệm, kết nối WiFi ổn định, không xảy ra hiện tượng mất gói. Tốc độ khung hình nhận được trên PC đạt 13–15 fps, đáp ứng yêu cầu hiển thị và xử lý ảnh thời gian thực của mô hình YOLOv11-seg.

- *Điều khiển xe qua WebSocket (ESP32 – điện thoại)*: Ở khoảng cách 10 m (trong không gian trong nhà), tín hiệu WiFi vẫn duy trì kết nối. Tuy nhiên, tốc độ khung hình video truyền về giảm xuống còn khoảng 8 fps do suy hao tín hiệu và giảm băng thông. Ở cự ly này, các lệnh điều khiển vẫn được thực thi nhưng có độ trễ nhẹ.

4.2. Đánh giá kết quả

4.2.1. Kết quả đạt được

Về điều khiển và bám làn: Robot tự hành bám làn ổn định trên cả đoạn thẳng và đoạn cong của sa hình (rộng 48 cm, dài 5 m). Sai số lệch tâm trung bình khoảng 2–4 cm, đáp ứng yêu cầu bám làn trong không gian hẹp. Cơ chế giảm tốc khi vào cua hoạt động đúng thiết kế.

Về phát hiện và đo kích thước ổ gà: Mô hình YOLOv11-seg chạy trên PC phát hiện chính xác ổ gà hình tròn (10/10 lần). Sai số diện tích trung bình +11,4%, sai số độ sâu –12,4%, tuy nhiên sai số thể tích chỉ –2,4% nhờ sự bù trừ. Khi loại bỏ hai lần đo bị nhiễu do chế độ boost, sai số độ sâu giảm còn –7,6% và thể tích sai số +1,7%.

Về truyền thông: Hệ thống WiFi hoạt động ổn định trong phòng thí nghiệm, không mất gói UDP. Tốc độ khung hình đạt 13–15 fps (gần xe) và 8 fps ở khoảng cách 10 m. Các lệnh điều khiển WebSocket vẫn thực thi được ở cự ly 10 m, đáp ứng yêu cầu thử nghiệm.

Về giao diện giám sát: Giao diện PyQt5 hiển thị trực quan ảnh BEV, bảng ROI, metadata, quản lý cơ sở dữ liệu SQLite và bản đồ tương tác. Hệ thống lưu trữ và tra cứu dữ liệu ổ gà hoạt động đúng chức năng.

4.2.2. Các vấn đề còn tồn tại

Độ chính xác đo diện tích và độ sâu chưa cao: Sai số diện tích và độ sâu riêng lẻ khá lớn (± 11 –12%), chủ yếu do giới hạn phần cứng (camera FPS thấp gây motion blur, tấm nhựa giả lập bị lún, cảm biến siêu âm đơn điểm).

Ảnh hưởng của chế độ boost: Khi kích hoạt boost (tăng tốc vượt dốc), tốc độ tăng đột ngột làm sai lệch đo độ sâu, cần có cơ chế vô hiệu hóa boost trong vùng đo.

Khoảng cách truyền thông hạn chế: WiFi chỉ hoạt động ổn định trong phạm vi ngắn (dưới 15 m), tốc độ khung hình giảm khi ở xa, chưa phù hợp cho khảo sát thực địa trên diện rộng.

Phụ thuộc vào PC để chạy YOLO: Nếu mất kết nối UDP, PC không thể phát hiện ổ gà; chưa tối ưu để chạy trực tiếp trên xe.

GPS độ chính xác thấp: Module G-Mouse có sai số lớn (5–15 m), chưa phù hợp để xác định vị trí chính xác của ổ gà trên bản đồ.

4.3. Hướng phát triển trong tương lai

Từ những kết quả thực nghiệm đã đạt được cùng các hạn chế còn tồn tại, hướng phát triển tiếp theo của đề tài sẽ tập trung vào các mục tiêu trọng tâm sau:

- *Nâng cao độ chính xác đo lường vật lý:*
 - + *Tối ưu hóa thu nhận hình ảnh:* Nâng cấp lên hệ thống camera có tốc độ khung hình cao hoặc sử dụng camera Global Shutter nhằm triệt tiêu hiện tượng mờ nhòe do chuyển động.
 - + *Cải tiến cảm biến độ sâu:* Tích hợp các cảm biến chuyên dụng có độ chính xác cao hơn như LiDAR hoặc Depth Camera, đồng thời tăng tần suất lấy mẫu để làm mịn dữ liệu đám mây điểm.
- *Đột phá năng lực truyền thông và điều khiển:*
 - + *Mở rộng phạm vi truyền tải dữ liệu:* Thay thế kết nối Wi-Fi bằng việc truyền dữ liệu video qua mạng di động băng thông rộng 4G/5G (tích hợp module SIM), đảm bảo hệ thống vận hành không giới hạn khoảng cách địa lý.
 - + *Tăng cường độ ổn định điều khiển:* Ứng dụng các giao thức truyền thông tầm xa bằng sóng radio (như LoRa, Zigbee hoặc các module RF chuyên dụng) nhằm tối ưu hóa độ trễ và nâng cao độ ổn định tín hiệu.
- *Tối ưu hóa thuật toán trí tuệ nhân tạo (AI):*
 - + *Nhúng hóa mô hình xử lý:* Chuyển đổi và tăng tốc mô hình YOLOv11-seg trên các phần cứng chuyên dụng tích hợp ngay trên xe như Google Coral TPU hoặc Hailo-8, giảm thiểu sự phụ thuộc vào PC ngoại vi.
 - + *Nâng cao năng lực tổng quát hóa:* Tiếp tục thu thập và huấn luyện lại mô hình với tập dữ liệu đa dạng hơn, bao gồm nhiều hình thái ổ gà, điều kiện ánh sáng phức tạp và các góc quan sát khác nhau.
- *Hoàn thiện hệ thống định vị:*

+ Ứng dụng công nghệ định vị động thời gian thực (RTK-GPS) thay thế cho module GPS G-Mouse hiện tại, nâng độ chính xác định vị lên hàng centimet nhằm gắn tọa độ chính xác tuyệt đối cho từng vị trí hư hỏng phát hiện được.

- *Thử nghiệm trên đường thực tế và liên kết đa thiết bị:*

+ Triển khai thử nghiệm robot trên các tuyến đường thực tế (đường nội bộ, đường nông thôn) để đánh giá độ bền bỉ dưới các tác động phức tạp của môi trường.

+ Nghiên cứu giải pháp phối hợp đa tác tử, kết hợp xe tự hành mặt đất với thiết bị bay không người lái (Drone) để tối ưu hóa tốc độ và lập bản đồ hư hỏng hạ tầng giao thông trên diện rộng.

Kết luận chương 4

Chương 4 đã trình bày toàn bộ quá trình thực nghiệm và đánh giá toàn diện hệ thống xe tự hành trên mô hình sa hình thu nhỏ (kích thước rộng 48 cm, dài 5 m, có thiết kế đoạn cong). Các kết quả thực nghiệm cho thấy hệ thống vận hành đạt độ ổn định cao: robot bám làn chính xác, phát hiện và ước lượng kích thước ổ gà hiệu quả với sai số thể tích nằm trong ngưỡng chấp nhận được (dưới 5%). Song song đó, các kênh truyền thông qua giao thức UDP và WebSocket cũng chứng minh được sự hoạt động mượt mà trong phạm vi phòng thí nghiệm.

Bên cạnh những kết quả khả quan, chương này cũng đã thẳng thắn chỉ ra một số hạn chế cốt lõi như: sai số cục bộ về diện tích và độ sâu ở một số phân cảnh còn lớn; sự ảnh hưởng của chế độ tăng tốc (boost) đến độ ổn định; phạm vi kết nối Wi-Fi còn hẹp và độ chính xác của module GPS chưa cao. Những phân tích chi tiết về mặt kỹ thuật này chính là cơ sở khoa học để đề xuất các giải pháp nâng cấp về phần cứng, thuật toán và phương thức truyền thông trong tương lai, hướng tới mục tiêu hoàn thiện một hệ thống có đủ khả năng ứng dụng thực tế trong công tác quản lý và bảo trì hạ tầng giao thông đường bộ.

TÀI LIỆU THAM KHẢO

- [1]. Lê Hùng Lân, Nguyễn Văn Tiềm, Hệ thống điều khiển nhiều chiều và phi tuyến, NXB Giao thông Vận tải, (2013).
- [2]. Lương Thái Lê, Machine Learning, Bài giảng Trường Đại học Giao thông Vận tải.
- [3]. Lê Thị Thúy Nga, Xử lý ảnh trong công nghiệp, Bài giảng Bộ môn Điều khiển học, Trường Đại học Giao thông Vận tải.
- [4]. Nguyễn Quốc Tuấn, Cơ sở dữ liệu, Bài giảng Bộ môn Mạng và Hệ thống thông tin, Trường Đại học Giao thông Vận tải.
- [5]. Che Quang Huy, Le Duc Tri, Pham Minh Quan, Nguyen Vinh Tiep, Lam Duc Khai, TwinLiteNet+: An Enhanced Multi-Task Segmentation Model for Autonomous Driving, (2024).
<https://arxiv.org/pdf/2403.16958>
- [6]. Shenzhen Jinshunlaite Motor Co., Ltd., 37mm round spur gear motor (JGB37-520B), Datasheet.
https://www.aslongdcmotor.com/photo/aslongdcmotor/document/26547/37mm%20Round%20Spur%20Gear%20Motor_PDF00.pdf
- [7]. Ultralytics, Ultralytics YOLO11, Ultralytics Documentation, (2024).
<https://docs.ultralytics.com/vi/models/yolo11>
- [8]. Roboflow, How to train a YOLOv11 object detection model on a custom dataset, Roboflow Blog, (2024).
<https://blog.roboflow.com/yolov11-how-to-train-custom-data/>
- [9]. Khuê Nguyễn Creator, Lập trình ESP32 websocket điều khiển đèn real time, Khuê Nguyễn Creator Blog, (2024).
<https://khuenguyencreator.com/lap-trinh-esp32-websocket-dieu-khien-den-real-time/>
- [10]. Martin Fitzpatrick, PyQt5 tutorial, Python GUIs, (2026).
<https://www.pythonguis.com/pyqt5-tutorial/>
- [11]. Roboticsbook.org, Kinematics for driving, Roboticsbook.org.
https://www.roboticsbook.org/S62_driving_actions.html
- [12]. chequanghuy, TwinLiteNet: efficient and lightweight model for drivable area & lane segmentation, GitHub Repository, (2024).
<https://github.com/chequanghuy/TwinLiteNet>

PHỤ LỤC

Phụ lục 1. Chương trình code trên ESP32

(Ngôn ngữ C++)

Config.h

```
#pragma once
#include <Arduino.h>

// ===== Pins =====
static const int PIN_RPWM = 25;
static const int PIN_LPWM = 26;
static const int PIN_REN = 27;
static const int PIN_LEN = 14;
static const int PIN_ENC_A = 32;
static const int PIN_ENC_B = 33;
static const int PIN_SERVO = 15;

// ===== UART =====
#define RPI_SERIAL Serial1
static const int PIN_TX1 = 17;
static const int PIN_RX1 = 16;
static const int UART_BAUD = 115200;

// ===== Encoder =====
static const int ENCODER_COUNTS_PER_REV = 618;

// ===== Wheel & Units =====
static constexpr float WHEEL_DIAM_M = 0.075f;
static constexpr float RPM2MPS = (float)M_PI * WHEEL_DIAM_M / 60.0f;
static constexpr float MPS2RPM = 1.0f / RPM2MPS;

// ===== Limits =====
static constexpr float MAX_MPS = 0.70f;
static constexpr float MAX_STEER_DEG = 30.0f;
static constexpr float DEAD_MPS = 0.02f;
static constexpr float DEAD_SERVO_DEG = 0.5f;

// ===== Servo geometry =====
static constexpr float SERVO_MIN = 41.0f;
static constexpr float SERVO_MAX = 135.0f;
static constexpr float SERVO_CENTER = 88.0f;
static constexpr float SERVO_RATE_DPS = 90.0f;
static constexpr float STEER_TO_SERVO = 47.0f / 30.0f;
static constexpr float SERVO_TO_STEER = 1.0f / STEER_TO_SERVO;

// ===== PWM/Motor =====
static const int CH_RPWM = 4;
static const int CH_LPWM = 5;
static const int PWM_FREQ = 20000;
static const int PWM_BITS = 8;
static const int PWM_DEADBAND = 5;

// ===== PID =====
static float Kp=1.86f, Ki=4.5f, Kd=0.046f;
// ===== Filters / timing =====
static const int FILTER_SIZE = 10;
static const float LOOP_DT_S = 0.050f;

// ===== Safety =====
static const unsigned long TIMEOUT_MS = 2000;
static const int RSSI_WEAK_THRESHOLD = -75;

// ===== Auto-hold =====
static constexpr float HOLD_RPM_TH = 0.30f;
static const unsigned long HOLD_PERIOD_MS = 100;
static const int HOLD_INIT_PWM = 15;
static const int HOLD_ADJ_STEP = 3;
static const int HOLD_MAX_PWM = 80;
static const unsigned long HOLD_ARM_MS = 500;
static const unsigned long HOLD_SP0_MS = 1000;
static const int HOLD_MIN_PWM = 35;
// ===== WiFi Station =====
static const char* WIFI_SSID = "Remote";
static const char* WIFI_PSK = "dung28092003";

// ===== Ultrasonic SRF05 =====
// Cảm biến 1: đo ổ gà
static const int PIN_SRF05_POTHOLE_TRIG = 18;
static const int PIN_SRF05_POTHOLE_ECHO = 19;
// Cảm biến 2: phía trước
static const int PIN_SRF05_FRONT_TRIG = 21;
static const int PIN_SRF05_FRONT_ECHO = 22;
// Cảm biến 3: phía sau (ECHO dùng GPIO5)
static const int PIN_SRF05_REAR_TRIG = 23;
static const int PIN_SRF05_REAR_ECHO = 5;
```

```
static const float SOUND_SPEED_CM_PER_US =
0.0343f; // 343 m/s
static const unsigned long SRF05_TIMEOUT_US =
10000;
```

```
// ===== Battery voltage =====
static const int PIN_BATTERY = 36;
static const float R1 = 100000.0f;
static const float R2 = 33000.0f;
static const float VOLTAGE_DIVIDER = R2 / (R1
+ R2);
static const float BATTERY_MIN_V = 11.1f;
static const float BATTERY_MAX_V = 12.62f;
```

Control_Motor.cpp

```
#include "Control_Motor.h"
#include "Config.h"
```

```
int g_lastPwm = 0;
static volatile long g_encoderCount = 0;
static portMUX_TYPE g_encMux =
portMUX_INITIALIZER_UNLOCKED;
```

```
inline void pwmWriteSafe(int ch, int val){
ledcWrite(ch, val); }
```

```
void motorDrivePWM(int pwm){
pwm = constrain(pwm, -255, 255);
if (abs(pwm) < PWM_DEADBAND) pwm = 0;
if (pwm >= 0){ pwmWriteSafe(CH_RPWM, 0);
pwmWriteSafe(CH_LPWM, pwm); }
else { pwmWriteSafe(CH_RPWM, -pwm);
pwmWriteSafe(CH_LPWM, 0); }
g_lastPwm = pwm;
}
```

```
void motorBrake(){
pwmWriteSafe(CH_RPWM, 255);
pwmWriteSafe(CH_LPWM, 255);
g_lastPwm = 0;
}
```

```
void IRAM_ATTR encoderISR(){
int b = digitalRead(PIN_ENC_B);
portENTER_CRITICAL_ISR(&g_encMux);
g_encoderCount += (b>0) ? 1 : -1;
portEXIT_CRITICAL_ISR(&g_encMux);
}
```

```
long getEncoderCount() {
long snap;
portENTER_CRITICAL(&g_encMux);
snap = g_encoderCount;
```

```
portEXIT_CRITICAL(&g_encMux);
return snap;
}
```

```
void motorHardwareBegin() {
pinMode(PIN_REN,OUTPUT);
pinMode(PIN_LEN,OUTPUT);
digitalWrite(PIN_REN,HIGH);
digitalWrite(PIN_LEN,HIGH);
ledcSetup(CH_RPWM, PWM_FREQ,
PWM_BITS); ledcAttachPin(PIN_RPWM,
CH_RPWM);
ledcSetup(CH_LPWM, PWM_FREQ,
PWM_BITS); ledcAttachPin(PIN_LPWM,
CH_LPWM);
pinMode(PIN_ENC_A,INPUT_PULLUP);
pinMode(PIN_ENC_B,INPUT_PULLUP);
```

```
attachInterrupt(digitalPinToInterrupt(PIN_ENC_A)
, encoderISR, RISING);
}
```

Control_Motor.h

```
#pragma once
#include <Arduino.h>
```

```
void motorHardwareBegin();
void motorDrivePWM(int pwm);
void motorBrake();
long getEncoderCount();
```

extern int g_lastPwm;

Control_Parser.cpp

```
#include "Control_Parser.h"
#include "Config.h"
#include "Control.h"
```

```
void handleSerialCommands() {
if (controllsLinkLost()) return;
while (RPI_SERIAL.available() > 0) {
String input =
RPI_SERIAL.readStringUntil('\n');
input.trim();

if (input.startsWith("S")) {
int aPos = input.indexOf('A');
if (aPos != -1) {
float dc_mps_in = input.substring(1,
aPos).toFloat();
float steer_deg_in = input.substring(aPos
+ 1).toFloat();
```

// Cập nhật lệnh điều khiển

```

        controlOnAutoSet(dc_mps_in,
        steer_deg_in);
    }
}
}
}

```

Control_Parser.h

```

#pragma once
#include <Arduino.h>
void handleSerialCommands();

```

Control_Safety.cpp

```

#include "Control_Safety.h"
#include "Config.h"
#include "Sensors.h"
#include <WiFi.h>
#include <math.h>

bool isWifiStrongEnough() {
    int rssi = WiFi.RSSI();
    return (rssi >= RSSI_WEAK_THRESHOLD);
}

float applySafetyLimit(float desired_mps) {
    float front_cm = getDistanceCm(1); // cảm biến
    trước
    float rear_cm = getDistanceCm(2); // cảm biến
    sau
    const float FRONT_STOP = 10.0f; // dừng hẳn
    nếu < 10cm
    const float FRONT_SAFE = 30.0f; // bắt đầu
    giảm tốc khi < 30cm
    const float REAR_STOP = 10.0f;
    const float REAR_SAFE = 30.0f;
    if (desired_mps > 0.0f) {
        if (isinf(front_cm) || front_cm <= 0) return
        desired_mps;
        if (front_cm <= FRONT_STOP) return 0.0f;
        if (front_cm >= FRONT_SAFE) return
        desired_mps;
        float factor = (front_cm - FRONT_STOP) /
        (FRONT_SAFE - FRONT_STOP);
        return desired_mps * factor;
    }
    else if (desired_mps < 0.0f) {
        if (isinf(rear_cm) || rear_cm <= 0) return
        desired_mps;
        if (rear_cm <= REAR_STOP) return 0.0f;
        if (rear_cm >= REAR_SAFE) return
        desired_mps;
        float factor = (rear_cm - REAR_STOP) /
        (REAR_SAFE - REAR_STOP);
        return desired_mps * factor;
    }
}

```

```

    }
    return desired_mps;
}

```

Control_Safety.h

```

#pragma once
#include <Arduino.h>
bool isWifiStrongEnough();
float applySafetyLimit(float desired_mps);

```

Control.cpp

```

#include "Control.h"
#include "Config.h"
#include "Sensors.h"
#include "Control_Safety.h"
#include "Control_Parser.h"
#include "Control_Motor.h"
#include <ESP32Servo.h>
#include <WiFi.h>
#include <math.h>

static Servo g_servo;
static float g_curRpm = 0.0f;
static float g_setpoint_rpm = 0.0f;
static float g_rampSet_rpm = 0.0f;
static float g_integral = 0.0f, g_lastErr = 0.0f;
static float g_speedBuf[FILTER_SIZE];
static int g_fi = 0;
static float g_servoTargetAbs = SERVO_CENTER;
static float g_servoPosAbs = SERVO_CENTER;
static bool g_softAuto = false;
static bool g_autoMode = false;
static bool g_lastAuto = false;
static volatile bool g_manualNeedsNeutral = false;
static unsigned long g_lastRxMs = 0;
static bool g_linkLost = false;
static unsigned long g_lastTick = 0;
static long g_lastEnc = 0;
static float g_dt = LOOP_DT_S;
static bool g_isReversingLock = false;
// auto-hold
static bool autoHoldActive = false, holdArming =
false, sp0Counting = false;
static unsigned long holdArmStart = 0,
lastHoldCheck = 0, sp0Start = 0;
static int holdStickPWM = 0;
// Ramps
static float rampUpStep = 10.0f;
static float rampDownStep = 5.0f;

static float filterRPM(float v) {
    g_speedBuf[g_fi] = v;
    g_fi = (g_fi + 1) % FILTER_SIZE;
}

```

```

float s = 0; for (int i = 0; i < FILTER_SIZE; i++)
s += g_speedBuff[i];
return s / FILTER_SIZE;
}

```

```

static void checkLinkAndModeTransitions(unsigned
long now) {
    bool wifiConnected = (WiFi.status() ==
WL_CONNECTED);
    bool rssiOk = isWifiStrongEnough();
    bool hasGoodLink = wifiConnected && rssiOk;
    bool isTimeout = (now - g_lastRxMs >
TIMEOUT_MS);
    bool linkBad = (!hasGoodLink || isTimeout);
    if (linkBad) {
        if (!g_linkLost) {
            g_linkLost = true;
            g_integral = 0; g_lastErr = 0;
            g_setpoint_rpm = 0; g_rampSet_rpm = 0;
            sp0Counting = true; sp0Start = now;
            g_servoTargetAbs = SERVO_CENTER;
        }
    } else {
        if (g_linkLost) g_linkLost = false;
    }
    if (g_autoMode != g_lastAuto) {
        g_integral = 0; g_lastErr = 0;
        if (!g_autoMode) {
            g_setpoint_rpm = 0; g_rampSet_rpm = 0;
            g_isReversingLock = false;
            autoHoldActive = false; holdArming = false;
            motorBrake();
            g_servo.write(SERVO_CENTER);
            g_manualNeedsNeutral = true;
        }
        g_lastAuto = g_autoMode;
    }
}

```

```

static void updateServoRamp() {
    float maxStep = SERVO_RATE_DPS * g_dt;
    float diff = g_servoTargetAbs - g_servoPosAbs;
    if (diff > maxStep) g_servoPosAbs += maxStep;
    else if (diff < -maxStep) g_servoPosAbs -=
maxStep;
    else g_servoPosAbs = g_servoTargetAbs;
    g_servoPosAbs = constrain(g_servoPosAbs,
SERVO_MIN, SERVO_MAX);
    g_servo.write(g_servoPosAbs);
}

```

```

static void calculateCurrentRPM() {

```

```

    long encSnap = getEncoderCount();
    long delta = encSnap - g_lastEnc; g_lastEnc =
encSnap;
    float rpm = -(float)delta /
(float)ENCODER_COUNTS_PER_REV / g_dt *
60.0f;
    g_curRpm = filterRPM(rpm);
}

```

```

static float processReverseLockAndSafety() {
    float stopThreshold = 20.0f;
    if (!g_isReversingLock) {
        if ((g_setpoint_rpm > 2.0f && g_curRpm < -
stopThreshold) ||
            (g_setpoint_rpm < -2.0f && g_curRpm >
stopThreshold)) {
            g_isReversingLock = true;
        }
    }
    float effectiveSetpoint = g_setpoint_rpm;
    if (g_isReversingLock) {
        effectiveSetpoint = 0.0f;
        if (fabsf(g_curRpm) < 2.0f) {
            g_isReversingLock = false;
            g_integral = 0; g_lastErr = 0;
        }
    }
    float desired_mps = effectiveSetpoint *
RPM2MPS;
    float safe_mps = applySafetyLimit(desired_mps);
    return safe_mps * MPS2RPM;
}

```

```

static void updateSetpointRamp(float
effectiveSetpoint) {
    if (g_linkLost) {
        float diff = 0.0f - g_rampSet_rpm;
        if (diff > 0) g_rampSet_rpm += min(diff,
rampDownStep);
        else g_rampSet_rpm += max(diff, -
rampDownStep);
    } else {
        if (fabsf(effectiveSetpoint) >
fabsf(g_rampSet_rpm)) {
            float diff = effectiveSetpoint - g_rampSet_rpm;
            if (diff > 0) g_rampSet_rpm += min(diff,
rampUpStep);
            else g_rampSet_rpm += max(diff, -
rampUpStep);
        } else {
            g_rampSet_rpm = effectiveSetpoint;
        }
    }
}

```

```

    }
    if (fabsf(g_rampSet_rpm) < 2.5f &&
    fabsf(effectiveSetpoint) < 2.5f) g_rampSet_rpm =
    0;
    }

static void handlePIDControl(float
effectiveSetpoint) {
    autoHoldActive = false; holdArming = false;
    holdStickPWM = 0; sp0Counting = false;
    static float lastSet = 0;
    if ((effectiveSetpoint > 0 && lastSet <= 0) ||
    (effectiveSetpoint < 0 && lastSet >= 0)) {
        g_integral = 0; g_lastErr = 0;
    }
    lastSet = effectiveSetpoint;
    float err = g_rampSet_rpm - g_curRpm;
    g_integral += err * g_dt;
    if (Ki > 0) {
        float I_MAX = 255.0f / Ki;
        if (g_integral > I_MAX) g_integral = I_MAX;
        if (g_integral < -I_MAX) g_integral = -I_MAX;
    }
    float deriv = (err - g_lastErr) / g_dt;
    float out = Kp * err + Ki * g_integral + Kd *
    deriv;
    g_lastErr = err;
    motorDrivePWM(constrain((int)lroundf(out), -
    255, 255));
    }

static void handleAutoHold(unsigned long now) {
    if (!sp0Counting) { sp0Counting = true; sp0Start
    = now; }
    else {
        if ((now - sp0Start) >= HOLD_SP0_MS &&
    !autoHoldActive) {
            autoHoldActive = true; holdArming = false;
            lastHoldCheck = now;
            int desiredSign = (g_curRpm > 0) ? -1 : +1;
            holdStickPWM = constrain(desiredSign *
    HOLD_INIT_PWM, -HOLD_MAX_PWM,
    HOLD_MAX_PWM);
        }
    }

    if (!autoHoldActive) {
        if (!holdArming) {
            if (fabsf(g_curRpm) < HOLD_RPM_TH) {
                holdArming = true; holdArmStart = now; }
            motorDrivePWM(0);
        } else {

```

```

            if (fabsf(g_curRpm) >= HOLD_RPM_TH) {
                holdArming = false; motorDrivePWM(0); }
            else if (now - holdArmStart >=
    HOLD_ARM_MS) { autoHoldActive = true;
            lastHoldCheck = now; }
            else motorDrivePWM(0);
        }
    } else {
        int pwmApply = (abs(holdStickPWM) <
    HOLD_MIN_PWM) ? 0 : holdStickPWM;
        motorDrivePWM(pwmApply);
        if (now - lastHoldCheck >=
    HOLD_PERIOD_MS) {
            lastHoldCheck = now;
            if (fabsf(g_curRpm) >= HOLD_RPM_TH) {
                int desiredSign = (g_curRpm > 0) ? -1 : +1;
                if (holdStickPWM == 0 || (holdStickPWM *
    desiredSign < 0)) holdStickPWM = desiredSign *
    HOLD_INIT_PWM;
                else holdStickPWM += desiredSign *
    HOLD_ADJ_STEP;
                holdStickPWM = constrain(holdStickPWM, -
    HOLD_MAX_PWM, HOLD_MAX_PWM);
            }
        }
    }
}

```

```

static void sendTelemetry(unsigned long now) {
    static unsigned long lastTeleUART = 0;
    if (now - lastTeleUART >= 100) {
        lastTeleUART = now;
        float real_mps = g_curRpm * RPM2MPS;
        float real_steer = (g_servoPosAbs -
    SERVO_CENTER) * SERVO_TO_STEER;
        float target_mps = g_rampSet_rpm *
    RPM2MPS;
        float original_mps = g_setpoint_rpm *
    RPM2MPS;
        float target_steer = (g_servoTargetAbs -
    SERVO_CENTER) * SERVO_TO_STEER;

        float batPct = getBatteryPercent();
        float distPothole = getDistanceCm(0);
        float distFront = getDistanceCm(1);
        float distRear = getDistanceCm(2);
        int mode = g_autoMode ? 1 : 0;

        RPI_SERIAL.printf("FB:S=%.2fA=%.2fB=%.0fP
    =%.1fF=%.1fR=%.1fM=%d\n",
            real_mps, real_steer, batPct,
            distPothole, distFront, distRear, mode);
    }
}

```

```

Serial.printf("FB:TocDo=%.2f|GocLai=%.2f|Ramp
TocDo=%.2f|Setpoint=%.2f|SetpointGocLai=%.2f|
Pin=%.0f|OGa=%.1f|CBTruoc=%.1f|CBSau=%.1f|
CheDo=%d\n",
    real_mps, real_steer, target_mps,
    original_mps, target_steer,
    batPct, distPothole, distFront, distRear,
    mode);
}
}

```

```

void controlGetState(float& rpm, int& pwm, float&
servoAbsDeg, float& set_rpm, bool& isAuto) {
    rpm      = g_curRpm;
    pwm      = g_lastPwm;
    servoAbsDeg = g_servoPosAbs;
    set_rpm   = g_rampSet_rpm;
    isAuto    = g_autoMode;
}

```

```

void controlFillStatus.JSON(char* buf, size_t len) {
    snprintf(buf, len,
        "{ \"rpm\": %.2f, \"pwm\": %d, \"sv\": %.1f, \"dc\": %.0f, \"md\": %d }",
        g_curRpm, g_lastPwm, g_servoPosAbs,
        (fabsf(g_rampSet_rpm) < 2.5f) ? 0 :
        roundf(g_rampSet_rpm),
        g_autoMode ? 1 : 0
    );
}

```

```

void controlSetSoftAuto(bool on) { g_softAuto =
on; }
bool controlIsAuto() { return g_autoMode; }
void controlForceManualNeutralGate() {
    g_manualNeedsNeutral = true; }
void controlOnHeartbeat() { g_lastRxMs = millis();
}

```

```

void controlOnAutoSet(float dc_mps, float
steer_deg) {
    if (g_linkLost) return;
    if (!isWifiStrongEnough()) return;
    dc_mps = constrain(dc_mps, -MAX_MPS,
MAX_MPS);
    steer_deg = constrain(steer_deg, -
MAX_STEER_DEG, MAX_STEER_DEG);
    float dc_rpm = (fabsf(dc_mps) < DEAD_MPS) ?
0.0f : (dc_mps * MPS2RPM);

```

```

    float servoAbs = SERVO_CENTER + steer_deg *
STEER_TO_SERVO;
    servoAbs = constrain(servoAbs, SERVO_MIN,
SERVO_MAX);
    static portMUX_TYPE cmdMux =
portMUX_INITIALIZER_UNLOCKED;
    portENTER_CRITICAL(&cmdMux);
    g_setpoint_rpm = dc_rpm;
    g_servoTargetAbs = servoAbs;
    portEXIT_CRITICAL(&cmdMux);
}

```

```

void controlOnManualWS(float dc_mps, float
steer_deg) {
    if (g_linkLost) return;
    if (g_autoMode) { g_lastRxMs = millis(); return; }
    if (!isWifiStrongEnough()) { g_lastRxMs =
millis(); return; }
    dc_mps = constrain(dc_mps, -MAX_MPS,
MAX_MPS);
    steer_deg = constrain(steer_deg, -
MAX_STEER_DEG, MAX_STEER_DEG);
    float dc_rpm = (fabsf(dc_mps) < DEAD_MPS) ?
0.0f : (dc_mps * MPS2RPM);
    float servoAbs = SERVO_CENTER + steer_deg *
STEER_TO_SERVO;
    servoAbs = constrain(servoAbs, SERVO_MIN,
SERVO_MAX);
    if (g_manualNeedsNeutral) {
        bool dcNeutral = fabsf(dc_mps) <=
DEAD_MPS;
        bool svNeutral = fabsf(steer_deg) <=
DEAD_SERVO_DEG;
        g_lastRxMs = millis();
        if (!(dcNeutral && svNeutral)) return;
        g_manualNeedsNeutral = false;
        dc_rpm = 0; servoAbs = SERVO_CENTER;
    }
    static portMUX_TYPE cmdMux =
portMUX_INITIALIZER_UNLOCKED;
    portENTER_CRITICAL(&cmdMux);
    g_setpoint_rpm = dc_rpm;
    g_servoTargetAbs = servoAbs;
    g_lastRxMs = millis();
    portEXIT_CRITICAL(&cmdMux);
}

```

```

bool controlIsLinkLost() {
    return g_linkLost;
}

```

```

void controlBegin() {

```

```

motorHardwareBegin();
g_servo.attach(PIN_SERVO);
g_servoPosAbs = SERVO_CENTER;
g_servo.write(g_servoPosAbs);
for (int i = 0; i < FILTER_SIZE; i++)
g_speedBuff[i] = 0;
g_lastTick = millis(); g_lastRxMs = millis();
motorBrake();
RPI_SERIAL.begin(UART_BAUD, SERIAL_8N1,
PIN_RX1, PIN_TX1);
}

```

```

void controlLoop50ms() {
g_autoMode = g_softAuto;
unsigned long now = millis();
checkLinkAndModeTransitions(now);
if (g_autoMode) {
handleSerialCommands();
}
updateServoRamp();
calculateCurrentRPM();
float effectiveSetpoint =
processReverseLockAndSafety();
updateSetpointRamp(effectiveSetpoint);
if (fabsf(effectiveSetpoint) >= 2.5f) {
handlePIDControl(effectiveSetpoint);
} else {
handleAutoHold(now);
}
sendTelemetry(now);
}

```

```

Control.h
#pragma once
#include <Arduino.h>

```

```

void controlBegin();
void controlLoop50ms();
void controlOnHeartbeat();

```

```

// Đầu vào từ UI/RPi: tốc độ m/s & góc đánh lái
bánh xe ( $\pm 30^\circ$ )

```

```

void controlOnManualWS(float dc_mps, float
steer_deg);
void controlOnAutoSet(float dc_mps, float
steer_deg);

```

```

// Mode
void controlSetSoftAuto(bool on);
bool controlsAuto();
void controlForceManualNeutralGate();

```

```

// Status helpers (cho Net/UI)

```

```

void controlFillStatusJSON(char* buf, size_t len);
void controlGetState(float& rpm, int& pwm, float&
servoAbsDeg,
float& set_rpm, bool& isAuto);

```

```

bool controlsLinkLost();

```

```

main.cpp

```

```

#include <Arduino.h>
#include "Config.h"
#include "Control.h"
#include "Net.h"
#include "Sensors.h"

```

```

void setup(){
Serial.begin(115200);
controlBegin();
netBegin();
initSensors();
}

```

```

void loop(){
unsigned long now = millis();

```

```

// --- ĐIỀU KHIỂN ĐỘNG CƠ (50ms) ---

```

```

static unsigned long lastControl = 0;
if (now - lastControl >= 50){
controlLoop50ms();
lastControl = now;
}

```

```

// --- MẠNG WIFI (20ms) ---

```

```

static unsigned long lastNet = 0;
if (now - lastNet >= 20){
netLoop();
lastNet = now;
}

```

```

// --- CẬP NHẬT CẢM BIẾN SIÊU ÂM---

```

```

updateUltrasonic();

```

```

// --- CẬP NHẬT PIN ---

```

```

static unsigned long lastBattery = 0;
if (now - lastBattery >= 3000) {
updateBattery();
lastBattery = now;
}
}

```

```

Net.cpp

```

```

#include <Arduino.h>
#include <WiFi.h>
#include <WebServer.h>
#include <WebSocketsServer.h>

```

```

#include <ESPmDNS.h>
#include "Config.h"
#include "Control.h"
#include "WebUI.h"
#include "Sensors.h"

static WebServer server(80);
static WebSocketsServer ws(81);
static unsigned long lastWiFiCheck = 0;
const unsigned long
WIFI_RECONNECT_INTERVAL = 2000;

// ===== Handlers =====
static void handleRoot() {
    server.send(200, "text/html", WEB_PAGE);
}

static void handleStatus() {
    char buf[160];
    controlFillStatusJSON(buf, sizeof(buf));
    server.sendHeader("Cache-Control", "no-cache,
no-store, must-revalidate");
    server.sendHeader("Pragma", "no-cache");
    server.sendHeader("Expires", "0");
    server.send(200, "application/json", buf);
}

static void handleMode() {
    bool softAuto = false;
    if (server.hasArg("auto")) softAuto =
(server.arg("auto").toInt() != 0);
    controlSetSoftAuto(softAuto);
    if (!softAuto) controlForceManualNeutralGate();
    server.send(200, "text/plain", softAuto ? "AUTO"
: "MANUAL");
}

static void handleAuto() {
    float dc_mps = 0.0f, sv_deg = 0.0f;
    if (server.hasArg("dc")) dc_mps =
atof(server.arg("dc").c_str());
    if (server.hasArg("sv")) sv_deg =
atof(server.arg("sv").c_str());
    controlOnAutoSet(constrain(dc_mps, -
MAX_MPS, MAX_MPS), constrain(sv_deg, -
MAX_STEER_DEG, MAX_STEER_DEG));
    controlOnHeartbeat();
    server.send(200, "text/plain", "OK");
}

static void handlePing() {
    controlOnHeartbeat();

```

```

    server.send(200, "text/plain", "OK");
}

// ===== WS =====
static void handleWsMsg(uint8_t *payload, size_t
len) {
    static char buf[64];
    size_t n = min(len, sizeof(buf) - 1);
    memcpy(buf, payload, n);
    buf[n] = 0;
    char *pMode = strstr(buf, "mode:");
    if (pMode) {
        bool softAuto = (atoi(pMode + 5) != 0);
        controlSetSoftAuto(softAuto);
        if (!softAuto)
controlForceManualNeutralGate();
        controlOnHeartbeat();
        return;
    }
    if (len == 2 && payload[0] == 'h' &&
payload[1] == 'b') {
        controlOnHeartbeat();
        return;
    }
    if (controlIsAuto()) {
        controlOnHeartbeat();
        return;
    }
    char *pdc = strstr(buf, "dc:");
    char *psv = strstr(buf, "sv:");
    float dc_mps = 0.0f, sv_deg = 0.0f;
    if (pdc) dc_mps = atof(pdc + 3);
    if (psv) {
        char *psvVal = strchr(psv, ':');
        if (psvVal) sv_deg = atof(psvVal + 1);
    }
    controlOnManualWS(constrain(dc_mps, -
MAX_MPS, MAX_MPS), constrain(sv_deg, -
MAX_STEER_DEG, MAX_STEER_DEG));
    controlOnHeartbeat();
}

static void onWsEvent(uint8_t num, WStype_t type,
uint8_t *payload, size_t length) {
    if (type == WStype_TEXT)
handleWsMsg(payload, length);
    else controlOnHeartbeat();
}

// ===== Public =====
void netBegin() {
    WiFi.mode(WIFI_STA);

```

```

WiFi.begin(WIFI_SSID, WIFI_PSK);
WiFi.setSleep(false);
WiFi.setAutoReconnect(true);
Serial.println("WiFi: Dang khoi tao ket noi...");
server.on("/", handleRoot);
server.on("/status", handleStatus);
server.on("/mode", handleMode);
server.on("/auto", handleAuto);
server.on("/hb", handlePing);
server.begin();

ws.begin();
ws.onEvent(onWsEvent);
}

void netLoop() {
  unsigned long now = millis();
  // --- Reconnect WiFi mỗi 2 giây nếu mất kết nối ---
  --
  if (WiFi.status() != WL_CONNECTED) {
    if (now - lastWiFiCheck >=
WIFI_RECONNECT_INTERVAL) {
      lastWiFiCheck = now;
      Serial.println("WiFi: Dang tim lai tin
hieu...");
      if (WiFi.status() == WL_IDLE_STATUS ||
WiFi.status() == WL_DISCONNECTED) {
        WiFi.begin(WIFI_SSID, WIFI_PSK);
      }
    }
    return;
  }

  // --- Chạy server nếu đã kết nối ---
  server.handleClient();
  ws.loop();

  static unsigned long lastSample = 0;
  int rssi = WiFi.RSSI();
  bool isAuto = controlsAuto();
  unsigned long sampleInterval = isAuto ? 150 :
(rssi > -65 ? 200 : 500);
  if (now - lastSample >= sampleInterval) {
    lastSample = now;
    float rpm, setrpm, svAbs;
    int pwm; bool md;
    controlGetState(rpm, pwm, svAbs, setrpm,
md);
    float batPercent = getBatteryPercent();
    float us1 = getDistanceCm(0), us2 =
getDistanceCm(1), us3 = getDistanceCm(2);
    if (isinf(us1) || isnan(us1)) us1 = -1.0f;

```

```

    if (isinf(us2) || isnan(us2)) us2 = -1.0f;
    if (isinf(us3) || isnan(us3)) us3 = -1.0f;
    char msg[160];
    snprintf(msg, sizeof(msg),

"state:sv=%0.2f,dc=%0.1f,rpm=%0.1f,pwm=%d,md=
%d,volt=%0.2f,bat=%0.0f,us1=%0.1f,us2=%0.1f,us3=
%0.1f,rssi=%d",
        svAbs, (fabsf(setrpm) < 2.5f) ? 0 :
roundf(setrpm),
        roundf(rpm), pwm, md ? 1 : 0,
        getBatteryVoltage(), batPercent, us1, us2,
us3, rssi);
    if (ws.connectedClients() > 0) {
      ws.broadcastTXT(msg);
    }
  }
}

Net.h
#pragma once
void netBegin();
void netLoop();

Sensors.cpp
#include "Sensors.h"
#include "Config.h"
#include <math.h>

// ===== SRF05
Ultrasonic =====
static unsigned long lastMeasureTime[3] = {0, 0,
0};
static float lastDistance[3] = {INFINITY,
INFINITY, INFINITY};
static int currentSensor = 0;
static const unsigned long
MEASURE_INTERVAL_MS = 100;

static float measureOneSensor(int trigPin, int
echoPin, unsigned long timeoutUs) {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  long duration = pulseIn(echoPin, HIGH,
timeoutUs);
  if (duration == 0) return -1.0f;

  float dist = duration *
SOUND_SPEED_CM_PER_US / 2.0f;
  if (dist > 400.0f) return -1.0f;

```

```

    return dist;
}

void initUltrasonic() {
    pinMode(PIN_SRF05_POTHOLE_TRIG,
    OUTPUT);
    pinMode(PIN_SRF05_POTHOLE_ECHO,
    INPUT);
    pinMode(PIN_SRF05_FRONT_TRIG, OUTPUT);
    pinMode(PIN_SRF05_FRONT_ECHO, INPUT);
    pinMode(PIN_SRF05_REAR_TRIG, OUTPUT);
    pinMode(PIN_SRF05_REAR_ECHO, INPUT);

    digitalWrite(PIN_SRF05_POTHOLE_TRIG,
    LOW);
    digitalWrite(PIN_SRF05_FRONT_TRIG, LOW);
    digitalWrite(PIN_SRF05_REAR_TRIG, LOW);

    for (int i = 0; i < 3; i++) {
        lastMeasureTime[i] = millis();
        lastDistance[i] = INFINITY;
    }
}

void updateUltrasonic() {
    unsigned long now = millis();
    if (now - lastMeasureTime[currentSensor] >=
    MEASURE_INTERVAL_MS) {
        float dist = -1.0f;
        switch (currentSensor) {
            case 0:
                dist =
                measureOneSensor(PIN_SRF05_POTHOLE_TRIG,
                PIN_SRF05_POTHOLE_ECHO,
                SRF05_TIMEOUT_US);
                break;
            case 1:
                dist =
                measureOneSensor(PIN_SRF05_FRONT_TRIG,
                PIN_SRF05_FRONT_ECHO,
                SRF05_TIMEOUT_US);
                break;
            case 2:
                dist =
                measureOneSensor(PIN_SRF05_REAR_TRIG,
                PIN_SRF05_REAR_ECHO,
                SRF05_TIMEOUT_US);
                break;
        }
        if (dist >= 0) {
            lastDistance[currentSensor] = dist;
        } else {

```

```

            lastDistance[currentSensor] = INFINITY;
        }
        lastMeasureTime[currentSensor] = now;
        currentSensor = (currentSensor + 1) % 3;
    }
}

float getDistanceCm(int sensorIdx) {
    if (sensorIdx < 0 || sensorIdx >= 3) return
    INFINITY;
    float raw = lastDistance[sensorIdx];
    if (isinf(raw)) return INFINITY;

    // Trừ offset theo từng cảm biến
    float offset = 0.0f;
    switch (sensorIdx) {
        case 0: offset = 7.4f; break; // ở gà
        case 1: offset = 6.7f; break; // trước
        case 2: offset = 4.1f; break; // sau
    }
    float corrected = raw - offset;
    if (corrected < 0) corrected = 0;
    return corrected;
}

// ===== Battery
// =====

static float batteryVoltage = 0.0f;
static float displayedPercent = -1.0f;

// Cấu hình của sổ trượt cho median
static const int WINDOW_SIZE = 5;
static float window[WINDOW_SIZE];
static int windowIndex = 0;
static int windowCount = 0;

// Tham số EMA
static const float EMA_ALPHA = 0.1f;
static float filteredVoltage = 0.0f;
static int medianCount = 0;
static const int MEDIAN_THRESHOLD = 5;

// Hàm so sánh cho qsort
static int cmpFloat(const void *a, const void *b) {
    float fa = *(const float*)a;
    float fb = *(const float*)b;
    return (fa > fb) - (fa < fb);
}

void initBattery() {
    analogReadResolution(12);
    pinMode(PIN_BATTERY, INPUT);

```

```

windowIndex = 0;
windowCount = 0;
medianCount = 0;

// Đọc một mẫu để có giá trị tạm ban đầu
int raw = analogRead(PIN_BATTERY);
float voltageAtPin = (raw / 4095.0f) * 3.3f;
float initialVoltage = (voltageAtPin /
VOLTAGE_DIVIDER) * 1.030f;
filteredVoltage = initialVoltage;
batteryVoltage = initialVoltage;
float percent = (batteryVoltage -
BATTERY_MIN_V) / (BATTERY_MAX_V -
BATTERY_MIN_V) * 100.0f;
displayedPercent = constrain(percent, 0.0f,
100.0f);
}

void updateBattery() {
// ===== ĐỌC ADC, CỬA SỐ TRƯỢT,
MEDIAN, EMA =====
int raw = analogRead(PIN_BATTERY);
float voltageAtPin = (raw / 4095.0f) * 3.3f;
float instantVoltage = (voltageAtPin /
VOLTAGE_DIVIDER) * 1.030f;
// Thêm vào cửa sổ trượt
window[windowIndex] = instantVoltage;
windowIndex = (windowIndex + 1) %
WINDOW_SIZE;
if (windowCount < WINDOW_SIZE)
windowCount++;
if (windowCount < WINDOW_SIZE) return;
// Tính median
float sorted[WINDOW_SIZE];
memcpy(sorted, window, sizeof(float) *
WINDOW_SIZE);
qsort(sorted, WINDOW_SIZE, sizeof(float),
cmpFloat);
float median;
if (WINDOW_SIZE % 2 == 0)
median = (sorted[WINDOW_SIZE/2 - 1] +
sorted[WINDOW_SIZE/2]) / 2.0f;
else
median = sorted[WINDOW_SIZE/2];
medianCount++;
float currentFiltered;
if (medianCount <= MEDIAN_THRESHOLD)
currentFiltered = median;
else
currentFiltered = EMA_ALPHA * median + (1
- EMA_ALPHA) * filteredVoltage;

```

```

filteredVoltage = currentFiltered;

// ===== TÍCH LŨY filteredVoltage
VÀO KHAY, TÍNH TRUNG BÌNH =====
static float avgBuffer[10];
static int avgCount = 0;
static float avgSum = 0.0f;
// Thêm filteredVoltage vào khay
avgBuffer[avgCount] = filteredVoltage;
avgSum += filteredVoltage;
avgCount++;
// Nếu đã đủ 10 mẫu thì tính trung bình và cập
nhật batteryVoltage
if (avgCount >= 10) {
float avgVoltage = avgSum / 10.0f;
batteryVoltage = avgVoltage;
float percent = (batteryVoltage -
BATTERY_MIN_V) / (BATTERY_MAX_V -
BATTERY_MIN_V) * 100.0f;
displayedPercent = constrain(percent, 0.0f,
100.0f);
// Reset khay để bắt đầu chu kỳ mới
avgCount = 0;
avgSum = 0.0f;
}
}

float getBatteryVoltage() {
return batteryVoltage;
}

float getBatteryPercent() {
return displayedPercent;
}

void initSensors() {
initUltrasonic();
initBattery();
}

Sensors.h
#ifndef SENSORS_H
#define SENSORS_H
#include <Arduino.h>

void initSensors();
void updateUltrasonic();
float getDistanceCm(int sensorIdx);
void updateBattery();
float getBatteryVoltage();
float getBatteryPercent();
#endif

WebUI.h

```

#pragma once

```
static const char* WEB_PAGE = R"rawliteral(  
<!DOCTYPE html>  
<html>  
<head>  
<meta charset="UTF-8">  
<meta name="viewport" content="width=device-  
width, initial-scale=1.0, maximum-scale=1.0, user-  
scalable=no">  
<title>ESP32 Car Control</title>
```

```
<style>  
:root{  
  --font-family: Arial, system-ui, -apple-system,  
  "Segoe UI", Roboto, "Helvetica Neue", sans-serif;  
  --c-bg:#5f7fb; --c-card-bd:#e2e8f0; --c-  
  track:#d7dee8; --c-switch-off:#bbb; --c-switch-  
  on:#22c55e;  
  --c-thumb-1:#2563eb; --c-thumb-2:#10b981; --c-  
  text:#333;  
  --radius-card:18px; --shadow-card:0 8px 22px  
  rgba(0,0,0,.08); --shadow-thumb:0 2px 6px  
  rgba(0,0,0,.25);  
  --slider-track-h:8px;  
  --thumb-vert-w:80px; --thumb-vert-h:80px; --  
  thumb-horz-w:80px; --thumb-horz-h:80px;  
  --sw-w: 7.2vw; --sw-h: 3.4vw; --sw-pad: 0.25vw;  
  --container-max-w: 1000px; --col-gap: 2vw;  
  --rect-pad: 1.2vw; --rect-vert-w: 12vw; --rect-vert-  
  min-w: 120px; --rect-vert-h: 70vh; --rect-vert-min-  
  h: 200px;  
  --rect-horz-w: 28vw; --rect-horz-min-w: 280px; --  
  rect-horz-h: 12vw; --rect-horz-min-h: 120px;  
}
```

```
html, body {  
  height:100%; margin:0; padding:0;  
  overflow:hidden;  
  font-family: var(--font-family); background-color:  
  var(--c-bg);  
  touch-action: none; overscroll-behavior: none;  
  -webkit-user-select: none; user-select: none;  
}  
* { -webkit-tap-highlight-color: transparent; user-  
select:none; }  
input[type=range] { touch-action: none; }
```

```
.container{  
  display:flex; justify-content:space-between; align-  
  items:center; gap: var(--col-gap);
```

```
  width:100%; max-width: var(--container-max-w);  
  margin:0 auto; height:100vh; padding:2vh 2vw;  
  box-sizing:border-box;  
}  
.column{ display:flex; flex-direction:column; align-  
  items:center; justify-content:center; }
```

```
#info{ font-family: Consolas, monospace; font-  
  size:1.5vw; text-align:center; padding:1vw; min-  
  width:20vw; max-width:40vw; color: var(--c-text);  
  pointer-events: none; }  
p{ font-size:2.2vw; margin:0.5vw 0; pointer-events:  
  none; }
```

```
.toggle{ display:flex; flex-direction:column; align-  
  items:center; margin-bottom:1.5vw; z-index: 10; }  
.toggle-label{ font-size:1.2vw; font-weight:700;  
  color: var(--c-text); margin-bottom: 0.5vw; text-  
  align: center; pointer-events: none; }
```

/* Switch Style */

```
.switch{  
  --w: var(--sw-w); --h: var(--sw-h); --pad: var(--  
  sw-pad);  
  width: var(--w); height: var(--h);  
  position:relative;  
  max-width:120px; max-height:56px; min-  
  width:70px; min-height:32px;  
  cursor: pointer;  
}  
.switch input{ display: none; }  
.switch .track{ position:absolute; inset:0; border-  
  radius: calc(var(--h)); background: var(--c-switch-  
  off); transition: background .22s; pointer-events:  
  none; }  
.switch .slider-round{ position:absolute; top:var(--  
  pad); left:var(--pad); width:calc(var(--h)*0.8);  
  height:calc(var(--h)*0.8); border-radius:50%;  
  background:#fff; box-shadow:0 0.25vw 0.6vw  
  rgba(0,0,0,.3); transition:left .22s; pointer-events:  
  none; }  
.switch.active .track{ background: var(--c-switch-  
  on); }  
.switch.active .slider-round{ left: calc(var(--w) -  
  var(--pad) - (var(--h)*0.8)); }
```

```
.control-rect{  
  background: var(--c-bg); border:2px solid var(--c-  
  card-bd); border-radius: var(--radius-card);  
  box-shadow: var(--shadow-card);  
  position:relative; display:flex; align-items:center;  
  justify-content:center; padding: var(--rect-pad);
```

```

}
.control-rect.vertical{ --rectHpx:150px; width:
var(--rect-vert-w); min-width: var(--rect-vert-min-
w); height: var(--rect-vert-h); min-height: var(--
rect-vert-min-h);}
.slider-vertical-wrap{ position:relative;
width:100%; height:100%; display:flex; align-
items:center; justify-content:center; }
.control-rect.horizontal{ width: var(--rect-horz-w);
min-width: var(--rect-horz-min-w); height: var(--
rect-horz-h); min-height: var(--rect-horz-min-h); }

/* CSS GỐC CHO iOS - GIỮ NGUYỄN */
input[type=range].vertical{-webkit-
appearance:none; appearance:none; width:
calc(var(--rectHpx) * 0.90) !important;
height:40px; transform:rotate(-90deg); transform-
origin:50% 50%; background:transparent;
outline:none; }
@supports (-webkit-appearance: slider-vertical){
input[type=range].vertical{ transform:none; -
webkit-appearance: slider-vertical; writing-mode:
bt-lr; width:40px !important; height: calc(var(--
rectHpx) * 0.90) !important; }
}
input[type=range].vertical::-webkit-slider-
runnable-track{ height: var(--slider-track-h);
background: var(--c-track); border-radius:999px; }
input[type=range].vertical::-webkit-slider-thumb{ -
webkit-appearance:none; width: var(--thumb-vert-
w); height: var(--thumb-vert-h); border-
radius:50%; background: var(--c-thumb-1); box-
shadow: var(--shadow-thumb); margin-top:0; }
input[type=range].vertical::-moz-range-track{
height: var(--slider-track-h); background: var(--c-
track); border-radius:999px; }
input[type=range].vertical::-moz-range-thumb{
width: var(--thumb-vert-w); height: var(--thumb-
vert-h); border:none; border-radius:50%;
background: var(--c-thumb-1); }

input[type=range].flat{-webkit-appearance:none;
appearance:none; width:90%; height:36px;
background:transparent; outline:none; }
input[type=range].flat::-webkit-slider-runnable-
track{ height: 80px; background: var(--c-track);
border-radius:999px; }
input[type=range].flat::-webkit-slider-thumb{ -
webkit-appearance:none; width: var(--thumb-horz-
w); height: var(--thumb-horz-h); border-
radius:50%; background: var(--c-thumb-2); box-
shadow: var(--shadow-thumb); margin-top:0; }

```

```

input[type=range].flat::-moz-range-track{ height:
80px; background: var(--c-track); border-
radius:999px; }
input[type=range].flat::-moz-range-thumb{ width:
var(--thumb-horz-w); height: var(--thumb-horz-h);
border:none; border-radius:50%; background:
var(--c-thumb-2); }

.disabled { filter: grayscale(0.5) opacity(0.6);
pointer-events: none; }

/* Vạch sóng WiFi */
.wifi-bars {
display: inline-flex;
gap: 2px;
align-items: flex-end;
margin-left: 4px;
}
.wifi-bars .bar {
width: 4px;
background-color: #aaa;
border-radius: 1px;
transition: background-color 0.1s;
}
.wifi-bars .bar:nth-child(1) { height: 3px; }
.wifi-bars .bar:nth-child(2) { height: 6px; }
.wifi-bars .bar:nth-child(3) { height: 9px; }
.wifi-bars .bar:nth-child(4) { height: 12px; }

@media (orientation:landscape){
.switch{ --w:26vh; --h:11vh; --pad:1.1vh; }
.toggle-label{ font-size:2.2vh; }
}
@media (orientation:portrait){
.switch{ --w:14vh; --h:8vh; --pad:0.6vh; }
.switch .slider-round{ width: calc(var(--h) * 0.86);
height: calc(var(--h) * 0.86); top: calc(var(--pad) -
0.3vh); }
.toggle-label{ font-size:1.8vh; }
}

/* ===== CSS RIÊNG CHO ANDROID
===== */
.android .control-rect.vertical {
height: 78vh;
min-height: 280px;
}
.android .slider-vertical-wrap {
display: flex;
align-items: center;
justify-content: center;
overflow: visible;
}

```

```

}

.android .control-rect.horizontal {
    width: 25vw;
    min-width: 250px;
    margin: 0 auto;
}

.android input[type=range].vertical {
    -webkit-appearance: none !important;
    appearance: none !important;
    width: calc(var(--rectHpx) - 40px) !important;
    height: 80px !important;
    background: transparent;
    position: absolute;
    transform: rotate(-90deg) !important;
    outline: none;
    margin: 0;
    padding: 0;
    cursor: pointer;
}

.android input[type=range].vertical::-webkit-
slider-runnable-track {
    height: 80px;
    background: var(--c-track);
    border-radius: 40px;
    border: none;
}

.android input[type=range].vertical::-webkit-
slider-thumb {
    -webkit-appearance: none;
    width: var(--thumb-vert-w);
    height: var(--thumb-vert-h);
    background: var(--c-thumb-l);
    border-radius: 50%;
    box-shadow: var(--shadow-thumb);
    margin-top: 0px !important;
}

.android #info {
    font-size: 1.2vw;
    max-width: 45vw;
    line-height: 1.3;
}

.android .container {
    height: calc(var(--vh, 1vh) * 100);
}

@media (max-width: 700px) {
    .android #info { font-size: 2vw; }
}

@media (max-width: 500px) {
    .android #info { font-size: 2.8vw; }
}

</style>

```

```

</head>

<body>
<div class="container">
    <div class="column">
        <div class="control-rect vertical">
            <div class="slider-vertical-wrap">
                <input type="range" id="dcSlider"
class="vertical" min="-0.70" max="0.70"
value="0.00" step="0.01">
            </div>
        </div>
    </div>

    <div class="column">
        <p style="text-
align:center;"><b>REMOTE</b></p>
        <div class="toggle">
            <div class="toggle-label">Autonomous</div>
            <div class="switch" id="swAuto">
                <input type="checkbox" id="autoToggle">
                <span class="track"></span>
                <span class="slider-round"></span>
            </div>
        </div>
        <div class="toggle">
            <div class="toggle-label">Cruise
Control</div>
            <div class="switch" id="swCruise">
                <input type="checkbox" id="cruiseToggle">
                <span class="track"></span>
                <span class="slider-round"></span>
            </div>
        </div>
        <div id="info">Loading...</div>
    </div>

    <div class="column">
        <div class="control-rect horizontal">
            <input type="range" id="servoSlider"
class="flat" min="-30" max="30" value="0"
step="1">
        </div>
    </div>
</div>

<script>
// Phát hiện trình duyệt
(function() {
    const isIOS =
/iPad|iPhone|iPod/.test(navigator.userAgent) &&
!window.MSStream;

```

```

const isSafari =
/^(?!chrome|android).*safari/i.test(navigator.userAgent);
if (isIOS || isSafari) {
    document.body.classList.add('ios');
} else {
    document.body.classList.add('android');
    function setVh() {
        let vh = window.innerHeight * 0.01;
        document.documentElement.style.setProperty('-vh', `${vh}px`);
    }
    window.addEventListener('resize', setVh);
    setVh();
}
})();

```

```

document.addEventListener('contextmenu', e =>
e.preventDefault());

```

```

const dcSlider =
document.getElementById('dcSlider');
const servoSlider =
document.getElementById('servoSlider');
const autoToggle =
document.getElementById('autoToggle');
const cruiseToggle =
document.getElementById('cruiseToggle');
const swAuto =
document.getElementById('swAuto');
const swCruise =
document.getElementById('swCruise');
const info = document.getElementById('info');

```

```

function syncDCSliderLength(){
    const rect = document.querySelector('.control-rect.vertical');
    if(rect) rect.style.setProperty('--rectHpx',
rect.clientHeight + 'px');
}
addEventListener('load', syncDCSliderLength);
addEventListener('resize', syncDCSliderLength);

```

```

const WHEEL_DIAM_M = 0.075;
const RPM2MS = Math.PI * WHEEL_DIAM_M /
60.0;
const SERVO_CENTER = 88;
const STEER_PER_SERVO = 30.0 / 47.0;
const DEAD_MPS = 0.02;
const HOLD_MPS = 0.00;

```

```

let ws;
// === Làm tròn dc đến 2 chữ số, sv thành số
nguyên ===
function sendCombined(dc, sv) {
    if (ws && ws.readyState === 1) {
        ws.send(`dc:${dc.toFixed(2)};sv:${sv.toFixed(0)}`);
        lastOutboundTime = Date.now();
    }
}

```

```

// Lưu giá trị đã gửi cuối cùng
let lastSentDC = 0, lastSentSV = 0;
let fastSendInterval = null;
let lastChangeTime = 0;
let lastOutboundTime = Date.now();

```

```

function sendIfChanged() {
    // Ưu tiên lấy giá trị thực tế đang hiển thị trên
    thanh trượt
    let targetDC = dcDrag ?
clampDC(+dcSlider.value) :
clampDC(rampCmdDC);
    let targetSV = svDrag ?
clampSV(+servoSlider.value) :
clampSV(rampCmdSV);

    if (Math.abs(targetDC) < DEAD_MPS) targetDC
= 0;

    // Kiểm tra thay đổi với ngưỡng nhạy hơn
    let dcChanged = Math.abs(targetDC -
lastSentDC) > 0.005;
    let svChanged = Math.abs(targetSV - lastSentSV)
> 0.5;

```

```

if (dcChanged || svChanged) {
    sendCombined(targetDC, targetSV);
    lastSentDC = targetDC;
    lastSentSV = targetSV;
    lastChangeTime = Date.now();
}
}

```

```

// === VÒNG LẶP: 30ms, heartbeat chỉ khi cần, tự
động dừng ===
function startFastSend() {
    if (fastSendInterval) return;
    fastSendInterval = setInterval(() => {
        const now = Date.now();

```

// 1. Gửi lệnh lái nếu có thay đổi

```

    sendIfChanged();

    // 2. Heartbeat chỉ gửi nếu trong 250ms không có
    // lệnh nào
    if (now - lastOutboundTime > 250) {
        if (ws && ws.readyState === 1) {
            ws.send('hb');
            lastOutboundTime = now;
        }
    }
}, 30); // 30ms (~33 fps)
}

function setSwitchState(swEl, checkbox, checked) {
    checkbox.checked = checked;
    if(checked) swEl.classList.add('active');
    else swEl.classList.remove('active');
}

function forceSyncVisuals() {
    if(autoToggle.checked &&
    !swAuto.classList.contains('active'))
    swAuto.classList.add('active');
    if(!autoToggle.checked &&
    swAuto.classList.contains('active'))
    swAuto.classList.remove('active');
    if(cruiseToggle.checked &&
    !swCruise.classList.contains('active'))
    swCruise.classList.add('active');
    if(!cruiseToggle.checked &&
    swCruise.classList.contains('active'))
    swCruise.classList.remove('active');
}

function updateUiByMode(isAuto){
    const vertRect =
    document.querySelector('.control-rect.vertical');
    const horzRect =
    document.querySelector('.control-rect.horizontal');
    if(isAuto){
        dcSlider.disabled = true;
        servoSlider.disabled = true;
        setSwitchState(swCruise, cruiseToggle, false);
        swCruise.style.pointerEvents = 'none';
        swCruise.style.opacity = '0.5';
        vertRect && vertRect.classList.add('disabled');
        horzRect && horzRect.classList.add('disabled');
    }else{
        dcSlider.disabled = false;
        servoSlider.disabled = false;
        swCruise.style.pointerEvents = 'auto';
        swCruise.style.opacity = '1.0';
    }
}

```

```

    vertRect &&
    vertRect.classList.remove('disabled');
    horzRect &&
    horzRect.classList.remove('disabled');
}

function wsConnect(){
    ws = new
    WebSocket(`ws://${location.hostname}:81/`);
    ws.onopen = () => {
        syncDCSliderLength();
        startFastSend(); // Kích hoạt vòng lặp quản lý
        ngay khi kết nối
    };
    ws.onclose = ()=>setTimeout(wsConnect, 1000);
    ws.onerror = ()=>ws.close();
    ws.onmessage = (evt)=>{
        const s = evt.data;
        if (!s || typeof s !== 'string' ||
        !s.startsWith('state:')) return;
        const kv =
        Object.fromEntries(s.slice(6).split(',').map(pair=>{
            const [k,v]=pair.split('='); return [k,v];
        }));
        const sv=parseFloat(kv.sv),
        rpm=parseFloat(kv.rpm), pwm=parseInt(kv.pwm),
        md=parseInt(kv.md||"0");

        if(!lastAutoInteraction) {
            const serverAuto = (md === 1);
            if(autoToggle.checked !== serverAuto) {
                setSwitchState(swAuto, autoToggle,
                serverAuto);
                updateUiByMode(serverAuto);
            }
        }
        forceSyncVisuals();

        const speedMs = isNaN(rpm) ? 0 : rpm *
        RPM2MS;
        const servoRel = (isNaN(sv)?0:(sv-
        SERVO_CENTER));
        const steerDeg = servoRel *
        STEER_PER_SERVO;

        let batVolt = '?V';
        if (kv.volt) {
            let voltNum = parseFloat(kv.volt);
            if (!isNaN(voltNum)) batVolt =
            voltNum.toFixed(2) + 'V';
        }
    }
}

```

```

let batPercent = (kv.bat !== undefined) ? kv.bat :
'?';

let wifiBarsHtml = "";
if (kv.rssi !== undefined) {
    let rssi = parseInt(kv.rssi);
    let level = 0;
    if (rssi > -50) level = 4;
    else if (rssi > -60) level = 3;
    else if (rssi > -70) level = 2;
    else if (rssi > -80) level = 1;
    else level = 0;
    let bars = "";
    for (let i = 0; i < 4; i++) {
        let color = (i < level) ? '#22c55e' : '#aaa';
        bars += `<div class="bar"
style="background:${color};"></div>`;
    }
    wifiBarsHtml = `<div class="wifi-
bars">${bars}</div>`;
} else {
    wifiBarsHtml = `<div class="wifi-bars"><div
class="bar"></div><div
class="bar"></div><div
class="bar"></div><div
class="bar"></div></div>`;
}

// Hàm nhỏ để chuyển đổi hiển thị -1 thành inf
const fmtDist = (v) => (parseFloat(v) < 0 ||
isNaN(v)) ? "inf" : v + " cm";

info.innerHTML = `Mode:
<b>${md===1?'AUTONOMOUS':'MANUAL'}</b>
<br>` +
    `WiFi: ${wifiBarsHtml}<br>` +
    `📶: ${batPercent}%<br>` +
    `Setpoint:
${(+dcSlider.value).toFixed(2)} m/s<br>` +
    `Speed: ${speedMs.toFixed(2)}
m/s<br>` +
    `DC PWM:
${isNaN(pwm)?0:pwm}<br>` +
    `Steer: ${steerDeg.toFixed(2)}°<br>`
+
    `Pothole: ${fmtDist(kv.us1)}<br>` +
    `Front: ${fmtDist(kv.us2)}<br>` +
    `Rear: ${fmtDist(kv.us3)}`;
};
}
wsConnect();

```

```

// === ĐÃ XÓA setInterval gửi hb cố định ===

// ===== ĐIỀU
KHIỂN RIÊNG DC & SV
=====

let dcDrag = false, svDrag = false;
let rampCmdDC = +dcSlider.value;
let rampCmdSV = +servoSlider.value;
let rampActiveDC = false, rampActiveSV = false;
let lastAutoInteraction = false;

function clampDC(x){ return Math.max(-0.70,
Math.min(0.70, x)); }
function clampSV(x){ return Math.max(-30,
Math.min(30, x)); }

function bindSliderPointer(slider, kind){
    slider.addEventListener('pointerdown', e=>{
        slider.setPointerCapture(e.pointerId);
        if(kind==='dc'){ dcDrag=true;
rampActiveDC=false; rampCmdDC =
clampDC(+dcSlider.value); }
        else { svDrag=true; rampActiveSV=false;
rampCmdSV = clampSV(+servoSlider.value); }
        sendIfChanged();
        startFastSend();
    });
    slider.addEventListener('pointermove', e=>{
        if(kind==='dc'){ rampCmdDC =
clampDC(+dcSlider.value); }
        else { rampCmdSV =
clampSV(+servoSlider.value); }
        lastChangeTime = Date.now();
        sendIfChanged();
    });
    slider.addEventListener('pointerup', e=>{
        if(kind==='dc'){
            dcDrag=false;
            if(cruiseToggle.checked) { rampActiveDC =
false; rampCmdDC = clampDC(+dcSlider.value); }
            else { rampActiveDC=true; dcSlider.value =
HOLD_MPS.toFixed(2); rampCmdDC =
HOLD_MPS; }
        }
        else {
            // Xử lý Servo khi buông tay
            svDrag = false;
            rampActiveSV = false;
            rampCmdSV = 0;
            servoSlider.value = "0";
        }
    }
}

```

```

    sendIfChanged();
  });
  slider.addEventListener('pointercancel', e=>{
    if(kind==='dc'){
      dcDrag=false;
      if(cruiseToggle.checked) { rampActiveDC =
false; rampCmdDC = clampDC(+dcSlider.value); }
      else { rampActiveDC=true; dcSlider.value =
"0.00"; }
    } else {
      svDrag = false;
      rampActiveSV = false;
      rampCmdSV = 0;
      servoSlider.value = "0";
    }
    sendIfChanged();
  });
}
bindSliderPointer(dcSlider,'dc');
bindSliderPointer(servoSlider,'sv');

function bindFastSwitch(swEl, checkbox,
onChange){
  swEl.addEventListener('pointerdown', (e) => {
    e.preventDefault(); e.stopPropagation();
    const newState = !checkbox.checked;
    setSwitchState(swEl, checkbox, newState);
    if(onChange) onChange(newState);
  });
  swEl.addEventListener('click', e => {
    e.preventDefault(); e.stopPropagation(); });
}

bindFastSwitch(swAuto, autoToggle, (state) => {
  lastAutoInteraction = true;
  updateUiByMode(state);
  rampActiveDC = false; rampActiveSV = false;
  rampCmdDC = 0; rampCmdSV = 0;
  dcSlider.value = "0.00"; servoSlider.value = "0";
  sendIfChanged();

  if (ws && ws.readyState === 1) {
    ws.send(`mode:${state ? 1 : 0}`);
  }

  setTimeout(() => lastAutoInteraction = false,
500);

```

```

});

bindFastSwitch(swCruise, cruiseToggle, (state) =>
{
  if(state) { rampActiveDC = false; rampCmdDC =
clampDC(+dcSlider.value); }
  else { if(!dcDrag) { rampActiveDC = true;
dcSlider.value = HOLD_MPS.toFixed(2);
rampCmdDC = HOLD_MPS; } }
  sendIfChanged();
});

// Ramp về 0 khi thả tay
setInterval(() => {
  if (autoToggle.checked) return;
  let changed = false;

  // --- Logic DC ---
  if (rampActiveDC && !dcDrag &&
!cruiseToggle.checked) {
    rampCmdDC = Math.abs(rampCmdDC -
HOLD_MPS) < 0.01 ? HOLD_MPS : rampCmdDC
+ (rampCmdDC > HOLD_MPS ? -0.04 : 0.04);
    dcSlider.value = rampCmdDC.toFixed(2);
    changed = true;
  }

  // --- Logic SERVO ---
  let targetSV = svDrag ?
clampSV(+servoSlider.value) : 0;
  if (rampCmdSV !== targetSV) {
    rampCmdSV = targetSV;
    if (!svDrag) servoSlider.value = "0"; // Ép thanh
trượt về 0 khi buông tay
    changed = true;
  }

  if (changed) {
    lastChangeTime = Date.now();
    sendIfChanged();
    if (!fastSendInterval) startFastSend();
  }
}, 50);
</script>
</body>
</html>
)rawliteral";

```

Phụ lục 2. Chương trình code trên Raspberry Pi 5 (Ngôn ngữ Python)

ai_thread.py

```
import threading
import time
import cv2
import numpy as np
import onnxruntime as ort
from config import (
    MODEL_PATH, FRAME_SIZE, INPUT_SIZE,
    MIN_AREA,
    NUM_SCAN_LINES, Y_START_RATIO,
    Y_END_RATIO, MAX_CENTER_SHIFT,
    FRONT_SAFE_DISTANCE
)
from utils import get_bird_eye_matrix, filter_noise,
compute_center_per_row, filter_outlier_centers
from serial_manager import SerialManager
from motion_control import MotionController
from ui_manager import UIManager

class AIThread(threading.Thread):
    def __init__(self, stop_event, shared_data):
        super().__init__()
        self.stop_event = stop_event
        self.shared = shared_data
        self.serial = SerialManager()
        self.motion = MotionController()
        self.ui = UIManager()

    def run(self):
        print("[AI] Khởi tạo AI model...")
        session =
ort.InferenceSession(MODEL_PATH,
providers=['CPUExecutionProvider'])
        M, M_inv = get_bird_eye_matrix()
        h, w = FRAME_SIZE[1], FRAME_SIZE[0]
        y_positions = np.linspace(int(h *
Y_START_RATIO), int(h * Y_END_RATIO),
NUM_SCAN_LINES, dtype=int)
        weights = np.linspace(1.0, 0.3,
NUM_SCAN_LINES)
        if not self.serial.connect():
            return
        self.ui.initialize()
        last_ai_time = 0
        min_interval = 1.0 / 30.0
        while not self.stop_event.is_set():
            # Đọc phản hồi từ ESP
            self.serial.read_feedback(self.shared)

            now = time.time()
            if now - last_ai_time < min_interval:
                time.sleep(0.001)
                continue
            last_ai_time = now
            # Lấy dữ liệu từ shared_data
            with self.shared['lock']:
                frame = self.shared['frame_orig']
                if frame is None:
                    continue
                speed = self.shared['speed']
                steer = self.shared['steer']
                front = self.shared['front_distance']
                auto_mode = self.shared['auto_mode']
            # Kiểm tra vật cản phía trước
            front_obstacle = False
            if not np.isinf(front) and not np.isnan(front):
                if 0.01 < front <
FRONT_SAFE_DISTANCE:
                    front_obstacle = True
            center_points = []
            final_speed = 0.0
            final_steer = 0.0

            if auto_mode and not front_obstacle:
                # Xử lý ảnh và AI
                img = cv2.resize(cv2.cvtColor(frame,
cv2.COLOR_BGR2RGB), INPUT_SIZE)
                img = (img.astype(np.float32) /
255.0).transpose(2, 0, 1)
                img = np.expand_dims(img, axis=0)
                outputs = session.run(None,
{session.get_inputs()[0].name: img})
                da_pred = np.argmax(outputs[0],
axis=1).squeeze().astype(np.uint8) * 255
                ll_pred = np.argmax(outputs[1],
axis=1).squeeze().astype(np.uint8) * 255

                da_mask = cv2.resize(da_pred,
FRAME_SIZE,
interpolation=cv2.INTER_NEAREST)
                ll_mask = cv2.resize(ll_pred,
FRAME_SIZE,
interpolation=cv2.INTER_NEAREST)

                clean_da = filter_noise(da_mask,
MIN_AREA)
```

```

        clean_ll = filter_noise(ll_mask,
MIN_AREA)

        has_da = np.sum(clean_da > 0) > 1000
        if not has_da:
            final_speed = 0.0
            final_steer = 0.0
            self.motion.boost_state = 0
            self.motion.stall_start_time = None
        else:
            warped_da =
cv2.warpPerspective(clean_da, M, FRAME_SIZE)
            warped_ll =
cv2.warpPerspective(clean_ll, M, FRAME_SIZE)

            centers = []
            valid_weights = []
            mid_w = warped_ll.shape[1] // 2
            center_points = []
            for idx, y in enumerate(y_positions):
                c, valid =
compute_center_per_row(y, warped_ll, warped_da,
mid_w)
                if valid:
                    centers.append(c)
                    center_points.append((int(c),
int(y)))

            valid_weights.append(weights[idx])

            if len(center_points) >= 2:
                center_points =
filter_outlier_centers(center_points,
MAX_CENTER_SHIFT)

            if len(center_points) == 0:
                final_speed = 0.0
                final_steer = 0.0
                self.motion.boost_state = 0
                self.motion.stall_start_time = None
            else:
                total_error = 0.0
                total_weight = 0.0
                for c, wgt in zip(centers,
valid_weights):
                    error = c - mid_w
                    total_error += error * wgt
                    total_weight += wgt
                current_error = total_error /
total_weight

```

```

        cmd_steer =
self.motion.compute_pid(current_error)
        cmd_speed =
self.motion.compute_speed_from_steer(cmd_steer)
        final_speed =
self.motion.update_boost(speed, cmd_speed,
cmd_steer, time.time())
        final_steer = cmd_steer

        # Ghi lại lệnh và trạng thái boost vào
shared_data
        with self.shared['lock']:
            self.shared['cmd_speed'] = final_speed
            self.shared['cmd_steer'] = final_steer
            self.shared['boost_state'] =
self.motion.get_boost_state()

            self.serial.send_command(final_speed,
final_steer)
        else:
            # Chế độ manual hoặc có vật cản
            self.serial.send_command(0.0, 0.0)
            with self.shared['lock']:
                self.shared['cmd_speed'] = 0.0
                self.shared['cmd_steer'] = 0.0
                self.shared['boost_state'] = 0
                self.motion.boost_state = 0
                self.motion.stall_start_time = None
                final_speed = 0.0
                final_steer = 0.0

            # Hiển thị
            battery = self.shared.get('battery', 0.0)
            boost_state = self.motion.get_boost_state()
            display_frame = self.ui.draw_info(frame,
auto_mode, center_points, M_inv,
speed, steer,
final_speed, final_steer,
front, battery,
boost_state)
            self.ui.show(display_frame)

            key = self.ui.get_key()
            if key == ord('q') or key == 27:
                self.stop_event.set()

            # Đọc lại feedback một lần nữa để cập nhật
nhanh
            self.serial.read_feedback(self.shared)

            self.serial.close()
            self.ui.destroy()

```

```

        print("[AI] Đã dùng")
bev_thread.py
import threading
import time
import cv2
import numpy as np
from datetime import datetime
from config import (
    FRAME_SIZE, BEV_W, BEV_TOP_H, BEV_H,
    DISPLAY_H,
    PIXEL_PER_METER, TRUC_SAU_Y_PIXEL
)
from utils import get_bev_matrix

class BEVThread(threading.Thread):
    def __init__(self, stop_event, shared_data):
        super().__init__()
        self.stop_event = stop_event
        self.shared = shared_data
        self.M_bev = get_bev_matrix()
        self.bev_w = BEV_W
        self.bev_h = BEV_H
        self.prev_bev = None
        self.prev_time = None
        self.L = 0.34
        self.accum_psi = 0.0
        self.axle_center = (BEV_W // 2,
            TRUC_SAU_Y_PIXEL)
        print(f"[BEV] Tâm xoay trục sau:
{self.axle_center} (pixel) (trong ảnh nội bộ
800px)")

    def compute_delta_pose(self, speed_mps,
        steer_deg, dt):
        if dt <= 0:
            return 0.0, 0.0
        steer_rad = np.deg2rad(steer_deg)
        if abs(steer_rad) < 1e-6:
            delta_psi = 0.0
        else:
            delta_psi = speed_mps * np.tan(steer_rad) /
self.L * dt
            dy = speed_mps * dt
            return delta_psi, dy

    def warp_prev_bev(self, prev_bev, delta_psi_rad,
        dy_meter):
        if prev_bev is None:
            return None
        h, w = prev_bev.shape[:2]
        angle_deg = -np.rad2deg(delta_psi_rad)

```

```

        M =
cv2.getRotationMatrix2D(self.axle_center,
        angle_deg, 1.0)
        dy_pixel = dy_meter * PIXEL_PER_METER
        M[1, 2] += dy_pixel
        warped = cv2.warpAffine(prev_bev, M, (w, h),

borderMode=cv2.BORDER_CONSTANT,
        borderValue=0)
        return warped

    def run(self):
        print("[BEV] Bắt đầu luồng BEV (nội bộ
800px, hiển thị 600px, tâm trục sau)...")
        cap = cv2.VideoCapture(0)
        cap.set(cv2.CAP_PROP_FRAME_WIDTH,
            FRAME_SIZE[0])
        cap.set(cv2.CAP_PROP_FRAME_HEIGHT,
            FRAME_SIZE[1])
        min_interval = 1.0 / 30.0 #
MAX_FPS_PROCESS
        last_process_time = 0
        while not self.stop_event.is_set():
            ret, frame = cap.read()
            if not ret:
                continue
            now = time.time()
            if now - last_process_time < min_interval:
                continue
            last_process_time = now
            with self.shared['lock']:
                speed = self.shared['speed']
                steer = self.shared['steer']
                self.shared['frame_orig'] = frame.copy()
                top_bev = cv2.warpPerspective(frame,
self.M_bev, (BEV_W, BEV_TOP_H))
                if self.prev_bev is not None and
self.prev_time is not None:
                    dt = now - self.prev_time
                    if dt > 0:
                        delta_psi, dy =
self.compute_delta_pose(speed, steer, dt)
                        self.accum_psi += delta_psi
                        warped_full =
self.warp_prev_bev(self.prev_bev, delta_psi, dy)
                        if warped_full is not None:
                            bev_full = warped_full
                        else:
                            bev_full = np.zeros((BEV_H,
BEV_W, 3), dtype=np.uint8)
                        else:
                            bev_full = self.prev_bev.copy()

```

```

else:
    bev_full = np.zeros((BEV_H, BEV_W, 3),
dtype=np.uint8)
    self.accum_psi = 0.0
    bev_full[0:BEV_TOP_H, :] = top_bev
    self.prev_bev = bev_full.copy()
    self.prev_time = now
    bev_display = bev_full[:DISPLAY_H, :]
    with self.shared['lock']:
        self.shared['bev_frame'] = bev_display
        self.shared['yaw_angle_deg'] =
np.rad2deg(self.accum_psi)
        metadata = {
            "timestamp":
datetime.now().strftime("%Y%m%d_%H%M%S"),
            "gps": {
                "latitude": self.shared.get('lat', 0.0),
                "longitude": self.shared.get('lon',
0.0),
                "valid": self.shared.get('gps_valid',
False)
            },
            "sensors": {
                "pothole_depth_cm":
self.shared.get('pothole_depth', 0.0),
                "speed_mps": speed,
                "steering_angle_deg": steer,
                "battery_percent":
self.shared.get('battery', 0.0),
                "front_distance_cm":
self.shared.get('front_distance', -1),
                "rear_distance_cm":
self.shared.get('rear_distance', -1),
                "yaw_angle_deg":
np.rad2deg(self.accum_psi)
            },
            "system": {
                "auto_mode":
self.shared.get('auto_mode', False),
                "frame_width": BEV_W,
                "frame_height": DISPLAY_H
            }
        }
        self.shared['metadata'] = metadata
    cap.release()
    print("[BEV] Đã dùng")
config.py
import os
BASE_DIR =
os.path.dirname(os.path.abspath(__file__))
MODEL_PATH = os.path.join(BASE_DIR,
"model_249p.onnx")

```

```

SERIAL_PORT = "/dev/ttyAMA0"
BAUD_RATE = 115200

INPUT_SIZE = (426, 240)
FRAME_SIZE = (640, 480)

MIN_AREA = 1200
MAX_SPEED = 0.15
MIN_SPEED = 0.1
MAX_STEER = 30.0
KP = 0.35
KD = 0.12
NUM_SCAN_LINES = 10
Y_START_RATIO = 0.95
Y_END_RATIO = 0.80
MAX_CENTER_SHIFT = 35

FRONT_SAFE_DISTANCE = 15.0
STEER_SLOW_THRESHOLD = 15.0

LAPTOP_IP = "192.168.137.1"
VIDEO_PORT = 5557
MAX_FPS_SEND = 15
MAX_FPS_PROCESS = 30

BEV_W = 400
BEV_H = 800
BEV_TOP_H = 400
BEV_BOTTOM_H = 400
PIXEL_PER_METER = 666.67

TRUC_SAU_DIST_CM = 50.5
TRUC_SAU_Y_PIXEL = BEV_TOP_H +
int((TRUC_SAU_DIST_CM / 100.0) *
PIXEL_PER_METER)

DISPLAY_H = 600

COLOR_DRIVABLE = (0, 255, 0)
COLOR_LANE = (255, 0, 0)
COLOR_CENTER_PATH = (0, 255, 255)
COLOR_CENTER_POINT = (0, 0, 255)
COLOR_DA_POLY = (0, 255, 0)

# === Boost / Stall parameters ===
BOOST_FORWARD_DURATION = 1.0
BOOST_BACKWARD_DURATION = 0.1
BOOST_FORWARD_SPEED = 0.70
BOOST_BACKWARD_SPEED = -0.1
SPEED_THRESHOLD_MOVING = 0.05
STALL_DELAY = 10.0
MAX_STEER_FOR_BOOST = 15

```

motion_control.py

```
import numpy as np
import time
from config import (
    KP, KD, MAX_STEER, MIN_SPEED,
    MAX_SPEED, STEER_SLOW_THRESHOLD,
    BOOST_FORWARD_DURATION,
    BOOST_BACKWARD_DURATION,
    BOOST_FORWARD_SPEED,
    BOOST_BACKWARD_SPEED,
    SPEED_THRESHOLD_MOVING, STALL_DELAY,
    MAX_STEER_FOR_BOOST
)

class MotionController:
    def __init__(self):
        self.last_error = 0.0
        self.boost_state = 0      # 0: normal, 1: boost
        self.boost_start_time = 0.0
        self.stall_start_time = None
        self.ramp_start_speed = 0.0
        self.ramp_target_speed = 0.0

    def compute_pid(self, current_error):
        """Tính góc lái từ sai số (PD)"""
        cmd_steer = KP * current_error + KD *
        (current_error - self.last_error)
        cmd_steer = np.clip(cmd_steer, -
        MAX_STEER, MAX_STEER)
        self.last_error = current_error
        return cmd_steer

    def compute_speed_from_steer(self, cmd_steer):
        """Tính tốc độ mong muốn dựa trên góc lái
        (càng cua chậm càng chậm)"""
        if abs(cmd_steer) >
        STEER_SLOW_THRESHOLD:
            speed_factor = 1 - ((abs(cmd_steer) -
            STEER_SLOW_THRESHOLD) / (MAX_STEER -
            STEER_SLOW_THRESHOLD))
            cmd_speed = MIN_SPEED +
            (MAX_SPEED - MIN_SPEED) * (speed_factor **
            2)
        else:
            cmd_speed = MAX_SPEED
        return cmd_speed

    def update_boost(self, actual_speed, cmd_speed,
        cmd_steer, current_time):
        """
```

Cập nhật trạng thái boost dựa trên vận tốc thực và lệnh mong muốn.

```
Trả về: final_speed (float)
"""
    need_move = (abs(actual_speed) <
    SPEED_THRESHOLD_MOVING and
    cmd_speed > 0.1 and
    abs(cmd_steer) <=
    MAX_STEER_FOR_BOOST)

    # Nếu đang boost nhưng góc lái quá lớn ->
    thoát boost
    if self.boost_state != 0 and abs(cmd_steer) >
    MAX_STEER_FOR_BOOST:
        self.boost_state = 0
        self.stall_start_time = None

    if need_move:
        if self.stall_start_time is None:
            self.stall_start_time = current_time
        elif (current_time - self.stall_start_time) >=
        STALL_DELAY:
            if self.boost_state == 0:
                self.boost_state = 1
                self.boost_start_time = current_time
                print("[BOOST] Start boost forward")
            else:
                self.stall_start_time = None
                if self.boost_state != 0 and self.boost_state
                != 3:
                    if self.boost_state == 1:
                        self.boost_state = 3
                        self.ramp_start_speed =
                        BOOST_FORWARD_SPEED
                        self.ramp_target_speed = cmd_speed
                        self.boost_start_time = current_time
                    elif self.boost_state == 2:
                        self.boost_state = 3
                        self.ramp_start_speed =
                        abs(BOOST_BACKWARD_SPEED)
                        self.ramp_target_speed = cmd_speed
                        self.boost_start_time = current_time
                    else:
                        self.boost_state = 0

    # Tính final_speed dựa trên trạng thái boost
    if self.boost_state == 0:
        final_speed = cmd_speed
    elif self.boost_state == 1:
        elapsed = current_time -
        self.boost_start_time
```

```

        if elapsed >=
BOOST_FORWARD_DURATION:
        self.boost_state = 2
        self.boost_start_time = current_time
        final_speed =
BOOST_BACKWARD_SPEED
        else:
        final_speed =
BOOST_FORWARD_SPEED
        elif self.boost_state == 2:
        elapsed = current_time -
self.boost_start_time
        if elapsed >=
BOOST_BACKWARD_DURATION:
        self.boost_state = 1
        self.boost_start_time = current_time
        final_speed =
BOOST_FORWARD_SPEED
        else:
        final_speed =
BOOST_BACKWARD_SPEED
        elif self.boost_state == 3:
        elapsed = current_time -
self.boost_start_time
        speed_decrement = (elapsed / 0.05) * 0.01
# ramp down logic giữ nguyên
        new_speed = self.ramp_start_speed -
speed_decrement
        if new_speed <= self.ramp_target_speed:
        self.boost_state = 0
        final_speed = self.ramp_target_speed
        else:
        final_speed = new_speed
        else:
        final_speed = cmd_speed
        return final_speed

def get_boost_state(self):
        return self.boost_state
serial_manager.py
import serial
import re
import time
from config import SERIAL_PORT, BAUD_RATE

class SerialManager:
    def __init__(self):
        self.ser = None
        self.connected = False

    def connect(self):
        """Kết nối serial và gửi lệnh dừng ban đầu"""

```

```

        try:
            self.ser = serial.Serial(SERIAL_PORT,
BAUD_RATE, timeout=0.01)
            print(f"[Serial] UART OK
{SERIAL_PORT}")
            self.send_command(0.0, 0.0)
            time.sleep(0.5)
            self.connected = True
            return True
        except Exception as e:
            print(f"[Serial] Lỗi UART: {e}")
            self.connected = False
            return False

def send_command(self, speed, steer):
        """Gửi lệnh Sxx.xxAxx.xx\r\n xuống ESP"""
        if self.ser:

self.ser.write(f"S{speed:.2f}A{steer:.2f}\r\n".encode
())

def read_feedback(self, shared_data):
        """Đọc dữ liệu từ ESP và cập nhật
shared_data (có lock)"""
        if not self.ser or not self.ser.in_waiting:
            return
        line = self.ser.readline().decode('utf-8',
errors='ignore').strip()
        if line.startswith('FB:'):
            m_match = re.search(r'M=(\d)', line)
            a_match = re.search(r'A=([+-]?\d+\.?\d*)',
line)
            s_match = re.search(r'S=([+-]?\d+\.?\d*)',
line)
            f_match = re.search(r'F=([+-
]?(?:\d+(?:\.\d+)?)|inf|INF))', line,
re.IGNORECASE)
            r_match = re.search(r'R=([+-
]?(?:\d+(?:\.\d+)?)|inf|INF))', line,
re.IGNORECASE)
            b_match = re.search(r'B=(\d+\.?\d*)', line)
            p_match = re.search(r'P=([+-
]?(?:\d+(?:\.\d+)?)|inf|INF))', line,
re.IGNORECASE)
            with shared_data['lock']:
                if m_match:
                    shared_data['auto_mode'] =
(m_match.group(1) == '1')
                if s_match:
                    shared_data['speed'] =
float(s_match.group(1))
                if a_match:

```

```

        shared_data['steer'] =
float(a_match.group(1))
    if f_match:
        val = f_match.group(1).lower()
        shared_data['front_distance'] =
float('inf') if val == 'inf' else float(val)
    if r_match:
        val = r_match.group(1).lower()
        shared_data['rear_distance'] =
float('inf') if val == 'inf' else float(val)
    if b_match:
        shared_data['battery'] =
float(b_match.group(1))
    if p_match:
        val = p_match.group(1).lower()
        shared_data['pothole_depth'] =
float('inf') if val == 'inf' else float(val)

def close(self):
    if self.ser:
        self.ser.close()
    self.connected = False
telemetry.py
import threading
import time
import serial
import pynmea2
from config import SERIAL_PORT

class TelemetryThread(threading.Thread):
    def __init__(self, stop_event, shared_data):
        super().__init__()
        self.stop_event = stop_event
        self.shared = shared_data

    def run(self):
        print("[Telemetry] Đã tắt đọc ESP (chỉ đọc
GPS nếu có)...")
        try:
            ser_gps = serial.Serial("/dev/ttyACM0",
baudrate=9600, timeout=1)
            print("[Telemetry] GPS OK")
        except Exception as e:
            print(f"[Telemetry] Lỗi GPS: {e}")
            ser_gps = None

        while not self.stop_event.is_set():
            if ser_gps and ser_gps.in_waiting > 0:
                line = ser_gps.readline().decode('ascii',
errors='ignore')
                if line.startswith('$GPGGA') or
line.startswith('$GNGGA'):

```

```

            try:
                msg = pynmea2.parse(line)
                if msg.latitude != 0.0 and
msg.longitude != 0.0:
                    with self.shared['lock']:
                        self.shared['lat'] = msg.latitude
                        self.shared['lon'] =
msg.longitude
                    self.shared['gps_valid'] = True
            else:
                with self.shared['lock']:
                    self.shared['gps_valid'] = False
        except:
            pass
        time.sleep(0.005)

    if ser_gps:
        ser_gps.close()
    print("[Telemetry] Đã dừng")
udp_sender.py
import threading
import time
import socket
import struct
import cv2
import json
import random
from config import LAPTOP_IP, VIDEO_PORT,
MAX_FPS_SEND

class UDPSenderThread(threading.Thread):
    def __init__(self, stop_event, shared_data):
        super().__init__()
        self.stop_event = stop_event
        self.shared = shared_data
        self.sock = None
        self.addr = (LAPTOP_IP, VIDEO_PORT)
        self.CHUNK_SIZE = 60 * 1024

    def connect(self):
        try:
            self.sock = socket.socket(socket.AF_INET,
socket.SOCK_DGRAM)
            self.sock.settimeout(None)
            print(f"[UDP] Đã tạo socket đến
{LAPTOP_IP}:{VIDEO_PORT}")
            return True
        except Exception as e:
            print(f"[UDP] Lỗi socket: {e}")
            return False

    def send_frame(self, frame, metadata):

```

```

if self.sock is None:
    if not self.connect():
        return False
    try:
        encode_param =
[int(cv2.IMWRITE_JPEG_QUALITY), 85]
        _, frame_encoded = cv2.imencode('.jpg',
frame, encode_param)
        frame_data = frame_encoded.tobytes()
        metadata_json =
json.dumps(metadata).encode('utf-8')
        meta_len = struct.pack('!I',
len(metadata_json))
        frame_len = struct.pack('!I',
len(frame_data))
        total_payload = meta_len + metadata_json
+ frame_len + frame_data
        total_len = len(total_payload)
        num_chunks = (total_len +
self.CHUNK_SIZE - 1) // self.CHUNK_SIZE
        frame_id = random.randint(0, 65535)

        for i in range(num_chunks):
            start = i * self.CHUNK_SIZE
            end = min(start + self.CHUNK_SIZE,
total_len)
            chunk_data = total_payload[start:end]
            header = struct.pack('!HHH', frame_id,
num_chunks, i)
            packet = header + chunk_data
            self.sock.sendto(packet, self.addr)
        return True
    except Exception as e:
        print(f"[UDP] Lỗi gửi: {e}")
        return False

def run(self):
    print("[UDP] Bắt đầu gửi BEV (15fps)...")
    send_interval = 1.0 / MAX_FPS_SEND
    last_send = 0

    while not self.stop_event.is_set():
        now = time.time()
        if now - last_send >= send_interval:
            with self.shared['lock']:
                frame = self.shared.get('bev_frame')
                metadata = self.shared.get('metadata')
                if frame is not None and metadata is not
None:
                    self.send_frame(frame, metadata)
                    last_send = now
            else:

```

```

time.sleep(0.001)

if self.sock:
    self.sock.close()
    print("[UDP] Đã dừng")

ui_manager.py
import cv2
import subprocess
import numpy as np
from utils import draw_center_path_on_frame

class UIManager:
    def __init__(self, window_name="Robot
Control"):
        self.window_name = window_name
        self.first_frame = True

    def initialize(self):
        """Tạo cửa sổ OpenCV"""
        cv2.namedWindow(self.window_name,
cv2.WINDOW_NORMAL)

        def draw_info(self, frame, auto_mode,
center_points, M_inv, speed, steer,
final_speed, final_steer, front_distance,
battery, boost_state):
            """Vẽ tất cả thông tin lên frame"""
            display = frame.copy()
            if auto_mode and center_points and
len(center_points) >= 2:
                display =
draw_center_path_on_frame(display,
center_points, M_inv)
                mode_text = "AUTO MODE" if auto_mode
else "MANUAL MODE"
                front_str = f"Front: {front_distance:.1f} cm" if
not np.isinf(front_distance) else "Front: ---"
                info_lines = [
                    f"Mode: {mode_text}",
                    f"Speed: {speed:.2f} m/s",
                    f"Steer: {steer:.1f} deg",
                    f"Cmd Speed: {final_speed:.2f} | Cmd
Steer: {final_steer:.1f}",
                    front_str,
                    f"Battery: {battery:.1f}%",
                    f"Boost: {boost_state}"
                ]
                y_offset = 30
                for i, line in enumerate(info_lines):
                    cv2.putText(display, line, (10, y_offset +
i*25),

```

```

        cv2.FONT_HERSHEY_SIMPLEX,
        0.5, (0, 0, 255), 1, cv2.LINE_AA)
    return display

def show(self, frame):
    """Hiển thị frame và xử lý fullscreen lần
    đầu"""
    cv2.imshow(self.window_name, frame)
    if self.first_frame:
        cv2.waitKey(500) # đợi cửa sổ hiện ra
    try:
        subprocess.Popen(['wmctrl', '-r',
            self.window_name, '-b', 'add,fullscreen'])
        print("[UI] Đã gửi lệnh fullscreen qua
        wmctrl")
    except Exception as e:
        print(f"[UI] Không thể dùng wmctrl: {e},
        thử lại cv2 fullscreen")
    cv2.setWindowProperty(self.window_name,
        cv2.WND_PROP_FULLSCREEN,
        cv2.WINDOW_FULLSCREEN)
    self.first_frame = False

def get_key(self):
    """Lấy phím nhấn từ bàn phím (non-
    blocking)"""
    return cv2.waitKey(1) & 0xFF

def destroy(self):
    cv2.destroyAllWindows()

utils.py
import cv2
import numpy as np
from config import (
    FRAME_SIZE, BEV_W, BEV_TOP_H,
    COLOR_DRIVABLE, COLOR_LANE,
    COLOR_CENTER_PATH,
    COLOR_CENTER_POINT, COLOR_DA_POLY,
    MAX_CENTER_SHIFT
)

def get_bird_eye_matrix():
    src = np.float32([[114, 27], [526, 27], [640,
    417], [0, 417]])
    dst = np.float32([[120, 0], [520, 0], [520, 400],
    [120, 400]])
    M = cv2.getPerspectiveTransform(src, dst)
    M_inv = cv2.getPerspectiveTransform(dst, src)
    return M, M_inv

def get_bev_matrix():

```

```

    src = np.float32([[114, 27], [526, 27], [640,
    417], [0, 417]])
    dst = np.float32([[0, 0], [BEV_W, 0], [BEV_W,
    BEV_TOP_H], [0, BEV_TOP_H]])
    return cv2.getPerspectiveTransform(src, dst)

def filter_noise(mask, min_area):
    clean_mask = np.zeros_like(mask)
    contours, _ = cv2.findContours(mask,
    cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE)
    for cnt in contours:
        if cv2.contourArea(cnt) > min_area:
            cv2.drawContours(clean_mask, [cnt], -1,
            255, -1)
    return clean_mask

def draw_drivable_and_lane(frame,
    da_mask_warped, ll_mask, M_inv):
    overlay = frame.copy()
    h_warp, w_warp = da_mask_warped.shape
    left_pts = []
    right_pts = []
    for y in range(0, h_warp, 5):
        row = da_mask_warped[y, :]
        indices = np.where(row == 255)[0]
        if len(indices) > 0:
            left_pts.append([indices[0], y])
            right_pts.append([indices[-1], y])
    if len(left_pts) > 0 and len(right_pts) > 0:
        left_pts = np.array(left_pts,
        dtype=np.float32).reshape(-1, 1, 2)
        right_pts = np.array(right_pts,
        dtype=np.float32).reshape(-1, 1, 2)
        left_warped =
        cv2.perspectiveTransform(left_pts,
        M_inv).reshape(-1, 2).astype(np.int32)
        right_warped =
        cv2.perspectiveTransform(right_pts,
        M_inv).reshape(-1, 2).astype(np.int32)
        polygon = np.vstack((left_warped,
        right_warped[:-1]))
        cv2.fillPoly(overlay, [polygon],
        COLOR_DA_POLY)
        cv2.addWeighted(overlay, 0.4, frame, 0.6, 0,
        frame)
        if ll_mask.shape != frame.shape[:2]:
            ll_mask = cv2.resize(ll_mask,
            (frame.shape[1], frame.shape[0]),
            interpolation=cv2.INTER_NEAREST)
        frame[ll_mask == 255] = COLOR_LANE
    return frame

```

```

def compute_center_per_row(y, warped_ll,
warped_da, mid_w):
    ll_row = warped_ll[y, :]
    da_row = warped_da[y, :]
    ll_indices = np.where(ll_row == 255)[0]
    da_indices = np.where(da_row == 255)[0]
    if len(da_indices) == 0:
        return None, False
    x_da_left = da_indices[0]
    x_da_right = da_indices[-1]
    center_da = (x_da_left + x_da_right) / 2.0
    ll_left_side = ll_indices[ll_indices < center_da]
    ll_right_side = ll_indices[ll_indices >
center_da]
    has_left_ll = len(ll_left_side) > 0
    has_right_ll = len(ll_right_side) > 0
    if has_left_ll and has_right_ll:
        x_tim = (np.mean(ll_left_side) +
np.mean(ll_right_side)) / 2.0
    elif has_left_ll:
        x_tim = (np.mean(ll_left_side) + x_da_right) /
2.0
    elif has_right_ll:
        x_tim = (x_da_left + np.mean(ll_right_side)) /
2.0
    else:
        x_tim = center_da
    return x_tim, True

def filter_outlier_centers(center_points, max_shift):
    if len(center_points) < 2:
        return center_points
    x_vals = np.array([p[0] for p in center_points],
dtype=np.float32)
    y_vals = np.array([p[1] for p in center_points],
dtype=np.float32)
    valid_mask = np.ones(len(x_vals), dtype=bool)
    for i in range(1, len(x_vals)):
        if abs(x_vals[i] - x_vals[i-1]) > max_shift:
            valid_mask[i] = False
    if not np.all(valid_mask):
        valid_indices = np.where(valid_mask)[0]
        if len(valid_indices) >= 2:
            x_interp =
np.interp(np.arange(len(x_vals)), valid_indices,
x_vals[valid_indices])
            return [(int(x_interp[i]), int(y_vals[i])) for i
in range(len(x_vals))]
        elif len(valid_indices) == 1:
            return [center_points[valid_indices[0]]]
    else:

```

```

        return []
    return center_points
def draw_center_path_on_frame(frame,
center_points_warped, M_inv):
    if len(center_points_warped) < 2:
        return frame
    pts_src = np.array(center_points_warped,
dtype=np.float32).reshape(-1,1,2)
    pts_dst = cv2.perspectiveTransform(pts_src,
M_inv).reshape(-1,2).astype(np.int32)
    cv2.polylines(frame, [pts_dst], False,
COLOR_CENTER_PATH, 3)
    for pt in pts_dst:
        cv2.circle(frame, tuple(pt), 5,
COLOR_CENTER_POINT, -1)
    return frame
main.py
# main.py
import threading
import time
import cv2
from ai_thread import AIThread
from bev_thread import BEVThread
from udp_sender import UDPSenderThread
from telemetry import TelemetryThread

def main():
    stop_event = threading.Event()
    shared = {
        'lock': threading.Lock(),
        'frame_orig': None,
        'bev_frame': None,
        'metadata': None,
        'speed': 0.0,
        'steer': 0.0,
        'auto_mode': False,
        'front_distance': float('inf'),
        'rear_distance': float('inf'),
        'pothole_depth': 0.0,
        'battery': 0.0,
        'lat': 0.0,
        'lon': 0.0,
        'gps_valid': False,
        'cmd_speed': 0.0,
        'cmd_steer': 0.0,
        'boost_state': 0,
        'yaw_angle_deg': 0.0,
    }

    tele_thread = TelemetryThread(stop_event,
shared)
    bev_thread = BEVThread(stop_event, shared)

```

```
ai_thread = AIThread(stop_event, shared)
udp_thread = UDPSenderThread(stop_event,
shared)
```

```
tele_thread.start()
```

```
bev_thread.start()
```

```
ai_thread.start()
```

```
udp_thread.start()
```

```
try:
```

```
    while not stop_event.is_set():
```

```
        time.sleep(0.1)
```

```
except KeyboardInterrupt:
```

```
    print("Nhận tín hiệu dừng...")
```

```
finally:
```

```
    stop_event.set()
```

```
    tele_thread.join(timeout=2)
```

```
    bev_thread.join(timeout=2)
```

```
    ai_thread.join(timeout=2)
```

```
    udp_thread.join(timeout=2)
```

```
    cv2.destroyAllWindows()
```

```
    print("Hệ thống đã dừng hoàn toàn.")
```

```
if __name__ == "__main__":
```

```
    main()
```

Phụ lục 3. Chương trình code nhận diện ổ gà và lưu trữ trên PC (Ngôn ngữ Python)

database.py

import sqlite3

import os

class DatabaseManager:

def __init__(self,

db_path="database/potholes.db"):

self.db_path = db_path

os.makedirs(os.path.dirname(self.db_path),

exist_ok=True)

self.init_database()

def init_database(self):

conn = sqlite3.connect(self.db_path)

cursor = conn.cursor()

cursor.execute("""

CREATE TABLE IF NOT EXISTS

pothole_records (

id INTEGER PRIMARY KEY

AUTOINCREMENT,

track_id INTEGER,

timestamp TEXT NOT NULL,

area_cm2 REAL NOT NULL,

depth_cm REAL NOT NULL,

volume_cm3 REAL NOT NULL,

gps_lat REAL NOT NULL,

gps_lon REAL NOT NULL,

image_path TEXT NOT NULL

)

""")

cursor.execute("PRAGMA

table_info(pothole_records)")

columns = [col[1] for col in cursor.fetchall()]

if 'volume_cm3' not in columns:

cursor.execute("ALTER TABLE

pothole_records ADD COLUMN volume_cm3

REAL DEFAULT 0")

conn.commit()

conn.close()

def load_all_records(self):

conn = sqlite3.connect(self.db_path)

cursor = conn.cursor()

cursor.execute("""

SELECT id, track_id, timestamp, area_cm2,

depth_cm, volume_cm3, gps_lat, gps_lon,

image_path

FROM pothole_records ORDER BY id

DESC

""")

rows = cursor.fetchall()

conn.close()

return rows

def get_pothole_by_id(self, pothole_id):

conn = sqlite3.connect(self.db_path)

cursor = conn.cursor()

Ép kiểu an toàn

try:

pothole_id = int(pothole_id)

except:

return None

cursor.execute("""

SELECT id, area_cm2, depth_cm,

volume_cm3, timestamp, gps_lat, gps_lon,

image_path

FROM pothole_records WHERE id = ?

""', (pothole_id,))

row = cursor.fetchone()

conn.close()

return row

def get_group_by_gps(self, lat, lon,

tolerance=0.000001):

conn = sqlite3.connect(self.db_path)

cursor = conn.cursor()

cursor.execute("""

SELECT id, area_cm2, depth_cm,

volume_cm3, timestamp, gps_lat, gps_lon,

image_path

FROM pothole_records

WHERE ABS(gps_lat - ?) < ? AND

ABS(gps_lon - ?) < ?

ORDER BY id

""', (lat, tolerance, lon, tolerance))

rows = cursor.fetchall()

conn.close()

return rows

def get_all_map_records(self):

conn = sqlite3.connect(self.db_path)

cursor = conn.cursor()

cursor.execute("""

SELECT id, gps_lat, gps_lon

```

        FROM pothole_records WHERE gps_lat !=
0 AND gps_lon != 0 ORDER BY id DESC
    """
    rows = cursor.fetchall()
    conn.close()
    return rows

```

```

def get_records_in_bbox(self, lat_min, lat_max,
lon_min, lon_max, limit=None):
    """

```

Trả về danh sách các tuple (id, gps_lat, gps_lon) của ổ gà nằm trong bounding box.

```

    """
    conn = sqlite3.connect(self.db_path)
    cursor = conn.cursor()
    query = """
        SELECT id, gps_lat, gps_lon FROM
        pothole_records
        WHERE gps_lat BETWEEN ? AND ?
        AND gps_lon BETWEEN ? AND ?
        AND gps_lat != 0 AND gps_lon != 0
    """

```

```

    params = [lat_min, lat_max, lon_min,
lon_max]

```

```

    if limit:
        query += " LIMIT ?"
        params.append(limit)
    cursor.execute(query, params)
    rows = cursor.fetchall()
    conn.close()
    return rows

```

TwinLite.py

```

import torch
import torch.nn as nn
from torch.nn import Module, Conv2d, Parameter,
Softmax

```

```

class PAM_Module(Module):
    """ Position attention module"""
    #Ref from SAGAN
    def __init__(self, in_dim):
        super(PAM_Module, self).__init__()
        self.chanel_in = in_dim
        self.query_conv =
Conv2d(in_channels=in_dim,
out_channels=in_dim//8, kernel_size=1)
        self.key_conv = Conv2d(in_channels=in_dim,
out_channels=in_dim//8, kernel_size=1)
        self.value_conv =
Conv2d(in_channels=in_dim,
out_channels=in_dim, kernel_size=1)
        self.gamma = Parameter(torch.zeros(1))

```

```

self.softmax = Softmax(dim=-1)

```

```

def forward(self, x):
    """

```

```

    inputs :
        x : input feature maps( B X C X H X W)
    returns :

```

```

        out : attention value + input feature
        attention: B X (HxW) X (HxW)
    """

```

```

    m_batchsize, C, height, width = x.size()
    proj_query =
self.query_conv(x).view(m_batchsize, -1,
width*height).permute(0, 2, 1)
    proj_key = self.key_conv(x).view(m_batchsize,
-1, width*height)
    energy = torch.bmm(proj_query, proj_key)
    attention = self.softmax(energy)
    proj_value =
self.value_conv(x).view(m_batchsize, -1,
width*height)
    out = torch.bmm(proj_value,
attention.permute(0, 2, 1))
    out = out.view(m_batchsize, C, height, width)
    out = self.gamma*out + x
    return out

```

```

class CAM_Module(Module):

```

```

    """ Channel attention module"""

```

```

    def __init__(self, in_dim):
        super(CAM_Module, self).__init__()
        self.chanel_in = in_dim
        self.gamma = Parameter(torch.zeros(1))
        self.softmax = Softmax(dim=-1)

```

```

    def forward(self,x):
        """

```

```

    inputs :
        x : input feature maps( B X C X H X W)
    returns :

```

```

        out : attention value + input feature
        attention: B X C X C
    """

```

```

    m_batchsize, C, height, width = x.size()
    proj_query = x.view(m_batchsize, C, -1)
    proj_key = x.view(m_batchsize, C, -
1).permute(0, 2, 1)
    energy = torch.bmm(proj_query, proj_key)
    energy_new = torch.max(energy, -1,
keepdim=True)[0].expand_as(energy)-energy
    attention = self.softmax(energy_new)
    proj_value = x.view(m_batchsize, C, -1)
    out = torch.bmm(attention, proj_value)
    out = out.view(m_batchsize, C, height, width)
    out = self.gamma*out + x

```

```

        return out

class UPx2(nn.Module):
    """
    This class defines the convolution layer with
    batch normalization and PReLU activation
    """
    def __init__(self, nIn, nOut):
        """
        :param nIn: number of input channels
        :param nOut: number of output channels
        :param kSize: kernel size
        :param stride: stride rate for down-sampling.
        Default is 1
        """
        super().__init__()
        self.deconv = nn.ConvTranspose2d(nIn, nOut,
            2, stride=2, padding=0, output_padding=0,
            bias=False)
        self.bn = nn.BatchNorm2d(nOut, eps=1e-03)
        self.act = nn.PReLU(nOut)

    def forward(self, input):
        """
        :param input: input feature map
        :return: transformed feature map
        """
        output = self.deconv(input)
        output = self.bn(output)
        output = self.act(output)
        return output

    def fuseforward(self, input):
        """
        :param input: input feature map
        :return: transformed feature map
        """
        output = self.deconv(input)
        output = self.act(output)
        return output

class CBR(nn.Module):
    """
    This class defines the convolution layer with
    batch normalization and PReLU activation
    """
    def __init__(self, nIn, nOut, kSize, stride=1):
        """
        :param nIn: number of input channels
        :param nOut: number of output channels
        :param kSize: kernel size
        :param stride: stride rate for down-sampling.
        Default is 1
        """
        super().__init__()
        padding = int((kSize - 1)/2)

```

```

        #self.conv = nn.Conv2d(nIn, nOut, kSize,
        stride=stride, padding=padding, bias=False)
        self.conv = nn.Conv2d(nIn, nOut, (kSize,
        kSize), stride=stride, padding=(padding, padding),
        bias=False)
        #self.conv1 = nn.Conv2d(nOut, nOut, (1,
        kSize), stride=1, padding=(0, padding),
        bias=False)
        self.bn = nn.BatchNorm2d(nOut, eps=1e-03)
        self.act = nn.PReLU(nOut)

    def forward(self, input):
        """
        :param input: input feature map
        :return: transformed feature map
        """
        output = self.conv(input)
        #output = self.conv1(output)
        output = self.bn(output)
        output = self.act(output)
        return output

    def fuseforward(self, input):
        """
        :param input: input feature map
        :return: transformed feature map
        """
        output = self.conv(input)
        output = self.act(output)
        return output

class CB(nn.Module):
    """
    This class groups the convolution and batch
    normalization
    """
    def __init__(self, nIn, nOut, kSize, stride=1):
        """
        :param nIn: number of input channels
        :param nOut: number of output channels
        :param kSize: kernel size
        :param stride: optimal stide for down-sampling
        """
        super().__init__()
        padding = int((kSize - 1)/2)
        self.conv = nn.Conv2d(nIn, nOut, (kSize,
        kSize), stride=stride, padding=(padding, padding),
        bias=False)
        self.bn = nn.BatchNorm2d(nOut, eps=1e-03)

    def forward(self, input):
        """
        :param input: input feature map
        :return: transformed feature map
        """
        output = self.conv(input)
        output = self.bn(output)

```

```

        return output

class C(nn.Module):
    """
    This class is for a convolutional layer.
    """
    def __init__(self, nIn, nOut, kSize, stride=1):
        """
        :param nIn: number of input channels
        :param nOut: number of output channels
        :param kSize: kernel size
        :param stride: optional stride rate for down-
        sampling
        """
        super().__init__()
        padding = int((kSize - 1)/2)
        # print(nIn, nOut, (kSize, kSize))
        self.conv = nn.Conv2d(nIn, nOut, (kSize,
        kSize), stride=stride, padding=(padding, padding),
        bias=False)

    def forward(self, input):
        """
        :param input: input feature map
        :return: transformed feature map
        """
        output = self.conv(input)
        return output

class CDilated(nn.Module):
    """
    This class defines the dilated convolution.
    """
    def __init__(self, nIn, nOut, kSize, stride=1,
    d=1):
        """
        :param nIn: number of input channels
        :param nOut: number of output channels
        :param kSize: kernel size
        :param stride: optional stride rate for down-
        sampling
        :param d: optional dilation rate
        """
        super().__init__()
        padding = int((kSize - 1)/2) * d
        self.conv = nn.Conv2d(nIn, nOut, (kSize,
        kSize), stride=stride, padding=(padding, padding),
        bias=False, dilation=d)

    def forward(self, input):
        """
        :param input: input feature map

```

```

        :return: transformed feature map
        """
        output = self.conv(input)
        return output

class DownSamplerB(nn.Module):
    def __init__(self, nIn, nOut):
        super().__init__()
        n = int(nOut/5)
        n1 = nOut - 4*n
        self.c1 = C(nIn, n, 3, 2)
        self.d1 = CDilated(n, n1, 3, 1, 1)
        self.d2 = CDilated(n, n, 3, 1, 2)
        self.d4 = CDilated(n, n, 3, 1, 4)
        self.d8 = CDilated(n, n, 3, 1, 8)
        self.d16 = CDilated(n, n, 3, 1, 16)
        self.bn = nn.BatchNorm2d(nOut, eps=1e-3)
        self.act = nn.PReLU(nOut)

    def forward(self, input):
        output1 = self.c1(input)
        d1 = self.d1(output1)
        d2 = self.d2(output1)
        d4 = self.d4(output1)
        d8 = self.d8(output1)
        d16 = self.d16(output1)
        add1 = d2
        add2 = add1 + d4
        add3 = add2 + d8
        add4 = add3 + d16
        combine = torch.cat([d1, add1, add2, add3,
        add4], 1)
        #combine_in_out = input + combine
        output = self.bn(combine)
        output = self.act(output)
        return output

class BR(nn.Module):
    """
    This class groups the batch normalization and
    PReLU activation
    """
    def __init__(self, nOut):
        """
        :param nOut: output feature maps
        """
        super().__init__()
        self.nOut=nOut
        self.bn = nn.BatchNorm2d(nOut, eps=1e-03)
        self.act = nn.PReLU(nOut)

    def forward(self, input):
        """

```

```

:param input: input feature map
:return: normalized and thresholded feature
map
'''
# print("bf bn :",input.size(),self.nOut)
output = self.bn(input)
# print("after bn :",output.size())
output = self.act(output)
# print("after act :",output.size())
return output
class DilatedParallelResidualBlockB(nn.Module):
'''
This class defines the ESP block, which is based
on the following principle
Reduce ---> Split ---> Transform --> Merge
'''
def __init__(self, nIn, nOut, add=True):
'''
:param nIn: number of input channels
:param nOut: number of output channels
:param add: if true, add a residual connection
through identity operation. You can use projection
too as
in ResNet paper, but we avoid to use it if
the dimensions are not the same because we do not
want to
increase the module complexity
'''
super().__init__()
n = max(int(nOut/5),1)
n1 = max(nOut - 4*n,1)
# print(nIn,n,n1,"--")
self.c1 = C(nIn, n, 1, 1)
self.d1 = CDilated(n, n1, 3, 1, 1) # dilation
rate of 2^0
self.d2 = CDilated(n, n, 3, 1, 2) # dilation rate
of 2^1
self.d4 = CDilated(n, n, 3, 1, 4) # dilation rate
of 2^2
self.d8 = CDilated(n, n, 3, 1, 8) # dilation rate
of 2^3
self.d16 = CDilated(n, n, 3, 1, 16) # dilation
rate of 2^4
# print("nOut bf :",nOut)
self.bn = BR(nOut)
# print("nOut at :",self.bn.size())
self.add = add

def forward(self, input):
'''
:param input: input feature map
:return: transformed feature map

```

```

'''
# reduce
output1 = self.c1(input)
# split and transform
d1 = self.d1(output1)
d2 = self.d2(output1)
d4 = self.d4(output1)
d8 = self.d8(output1)
d16 = self.d16(output1)
# heirarchical fusion for de-gridding
add1 = d2
add2 = add1 + d4
add3 = add2 + d8
add4 = add3 + d16
#
print(d1.size(),add1.size(),add2.size(),add3.size(),a
dd4.size())

#merge
combine = torch.cat([d1, add1, add2, add3,
add4], 1)
# print("combine :",combine.size())
# if residual version
if self.add:
# print("add :",combine.size())
combine = input + combine
# print(combine.size(),"-----")
output = self.bn(combine)
return output

class InputProjectionA(nn.Module):
'''
This class projects the input image to the same
spatial dimensions as the feature map.
For example, if the input image is 512 x512 x3
and spatial dimensions of feature map size are
56x56xF, then
this class will generate an output of 56x56x3
'''
def __init__(self, samplingTimes):
'''
:param samplingTimes: The rate at which you
want to down-sample the image
'''
super().__init__()
self.pool = nn.ModuleList()
for i in range(0, samplingTimes):
#pyramid-based approach for down-
sampling
self.pool.append(nn.AvgPool2d(3, stride=2,
padding=1))

```

```

def forward(self, input):
    """
    :param input: Input RGB Image
    :return: down-sampled image (pyramid-based
    approach)
    """
    for pool in self.pool:
        input = pool(input)
    return input

class ESPNet_Encoder(nn.Module):
    """
    This class defines the ESPNet-C network in the
    paper
    """
    def __init__(self, p=5, q=3):
        # def __init__(self, classes=20, p=1, q=1):
        """
        :param classes: number of classes in the
        dataset. Default is 20 for the cityscapes
        :param p: depth multiplier
        :param q: depth multiplier
        """
        super().__init__()
        self.level1 = CBR(3, 16, 3, 2)
        self.sample1 = InputProjectionA(1)
        self.sample2 = InputProjectionA(2)
        self.b1 = CBR(16 + 3, 19, 3)
        self.level2_0 = DownSamplerB(16 + 3, 64)
        self.level2 = nn.ModuleList()
        for i in range(0, p):

self.level2.append(DilatedParallelResidualBlockB(6
4, 64))
        self.b2 = CBR(128 + 3, 131, 3)
        self.level3_0 = DownSamplerB(128 + 3, 128)
        self.level3 = nn.ModuleList()
        for i in range(0, q):

self.level3.append(DilatedParallelResidualBlockB(1
28, 128))
        # self.mixstyle = MixStyle2(p=0.5, alpha=0.1)
        self.b3 = CBR(256, 32, 3)
        self.sa = PAM_Module(32)
        self.sc = CAM_Module(32)
        self.conv_sa = CBR(32, 32, 3)
        self.conv_sc = CBR(32, 32, 3)
        self.classifier = CBR(32, 32, 1, 1)

def forward(self, input):
    """
    :param input: Receives the input RGB image

```

```

        :return: the transformed feature map with
        spatial dimensions 1/8th of the input image
        """
        output0 = self.level1(input)
        inp1 = self.sample1(input)
        inp2 = self.sample2(input)
        output0_cat = self.b1(torch.cat([output0,
        inp1], 1))
        output1_0 = self.level2_0(output0_cat) #
        down-sampled
        for i, layer in enumerate(self.level2):
            if i==0:
                output1 = layer(output1_0)
            else:
                output1 = layer(output1)
            output1_cat = self.b2(torch.cat([output1,
        output1_0, inp2], 1))
            output2_0 = self.level3_0(output1_cat) #
            down-sampled
            for i, layer in enumerate(self.level3):
                if i==0:
                    output2 = layer(output2_0)
                else:
                    output2 = layer(output2)
            cat_ = torch.cat([output2_0, output2], 1)
            output2_cat = self.b3(cat_)
            out_sa=self.sa(output2_cat)
            out_sc=self.sc(output2_cat)
            out_sc=self.conv_sc(out_sc)
            out_s=out_sa+out_sc
            classifier = self.classifier(out_s)
            return classifier

class TwinLiteNet(nn.Module):
    """
    This class defines the ESPNet network
    """
    def __init__(self, p=2, q=3, ):
        super().__init__()
        self.encoder = ESPNet_Encoder(p, q)
        self.up_1_1 = UPx2(32, 16)
        self.up_2_1 = UPx2(16, 8)
        self.up_1_2 = UPx2(32, 16)
        self.up_2_2 = UPx2(16, 8)
        self.classifier_1 = UPx2(8, 2)
        self.classifier_2 = UPx2(8, 2)

def forward(self, input):
    x=self.encoder(input)
    x1=self.up_1_1(x)
    x1=self.up_2_1(x1)

```

```

        classifier1=self.classifier_1(x1)
        x2=self.up_1_2(x)
        x2=self.up_2_2(x2)
        classifier2=self.classifier_2(x2)
        return (classifier1,classifier2)

metadata_panel.py
import numpy as np
from datetime import datetime
from PyQt5.QtWidgets import QGroupBox,
QFormLayout, QLabel, QFrame
from PyQt5.QtGui import QFont
from PyQt5.QtCore import Qt
from widgets.switch import Switch

class MetadataPanel(QGroupBox):
    def __init__(self, parent=None):
        super().__init__("Thông tin hệ thống", parent)
        self.setMinimumWidth(400)
        self.setStyleSheet("color: #ecf0f1; font-weight:
bold;")
        self.layout = QFormLayout(self)
        data_font = QFont("Consolas", 12,
QFont.Bold)
        value_style = "color: #1abc9c;"
        self.lbl_status = QLabel("Chờ kết nối")
        self.lbl_fps = QLabel("0")
        self.lbl_timestamp = QLabel("00-00-0000
00:00:00")
        self.lbl_mode = QLabel("Thủ công")
        self.lbl_speed = QLabel("0.00 m/s")
        self.lbl_steering = QLabel("0.0°")
        self.lbl_battery = QLabel("0%")
        self.lbl_pothole = QLabel("0.0 cm")
        self.lbl_front = QLabel("--- cm")
        self.lbl_rear = QLabel("--- cm")
        self.lbl_lat = QLabel("0.000000")
        self.lbl_lon = QLabel("0.000000")
        self.lbl_gps_valid = QLabel("Mất kết nối
GPS")
        self.lbl_pothole_count = QLabel("0")
        self.lbl_pothole_area = QLabel("0.0 cm²")
        for lbl in [self.lbl_status, self.lbl_fps,
self.lbl_timestamp, self.lbl_mode,
self.lbl_speed, self.lbl_steering,
self.lbl_battery, self.lbl_pothole,
self.lbl_front, self.lbl_rear, self.lbl_lat,
self.lbl_lon, self.lbl_gps_valid,
self.lbl_pothole_count,
self.lbl_pothole_area]:
            lbl.setFont(data_font)
            lbl.setStyleSheet(value_style)
        self.layout.addRow("Kết nối:", self.lbl_status)

```

```

        self.layout.addRow("FPS:", self.lbl_fps)
        self.layout.addRow("Thời gian:",
self.lbl_timestamp)
        self.layout.addRow("Chế độ:", self.lbl_mode)
        self.layout.addRow("Tốc độ:", self.lbl_speed)
        self.layout.addRow("Góc lái:",
self.lbl_steering)
        self.layout.addRow("Pin:", self.lbl_battery)
        self.layout.addRow("Độ sâu ổ gà:",
self.lbl_pothole)
        self.layout.addRow("Khoảng cách trước:",
self.lbl_front)
        self.layout.addRow("Khoảng cách sau:",
self.lbl_rear)
        line = QFrame()
        line.setFrameShape(QFrame.HLine)
        line.setStyleSheet("color: #7f8c8d;")
        self.layout.addRow(line)
        self.layout.addRow("Số ổ gà:",
self.lbl_pothole_count)
        self.layout.addRow("Tổng diện tích:",
self.lbl_pothole_area)
        self.layout.addRow("Vĩ độ:", self.lbl_lat)
        self.layout.addRow("Kinh độ:", self.lbl_lon)
        self.layout.addRow("Trạng thái GPS:",
self.lbl_gps_valid)
        self.pothole_toggle = Switch("Phát hiện ổ gà")
        self.layout.addRow(self.pothole_toggle)

    def update_metadata(self, metadata):
        pothole_detection_enabled =
self.pothole_toggle.isChecked()
        raw_ts = metadata.get('timestamp_full',
metadata.get('timestamp', 'N/A'))
        ts_str = self.format_timestamp(raw_ts)
        self.lbl_timestamp.setText(ts_str)
        system = metadata.get('system', {})
        auto = system.get('auto_mode', False)
        self.lbl_mode.setText("Tự động" if auto else
"Thủ công")
        self.lbl_mode.setStyleSheet("color: #2ecc71;"
if auto else "color: #f39c12;")
        sensors = metadata.get('sensors', {})
        speed_mps = sensors.get('speed_mps', 0.0)
        steering_angle =
sensors.get('steering_angle_deg', 0.0)
        self.lbl_speed.setText(f"{speed_mps:.2f} m/s")
        self.lbl_steering.setText(f"{steering_angle:.1f}°")
        self.lbl_battery.setText(f"{sensors.get('battery_perc
ent', 0.0):.0f}%")

```

```

        depth_cm = sensors.get('pothole_depth_cm',
0.0)
        self.lbl_pothole.setText(f"{depth_cm:.1f} cm"
if pothole_detection_enabled else "--- cm")
        f = sensors.get('front_distance_cm', -1)
        r = sensors.get('rear_distance_cm', -1)
        self.lbl_front.setText(f"{f:.1f} cm" if f != -1 and
not np.isinf(f) else "--- cm")
        self.lbl_rear.setText(f"{r:.1f} cm" if r != -1 and
not np.isinf(r) else "--- cm")
        gps = metadata.get('gps', {})
        valid = gps.get('valid', False)
        if valid:
            lat = gps.get('latitude', 0.0)
            lon = gps.get('longitude', 0.0)
        else:
            lat = 0.0
            lon = 0.0
        self.lbl_lat.setText(f"{lat:.6f}")
        self.lbl_lon.setText(f"{lon:.6f}")
        self.lbl_gps_valid.setText("GPS hợp lệ" if
valid else "Mất kết nối GPS")
        self.lbl_gps_valid.setStyleSheet("color:
#2ecc71;" if valid else "color: #e74c3c;")

def update_pothole_stats(self, count, total_area):
    self.lbl_pothole_count.setText(str(count))
    self.lbl_pothole_area.setText(f"{total_area:.1f}
cm²")

def set_status(self, text):
    # Xử lý các thông báo từ UDP thread
    if text == "Connected":
        self.lbl_status.setText("Đã kết nối")
        self.lbl_status.setStyleSheet("color:
#2ecc71;") # xanh
    elif text == "Disconnected":
        self.lbl_status.setText("Mất kết nối")
        self.lbl_status.setStyleSheet("color:
#e74c3c;") # đỏ
    elif "UDP listening on port" in text:
        self.lbl_status.setText("Chờ kết nối")
        self.lbl_status.setStyleSheet("color:
#f39c12;") # vàng
    else:
        self.lbl_status.setText(text)

def set_fps(self, fps):
    self.lbl_fps.setText(str(fps))

def reset_all_display(self):

```

```

        """Reset tất cả các giá trị hiển thị về trạng
thái ban đầu khi mất kết nối."""
        self.lbl_fps.setText("0")

self.lbl_timestamp.setText(datetime.now().strftime("
%d-%m-%Y %H:%M:%S"))
        self.lbl_mode.setText("Thủ công")
        self.lbl_speed.setText("0.00 m/s")
        self.lbl_steering.setText("0.0°")
        self.lbl_battery.setText("0%")
        if self.pothole_toggle.isChecked():
            self.lbl_pothole.setText("0.0 cm")
        else:
            self.lbl_pothole.setText("--- cm")
            self.lbl_front.setText("--- cm")
            self.lbl_rear.setText("--- cm")
            self.lbl_lat.setText("0.000000")
            self.lbl_lon.setText("0.000000")
            self.lbl_gps_valid.setText("Mất kết nối GPS")
            self.lbl_gps_valid.setStyleSheet("color:
#e74c3c;")
            self.lbl_pothole_count.setText("0")
            self.lbl_pothole_area.setText("0.0 cm²")

@staticmethod
def format_timestamp(ts_str):
    if not isinstance(ts_str, str) or ts_str == "N/A":
        return datetime.now().strftime("%d-%m-%Y
%H:%M:%S")
    try:
        if "_" in ts_str and len(ts_str) >= 15:
            date_part, time_part = ts_str.split("_")
            year, month, day = date_part[:4],
date_part[4:6], date_part[6:8]
            hour, minute, second = time_part[:2],
time_part[2:4], time_part[4:6]
            return f"{day}-{month}-{year}
{hour}:{minute}:{second}"
        cleaned = ts_str.replace('T', ' ').replace('Z',
").split('.')[0]
        dt = datetime.strptime(cleaned, "%Y-%m-
%d %H:%M:%S")
        return dt.strftime("%d-%m-%Y
%H:%M:%S")
    except:
        return ts_str

processor.py
import os
import time
import uuid
import sqlite3
import cv2

```

```

import numpy as np
from datetime import datetime
from PyQt5.QtCore import QObject, pyqtSignal,
pyqtSlot
from ultralytics import YOLO
from .twinlite import TwinLiteProcessor
from .tracker import PotholeTracker

class PotholeProcessor(QObject):
    image_processed = pyqtSignal(np.ndarray, list)
    roi1_updated = pyqtSignal(list)
    pothole_saved = pyqtSignal(dict)
    data_refresh = pyqtSignal()
    map_refresh = pyqtSignal()
    metadata_updated = pyqtSignal(dict)

    def __init__(self, pixel_to_cm2, db_path,
images_dir):
        super().__init__()
        self.pixel_to_cm2 = pixel_to_cm2
        self.db_path = db_path
        self.images_dir = images_dir
        self.tracker = None
        self.yolo_model = None
        self.twinlite = None
        self.pothole_detection_enabled = False
        self.conf_threshold = 0.5
        self.roi1_rect = (0, 0, 400, 600)
        self.roi2_rect = (0, 0, 400, 600)
        self.bev_w = 400
        self.bev_h = 600

    def init_models(self, yolo_path, twinlite_path):
        self.yolo_model = YOLO(yolo_path)
        self.twinlite =
TwinLiteProcessor(twinlite_path)
        self.tracker =
PotholeTracker(pixel_to_cm2=self.pixel_to_cm2,
iou_threshold=0.3)

    def set_rois(self, roi1_rect, roi2_rect, bev_w,
bev_h):
        self.roi1_rect = roi1_rect
        self.roi2_rect = roi2_rect
        self.bev_w = bev_w
        self.bev_h = bev_h

    def set_pothole_detection(self, enabled):
        self.pothole_detection_enabled = enabled
        if not enabled:
            if self.tracker:
                self.tracker.tracks.clear()

```

```

        self.tracker.next_id = 1

    def is_center_in_roi2(self, box):
        x1, y1, x2, y2 = box
        cx = (x1 + x2) // 2
        cy = (y1 + y2) // 2
        rx1, ry1, rx2, ry2 = self.roi2_rect
        return (rx1 <= cx <= rx2 and ry1 <= cy <=
ry2)

    def is_fully_in_roi1(self, box):
        x1, y1, x2, y2 = box
        rx1, ry1, rx2, ry2 = self.roi1_rect
        return (x1 >= rx1 and x2 <= rx2 and y1 >=
ry1 and y2 <= ry2)

    def detect_potholes(self, bev_img):
        results = self.yolo_model(bev_img,
conf=self.conf_threshold, verbose=False)
        h, w = bev_img.shape[:2]
        detections = []
        boxes = results[0].boxes
        masks = results[0].masks
        if boxes is not None:
            for i, box in enumerate(boxes):
                conf = box.conf.item()
                if conf < self.conf_threshold:
                    continue
                x1, y1, x2, y2 = map(int,
box.xyxy[0].tolist())
                mask = None
                if masks is not None:
                    m = masks.data[i].cpu().numpy()
                    m = cv2.resize(m, (w, h))
                    mask = (m > 0.5).astype(np.uint8)
                    area_px = mask.sum() if mask is not
None else (x2 - x1) * (y2 - y1)
                    detections.append({'box': (x1, y1, x2, y2),
'confidence': conf, 'mask': mask, 'area_px':
area_px})
            return detections

    def filter_potholes_by_da(self, detections,
da_mask, overlap_threshold=0.3):
        filtered = []
        for det in detections:
            if det['mask'] is None:
                x1, y1, x2, y2 = det['box']
                if y2 <= y1 or x2 <= x1:
                    continue
                box_region = da_mask[y1:y2, x1:x2]
                if box_region.size == 0:

```

```

        continue
        overlap = (box_region > 0).sum() /
box_region.size
    else:
        overlap = np.logical_and(det['mask'],
da_mask).sum() / (det['mask'].sum() + 1e-5)
        if overlap >= overlap_threshold:
            filtered.append(det)
    return filtered

def save_pothole_record(self, track_id,
image_crop, area_cm2, depth_cm, lat, lon,
timestamp_str):
    volume_cm3 = area_cm2 * depth_cm
    safe_timestamp = timestamp_str.replace(" ",
"_").replace(":", "-").replace("/", "-")
    unique_id = str(uuid.uuid4())[8]
    filename =
f"pothole_track{track_id}_{safe_timestamp}_{unique_id}.jpg"
    # 1. Lấy ngày hiện tại
    today_str = datetime.now().strftime("%Y-%m-%d")
    # 2. Tạo đường dẫn đến thư mục ngày
(C:\...\OGa\2026-05-06)
    day_dir = os.path.join(self.images_dir,
today_str)
    # 3. Tạo thư mục nếu chưa tồn tại
if not os.path.exists(day_dir):
        os.makedirs(day_dir, exist_ok=True)
    # 4. Tạo đường dẫn đầy đủ của file ảnh
filepath = os.path.join(day_dir, filename)
    # 5. Lưu file vật lý
success = cv2.imwrite(filepath, image_crop)
if not success:
    print(f"[Processor] Cannot save image:
{filepath}")
    return None
    # 6. Lưu filepath vào Database
conn = sqlite3.connect(self.db_path)
    cursor = conn.cursor()
    cursor.execute("""
INSERT INTO pothole_records (track_id,
timestamp, area_cm2, depth_cm, volume_cm3,
gps_lat, gps_lon, image_path)
VALUES (?, ?, ?, ?, ?, ?, ?, ?)
""", (track_id, timestamp_str, area_cm2,
depth_cm, volume_cm3, lat, lon, filepath))
    conn.commit()
    record_id = cursor.lastrowid
    conn.close()
    return record_id

```

```

def format_timestamp(self, ts_str):
    if not isinstance(ts_str, str) or ts_str == "N/A":
        return datetime.now().strftime("%d-%m-%Y
%H:%M:%S")
    try:
        if "_" in ts_str and len(ts_str) >= 15:
            date_part, time_part = ts_str.split("_")
            year, month, day = date_part[:4],
date_part[4:6], date_part[6:8]
            hour, minute, second = time_part[:2],
time_part[2:4], time_part[4:6]
            return f"{day}-{month}-{year}
{hour}:{minute}:{second}"
            cleaned = ts_str.replace("T", " ").replace("Z",
").split('.')[0]
            dt = datetime.strptime(cleaned, "%Y-%m-
%d %H:%M:%S")
            return dt.strftime("%d-%m-%Y
%H:%M:%S")
        except:
            return ts_str

@pyqtSlot(np.ndarray, dict)
def process_frame(self, cv_img, metadata):
    self.metadata_updated.emit(metadata)
    if not self.pothole_detection_enabled:
        self.image_processed.emit(cv_img, [])
        return
    da_mask = self.twinlite.predict(cv_img)
    all_potholes = self.detect_potholes(cv_img)
    potholes_in_da =
self.filter_potholes_by_da(all_potholes, da_mask)
    self.tracker.update(potholes_in_da, cv_img)
    active_tracks =
self.tracker.get_active_tracks()
    gps = metadata.get('gps', {})
    valid = gps.get('valid', False)
    if valid:
        lat = gps.get('latitude', 0.0)
        lon = gps.get('longitude', 0.0)
    else:
        lat = 0.0
        lon = 0.0
    depth_cm = metadata.get('sensors',
{}).get('pothole_depth_cm', 0.0)
    raw_ts = metadata.get('timestamp_full',
metadata.get('timestamp',
datetime.now().strftime("%Y-%m-%d
%H:%M:%S")))
    ts_str = self.format_timestamp(raw_ts)
    roi1_tracks_data = []

```

```

for track in active_tracks:
    # Xử lý ROI1
    if self.is_fully_in_roi1(track.box):
        track.update_max_area()
        if not track.has_captured:
            track.capture_image_from_roi1(cv_img,
            expand=30)
            roi1_tracks_data.append({
                'id': track.id,
                'area_cm2': track.area_cm2,
                'max_area_cm2': track.max_area_cm2,
                'box': track.box,
                'has_captured': track.has_captured,
                'captured_image':
            track.captured_image
            })

    # Xử lý ROI2: nếu center nằm trong ROI2
    thì cập nhật max depth và đánh dấu đã vào ROI2
    currently_in_roi2 =
    self.is_center_in_roi2(track.box)
    if currently_in_roi2:
        track.update_max_depth(depth_cm)
        if not track.has_entered_roi2:
            track.has_entered_roi2 = True

    # Nếu track đã từng vào ROI2, hiện tại
    không còn trong ROI2, và chưa được lưu -> lưu
    if track.has_entered_roi2 and not
    currently_in_roi2 and not track.saved and
    track.has_captured:
        area_cm2 = track.max_area_cm2
        crop = track.captured_image
        if crop is not None:
            max_depth = track.max_depth_cm
            record_id =
            self.save_pothole_record(track.id, crop, area_cm2,
            max_depth, lat, lon, ts_str)
            if record_id is not None:
                track.saved = True
                saved_info = {
                    'track_id': track.id,
                    'area': area_cm2,
                    'depth': max_depth,
                    'volume': area_cm2 * max_depth,
                    'lat': lat,
                    'lon': lon,
                    'timestamp': ts_str,
                    'image': crop
                }
                self.pothole_saved.emit(saved_info)

```

```

self.data_refresh.emit()
self.map_refresh.emit()
display_img = cv_img.copy()
for track in active_tracks:
    x1, y1, x2, y2 = track.box
    cv2.rectangle(display_img, (x1, y1), (x2,
    y2), (0, 255, 0), 2)
    cv2.putText(display_img, f"ID:{track.id}",
    (x1, y1-5), cv2.FONT_HERSHEY_SIMPLEX, 0.4,
    (0,255,0), 1)
    display_img =
    self.twilight.overlay_masks(display_img, da_mask,
    alpha=0.3)
    self.image_processed.emit(display_img,
    active_tracks)
    self.roi1_updated.emit(roi1_tracks_data)

```

tracker.py

```

import time
class PotholeTrack:
    def __init__(self, track_id, detection,
    pixel_to_cm2, expand=30):
        self.id = track_id
        self.box = detection['box']
        self.area_px = detection['area_px']
        self.area_cm2 = self.area_px * pixel_to_cm2
        self.max_area_cm2 = self.area_cm2
        self.confidence = detection['confidence']
        self.last_seen = time.time()
        self.saved = False
        self.captured_image = None
        self.has_captured = False
        self.max_depth_cm = 0.0
        self.has_entered_roi2 = False

    def update(self, detection, pixel_to_cm2):
        self.box = detection['box']
        self.area_px = detection['area_px']
        self.area_cm2 = self.area_px * pixel_to_cm2
        self.confidence = detection['confidence']
        self.last_seen = time.time()

    def capture_image_from_roi1(self, bev_img,
    expand=30):
        if self.has_captured:
            return
        x1, y1, x2, y2 = self.box
        x1e = max(0, x1 - expand)
        y1e = max(0, y1 - expand)
        x2e = min(bev_img.shape[1], x2 + expand)
        y2e = min(bev_img.shape[0], y2 + expand)
        crop = bev_img[y1e:y2e, x1e:x2e]
        if crop.size > 0:

```

```

        self.captured_image = crop.copy()
        self.has_captured = True

    def update_max_area(self):
        if self.area_cm2 > self.max_area_cm2:
            self.max_area_cm2 = self.area_cm2

    def update_max_depth(self, depth_cm):
        if depth_cm > self.max_depth_cm:
            self.max_depth_cm = depth_cm

class PotholeTracker:
    def __init__(self, pixel_to_cm2,
        iou_threshold=0.3, lost_timeout=1.0):
        self.next_id = 1
        self.tracks = []
        self.iou_threshold = iou_threshold
        self.lost_timeout = lost_timeout
        self.pixel_to_cm2 = pixel_to_cm2

    def compute_iou(self, box1, box2):
        x1, y1, x2, y2 = box1
        x1b, y1b, x2b, y2b = box2
        inter_x1 = max(x1, x1b)
        inter_y1 = max(y1, y1b)
        inter_x2 = min(x2, x2b)
        inter_y2 = min(y2, y2b)
        inter_area = max(0, inter_x2 - inter_x1) *
        max(0, inter_y2 - inter_y1)
        area1 = (x2 - x1) * (y2 - y1)
        area2 = (x2b - x1b) * (y2b - y1b)
        return inter_area / (area1 + area2 - inter_area
        + 1e-5)

    def update(self, detections, bev_img):
        now = time.time()
        self.tracks = [t for t in self.tracks if now -
        t.last_seen < self.lost_timeout]
        if not detections:
            return
        unmatched_detections =
        list(range(len(detections)))
        unmatched_tracks =
        list(range(len(self.tracks)))
        for i, track in enumerate(self.tracks):
            best_iou = 0
            best_det = -1
            for j, det in enumerate(detections):
                if j in unmatched_detections:
                    iou = self.compute_iou(track.box,
                    det['box'])

```

```

                    if iou > best_iou and iou >
                    self.iou_threshold:
                        best_iou = iou
                        best_det = j
            if best_det != -1:
                unmatched_detections.remove(best_det)
                unmatched_tracks.remove(i)
                track.update(detections[best_det],
                self.pixel_to_cm2)
            for j in unmatched_detections:
                new_track = PotholeTrack(self.next_id,
                detections[j], self.pixel_to_cm2)
                self.tracks.append(new_track)
                self.next_id += 1

    def get_active_tracks(self):
        now = time.time()
        return [t for t in self.tracks if now - t.last_seen
        < 0.5]

```

twinlite.py

```

import sys
import os
import cv2
import torch
import numpy as np

sys.path.insert(0,
    os.path.dirname(os.path.dirname(__file__))) #
    thêm thư mục gốc để import model
try:
    from model.TwinLite import TwinLiteNet
except ImportError:
    print("Lỗi: Không tìm thấy model/TwinLite.py.
    Hãy kiểm tra lại.")
    sys.exit(1)

class TwinLiteProcessor:
    def __init__(self, weight_path, img_size=(240,
    426)):
        self.img_size = img_size
        self.device = 'cuda' if torch.cuda.is_available()
        else 'cpu'
        self.model = TwinLiteNet()
        state_dict = torch.load(weight_path,
        map_location=self.device, weights_only=False)
        self.model.load_state_dict(state_dict)
        self.model.eval()
        self.model.to(self.device)
        print(f"[TwinLite] Loaded from {weight_path}
        on {self.device}")

    def predict(self, img_bgr):

```

```

        h, w = img_bgr.shape[:2]
        img_resized = cv2.resize(img_bgr,
            (self.img_size[1], self.img_size[0]))
        img_rgb = cv2.cvtColor(img_resized,
            cv2.COLOR_BGR2RGB)
        img_tensor =
            torch.from_numpy(img_rgb).permute(2, 0,
            1).unsqueeze(0).float() / 255.0
        img_tensor = img_tensor.to(self.device)
        with torch.no_grad():
            da_out, ll_out = self.model(img_tensor)
            da_pred = torch.argmax(da_out,
            dim=1).squeeze(0).cpu().numpy().astype(np.uint8)
            * 255
            da_mask = cv2.resize(da_pred, (w, h),
            interpolation=cv2.INTER_NEAREST)
            return da_mask

```

```

    @staticmethod
    def overlay_masks(img_bgr, da_mask,
        da_color=(0, 255, 0), alpha=0.4):
        overlay = img_bgr.copy()
        overlay[da_mask == 255] = da_color
        return cv2.addWeighted(img_bgr, 1 - alpha,
            overlay, alpha, 0)

```

data_tab.py

```

import os
import sqlite3
from datetime import datetime
from PyQt5.QtWidgets import *
from PyQt5.QtCore import pyqtSignal, Qt,
    QDateTime
from PyQt5.QtGui import QPixmap
from shapely.geometry import Point
from PyQt5.QtGui import QPixmap, QPalette,
    QColor

try:
    import pandas as pd
    PANDAS_AVAILABLE = True
except ImportError:
    PANDAS_AVAILABLE = False

from database.database import DatabaseManager
from location_manager import LocationManager

```

```

class DataTab(QWidget):
    image_clicked = pyqtSignal(int, float, float)
    data_changed = pyqtSignal()

```

```

    def __init__(self, db_path, location_mgr=None,
        parent=None):
        super().__init__(parent)
        self.db = DatabaseManager(db_path)
        self.location_mgr = location_mgr
        self.current_rows = []
        self.filtered_rows = []

```

```

        self.setup_ui()
        self.load_data()

```

```

    def setup_ui(self):
        layout = QVBoxLayout(self)
        layout.setContentsMargins(5, 5, 5, 5)
        layout.setSpacing(5)

```

```

        # Khung lọc dữ liệu
        filter_group = QGroupBox("Bộ lọc dữ liệu")
        filter_group.setStyleSheet("QGroupBox { font-
            weight: bold; color: #1abc9c; }")
        group_layout = QVBoxLayout(filter_group)
        group_layout.setSpacing(10)
        group_layout.setContentsMargins(10, 20, 10,
            10)

```

```

        # Hàng 1: thời gian
        row_time = QHBoxLayout()
        row_time.addWidget(QLabel("Thời gian từ:"))

```

```

    def create_styled_calendar():
        cal = QCalendarWidget()

```

```

        cal.setVerticalHeaderFormat(QCalendarWidget.No
            VerticalHeader)

```

```

        # Thiết lập Palette cơ bản
        palette = cal.palette()
        palette.setColor(QPalette.WindowText,
            Qt.black)
        palette.setColor(QPalette.Text, Qt.black)
        cal.setPalette(palette)

```

```

        # CSS tập trung vào độ tương phản cao cho
        Header

```

```

        cal.setStyleSheet("""
            /* Toàn bộ bảng ngày */
            QCalendarWidget QTableView {
                background-color: #2c3e50;
                color: #ecf0f1;
                selection-background-color: #1abc9c;
                selection-color: #ffffff;
                font: 11pt "Segoe UI";
            }

```

```

        outline: 0;
    }

    /* Thanh tiêu đề chứa Thứ (T2-CN) */
    QCalendarWidget QHeaderView::section
{
    background-color: #ffffff; /* Nền
trắng */
    color: #000000; /* Chữ đen */
    font: bold 10pt "Segoe UI";
    padding: 5px;
    border: none;
}

```

```

/* Thanh điều hướng phía trên
(Tháng/Năm) */

```

```

    QCalendarWidget
QWidget#qt_calendar_navigationbar {
    background-color: #1e2a36;
    min-height: 35px;
}

```

```

    QCalendarWidget QToolButton {
        color: #ffffff;
        font: bold 10pt "Segoe UI";
        background-color: transparent;
        border: none;
        margin: 5px;
    }
    """)

```

```

from PyQt5.QtGui import
QTextCharFormat, QColor

```

```

header_fmt = QTextCharFormat()

header_fmt.setForeground(QColor("black"))
header_fmt.setFontWeight(75) # Bold
cal.setHeaderTextFormat(header_fmt)

workday_fmt = QTextCharFormat()

workday_fmt.setForeground(QColor("#f94343"))
workday_fmt.setFontWeight(75)

weekend_fmt = QTextCharFormat()

weekend_fmt.setForeground(QColor("#e74c3c"))
weekend_fmt.setFontWeight(75)

cal.setWeekdayTextFormat(Qt.Monday,
workday_fmt)

```

```

        cal.setWeekdayTextFormat(Qt.Tuesday,
workday_fmt)
        cal.setWeekdayTextFormat(Qt.Wednesday,
workday_fmt)
        cal.setWeekdayTextFormat(Qt.Thursday,
workday_fmt)
        cal.setWeekdayTextFormat(Qt.Friday,
workday_fmt)
        cal.setWeekdayTextFormat(Qt.Saturday,
weekend_fmt)
        cal.setWeekdayTextFormat(Qt.Sunday,
weekend_fmt)
        return cal

```

```

self.start_date = QDateTimeEdit()
self.start_date.setCalendarPopup(True)

```

```

self.start_date.setCalendarWidget(create_styled_ca
lendar())

```

```

self.start_date.setDateTime(QDateTime.currentDat
eTime().addDays(-7))

```

```

self.start_date.setDisplayFormat("dd/MM/yyyy
HH:mm")

```

```

self.start_date.setMinimumWidth(160)
row_time.addWidget(self.start_date)
row_time.addWidget(QLabel(" đến: "))
self.end_date = QDateTimeEdit()
self.end_date.setCalendarPopup(True)

```

```

self.end_date.setCalendarWidget(create_styled_cal
endar())

```

```

self.end_date.setDateTime(QDateTime.currentDate
Time())

```

```

self.end_date.setDisplayFormat("dd/MM/yyyy
HH:mm")

```

```

self.end_date.setMinimumWidth(160)
row_time.addWidget(self.end_date)
row_time.addStretch()
group_layout.addLayout(row_time)

```

```

row_pos = QHBoxLayout()

```

```

self.cbo_province = QComboBox()

```

```

self.cbo_province.addItem("-- Tỉnh/Thành phố
--")

```

```

self.cbo_province.setMinimumWidth(200)

```

```

self.cbo_ward = QComboBox()

```

```

self.cbo_ward.addItem("-- Phường/Xã --")

```

```

self.cbo_ward.setMinimumWidth(200)

```

```

if self.location_mgr:

self.cbo_province.addItem(self.location_mgr.get_p
rovince_names())

self.cbo_province.currentIndexChanged.connect(sel
f.on_province_changed)

    row_pos.addWidget(QLabel("Vị trí: "))
    row_pos.addWidget(self.cbo_province)
    row_pos.addWidget(self.cbo_ward)

self.btn_filter = QPushButton("Lọc")
self.btn_filter.setFixedWidth(70)
self.btn_reset = QPushButton("Reset")
self.btn_reset.setFixedWidth(70)
self.btn_export = QPushButton("Xuất Excel")
self.btn_export.setStyleSheet("background-
color: #27ae60; color: white; font-weight: bold;")
if not PANDAS_AVAILABLE:
    self.btn_export.setEnabled(False)

    row_pos.addWidget(self.btn_filter)
    row_pos.addWidget(self.btn_reset)
    row_pos.addWidget(self.btn_export)
    row_pos.addStretch()
    group_layout.addLayout(row_pos)
    layout.addWidget(filter_group)

# Bảng dữ liệu
self.table = QTableWidgetItem()
self.table.setColumnCount(9)

self.table.setHorizontalHeaderLabels(["Chọn",
"ID", "Track ID", "Thời gian phát hiện",
"Diện tích", "Độ sâu",
"Thể tích", "GPS", "Đường dẫn ảnh"])

self.table.setSelectionBehavior(QAbstractItemView.
SelectRows)

self.table.setEditTriggers(QAbstractItemView.NoEd
itTriggers)
    self.table.setStyleSheet("""
        QTableWidgetItem {
            background-color: #f5f6fa;
            color: #2f3640;
            gridline-color: #dcdde1;
            border: 1px solid #7f8c8d;
        }
        QHeaderView::section {
            background-color: #dcdde1;
            color: black;

```

```

font-weight: bold;
        }
        """)

self.table.itemSelectionChanged.connect(self.on_sel
ection_changed)

self.table.setContextMenuPolicy(Qt.CustomContext
Menu)

self.table.customContextMenuRequested.connect(se
lf.show_context_menu)
    layout.addWidget(self.table, stretch=1)

# Chi tiết ảnh và text
detail_layout = QHBoxLayout()
self.image_label = QLabel()
self.image_label.setFixedSize(250, 250)
self.image_label.setStyleSheet("background-
color: black; border: 1px solid #1abc9c; color:
white;")
    self.image_label.setAlignment(Qt.AlignCenter)
    self.image_label.setText("Chọn bản ghi để xem
ảnh")
    self.image_label.mousePressEvent =
self.on_image_click
    self.info_text = QTextEdit()
    self.info_text.setReadOnly(True)
    self.info_text.setFixedHeight(250)
    self.info_text.setStyleSheet("background-color:
#34495e; color: #ecf0f1; font-family: Consolas;
font-size: 10pt;")
    detail_layout.addWidget(self.image_label)
    detail_layout.addWidget(self.info_text)
    layout.addLayout(detail_layout)

# Hàng nút chức năng
btn_action_layout = QHBoxLayout()
self.btn_select_all = QPushButton("Chọn tất
cả")
    self.btn_clear_all = QPushButton("Bỏ chọn
tất cả")
    self.btn_edit = QPushButton("Sửa bản ghi")
    self.btn_delete_selected = QPushButton("Xóa
các mục đã chọn")

self.btn_delete_selected.setStyleSheet("background-
color: #c0392b; color: white;")

    btn_action_layout.addWidget(self.btn_select_all)

    btn_action_layout.addWidget(self.btn_clear_all)

```

```

        btn_action_layout.addWidget(self.btn_edit)

    btn_action_layout.addWidget(self.btn_delete_selected)

    layout.addLayout(btn_action_layout)

    # Kết nối sự kiện

    self.start_date.dateTimeChanged.connect(self.validate_dates)

    self.end_date.dateTimeChanged.connect(self.validate_dates)

    self.btn_edit.clicked.connect(self.edit_record)

    self.btn_delete_selected.clicked.connect(self.delete_selected_records)

    self.btn_select_all.clicked.connect(self.select_all)

    self.btn_clear_all.clicked.connect(self.clear_all)
    self.btn_filter.clicked.connect(self.apply_filter)
    self.btn_reset.clicked.connect(self.reset_filter)

    self.btn_export.clicked.connect(self.export_to_excel)

    def validate_dates(self):
        if self.start_date.dateTime() > self.end_date.dateTime():

    self.end_date.setDateTime(self.start_date.dateTime())

    def on_province_changed(self):
        prov = self.cbo_province.currentText()
        self.cbo_ward.blockSignals(True)
        self.cbo_ward.clear()
        self.cbo_ward.addItem("-- Phường/Xã --")
        if prov != "-- Tỉnh/Thành phố --" and
    self.location_mgr:
        wards =
    self.location_mgr.get_wards_by_province(prov)
        self.cbo_ward.addItem(wards)
        self.cbo_ward.blockSignals(False)

    def apply_filter(self):
        start_dt =
    self.start_date.dateTime().toPyDateTime()
        end_dt =
    self.end_date.dateTime().toPyDateTime()
        if start_dt > end_dt:

```

```

        QMessageBox.warning(self, "Lỗi thời gian",
        "Thời gian bắt đầu phải trước thời gian kết thúc!")
        return

    prov = self.cbo_province.currentText()
    ward = self.cbo_ward.currentText()

    location_geom = None
    if self.location_mgr:
        if ward != "-- Phường/Xã --":
            location_geom =
    self.location_mgr.get_geometry(ward, 'ward')
        elif prov != "-- Tỉnh/Thành phố --":
            location_geom =
    self.location_mgr.get_geometry(prov, 'province')
        if location_geom is not None:
            location_geom = location_geom.buffer(0)
            location_geom =
    location_geom.simplify(0.001,
    preserve_topology=True)

    filtered = []
    for row in self.current_rows:
        ts_str = row[2]
        lat = row[6]
        lon = row[7]
        try:
            ts_dt = datetime.strptime(ts_str, "%d-%m-%Y %H:%M:%S")
        except Exception:
            continue
        if not (start_dt <= ts_dt <= end_dt):
            continue
        if location_geom is not None:
            point = Point(lon, lat)
            if not location_geom.buffer(1e-5).intersects(point):
                continue
            filtered.append(row)
    self.filtered_rows = filtered
    self.display_rows(self.filtered_rows)

    def reset_filter(self):

    self.start_date.setDateTime(QDateTime.currentDateTime().addDays(-7))

    self.end_date.setDateTime(QDateTime.currentDateTime())

    if self.location_mgr:
        self.cbo_province.setCurrentIndex(0)
        self.cbo_ward.clear()

```

```

        self.cbo_ward.addItem("-- Phường/Xã --")
        self.filtered_rows = self.current_rows[:]
        self.display_rows(self.filtered_rows)

    def load_data(self):
        self.current_rows = self.db.load_all_records()
        self.filtered_rows = self.current_rows[:]
        self.display_rows(self.filtered_rows)
        self.data_changed.emit()

    def display_rows(self, rows):
        self.table.setRowCount(0)
        for row in rows:
            record_id, track_id, ts, area, depth, volume,
            lat, lon, img_path = row
            idx = self.table.rowCount()
            self.table.insertRow(idx)
            chk_item = QTableWidgetItem()
            chk_item.setFlags(Qt.ItemIsUserCheckable
| Qt.ItemIsEnabled)
            chk_item.setCheckState(Qt.Unchecked)
            self.table.setItem(idx, 0, chk_item)
            self.table.setItem(idx, 1,
QTableWidgetItem(str(record_id)))
            self.table.setItem(idx, 2,
QTableWidgetItem(str(track_id)))
            self.table.setItem(idx, 3,
QTableWidgetItem(ts))
            self.table.setItem(idx, 4,
QTableWidgetItem(f"{area:.1f}")
            self.table.setItem(idx, 5,
QTableWidgetItem(f"{depth:.1f}")
            self.table.setItem(idx, 6,
QTableWidgetItem(f"{volume:.1f}")
            self.table.setItem(idx, 7,
QTableWidgetItem(f"{lat:.6f}, {lon:.6f}")
            self.table.setItem(idx, 8,
QTableWidgetItem(img_path))
            self.table.resizeColumnsToContents()

    def show_context_menu(self, pos):
        menu = QMenu()
        edit_action = menu.addAction("Sửa bản ghi")
        action =
menu.exec_(self.table.viewport().mapToGlobal(pos)
)

        if action == edit_action:
            self.edit_record()

    def get_selected_record_ids(self):
        ids = []
        for row in range(self.table.rowCount()):

```

```

            chk = self.table.item(row, 0)
            if chk is not None and chk.checkState() ==
Qt.Checked:
                id_item = self.table.item(row, 1)
                if id_item is not None:
                    ids.append(int(id_item.text()))
            return ids

    def select_all(self):
        for row in range(self.table.rowCount()):
            item = self.table.item(row, 0)
            if item is not None:
                item.setCheckState(Qt.Checked)

    def clear_all(self):
        for row in range(self.table.rowCount()):
            item = self.table.item(row, 0)
            if item is not None:
                item.setCheckState(Qt.Unchecked)

    def delete_selected_records(self):
        ids = self.get_selected_record_ids()
        if not ids:
            QMessageBox.warning(self, "Chọn bản
ghi", "Vui lòng chọn ít nhất một bản ghi để xóa.")
            return

        conn = sqlite3.connect(self.db.db_path)
        cursor = conn.cursor()
        placeholders = ','.join('?' * len(ids))
        cursor.execute(f"SELECT id, image_path
FROM pothole_records WHERE id IN
({placeholders})", ids)
        rows = cursor.fetchall()
        conn.close()
        if not rows:
            return

        msg = f"Bạn có chắc muốn xóa {len(ids)} bản
ghi?\nID: {' , '.join(map(str, ids))}"
        reply = QMessageBox.question(self, "Xác
nhận xóa", msg, QMessageBox.Yes |
QMessageBox.No)
        if reply == QMessageBox.Yes:
            for rec_id, img_path in rows:
                if img_path and os.path.exists(img_path):
                    try:
                        os.remove(img_path)
                        print(f"Đã xóa ảnh: {img_path}")
                    except Exception as e:
                        print(f"Lỗi xóa ảnh {img_path}:
{e}")

```

```

        conn = sqlite3.connect(self.db.db_path)
        cursor = conn.cursor()
        cursor.execute(f"DELETE FROM
pothole_records WHERE id IN ({placeholders})",
ids)

        conn.commit()
        conn.close()
        self.load_data()
        QMessageBox.information(self, "Thành
công", f"Đã xóa {len(ids)} bản ghi")

    def edit_record(self):
        ids = self.get_selected_record_ids()
        if len(ids) != 1:
            QMessageBox.warning(self, "Chọn bản
ghi", "Vui lòng chọn đúng một bản ghi để sửa.")
            return
        rec_id = ids[0]
        conn = sqlite3.connect(self.db.db_path)
        cursor = conn.cursor()
        cursor.execute("SELECT timestamp,
area_cm2, depth_cm, gps_lat, gps_lon FROM
pothole_records WHERE id=?", (rec_id,))
        row = cursor.fetchone()
        conn.close()
        if not row:
            return
        ts, area, depth, lat, lon = row
        dialog = QDialog(self)
        dialog.setWindowTitle(f"Sửa bản ghi ID
{rec_id}")
        layout = QFormLayout(dialog)
        ts_edit = QDateTimeEdit()
        try:
            dt = datetime.strptime(ts, "%d-%m-%Y
%H:%M:%S")
        except:
            dt = datetime.now()
        ts_edit.setDateTime(QDateTime(dt))
        ts_edit.setDisplayFormat("dd/MM/yyyy
HH:mm:ss")
        area_edit = QLineEdit(str(area))
        depth_edit = QLineEdit(str(depth))
        lat_edit = QLineEdit(str(lat))
        lon_edit = QLineEdit(str(lon))
        layout.addRow("Thời gian phát hiện:",
ts_edit)
        layout.addRow("Diện tích (cm²):", area_edit)
        layout.addRow("Độ sâu (cm):", depth_edit)
        layout.addRow("Vĩ độ:", lat_edit)
        layout.addRow("Kinh độ:", lon_edit)

```

```

        buttons =
QDialogButtonBox(QDialogButtonBox.Ok |
QDialogButtonBox.Cancel)
        buttons.accepted.connect(dialog.accept)
        buttons.rejected.connect(dialog.reject)
        layout.addRow(buttons)

        if dialog.exec_() == QDialog.Accepted:
            try:
                new_ts =
ts_edit.dateTime().toPyDateTime().strftime("%d-
%m-%Y %H:%M:%S")
                new_area = float(area_edit.text())
                new_depth = float(depth_edit.text())
                new_volume = new_area * new_depth
                new_lat = float(lat_edit.text())
                new_lon = float(lon_edit.text())
            except ValueError:
                QMessageBox.warning(self, "Lỗi nhập
liệu", "Vui lòng nhập số hợp lệ cho các trường.")
                return
            conn = sqlite3.connect(self.db.db_path)
            cursor = conn.cursor()
            cursor.execute("""
UPDATE pothole_records
SET timestamp=?, area_cm2=?,
depth_cm=?, volume_cm3=?, gps_lat=?,
gps_lon=?
WHERE id=?
""", (new_ts, new_area, new_depth,
new_volume, new_lat, new_lon, rec_id))
            conn.commit()
            conn.close()
            self.load_data()
            QMessageBox.information(self, "Thành
công", f"Đã cập nhật bản ghi ID {rec_id}")

    def on_selection_changed(self):
        selected = self.table.selectedItems()
        if not selected:
            return
        row = selected[0].row()
        record_id = self.table.item(row, 1).text()
        track_id = self.table.item(row, 2).text()
        timestamp = self.table.item(row, 3).text()
        area = self.table.item(row, 4).text()
        depth = self.table.item(row, 5).text()
        volume = self.table.item(row, 6).text()
        gps = self.table.item(row, 7).text()
        img_path = self.table.item(row, 8).text()
        try:
            lat_str, lon_str = gps.split(',')

```

```

        lat = float(lat_str.strip())
        lon = float(lon_str.strip())
    except:
        lat, lon = 0.0, 0.0
        self.view_pothole_image(img_path, record_id,
                                track_id, timestamp, area, depth, volume, gps, lat,
                                lon)

    def view_pothole_image(self, img_path,
                           record_id, track_id, timestamp, area, depth,
                           volume, gps, lat, lon):
        if os.path.exists(img_path):
            pixmap = QPixmap(img_path)

        self.image_label.setPixmap(pixmap.scaled(self.image_label.size(), Qt.KeepAspectRatio,
                                                    Qt.SmoothTransformation))
        else:
            self.image_label.setText("Không tìm thấy ảnh")
            location_text = "Không xác định"
            if self.location_mgr and lat != 0.0 and lon != 0.0:
                try:
                    ward =
self.location_mgr.get_location_name_by_point(lat,
lon, 'ward')
                    province =
self.location_mgr.get_location_name_by_point(lat,
lon, 'province')
                    if province:
                        if ward:
                            location_text = f"{ward},
{province}"
                        else:
                            location_text = province
                except:
                    pass

            info_text = f"""<b>ID:</b> {record_id}<br>
<b>Track ID:</b> {track_id}<br>
<b>Thời gian phát hiện:</b> {timestamp}<br>
<b>Diện tích:</b> {area} cm²<br>
<b>Độ sâu:</b> {depth} cm<br>
<b>Thể tích:</b> {volume} cm³<br>
<b>GPS:</b> {gps}<br>
<b>Địa chỉ:</b> {location_text}<br>
<b>Đường dẫn ảnh:</b> {img_path}"""
            self.info_text.setHtml(info_text)

    def on_image_click(self, event):
        selected = self.table.selectedItems()

```

```

        if not selected:
            return
        row = selected[0].row()
        try:
            db_id = int(self.table.item(row, 1).text())
            gps_text = self.table.item(row, 7).text()
            lat_str, lon_str = gps_text.split(',')
            lat = float(lat_str.strip())
            lon = float(lon_str.strip())
            if lat == 0.0 and lon == 0.0:
                QMessageBox.warning(self, "Không có
tọa độ GPS",
                                "Bản ghi này không có tọa
độ GPS.")
            return
            self.image_clicked.emit(db_id, lat, lon)
        except Exception as e:
            print(f"Lỗi khi xử lý click ảnh: {e}")

    def export_to_excel(self):
        if not PANDAS_AVAILABLE:
            QMessageBox.warning(self, "Thiếu thư
viện",
                                "Cần cài đặt pandas và
openpyxl.\nChạy lệnh: pip install pandas
openpyxl")
            return
        if not self.filtered_rows:
            QMessageBox.information(self, "Xuất
Excel", "Không có dữ liệu để xuất.")
            return
        data = []
        for row in self.filtered_rows:
            record_id, track_id, ts, area, depth, volume,
lat, lon, img_path = row
            location_str = "N/A"
            if self.location_mgr and lat != 0 and lon !=
0:
                ward_name =
self.location_mgr.get_location_name_by_point(lat,
lon, 'ward')
                province_name =
self.location_mgr.get_location_name_by_point(lat,
lon, 'province')
                if province_name:
                    if ward_name:
                        location_str = f"{ward_name},
{province_name}"
                    else:
                        location_str = province_name
            data.append({
                "DB ID": record_id,

```

```

        "Thời gian phát hiện": ts,
        "Diện tích (cm²)": area,
        "Độ sâu (cm)": depth,
        "Thể tích (cm³)": volume,
        "Vĩ độ": lat,
        "Kinh độ": lon,
        "Địa điểm": location_str,
        "Đường dẫn ảnh": img_path
    })
    df = pd.DataFrame(data)
    file_path, _ =
    QFileDialog.getSaveFileName(self, "Xuất Excel",
    "potholes_data.xlsx", "Excel files (*.xlsx)")
    if file_path:
        try:
            df.to_excel(file_path, index=False,
            engine='openpyxl')
            QMessageBox.information(self, "Thành
            công", f"Đã xuất {len(df)} bản ghi ra {file_path}")
        except Exception as e:
            QMessageBox.critical(self, "Lỗi xuất
            file", str(e))
map_tab.py
import os
import json
from PyQt5.QtWidgets import *
from PyQt5.QtCore import pyqtSignal, Qt,
QObject, PyQtSlot
from PyQt5.QtGui import QPixmap
from PyQt5.QtWebEngineWidgets import
QWebEngineView
from PyQt5.QtWebChannel import QWebChannel
from shapely.geometry import Point, mapping, box
from bridge import PotholeBridge
from database.database import DatabaseManager
from location_manager import LocationManager

class MapBridge(QObject):
    markers_ready = pyqtSignal(str)
    geometry_ready = pyqtSignal(str)

    def __init__(self, db: DatabaseManager,
    parent=None):
        super().__init__(parent)
        self.db = db
        self.full_geometry = None
        self.filtered_ids = None

    def set_full_geometry(self, geom):
        self.full_geometry = geom

    def set_filtered_ids(self, ids_set):

```

```

        self.filtered_ids = ids_set

    @pyqtSlot(float, float, float, float, int)
    def load_markers(self, sw_lat, sw_lng, ne_lat,
    ne_lng, zoom):
        if zoom < 10:
            self.markers_ready.emit(json.dumps([]))
            return
        if zoom < 12:
            limit = 500
        elif zoom < 14:
            limit = 800
        else:
            limit = None
        rows = self.db.get_records_in_bbox(sw_lat,
        ne_lat, sw_lng, ne_lng, limit)
        markers = [{"id": r[0], "lat": r[1], "lon":
        r[2]} for r in rows if r[1] != 0 and r[2] != 0]
        if self.filtered_ids is not None:
            markers = [m for m in markers if m["id"] in
            self.filtered_ids]
        self.markers_ready.emit(json.dumps(markers))

    @pyqtSlot(float, float, float, float, int)
    def load_geometry(self, sw_lat, sw_lng, ne_lat,
    ne_lng, zoom):
        if self.full_geometry is None or
        self.full_geometry.is_empty:
            self.geometry_ready.emit("null")
            return
        if zoom <= 9:
            tolerance = 0.01
        elif zoom <= 11:
            tolerance = 0.005
        elif zoom <= 13:
            tolerance = 0.001
        elif zoom <= 15:
            tolerance = 0.0001
        else:
            tolerance = 0.000001
        bbox = box(sw_lng, sw_lat, ne_lng, ne_lat)
        try:
            clipped =
            self.full_geometry.intersection(bbox)
            if clipped.is_empty:
                self.geometry_ready.emit("null")
            return
            simplified = clipped.simplify(tolerance,
            preserve_topology=True)
            if simplified.geom_type in ['Polygon',
            'MultiPolygon']:
                geojson_feature = {

```

```

        "type": "FeatureCollection",
        "features": [{
            "type": "Feature",
            "geometry": mapping(simplified),
            "properties": {}
        }]
    }
    else:
        self.geometry_ready.emit("null")
        return
    geojson_str = json.dumps(geojson_feature)
    if len(geojson_str) > 1.5 * 1024 * 1024:
        simplified = clipped.simplify(tolerance *
2, preserve_topology=True)

```

```

    geojson_feature["features"][0]["geometry"] =
mapping(simplified)
    geojson_str =
json.dumps(geojson_feature)
    self.geometry_ready.emit(geojson_str)
    except Exception as e:
        print(f"Lỗi geometry: {e}")
        self.geometry_ready.emit("null")

```

```

class MapTab(QWidget):
    focus_pothole = pyqtSignal(float, float)

    def __init__(self, db_path, location_mgr=None,
parent=None):
        super().__init__(parent)
        self.db = DatabaseManager(db_path)
        self.location_mgr = location_mgr
        self.current_group_records = []
        self.all_records = []
        self.filtered_records = []
        self.current_highlight_geom = None
        self.setup_ui()
        self.load_all_records()
        self.refresh_map()

```

```

def setup_ui(self):
    layout = QVBoxLayout(self)
    layout.setSpacing(0)
    layout.setContentsMargins(5, 5, 5, 5)
    if self.location_mgr:
        filter_widget = QWidget()
        filter_widget.setMaximumHeight(50)
        filter_layout = QHBoxLayout(filter_widget)
        filter_layout.setContentsMargins(5, 2, 5, 2)
        self.cbo_province = QComboBox()
        self.cbo_province.addItem("-- Tất cả --")

```

```

        self.cbo_province.addItem(self.location_mgr.get_p
rovince_names())
        self.cbo_province.setMinimumWidth(200)

        self.cbo_province.currentIndexChanged.connect(sel
f.on_province_changed)
        self.cbo_ward = QComboBox()
        self.cbo_ward.addItem("-- Tất cả --")
        self.cbo_ward.setMinimumWidth(200)
        self.btn_apply = QPushButton("Lọc")
        self.btn_apply.setFixedWidth(80)
        self.btn_reset = QPushButton("Reset")
        self.btn_reset.setFixedWidth(80)

```

```

        filter_layout.addWidget(QLabel("Tỉnh/Thành
phố:"))
        filter_layout.addWidget(self.cbo_province)

        filter_layout.addWidget(QLabel("Xã/Phường:"))
        filter_layout.addWidget(self.cbo_ward)
        filter_layout.addWidget(self.btn_apply)
        filter_layout.addWidget(self.btn_reset)
        filter_layout.addStretch()
        layout.addWidget(filter_widget)

```

```

        self.btn_apply.clicked.connect(self.apply_filter)

```

```

        self.btn_reset.clicked.connect(self.reset_filter)
        splitter = QSplitter(Qt.Horizontal)
        self.map_view = QWebEngineView()
        self.map_view.setMinimumWidth(500)
        self.channel = QWebChannel()

```

```

        self.map_view.page().setWebChannel(self.channel)
        self.map_bridge = MapBridge(self.db, self)
        self.channel.registerObject("mapBridge",
self.map_bridge)

```

```

        self.map_bridge.markers_ready.connect(self.on_ma
rkers_ready)

```

```

        self.map_bridge.geometry_ready.connect(self.on_g
eometry_ready)
        self.pothole_bridge = PotholeBridge(self)
        self.channel.registerObject("potholeBridge",
self.pothole_bridge)
        right_widget = QWidget()
        right_widget.setMaximumWidth(305)
        right_layout = QVBoxLayout(right_widget)
        right_layout.setContentsMargins(5, 0, 5, 0)
        self.map_combo = QComboBox()

```

```

self.map_combo.currentIndexChanged.connect(self.
on_combo_changed)
    self.map_info_image = QLabel()
    self.map_info_image.setFixedSize(300, 300)

self.map_info_image.setStyleSheet("background-
color: black; border: 1px solid #1abc9c;")
    self.map_info_image.mousePressEvent =
self.on_image_click
    self.map_info_text = QTextEdit()
    self.map_info_text.setReadOnly(True)
    self.map_info_text.setStyleSheet("font-size:
10pt; font-family: Consolas;")
    right_layout.addWidget(QLabel("Chọn ổ gà
tại điểm này:"))
    right_layout.addWidget(self.map_combo)
    right_layout.addWidget(QLabel("Ảnh:"))
    right_layout.addWidget(self.map_info_image)
    right_layout.addWidget(QLabel("Chi tiết:"))
    right_layout.addWidget(self.map_info_text)
    splitter.addWidget(self.map_view)
    splitter.addWidget(right_widget)
    splitter.setSizes([700, 400])
    layout.addWidget(splitter)
    layout.setStretch(0, 0)
    layout.setStretch(1, 1)

def load_all_records(self):
    all_rows = self.db.get_all_map_records()
    self.all_records = [(r[0], r[1], r[2]) for r in
all_rows if r[1] != 0 and r[2] != 0]
    self.filtered_records = self.all_records[:.]

def on_province_changed(self):
    prov = self.cbo_province.currentText()
    self.cbo_ward.blockSignals(True)
    self.cbo_ward.clear()
    self.cbo_ward.addItem("-- Tất cả --")
    if prov != "-- Tất cả --" and self.location_mgr:
        wards =
self.location_mgr.get_wards_by_province(prov)
        self.cbo_ward.addItem(wards)
        self.cbo_ward.blockSignals(False)

def apply_filter(self):
    prov = self.cbo_province.currentText()
    ward = self.cbo_ward.currentText()
    if self.location_mgr is None:
        return
    if ward != "-- Tất cả --":
        location_name = ward

```

```

        level = 'ward'
    elif prov != "-- Tất cả --":
        location_name = prov
        level = 'province'
    else:
        self.filtered_records = self.all_records[:]
        self.current_highlight_geom = None
        self.map_bridge.set_full_geometry(None)
        self.map_bridge.set_filtered_ids(None)
        self.refresh_map()
        return
    geom =
self.location_mgr.get_geometry(location_name,
level)
    if geom is None:
        QMessageBox.warning(self, "Lỗi", f"Không
tìm thấy geometry cho {location_name}")
        self.filtered_records = []
        self.current_highlight_geom = None
        self.map_bridge.set_full_geometry(None)
        self.map_bridge.set_filtered_ids(set())
        self.refresh_map()
        return
    self.current_highlight_geom = geom
    self.map_bridge.set_full_geometry(geom)
    geom_simple = geom.simplify(0.001,
preserve_topology=True)
    epsilon = 1e-5
    filtered = []
    for rec in self.all_records:
        rec_id, lat, lon = rec
        point = Point(lon, lat)
        if
geom_simple.buffer(epsilon).intersects(point):
            filtered.append(rec)
    self.filtered_records = filtered
    self.map_bridge.set_filtered_ids({rec[0] for
rec in filtered})
    self.refresh_map()

def reset_filter(self):
    if self.location_mgr:
        self.cbo_province.setCurrentIndex(0)
        self.cbo_ward.clear()
        self.cbo_ward.addItem("-- Tất cả --")
        self.filtered_records = self.all_records
        self.current_highlight_geom = None
        self.map_bridge.set_full_geometry(None)
        self.map_bridge.set_filtered_ids(None)
        self.refresh_map()
    else:
        self.refresh_map()

```

```

def refresh_map(self):
    html = self.build_map_html()
    self.map_view.setHtml(html)

def build_map_html(self):
    vietnam_bounds = [[6.0, 102.0], [24.0,
118.0]]
    if self.current_highlight_geom:
        bounds =
self.current_highlight_geom.bounds
        fit_js =
f"window.map.fitBounds([[{bounds[1]},
{bounds[0]}], [{bounds[3]}, {bounds[2]}]]);"
    else:
        fit_js =
f"window.map.fitBounds({vietnam_bounds});"

    html = f"""
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <link rel="stylesheet"
href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.c
ss" />
    <script
src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js
"></script>
    <script
src="qrc:///qtwebchannel/qwebchannel.js"></scrip
t>

<style>#map{height:100%;width:100%;position:
absolute;}}
body{margin:0;padding:0;height:100%;}</style>
>
</head>
<body>
    <div id="map"></div>
    <script>
        var map;
        function initMap() {{
            map = L.map('map').setView([15.0,
108.0], 6);
            window.map = map;
            map.setMaxBounds([[6.0, 102.0], [24.0,
118.0]]);
            map.setMinZoom(5);
            map.setMaxZoom(18);

```

```

L.tileLayer('https://mt1.google.com/vt/lyrs=m&x={
{x}}&y={y}&z={z}&hl=vi', {{
    attribution: '© Google Maps'
}}).addTo(map);
var markerGroup =
L.featureGroup().addTo(map);
var geometryLayer = null;
var currentGeojson = null;
function groupMarkers(markers) {{
    var groups = new Map();
    markers.forEach(function(m) {{
        var key = m.lat.toFixed(6) + ',' +
m.lon.toFixed(6);
        if (!groups.has(key)) {{
            groups.set(key, {{ ids: [], lat:
m.lat, lon: m.lon }});
        }}
        groups.get(key).ids.push(m.id);
    }});
    var result = [];
    for (var [, g] of groups.entries()) {{
        result.push({{ id: g.ids[0], lat: g.lat,
lon: g.lon, count: g.ids.length }});
    }}
    return result;
}}

window.updateMarkers =
function(markersJson) {{
    var markers =
JSON.parse(markersJson);
    var grouped =
groupMarkers(markers);
    markerGroup.clearLayers();
    grouped.forEach(function(m) {{
        var popupText = m.count === 1 ?
' 1 ổ gà': '  ' + m.count + ' ổ gà';
        var marker = L.marker([m.lat,
m.lon])
            .bindPopup(popupText)
            .on('click', function() {{
                if (window.potholeBridge)
window.potholeBridge.select_pothole(m.id);
            }});
        markerGroup.addLayer(marker);
    }});
}};

function getPolygonStyle() {{
    var zoom = map.getZoom();

```

```

        if (zoom >= 16) {{
            return {{
                color: 'transparent',
                weight: 2,
                fillOpacity: 0,
                fillColor: '#3388ff'
            }};
        }} else {{
            return {{
                color: '#3388ff',
                weight: 2,
                fillOpacity: 0.3,
                fillColor: '#3388ff'
            }};
        }}
    }}

    function
    updateGeometryLayer(geojsonStr) {{
        if (geometryLayer) {{
            map.removeLayer(geometryLayer);
            geometryLayer = null;
        }}
        if (geojsonStr && geojsonStr !==
        'null') {{
            try {{
                var data =
                JSON.parse(geojsonStr);
                var style = getPolygonStyle();
                geometryLayer =
                L.geoJSON(data, {{ style: style }}).addTo(map);
            }} catch(e) {{
                console.log('Geometry error:', e);
            }}
        }}
    }}

    window.updateGeometry =
    function(geojsonStr) {{
        currentGeojson = geojsonStr;
        updateGeometryLayer(geojsonStr);
    }};

    function updatePolygonStyleByZoom() {{
        if (geometryLayer && currentGeojson
        && currentGeojson !== 'null') {{
            var newStyle = getPolygonStyle();
            geometryLayer.setStyle(newStyle);
        }}
    }}

    function loadMarkers() {{
        if (!window.mapBridge) return;

        var bounds = map.getBounds();
        var sw = bounds.getSouthWest();
        var ne = bounds.getNorthEast();
        var zoom = map.getZoom();

        window.mapBridge.load_markers(sw.lat, sw.lng,
        ne.lat, ne.lng, zoom);
    }}

    function loadGeometry() {{
        if (!window.mapBridge) return;
        var bounds = map.getBounds();
        var sw = bounds.getSouthWest();
        var ne = bounds.getNorthEast();
        var zoom = map.getZoom();

        window.mapBridge.load_geometry(sw.lat, sw.lng,
        ne.lat, ne.lng, zoom);
    }}

    window.loadMarkers = loadMarkers;
    window.loadGeometry = loadGeometry;
    map.on('moveend', function() {{
        loadMarkers();
        loadGeometry();
    }});
    map.on('zoomend', function() {{
        loadMarkers();
        loadGeometry();
        updatePolygonStyleByZoom();
    }});
    new
    QWebChannel(qt.webChannelTransport,
    function(channel) {{
        window.mapBridge =
        channel.objects.mapBridge;
        window.potholeBridge =
        channel.objects.potholeBridge;
        setTimeout(function() {{
            map.invalidateSize();
            {fit_js}
            loadMarkers();
            loadGeometry();
        }}, 200);
    }});
}}

document.addEventListener('DOMContentLoaded',
initMap);
</script>
</body>
</html>

```

```

"""
return html

def on_markers_ready(self, markers_json):
    js =
f"window.updateMarkers({json.dumps(markers_json)});"
    self.map_view.page().runJavaScript(js)

def on_geometry_ready(self, geojson_str):
    if geojson_str == "null":
        js = "window.updateGeometry(null);"
    else:
        js =
f"window.updateGeometry({json.dumps(geojson_str)});"
    self.map_view.page().runJavaScript(js)

def display_pothole_by_id(self, pothole_id):
    row = self.db.get_pothole_by_id(pothole_id)
    if not row:
        return
    lat, lon = row[5], row[6]
    if lat == 0 or lon == 0:
        return
    records = self.db.get_group_by_gps(lat, lon)
    if not records:
        return
    self.current_group_records = records
    self.map_combo.clear()
    for rec in records:
        self.map_combo.addItem(f"ID {rec[0]}",
rec[0])
    self.map_combo.setCurrentIndex(0)

def on_combo_changed(self, index):
    if index < 0 or not self.current_group_records:
        return
    rec_id = self.map_combo.itemData(index)
    for rec in self.current_group_records:
        if rec[0] == rec_id:
            img_path = rec[7]
            if img_path and os.path.exists(img_path):
                pixmap = QPixmap(img_path)

self.map_info_image.setPixmap(pixmap.scaled(
    self.map_info_image.size(),
Qt.KeepAspectRatio, Qt.SmoothTransformation))
    else:
        self.map_info_image.clear()
        self.map_info_image.setText("Không
tìm thấy ảnh")

```

```

        area, depth, volume, ts, lat, lon = rec[1],
rec[2], rec[3], rec[4], rec[5], rec[6]
        location_text = ""
        if self.location_mgr and lat != 0 and lon
!= 0:
            try:
                province =
self.location_mgr.get_location_name_by_point(lat,
lon, 'province')
                ward =
self.location_mgr.get_location_name_by_point(lat,
lon, 'ward')
                if province:
                    if ward:
                        location_text = f"<br><b>Địa
chỉ:</b> {ward}, {province}"
                    else:
                        location_text = f"<br><b>Địa
chỉ:</b> {province}"
            except Exception as e:
                print(f"Lỗi địa chỉ MapTab: {e}")
            info = f"<b>ID:</b>
{rec_id}<br><b>Diện tích:</b> {area:.1f}
cm²<br><b>Độ sâu:</b> {depth:.1f}
cm<br><b>Thể tích:</b> {volume:.1f}
cm³<br><b>Thời gian phát hiện:</b>
{ts}<br><b>GPS:</b> ({lat:.6f},
{lon:.6f}){location_text}"
            self.map_info_text.setHtml(info)
            break

def on_image_click(self, event):
    idx = self.map_combo.currentIndex()
    if idx < 0:
        return
    rec_id = self.map_combo.itemData(idx)
    if rec_id is None:
        return
    for rec in self.current_group_records:
        if rec[0] == rec_id:
            lat, lon = rec[5], rec[6]
            if lat != 0 and lon != 0:
                js = f""
                if (window.map &&
window.map.setView) {{
                    window.map.setView([lat], [lon],
19);
                    if (window.tempMarker)
window.tempMarker.remove();
                    window.tempMarker =
L.marker([lat], [lon]).addTo(window.map)
                    .bindPopup('Ồ gà ID {rec_id}')

```

```

        .openPopup();
        setTimeout(function(){
            if(window.tempMarker)
                window.tempMarker.remove();
            }, 8000);
        }
        """
        self.map_view.page().runJavaScript(js)
        break

def fill_panel_by_db_id(self, db_id):
    row = self.db.get_pothole_by_id(db_id)
    if not row:
        return
    lat, lon = row[5], row[6]
    if lat == 0 or lon == 0:
        return
    records = self.db.get_group_by_gps(lat, lon)
    if not records:
        records = [row]
    self.current_group_records = records
    self.map_combo.blockSignals(True)
    self.map_combo.clear()
    target_index = 0
    for i, rec in enumerate(records):
        self.map_combo.addItem(f"ID {rec[0]}",
rec[0])
        if rec[0] == db_id:
            target_index = i
    self.map_combo.blockSignals(False)

self.map_combo.setCurrentIndex(target_index)
self.on_combo_changed(target_index)

def focus_on_pothole(self, lat, lon):
    if lat == 0 or lon == 0:
        return
    js = f"""
    if (window.map && window.map.setView) {{
        window.map.setView([{{lat}}, {{lon}}], 18);
        if (window.tempMarker)
            window.tempMarker.remove();
        window.tempMarker = L.marker([{{lat}},
{{lon}}]).addTo(window.map)
        .bindPopup('Ồ gà tại đây')
        .openPopup();
        setTimeout(function(){
            if(window.tempMarker)
                window.tempMarker.remove();
            }, 8000);
        }} else {{
            console.log('Map chưa sẵn sàng');

```

```

        }}
        """
        self.map_view.page().runJavaScript(js)
        if self.location_mgr:
            province_name =
self.location_mgr.get_location_name_by_point(lat,
lon, 'province')
            ward_name =
self.location_mgr.get_location_name_by_point(lat,
lon, 'ward')
            if province_name:
                self.cbo_province.blockSignals(True)

self.cbo_province.setCurrentText(province_name)
self.cbo_province.blockSignals(False)
self.on_province_changed()
            if ward_name:
                self.cbo_ward.blockSignals(True)

self.cbo_ward.setCurrentText(ward_name)
self.cbo_ward.blockSignals(False)

def refresh_markers_only(self):
    self.map_view.page().runJavaScript("if(window loa
dMarkers) window.loadMarkers();")

def add_new_pothole(self, pothole_id, lat, lon):
    self.all_records.append((pothole_id, lat, lon))
    if self.current_highlight_geom is not None:
        point = Point(lon, lat)
        if self.current_highlight_geom.buffer(1e-
5).intersects(point):
            self.filtered_records.append((pothole_id,
lat, lon))
            if self.map_bridge.filtered_ids is not
None:
                self.map_bridge.filtered_ids.add(pothole_id)
            else:
                self.filtered_records.append((pothole_id,
lat, lon))
            self.refresh_markers_only()
processor_thread.py
import os
import time
import uuid
import sqlite3
import cv2
import numpy as np
from datetime import datetime

```

```

from PyQt5.QtCore import QObject, pyqtSignal,
pyqtSlot
from ultralytics import YOLO
from processors.twinlite import TwinLiteProcessor
from processors.tracker import PotholeTracker

```

```

class PotholeProcessor(QObject):
    image_processed = pyqtSignal(np.ndarray, list)
    roi1_updated = pyqtSignal(list)
    pothole_saved = pyqtSignal(dict)
    data_refresh = pyqtSignal()
    map_refresh = pyqtSignal()
    metadata_updated = pyqtSignal(dict)

```

```

    def __init__(self, pixel_to_cm2, db_path,
images_dir):

```

```

        super().__init__()
        self.pixel_to_cm2 = pixel_to_cm2
        self.db_path = db_path
        self.images_dir = images_dir
        self.tracker = None
        self.yolo_model = None
        self.twinlite = None
        self.pothole_detection_enabled = True
        self.conf_threshold = 0.5
        self.roi1_rect = (0, 0, 400, 600)
        self.roi2_rect = (0, 0, 400, 600)
        self.bev_w = 400
        self.bev_h = 600

```

```

    def init_models(self, yolo_path, twinlite_path):

```

```

        self.yolo_model = YOLO(yolo_path)
        self.twinlite =

```

```

TwinLiteProcessor(twinlite_path)

```

```

        self.tracker =

```

```

PotholeTracker(pixel_to_cm2=self.pixel_to_cm2,
iou_threshold=0.3)

```

```

    def set_rois(self, roi1_rect, roi2_rect, bev_w,
bev_h):

```

```

        self.roi1_rect = roi1_rect
        self.roi2_rect = roi2_rect
        self.bev_w = bev_w
        self.bev_h = bev_h

```

```

    def set_pothole_detection(self, enabled):

```

```

        self.pothole_detection_enabled = enabled
        if not enabled and self.tracker:
            self.tracker.tracks.clear()
            self.tracker.next_id = 1

```

```

    def is_center_in_roi2(self, box):

```

```

        x1, y1, x2, y2 = box
        cx = (x1 + x2) // 2
        cy = (y1 + y2) // 2
        rx1, ry1, rx2, ry2 = self.roi2_rect
        return (rx1 <= cx <= rx2 and ry1 <= cy <=
ry2)

```

```

    def is_fully_in_roi1(self, box):

```

```

        x1, y1, x2, y2 = box
        rx1, ry1, rx2, ry2 = self.roi1_rect
        return (x1 >= rx1 and x2 <= rx2 and y1 >=
ry1 and y2 <= ry2)

```

```

    def detect_potholes(self, bev_img):

```

```

        results = self.yolo_model(bev_img,
conf=self.conf_threshold, verbose=False)
        h, w = bev_img.shape[:2]
        detections = []
        boxes = results[0].boxes
        masks = results[0].masks
        if boxes is not None:
            for i, box in enumerate(boxes):
                conf = box.conf.item()
                if conf < self.conf_threshold:
                    continue
                x1, y1, x2, y2 = map(int,
box.xyxy[0].tolist())
                mask = None
                if masks is not None:
                    m = masks.data[i].cpu().numpy()
                    m = cv2.resize(m, (w, h))
                    mask = (m > 0.5).astype(np.uint8)
                    area_px = mask.sum() if mask is not
None else (x2 - x1) * (y2 - y1)
                detections.append({'box': (x1, y1, x2, y2),
'confidence': conf, 'mask': mask, 'area_px':
area_px})
            return detections

```

```

    def filter_potholes_by_da(self, detections,
da_mask, overlap_threshold=0.3):

```

```

        filtered = []
        for det in detections:
            if det['mask'] is None:
                x1, y1, x2, y2 = det['box']
                if y2 <= y1 or x2 <= x1:
                    continue
                box_region = da_mask[y1:y2, x1:x2]
                if box_region.size == 0:
                    continue
                overlap = (box_region > 0).sum() /
box_region.size

```

```

else:
    overlap = np.logical_and(det['mask'],
da_mask).sum() / (det['mask'].sum() + 1e-5)
    if overlap >= overlap_threshold:
        filtered.append(det)
    return filtered

def save_pothole_record(self, track_id,
image_crop, area_cm2, depth_cm, lat, lon,
timestamp_str):
    volume_cm3 = area_cm2 * depth_cm
    try:
        dt = datetime.strptime(timestamp_str, "%d-
%m-%Y %H:%M:%S")
        date_folder = dt.strftime("%Y-%m-%d") #
vi du: "2025-01-15"
    except:
        date_folder = datetime.now().strftime("%Y-
%m-%d")
    save_dir = os.path.join(self.images_dir,
date_folder)
    os.makedirs(save_dir, exist_ok=True)
    safe_timestamp = timestamp_str.replace(" ",
"_").replace(":", "-").replace("/", "-")
    unique_id = str(uuid.uuid4())[8]
    filename =
f"pothole_track{track_id}_{safe_timestamp}_{uni
ue_id}.jpg"
    filepath = os.path.join(save_dir, filename)
    success = cv2.imwrite(filepath, image_crop)
    if not success:
        print(f"[Processor] Cannot save image:
{filepath}")
        return None
    conn = sqlite3.connect(self.db_path)
    cursor = conn.cursor()
    cursor.execute("""
INSERT INTO pothole_records (track_id,
timestamp, area_cm2, depth_cm, volume_cm3,
gps_lat, gps_lon, image_path)
VALUES (?, ?, ?, ?, ?, ?, ?, ?)
""", (track_id, timestamp_str, area_cm2,
depth_cm, volume_cm3, lat, lon, filepath))
    conn.commit()
    record_id = cursor.lastrowid
    conn.close()
    print(f"[Processor] Saved pothole
ID={record_id}, max depth={depth_cm:.1f} cm,
volume={volume_cm3:.1f} cm³")
    return record_id

def format_timestamp(self, ts_str):

```

```

if not isinstance(ts_str, str) or ts_str == "N/A":
    return datetime.now().strftime("%d-%m-%Y
%H:%M:%S")
    try:
        if " " in ts_str and len(ts_str) >= 15:
            date_part, time_part = ts_str.split(" ")
            year, month, day = date_part[:4],
date_part[4:6], date_part[6:8]
            hour, minute, second = time_part[:2],
time_part[2:4], time_part[4:6]
            return f"{day}-{month}-{year}
{hour}:{minute}:{second}"
            cleaned = ts_str.replace('T', ' ').replace('Z',
").split('.')[0]
            dt = datetime.strptime(cleaned, "%Y-%m-
%d %H:%M:%S")
            return dt.strftime("%d-%m-%Y
%H:%M:%S")
        except:
            return ts_str

@pytestSlot(np.ndarray, dict)
def process_frame(self, cv_img, metadata):
    self.metadata_updated.emit(metadata)
    if not self.pothole_detection_enabled:
        self.image_processed.emit(cv_img, [])
        return
    da_mask = self.twinlite.predict(cv_img)
    all_potholes = self.detect_potholes(cv_img)
    potholes_in_da =
self.filter_potholes_by_da(all_potholes, da_mask)
    self.tracker.update(potholes_in_da, cv_img)
    active_tracks =
self.tracker.get_active_tracks()
    gps = metadata.get('gps', {})
    valid = gps.get('valid', False)
    if valid:
        lat = gps.get('latitude', 0.0)
        lon = gps.get('longitude', 0.0)
    else:
        lat = 0.0
        lon = 0.0
    depth_cm = metadata.get('sensors',
{}).get('pothole_depth_cm', 0.0)
    raw_ts = metadata.get('timestamp_full',
metadata.get('timestamp',
datetime.now().strftime("%Y-%m-%d
%H:%M:%S")))
    ts_str = self.format_timestamp(raw_ts)
    roi1_tracks_data = []
    for track in active_tracks:
        if self.is_fully_in_roi1(track.box):

```

```

        track.update_max_area()
        if not track.has_captured:

track.capture_image_from_roi1(cv_img,
expand=30)
        roi1_tracks_data.append({
            'id': track.id,
            'area_cm2': track.area_cm2,
            'max_area_cm2': track.max_area_cm2,
            'box': track.box,
            'has_captured': track.has_captured,
            'captured_image':
track.captured_image
        })
        currently_in_roi2 =
self.is_center_in_roi2(track.box)
        if currently_in_roi2:
            track.update_max_depth(depth_cm)
            if not track.has_entered_roi2:
                track.has_entered_roi2 = True
            if track.has_entered_roi2 and not
currently_in_roi2 and not track.saved and
track.has_captured:
                area_cm2 = track.max_area_cm2
                crop = track.captured_image
                if crop is not None:
                    max_depth = track.max_depth_cm
                    if max_depth > 0.5:
                        self.save_pothole_record(track.id,
crop, area_cm2, max_depth, lat, lon, ts_str)
                        track.saved = True
                        saved_info = {
                            'track_id': track.id,
                            'area': area_cm2,
                            'depth': max_depth,
                            'volume': area_cm2 * max_depth,
                            'lat': lat,
                            'lon': lon,
                            'timestamp': ts_str,
                            'image': crop
                        }
                        self.pothole_saved.emit(saved_info)
                        self.data_refresh.emit()
                        self.map_refresh.emit()
                    display_img = cv_img.copy()
                    for track in active_tracks:
                        x1, y1, x2, y2 = track.box
                        cv2.rectangle(display_img, (x1, y1), (x2,
y2), (0, 255, 0), 2)
                        cv2.putText(display_img, f"ID:{track.id}",
(x1, y1-5), cv2.FONT_HERSHEY_SIMPLEX, 0.4,
(0,255,0), 1)

```

```

        display_img =
self.twinlute.overlay_masks(display_img, da_mask,
alpha=0.3)
        self.image_processed.emit(display_img,
active_tracks)
        self.roi1_updated.emit(roi1_tracks_data)
udp_thread.py
import socket
import struct
import cv2
import numpy as np
import json
import time
from PyQt5.QtCore import QThread, pyqtSignal

class UDPVideoThread(QThread):
    change_pixmap_signal = pyqtSignal(np.ndarray,
dict)
    connection_status_signal = pyqtSignal(str)

    def __init__(self, port=5557):
        super().__init__()
        self.port = port
        self._run_flag = True
        self.fragments = {}
        self.cleanup_interval = 5.0
        self.last_cleanup = time.time()
        self.connected = False
        self.last_received = time.time()
        self.last_frame_shape = None
        self.sent_black_on_disconnect = False

    def run(self):
        sock = socket.socket(socket.AF_INET,
socket.SOCK_DGRAM)
        sock.setsockopt(socket.SOL_SOCKET,
socket.SO_REUSEADDR, 1)
        sock.bind(('0.0.0.0', self.port))
        sock.settimeout(1.0)
        self.connection_status_signal.emit(f"UDP
listening on port {self.port} (fragmented)")
        while self._run_flag:
            try:
                data, addr = sock.recvfrom(65536)
                if len(data) < 6:
                    continue
                self.last_received = time.time()
                if not self.connected:
                    self.connected = True
                    self.sent_black_on_disconnect = False
                self.connection_status_signal.emit("Connected")

```

```

        frame_id, num_chunks, chunk_idx =
struct.unpack('!HHH', data[:6])
        chunk_data = data[6:]
        if frame_id not in self.fragments:
            self.fragments[frame_id] = {
                'num_chunks': num_chunks,
                'chunks': {},
                'last_update': time.time()
            }

        self.fragments[frame_id]['chunks'][chunk_idx] =
        chunk_data
        self.fragments[frame_id]['last_update'] =
        time.time()
        frag = self.fragments[frame_id]
        if len(frag['chunks']) == num_chunks:
            total_data = b''.join(frag['chunks'][i]
        for i in range(num_chunks))
            del self.fragments[frame_id]
            if len(total_data) < 8:
                continue
            meta_len = struct.unpack('!I',
        total_data[:4])[0]
            if meta_len <= 0 or meta_len >
        10000000:
                continue
            if len(total_data) < 4 + meta_len + 4:
                continue
            metadata_json =
        total_data[4:4+meta_len].decode('utf-8')
            metadata = json.loads(metadata_json)
            frame_len = struct.unpack('!I',
        total_data[4+meta_len:8+meta_len])[0]
            if frame_len <= 0 or frame_len >
        50000000:
                continue
            frame_start = 8 + meta_len
            if len(total_data) < frame_start +
        frame_len:
                continue
            frame_data =
        total_data[frame_start:frame_start+frame_len]
            frame =
        cv2.imdecode(np.frombuffer(frame_data, np.uint8),
        cv2.IMREAD_COLOR)
            if frame is not None:
                self.last_frame_shape = frame.shape

        self.change_pixmap_signal.emit(frame, metadata)
        now = time.time()
        if now - self.last_cleanup >=
        self.cleanup_interval:

```

```

        expired = [fid for fid, info in
        self.fragments.items() if now - info['last_update'] >
        5.0]
        for fid in expired:
            del self.fragments[fid]
            self.last_cleanup = now
        except socket.timeout:
            if self.connected and (time.time() -
        self.last_received > 10.0):
                self.connected = False

        self.connection_status_signal.emit("Disconnected")
            if self.last_frame_shape is not None
        and not self.sent_black_on_disconnect:
                black_frame =
        np.zeros(self.last_frame_shape, dtype=np.uint8)
        self.change_pixmap_signal.emit(black_frame, {})
            self.sent_black_on_disconnect =
        True
            continue
        except Exception as e:

        self.connection_status_signal.emit(f"Error: {e}")
            sock.close()

        def stop(self):
            self._run_flag = False
            self.wait()

bird_eye_label.py
        from PyQt5.QtWidgets import QLabel
        from PyQt5.QtGui import QPainter, QPen, QColor
        from PyQt5.QtCore import Qt

        class BirdEyeLabel(QLabel):
            def __init__(self, roi_rects=None, parent=None):
                super().__init__(parent)
                self.roi_rects = roi_rects if roi_rects is not
        None else []
                self.setAlignment(Qt.AlignCenter)
                self.setStyleSheet("background-color: black;
        border: 2px solid #34495e;")
                self.setScaledContents(False)
                self.pixmap = None

            def setPixmap(self, pixmap):
                self.pixmap = pixmap
                self.update()

            def paintEvent(self, event):
                if self.pixmap and not self.pixmap.isNull():
                    painter = QPainter(self)

```

```

        scaled_pix = self.pixmap.scaled(self.size(),
Qt.KeepAspectRatio, Qt.SmoothTransformation)
        x = (self.width() - scaled_pix.width()) // 2
        y = (self.height() - scaled_pix.height()) // 2
        painter.drawPixmap(x, y, scaled_pix)
        img_w = self.pixmap.width()
        img_h = self.pixmap.height()
        scale = min(self.width() / img_w,
self.height() / img_h)
        offset_x = (self.width() - img_w * scale) / 2
        offset_y = (self.height() - img_h * scale) / 2

```

```

        for (rect, color) in self.roi_rects:
            x1, y1, x2, y2 = rect
            rx1 = offset_x + x1 * scale
            ry1 = offset_y + y1 * scale
            rx2 = offset_x + x2 * scale
            ry2 = offset_y + y2 * scale
            painter.setPen(QPen(color, 2))
            painter.drawRect(int(rx1), int(ry1),
int(rx2 - rx1), int(ry2 - ry1))
        else:
            super().paintEvent(event)

```

roi_table_widget.py

```

from PyQt5.QtWidgets import QGroupBox,
QVBoxLayout, QTableWidget, QTableWidgetItem,
QAbstractItemView
class ROITableWidget(QGroupBox):
    def __init__(self, parent=None):
        super().__init__("Các ổ gà trong vùng 1",
parent)
        self.setStyleSheet("color: #0ff; font-weight:
bold;")
        layout = QVBoxLayout(self)
        self.table = QTableWidget()
        self.table.setColumnCount(4)
        self.table.setHorizontalHeaderLabels(["ID",
"Area (cm²)", "Max Area (cm²)", "BBox"])
        self.table.setSelectionBehavior(QAbstractItemView.
SelectRows)
        self.table.setEditTriggers(QAbstractItemView.NoEd
itTriggers)
        self.table.setStyleSheet("""
QTableWidget {
    background-color: #f5f6fa;
    color: #2f3640;
    gridline-color: #dcdde1;
    border: 1px solid #7f8c8d;
    selection-background-color: #d1d8e0;
    selection-color: black;

```

```

}
QHeaderView::section {
    background-color: #dcdde1;
    color: #2f3640;
    padding: 5px;
    border: 1px solid #bdc3c7;
    font-weight: bold;
}
""")
layout.addWidget(self.table)

```

```

def update_data(self, roi1_tracks_data):
    self.table.setRowCount(0)
    total_area = 0
    for idx, t in enumerate(roi1_tracks_data):
        self.table.insertRow(idx)
        self.table.setItem(idx, 0,
QTableWidgetItem(str(t['id'])))
        self.table.setItem(idx, 1,
QTableWidgetItem(f"{t['area_cm2']:.1f}"))
        self.table.setItem(idx, 2,
QTableWidgetItem(f"{t['max_area_cm2']:.1f}"))
        self.table.setItem(idx, 3,
QTableWidgetItem(str(t['box'])))
        total_area += t['area_cm2']
    return len(roi1_tracks_data), total_area

```

saved_pothole_widget.py

```

import cv2
from PyQt5.QtWidgets import QGroupBox,
QVBoxLayout, QLabel, QTextEdit
from PyQt5.QtCore import pyqtSignal, Qt
from PyQt5.QtGui import QPixmap, QImage
class SavedPotholeWidget(QGroupBox):
    image_clicked = pyqtSignal(float, float, int) #
lat, lon, track_id
    def __init__(self, parent=None):
        super().__init__("Ổ gà vừa được lưu", parent)
        self.setStyleSheet("color: #1abc9c; font-
weight: bold;")
        layout = QVBoxLayout(self)
        self.image_label = QLabel()
        self.image_label.setAlignment(Qt.AlignCenter)
        self.image_label.setFixedHeight(200)
        self.image_label.setStyleSheet("background-
color: black; border: 1px solid #1abc9c;")
        self.image_label.setText("Chưa có ổ gà nào
được lưu")
        self.image_label.mousePressEvent =
self.on_image_click
        self.info_label = QTextEdit()

```

```

self.info_label.setReadOnly(True)
self.info_label.setMaximumHeight(150)
self.info_label.setStyleSheet("background-
color: #34495e; color: #ecf0f1;")

layout.addWidget(self.image_label)
layout.addWidget(self.info_label)

self.last_pothole = None # (lat, lon, track_id)

def update_saved_pothole(self, info):
    if info['image'] is not None:
        crop_rgb = cv2.cvtColor(info['image'],
cv2.COLOR_BGR2RGB)
        h, w, ch = crop_rgb.shape
        qimg = QImage(crop_rgb.data, w, h, ch*w,
QImage.Format_RGB888)
        pix = QPixmap.fromImage(qimg)

self.image_label.setPixmap(pix.scaled(self.image_l
abel.size(), Qt.KeepAspectRatio,
Qt.SmoothTransformation))
        text = f"Track ID: {info['track_id']}
GPS: ({info['lat']:.6f}, {info['lon']:.6f})
Diện tích max: {info['area']:.1f} cm2
Độ sâu: {info['depth']:.1f} cm
Thể tích: {info['volume']:.1f} cm3
Thời gian phát hiện: {info['timestamp']}""
        self.info_label.setPlainText(text)
        self.last_pothole = (info['lat'], info['lon'],
info['track_id'])

def on_image_click(self, event):
    if self.last_pothole:
        lat, lon, track_id = self.last_pothole
        self.image_clicked.emit(lat, lon, track_id)
switch.py
from PyQt5.QtWidgets import QCheckBox
from PyQt5.QtGui import QPainter, QColor, QPen
from PyQt5.QtCore import Qt, QRect

class Switch(QCheckBox):
    def __init__(self, text="", parent=None):
        super().__init__(text, parent)
        self.setCursor(Qt.PointingHandCursor)

    def paintEvent(self, event):
        painter = QPainter(self)
        painter.setRenderHint(QPainter.Antialiasing)
        switch_width = 50
        switch_height = 28
        spacing = 8

```

```

    if self.isChecked():
        bg_color = QColor(52, 199, 89)
    else:
        bg_color = QColor(224, 224, 224)
    painter.setBrush(bg_color)
    painter.setPen(Qt.NoPen)
    painter.drawRoundedRect(0, 0, switch_width,
switch_height, switch_height/2, switch_height/2)
    handle_radius = switch_height - 4
    if self.isChecked():
        handle_x = switch_width - handle_radius -
2
    else:
        handle_x = 2
    painter.setBrush(QColor(255, 255, 255))
    painter.drawEllipse(handle_x, 2,
handle_radius, handle_radius)
    text = self.text()
    if text:
        painter.setPen(QPen(QColor(236, 240,
241)))
        font = painter.font()
        font.setPointSize(10)
        font.setBold(True)
        painter.setFont(font)
        text_rect = QRect(switch_width + spacing,
0, self.width() - switch_width - spacing,
self.height())
        painter.drawText(text_rect, Qt.AlignLeft |
Qt.AlignVCenter, text)
        self.setFixedSize(switch_width + spacing +
painter.fontMetrics().width(text) + 10,
switch_height)

def mousePressEvent(self, event):
    self.setChecked(not self.isChecked())
    self.update()

def mouseReleaseEvent(self, event):
    self.toggled.emit(self.isChecked())
bridge.py
from PyQt5.QtCore import QObject, pyqtSlot
class PotholeBridge(QObject):
    def __init__(self, map_tab):
        super().__init__()
        self.map_tab = map_tab

    @pyqtSlot(int)
    def select_pothole(self, pothole_id):
        print(f"[Bridge] Received pothole ID:
{pothole_id}")

```

```

self.map_tab.display_pothole_by_id(pothole_id)
location_manager.py
import os
import json
from shapely.geometry import shape, Point

class LocationManager:
    def __init__(self, geojson_base):
        self.geojson_base = geojson_base
        self.province_geometries = {}
        self.ward_geometries = {}
        self.wards_by_province = {}
        self._load_all()

    def _load_geojson(self, filepath, target_dict,
is_ward=False):
        with open(filepath, 'r', encoding='utf-8') as f:
            data = json.load(f)
            for feature in data['features']:
                props = feature['properties']
                name = None
                province_name = None
                if is_ward:
                    name = props.get('ten_xa') or
props.get('name')
                    province_name = props.get('ten_tinh') or
props.get('province_name')
                else:
                    name = props.get('name') or
props.get('ten_tinh')
                if name:
                    geometry = shape(feature['geometry'])
                    target_dict[name] = geometry
                    if is_ward and province_name:

self.wards_by_province.setdefault(province_name,
[]).append(name)

    def _load_all(self):
        for root, _, files in os.walk(self.geojson_base):
            for f in files:
                if f.endswith("-tinh-thanh-34.geojson"):
                    self._load_geojson(os.path.join(root,
f), self.province_geometries, is_ward=False)
                elif f.endswith("-phuong-xa-34.geojson"):
                    self._load_geojson(os.path.join(root,
f), self.ward_geometries, is_ward=True)

    def get_province_names(self):
        return sorted(self.province_geometries.keys())

```

```

    def get_wards_by_province(self,
province_name):
        return
sorted(self.wards_by_province.get(province_name,
[]))

    def get_geometry(self, name, level='province'):
        if level == 'province':
            return self.province_geometries.get(name)
        else:
            return self.ward_geometries.get(name)

    def is_point_in_location(self, lat, lon,
location_name, level='province'):
        geom = self.get_geometry(location_name,
level)
        if not geom:
            return False
        return geom.contains(Point(lon, lat))

    def filter_records(self, records, location_name,
level='province'):
        geom = self.get_geometry(location_name,
level)
        if not geom:
            return []
        result = []
        for rec in records:
            lat = rec[-2]
            lon = rec[-1]
            if geom.contains(Point(lon, lat)):
                result.append(rec)
        return result

    def get_location_name_by_point(self, lat, lon,
level='province'):
        from shapely.geometry import Point
        point = Point(lon, lat)
        if level == 'province':
            target_dict = self.province_geometries
        else:
            target_dict = self.ward_geometries
        for name, geom in target_dict.items():
            if geom.contains(point):
                return name
        return None
main_window.py
import sys
import os
import time
import sqlite3
import cv2

```

```

import numpy as np
from PyQt5.QtWidgets import *
from PyQt5.QtCore import *
from PyQt5.QtGui import *

from threads.udp_thread import UDPVideoThread
from threads.processor_thread import
PotholeProcessor
from widgets.bird_eye_label import BirdEyeLabel
from widgets.saved_pothole_widget import
SavedPotholeWidget
from widgets.roi_table_widget import
ROIWidget
from tabs.data_tab import DataTab
from tabs.map_tab import MapTab
from panels.metadata_panel import MetadataPanel
from bridge import PotholeBridge
from location_manager import LocationManager
from database.database import DatabaseManager

class MainWindow(QMainWindow):
    def __init__(self, init_data=None):
        super().__init__()
        self.setWindowTitle("Phần mềm phát hiện ổ
gà")
        self.resize(1400, 800)
        self.setStyleSheet("background-color:
#2c3e50; color: white;")
        if init_data is not None:
            # ----- DỪNG DỮ LIỆU TỪ SPLASH --
            -----
            self.db_path = init_data['db_path']
            self.images_dir = init_data['images_dir']
            self.location_mgr =
init_data['location_mgr']
            self.processor = init_data['processor']
            self.processor_thread =
init_data['processor_thread']
            self.udp_thread = init_data['udp_thread']
            self.bev_w = init_data['bev_w']
            self.bev_h = init_data['bev_h']
            self.pixel_to_cm2 =
init_data['pixel_to_cm2']
            self.roi1_rect = init_data['roi1_rect']
            self.roi2_rect = init_data['roi2_rect']
            self.roi1_center_y_ratio =
init_data['roi1_center_y_ratio']
            self.roi1_height_ratio =
init_data['roi1_height_ratio']
            self.roi2_center_y_ratio =
init_data['roi2_center_y_ratio']

```

```

        self.roi2_height_ratio =
init_data['roi2_height_ratio']
        self.db = DatabaseManager(self.db_path)
        self.pothole_detection_enabled = False
        # Tạo metadata panel
        self.meta_panel = MetadataPanel()

        self.meta_panel.pothole_toggle.setChecked(self.pot
hole_detection_enabled)
        # Tạo giao diện
        self._setup_gui()
        # Cập nhật ROI cho bird_label
        self.update_rois()
        # Kết nối signals
        self._connect_signals()
        # Bật chế độ phát hiện (mặc định tắt)
        self.toggle_pothole_detection(False)
        # Bắt đầu UDP thread (chưa start ở splash)
        self.udp_thread.start()
        self.last_time = time.time()
        self.frame_count = 0
        self.load_data()
    else:
        # ----- KHỞI TẠO THÔNG THƯỜNG
        (KHÔNG SPLASH) -----
        # BEV config
        self.bev_w = 400
        self.bev_h = 600
        self.pixel_to_cm2 = (60.0 / 400.0) ** 2
        self.roi1_center_y_ratio = 0.42
        self.roi1_height_ratio = 0.2
        self.roi2_center_y_ratio = 0.82
        self.roi2_height_ratio = 0.3
        self.pothole_detection_enabled = False
        self.update_rois()
        # Database & folder
        self.db_path = os.path.join("database",
"potholes.db")
        self.db = DatabaseManager(self.db_path)
        os.makedirs("database", exist_ok=True)
        self.images_dir = os.path.abspath("OGa")
        os.makedirs(self.images_dir, exist_ok=True)
        self.init_database()
        # LocationManager
        geojson_base = os.path.join("database",
"geojson")
        if os.path.exists(geojson_base):
            self.location_mgr =
LocationManager(geojson_base)
        else:
            print(f"Warning: Geojson folder not
found at {geojson_base}")

```

```

        self.location_mgr = None
        # Metadata panel
        self.meta_panel = MetadataPanel()

    self.meta_panel.pothole_toggle.setChecked(self.pothole_detection_enabled)
    # Worker thread
    self.processor_thread = QThread()
    self.processor = PotholeProcessor(self.pixel_to_cm2, self.db_path, self.images_dir)

    self.processor.moveToThread(self.processor_thread)

    self.processor_thread.start()
    self.processor.init_models(
        os.path.join("model", "epoch99.pt"),
        os.path.join("model", "model_249.pth")
    )
    self.processor.set_rois(self.roi1_rect, self.roi2_rect, self.bev_w, self.bev_h)
    self.toggle_pothole_detection(False)
    # UDP thread
    self.udp_thread = UDPVideoThread(5557)
    # GUI
    self._setup_gui()
    self._connect_signals()
    self.last_time = time.time()
    self.frame_count = 0
    self.udp_thread.start()
    self.load_data()

# ----- CÁC PHƯƠNG THỨC CHUNG -----
def _setup_gui(self):
    """Tạo toàn bộ giao diện (được dùng chung cho cả hai trường hợp)"""
    self.central_widget = QWidget()
    self.setCentralWidget(self.central_widget)
    self.main_layout = QHBoxLayout(self.central_widget)
    self.tab_widget = QTabWidget()
    self.tab_widget.setStyleSheet("""
        QTabWidget::pane { border: 1px solid
#34495e; background: #2c3e50; }
        QTabBar::tab { height: 35px; width: 140px;
background-color: #34495e; color: #bdc3c7;
font-weight: bold; padding: 5px;
margin: 2px;
border-top-left-radius: 5px;
border-top-right-radius: 5px; }
    """)

```

```

        QTabBar::tab:selected { background-color:
#1abc9c; color: white; }
        QTabBar::tab:hover { background-color:
#16a085; color: white; }
    """)
    # Tab Camera (BEV)
    self.bev_tab = QWidget()
    be_v_layout = QHBoxLayout(self.bev_tab)
    left_widget = QWidget()
    left_layout = QVBoxLayout(left_widget)
    self.bev_container = QGroupBox("Camera
biến đổi phối cảnh")
    self.bev_container.setStyleSheet("color:
#ecf0f1; font-weight: bold;")
    self.bird_label = BirdEyeLabel(roi_rects=[],
parent=self.bev_container)
    be_v_inner = QVBoxLayout()
    be_v_inner.addWidget(self.bird_label)
    self.bev_container.setLayout(be_v_inner)
    left_layout.addWidget(self.bev_container)
    right_widget = QWidget()
    right_layout = QVBoxLayout(right_widget)
    self.saved_widget = SavedPotholeWidget()
    self.roi_table_widget = ROITableWidget()
    right_layout.addWidget(self.saved_widget, 3)
    right_layout.addWidget(self.roi_table_widget,
1)

    splitter = QSplitter(Qt.Horizontal)
    splitter.addWidget(left_widget)
    splitter.addWidget(right_widget)
    splitter.setSizes([500, 400])
    be_v_layout.addWidget(splitter)
    self.tab_widget.addTab(self.bev_tab,
"Camera")
    # Data Tab
    self.data_tab = DataTab(self.db_path,
self.location_mgr)
    self.tab_widget.addTab(self.data_tab, "Dữ
liệu")
    # Map Tab
    self.map_tab = MapTab(self.db_path,
self.location_mgr)
    self.tab_widget.addTab(self.map_tab, "Bản
đồ")
    self.main_layout.addWidget(self.tab_widget,
stretch=3)
    self.main_layout.addWidget(self.meta_panel,
stretch=1)

def _connect_signals(self):
    """Kết nối tất cả signals (chung cho cả hai
trường hợp)"""

```

```

self.processor.metadata_updated.connect(self.meta
_panel.update_metadata)

self.processor.image_processed.connect(self.update
_bev_display)

self.processor.roi1_updated.connect(self.update_roi
_table)

self.processor.pothole_saved.connect(self.saved_wi
dget.update_saved_pothole)

self.processor.pothole_saved.connect(self.on_pothol
e_saved)

self.processor.data_refresh.connect(self.data_tab.lo
ad_data)

self.saved_widget.image_clicked.connect(self.on_sa
ved_image_click)

self.data_tab.image_clicked.connect(self.on_data_i
mage_clicked)

self.meta_panel.pothole_toggle.toggled.connect(sel
f.toggle_pothole_detection)

self.udp_thread.change_pixmap_signal.connect(self
.processor.process_frame)

self.udp_thread.connection_status_signal.connect(s
elf.meta_panel.set_status)

self.udp_thread.connection_status_signal.connect(s
elf.on_connection_status)

def update_rois(self):
    center_y1 = int(self.roi1_center_y_ratio *
self.bev_h)
    half_h1 = int(self.roi1_height_ratio *
self.bev_h / 2)
    self.roi1_rect = (0, max(0, center_y1 -
half_h1), self.bev_w, min(self.bev_h, center_y1 +
half_h1))
    center_y2 = int(self.roi2_center_y_ratio *
self.bev_h)
    half_h2 = int(self.roi2_height_ratio *
self.bev_h / 2)
    self.roi2_rect = (0, max(0, center_y2 -
half_h2), self.bev_w, min(self.bev_h, center_y2 +
half_h2))

```

```

if hasattr(self, 'bird_label'):
    if self.pothole_detection_enabled:
        self.bird_label.roi_rects =
[(self.roi1_rect, QColor(0, 255, 255)),
(self.roi2_rect, QColor(255, 255, 0))]
    else:
        self.bird_label.roi_rects = []
        self.bird_label.update()
if hasattr(self, 'processor'):
    self.processor.set_rois(self.roi1_rect,
self.roi2_rect, self.bev_w, self.bev_h)

def init_database(self):
    conn = sqlite3.connect(self.db_path)
    cursor = conn.cursor()
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS
pothole_records (
            id INTEGER PRIMARY KEY
AUTOINCREMENT,
            track_id INTEGER,
            timestamp TEXT NOT NULL,
            area_cm2 REAL NOT NULL,
            depth_cm REAL NOT NULL,
            volume_cm3 REAL NOT NULL,
            gps_lat REAL NOT NULL,
            gps_lon REAL NOT NULL,
            image_path TEXT NOT NULL
        )
    """)
    cursor.execute("PRAGMA
table_info(pothole_records)")
    columns = [col[1] for col in cursor.fetchall()]
    if 'volume_cm3' not in columns:
        cursor.execute("ALTER TABLE
pothole_records ADD COLUMN volume_cm3
REAL DEFAULT 0")
    conn.commit()
    conn.close()

def load_data(self):
    self.data_tab.load_data()
    self.map_tab.refresh_map()

def toggle_pothole_detection(self, enabled):
    self.pothole_detection_enabled = enabled
    self.processor.set_pothole_detection(enabled)

if hasattr(self, 'bird_label'):
    if not enabled:
        self.bird_label.roi_rects = []

```

```

self.saved_widget.image_label.setText("Tất phát
hiện")
    self.saved_widget.info_label.clear()
    self.meta_panel.update_pothole_stats(0,
0.0)
    self.meta_panel.lbl_pothole.setText("0.0
cm")
    else:
        self.bird_label.roi_rects =
[(self.roi1_rect, QColor(0, 255, 255)),
        (self.roi2_rect,
QColor(255, 255, 0))]

self.saved_widget.image_label.setText("Chưa có ổ
gà nào được lưu")
    self.bird_label.update()

def update_bev_display(self, image,
active_tracks):
    self.frame_count += 1
    now = time.time()
    if now - self.last_time >= 1.0:
        self.meta_panel.set_fps(self.frame_count)
        self.frame_count, self.last_time = 0, now
        pixmap = self.convert_cv_qt(image)
        self.bird_label.setPixmap(pixmap)

def update_roi_table(self, roi1_tracks_data):
    count, total_area =
self.roi_table_widget.update_data(roi1_tracks_data
)
    self.meta_panel.update_pothole_stats(count,
total_area)

def convert_cv_qt(self, cv_img):
    rgb = cv2.cvtColor(cv_img,
cv2.COLOR_BGR2RGB)
    h, w, ch = rgb.shape
    return QPixmap.fromImage(QImage(rgb.data,
w, h, ch*w, QImage.Format_RGB888))

def on_saved_image_click(self, lat, lon,
track_id):
    self.tab_widget.setCurrentIndex(2) # Map tab
    self.map_tab.focus_on_pothole(lat, lon)

def on_data_image_clicked(self, db_id, lat, lon):
    self.map_tab.fill_panel_by_db_id(db_id)
    self.tab_widget.setCurrentIndex(2)
    self.map_tab.focus_on_pothole(lat, lon)

```

```

def on_pothole_saved(self, pothole_id):
    row = self.db.get_pothole_by_id(pothole_id)
    if row:
        lat = row[5]
        lon = row[6]
        self.map_tab.add_new_pothole(pothole_id,
lat, lon)

```

```

def on_connection_status(self, status):
    if status == "Disconnected":
        self.meta_panel.reset_all_display()
        self.frame_count = 0
        self.last_time = time.time()
        self.meta_panel.set_fps(0)
        if hasattr(self.roi_table_widget, 'clear'):
            self.roi_table_widget.clear()

```

```

self.saved_widget.image_label.setText("Chưa có ổ
gà nào được lưu")
    self.saved_widget.info_label.clear()

```

```

def closeEvent(self, event):
    self.udp_thread.stop()
    self.processor_thread.quit()
    self.processor_thread.wait()
    event.accept()

```

splash.py

```

import sys
import os
import sqlite3
import traceback
from PyQt5.QtWidgets import QDialog,
QVBoxLayout, QLabel, QProgressBar,
QApplication
from PyQt5.QtCore import Qt, QThread,
pyqtSignal, QObject
from PyQt5.QtGui import QPixmap
# Import các module của dự án
from threads.udp_thread import UDPVideoThread
from threads.processor_thread import
PotholeProcessor
from database.database import DatabaseManager
from location_manager import LocationManager

class SplashScreen(QDialog):
    """Cửa sổ splash hiển thị logo với thanh tiến
trình và phần trăm nằm đè bên trong ảnh"""
    def __init__(self, image_path):
        super().__init__()

self.setWindowFlags(Qt.FramelessWindowHint)
self.setModal(False)

```

```

self.setAttribute(Qt.WA_TranslucentBackground)
    main_layout = QVBoxLayout()
    main_layout.setContentsMargins(0, 0, 0, 0)
    main_layout.setSpacing(0)
    self.setLayout(main_layout)
    self.image_label = QLabel()
    pixmap = QPixmap(image_path)
    self.img_w, self.img_h = 800, 450
    if not pixmap.isNull():
        pixmap = pixmap.scaled(self.img_w,
self.img_h, Qt.IgnoreAspectRatio,
Qt.SmoothTransformation)
    else:
        pixmap = QPixmap(self.img_w, self.img_h)
        pixmap.fill(Qt.gray)
    self.image_label.setPixmap(pixmap)
    main_layout.addWidget(self.image_label)
    self.progress_bar =
QProgressBar(self.image_label)
    self.progress_bar.setRange(0, 100)
    self.progress_bar.setValue(0)
    self.pb_height = 10

self.progress_bar.setFixedHeight(self.pb_height)
self.progress_bar.setFixedWidth(self.img_w)
self.progress_bar.move(0, self.img_h -
self.pb_height)
    self.progress_bar.setStyleSheet(f"""
QProgressBar {{
    border: none;
    background-color: rgba(255, 255, 255,
30);

    text-align: center;
    color: transparent;
}}
QProgressBar::chunk {{
    background-color: #1abc9c;
}}
""")
    self.percent_label = QLabel("0%",
self.image_label)
    self.percent_label.setStyleSheet("""
color: white;
font-weight: bold;
font-size: 13px;
background: rgba(0, 0, 0, 160);
padding: 2px 12px;
border-radius: 5px;
""")
    self.update_label_position()
    self.setFixedSize(self.img_w, self.img_h)

```

```

self.center_on_screen()

def center_on_screen(self):
    """Căn giữa cửa sổ trên màn hình (Sửa lỗi
AttributeError)"""
    screen =
QApplication.primaryScreen().availableGeometry()
    self.move((screen.width() - self.width()) // 2,
(screen.height() - self.height()) // 2)

def update_label_position(self):
    """Tính toán tọa độ để đưa nhãn phần trăm ra
CHÍNH GIỮA chiều ngang"""
    self.percent_label.adjustSize()
    x = (self.img_w - self.percent_label.width()) //
2
    y = self.img_h - self.pb_height -
self.percent_label.height() - 5
    self.percent_label.move(x, y)

def update_progress(self, value):
    """Cập nhật giá trị tiến trình từ thread khởi
tạo"""
    self.progress_bar.setValue(value)
    self.percent_label.setText(f"{value}%")
    self.update_label_position()
    QApplication.processEvents()

class AppInitializer(QObject):
    """Chạy các tác vụ khởi tạo nặng trong thread
riêng, emit tiến trình"""
    progress = pyqtSignal(int)
    finished = pyqtSignal(dict)

    def run(self):
        try:
            # Bước 1: Database
            self.progress.emit(5)
            db_path = os.path.join("database",
"potholes.db")
            os.makedirs("database", exist_ok=True)
            self._init_database_schema(db_path)
            self.progress.emit(15)
            # Bước 2: Thư mục ảnh
            images_dir = os.path.abspath("OGa")
            os.makedirs(images_dir, exist_ok=True)
            self.progress.emit(25)
            # Bước 3: Location Manager
            geojson_base = os.path.join("database",
"geojson")
            location_mgr = None
            if os.path.exists(geojson_base):

```

```

        location_mgr =
LocationManager(geojson_base)
        self.progress.emit(35)
        # Bước 4: Cấu hình BEV/ROI
        bev_w, bev_h = 400, 600
        pixel_to_cm2 = (60.0 / 400.0) ** 2
        roi1_ratio, roi1_h_ratio = 0.42, 0.2
        roi2_ratio, roi2_h_ratio = 0.82, 0.3
        roi1_rect = (0, max(0, int(roi1_ratio *
bev_h) - int(roi1_h_ratio * bev_h / 2)),
                    bev_w, min(bev_h, int(roi1_ratio *
bev_h) + int(roi1_h_ratio * bev_h / 2)))
        roi2_rect = (0, max(0, int(roi2_ratio *
bev_h) - int(roi2_h_ratio * bev_h / 2)),
                    bev_w, min(bev_h, int(roi2_ratio *
bev_h) + int(roi2_h_ratio * bev_h / 2)))
        self.progress.emit(45)
        # Bước 5: Khởi tạo AI Processor
        processor_thread = QThread()
        processor =
PotholeProcessor(pixel_to_cm2, db_path,
images_dir)
        processor.moveToThread(processor_thread)
        processor_thread.start()
        self.progress.emit(55)
        processor.init_models(
            os.path.join("model", "epoch62.pt"),
            os.path.join("model", "model_249.pth")
        )
        self.progress.emit(85)
        processor.set_rois(roi1_rect, roi2_rect,
bev_w, bev_h)
        # Bước 6: UDP Video
        udp_thread = UDPVideoThread(5557)
        self.progress.emit(100)
        # Đóng gói dữ liệu trả về cho MainWindow
        result = {
            'db_path': db_path, 'images_dir':
images_dir, 'location_mgr': location_mgr,
            'processor': processor,
            'processor_thread': processor_thread, 'udp_thread':
udp_thread,
            'bev_w': bev_w, 'bev_h': bev_h,
            'pixel_to_cm2': pixel_to_cm2,
            'roi1_rect': roi1_rect, 'roi2_rect':
roi2_rect,
            'roi1_center_y_ratio': roi1_ratio,
            'roi1_height_ratio': roi1_h_ratio,
            'roi2_center_y_ratio': roi2_ratio,
            'roi2_height_ratio': roi2_h_ratio,
        }
        self.finished.emit(result)

```

```

except Exception:
    traceback.print_exc()
    self.finished.emit({})

def _init_database_schema(self, db_path):
    conn = sqlite3.connect(db_path)
    cursor = conn.cursor()
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS
pothole_records (
            id INTEGER PRIMARY KEY
AUTOINCREMENT,
            track_id INTEGER,
            timestamp TEXT NOT NULL,
            area_cm2 REAL NOT NULL,
            depth_cm REAL NOT NULL,
            volume_cm3 REAL NOT NULL,
            gps_lat REAL NOT NULL,
            gps_lon REAL NOT NULL,
            image_path TEXT NOT NULL
        )
""")
    cursor.execute("PRAGMA
table_info(pothole_records)")
    cols = [col[1] for col in cursor.fetchall()]
    if 'volume_cm3' not in cols:
        cursor.execute("ALTER TABLE
pothole_records ADD COLUMN volume_cm3
REAL DEFAULT 0")
    conn.commit()
    conn.close()

main.py
import sys
import os
import traceback
from PyQt5.QtWidgets import QApplication
from PyQt5.QtCore import QThread
from splash import SplashScreen, AppInitializer
from main_window import MainWindow
main_window = None

def on_init_finished(init_data, splash, init_thread,
app):
    global main_window
    init_thread.quit()
    init_thread.wait()
    try:
        main_window =
MainWindow(init_data=init_data)
        main_window.showMaximized()
        splash.close()
        splash.deleteLater()

```

```

except Exception as e:
    traceback.print_exc()
    splash.close()

if __name__ == "__main__":
    app = QApplication(sys.argv)
    logo_path = os.path.join("database",
"logo_start.png")
    if not os.path.exists(logo_path):
        logo_path =
r"C:\Users\dung2\Desktop\GiaoDienOGa\database
\logo_start.png"
    splash = SplashScreen(logo_path)
    splash.show()
    init_thread = QThread()
    initializer = AppInitializer()
    initializer.moveToThread(init_thread)

    initializer.progress.connect(splash.update_progress
)
    initializer.finished.connect(lambda data:
on_init_finished(data, splash, init_thread, app))
    init_thread.started.connect(initializer.run)
    init_thread.start()
    sys.exit(app.exec_())

```